

From Platonic Solids to Regular 4-Polytopes: A Topological and Coxeter-Theoretic Classification with Lean 4 Certificate generated by LLM

Lars Warren Ericson¹

¹ORCID: 0000-0001-8299-9361

¹Catskills Research Company

¹lars.ericson@catskillsresearch.com

September 14, 2026

Github: <https://github.com/catskillsresearch/platonic-solids>

Abstract

We present an expository and formally verified classification of the regular convex polyhedra in $\mathbb{R}^3$ (the Platonic solids) and the regular convex polychora in $\mathbb{R}^4$, formalized in **Lean 4** against Mathlib. In dimension 3, Euler's formula $V - E + F = 2$ together with regularity incidence $pF = 2E$, $qV = 2E$ yields the Diophantine inequality $(p - 2)(q - 2) < 4$ and exactly five solutions; the same identities algebraically force $E > 0$, so non-degeneracy is a theorem rather than an extra axiom. The compared 3D theorems take $V + F = E + 2$ as a hypothesis. The development also proves algebraic Euler–Poincaré, the radial-projection homeomorphism, cellular Betti numbers of the tetrahedron surface ($\chi = 2$) and of the 4-simplex boundary ($\chi = 0$), the open-cover quasi-isomorphism in **Platonic Solids/SingularExcision/**, and singular $H_n(S^n; R) \cong R$ for $n > 0$. A chain-level bridge identifies the matrix Betti definitions with categorical homology and proves that every finite 2-complex quasi-isomorphic to singular chains of a space homotopy-equivalent to S^2 has Betti profile $(1, 0, 1)$, hence satisfies the homological Platonic classification. A fourth Palomar-compared theorem, **simplicialSingularComparison_boundaryTwo_quasiIso**, constructs the canonical realization–singular adjunction-unit chain map on the simplicial circle $\partial\Delta[2]$ and proves it is a quasi-isomorphism by comparing the simplicial Mayer–Vietoris sequence with an open-cover sequence of punctured-circle neighborhoods. This is a specific finite gluing example, not a general comparison theorem for arbitrary simplicial or polyhedral cellulations. In dimension 4, $\chi(S^3) = 0$ yields no flag bound. The compared 4D theorem says that a Platonic-cell / Platonic-vertex-figure triple with $\Delta > 0$ is one of the six regular convex 4-polytopes; the development also proves the converse and that those six are exactly the triples with $p, q, r \geq 3$ whose Gram matrix is positive definite (**regular_polychora_iff**). The classification uses the rank-4 Schläfli Gram matrix and Sylvester's criterion; it does not construct Coxeter groups as Lean objects. There is no **sorry** outside the deliberate holes in **Challenge.lean** and no project-defined axiom; the compared theorems may depend on Mathlib's **propext**, **Classical.choice**, and **Quot.sound**. Companion artifact: **Platonic Solids.lean** (standalone Lake package in this directory).

Contents

1 1. Introduction: The Dimensional Hierarchy

4

2	2. Dimension 3: Topology, Incidence, and the Non-Degeneracy Insight	4
2.1	2.1. Topological Derivation of Euler’s Formula	6
2.1.1	2.1.1 The canonical comparison on the simplicial circle	7
2.2	2.2. Incidence Combinatorics	9
2.3	2.3. The Diophantine Reduction	9
2.4	2.4. Formalization Insight: Elimination of the Non-Degeneracy Axiom	10
3	3. Dimension 4: Why Topology Fails and the Gram Matrix Takes Over	10
3.1	3.1. The Topological Barrier: $\chi(S^3) = 0$	12
3.2	3.2. Coxeter Systems and the Schläfli Gram Matrix	12
3.3	3.3. Sylvester’s Criterion and Principal Minors	12
3.4	3.4. The Critical Dimension-4 Minor: D_4	13
4	4. Evaluation of the 11 Candidate Triples	13
4.1	4.1 Group I: Spherical Polychora ($\Delta > 0$)	14
4.2	4.2 Group II: Euclidean Honeycomb ($\Delta = 0$)	14
4.3	4.3 Group III: Hyperbolic Honeycombs ($\Delta < 0$)	14
5	5. Complete Lean 4 Formal Verification	15
6	6. Pedagogical and Formalization Significance	17
7	7. Acknowledgements	18
7.1	7.1 Declaration of Generative AI and AI-Assisted Technologies	18
8	8. References	18
A	A. Complete Lean source	19
A.1	A.1 PlatonicSolids.lean	19
A.2	A.2 PlatonicSolids/RadialProjection.lean	20
A.3	A.3 PlatonicSolids/EulerPoincare.lean	20
A.4	A.4 PlatonicSolids/CellularHomology.lean	22
A.5	A.5 PlatonicSolids/SingularHomology.lean	26
A.6	A.6 PlatonicSolids/RelativeHomology.lean	29
A.7	A.7 PlatonicSolids/MayerVietoris.lean	32
A.8	A.8 PlatonicSolids/SingularExcision.lean	37
A.9	A.9 PlatonicSolids/SingularExcision/Algebra.lean	37
A.10	A.10 PlatonicSolids/SingularExcision/AffineSubdivision.lean	40
A.11	A.11 PlatonicSolids/SingularExcision/AffineRealization.lean	53
A.12	A.12 PlatonicSolids/SingularExcision/Mesh.lean	58
A.13	A.13 PlatonicSolids/SingularExcision/BarycentricGeometry.lean	60
A.14	A.14 PlatonicSolids/SingularExcision/SingularChains.lean	66
A.15	A.15 PlatonicSolids/SingularExcision/ModuleSupport.lean	73
A.16	A.16 PlatonicSolids/SingularExcision/Subcomplexes.lean	74
A.17	A.17 PlatonicSolids/SingularExcision/DegreewisePushout.lean	77
A.18	A.18 PlatonicSolids/SingularExcision/CokernelPushout.lean	79
A.19	A.19 PlatonicSolids/SingularExcision/CoverChains.lean	81
A.20	A.20 PlatonicSolids/SingularExcision/SmallChains.lean	83
A.21	A.21 PlatonicSolids/SingularExcision/OpenCoverQI.lean	100
A.22	A.22 PlatonicSolids/SimplicialSingularComparison.lean	109
A.23	A.23 PlatonicSolids/SimplicialSingularBoundaryTwo.lean	157
A.24	A.24 PlatonicSolids/SphereSingularHomology.lean	182
A.25	A.25 PlatonicSolids/Sphere2Homology.lean	197

A.26 PlatonicSolids/Sphere3Homology.lean	199
A.27 PlatonicSolids/EulerBetti.lean	202
A.28 PlatonicSolids/PlatonicPair.lean	203
A.29 PlatonicSolids/Reciprocal.lean	203
A.30 PlatonicSolids/Incidence.lean	204
A.31 PlatonicSolids/Angles.lean	206
A.32 PlatonicSolids/GramMatrix.lean	206
A.33 PlatonicSolids/LeadingMinors.lean	207
A.34 PlatonicSolids/TrailingMinor.lean	207
A.35 PlatonicSolids/D3IffPlatonic.lean	208
A.36 PlatonicSolids/QuadForm.lean	209
A.37 PlatonicSolids/CompletingSquare.lean	209
A.38 PlatonicSolids/DetFormula.lean	210
A.39 PlatonicSolids/Sylvester.lean	211
A.40 PlatonicSolids/TrigValues.lean	213
A.41 PlatonicSolids/CompatibleTriples.lean	214
A.42 PlatonicSolids/SignTable.lean	215
A.43 PlatonicSolids/Regular4.lean	217
A.44 PlatonicSolids/Classification.lean	218
A.45 Challenge.lean	219
A.46 Solution.lean	222

1 1. Introduction: The Dimensional Hierarchy

In Euclidean geometry, a regular polytope is a figure whose symmetry group acts transitively on its flags (incident chains of vertices, edges, faces, and higher-dimensional cells).

- In **dimension 2**, there exist infinitely many regular polygons $\{p\}$ for each $p \geq 3$.
- In **dimension 3**, Euclid's *Elements* (Book XIII) establishes that there exist exactly **5 Platonic solids**.
- In **dimension 4**, Ludwig Schläfli proved in 1852 that there exist exactly **6 regular convex 4-polytopes** (polychora).
- In **dimension $n \geq 5$** , the classification collapses permanently to **3 families**: the regular simplex, the hypercube (measure polytope), and the cross-polytope.

This distribution raises foundational mathematical and pedagogical questions: **Why does dimension 3 permit 5 shapes, dimension 4 permit 6 (admitting the exceptional self-dual 24-cell), and dimension $n \geq 5$ permit only 3? Why does topology suffice to bound the classification in 3D, while failing completely in 4D?**

This paper addresses these questions through an analytical exposition coupled with a fully verified formal certificate in Lean 4. Figure 1 is the coarsest dependency blueprint: the compared 3D theorems assume Euler $V + F = E + 2$ and regularity incidence, the sign of the Schläfli determinant Δ bounds the compared 4D census, and a fourth compared theorem records the canonical simplicial-to-singular comparison for $\partial\Delta[2]$.

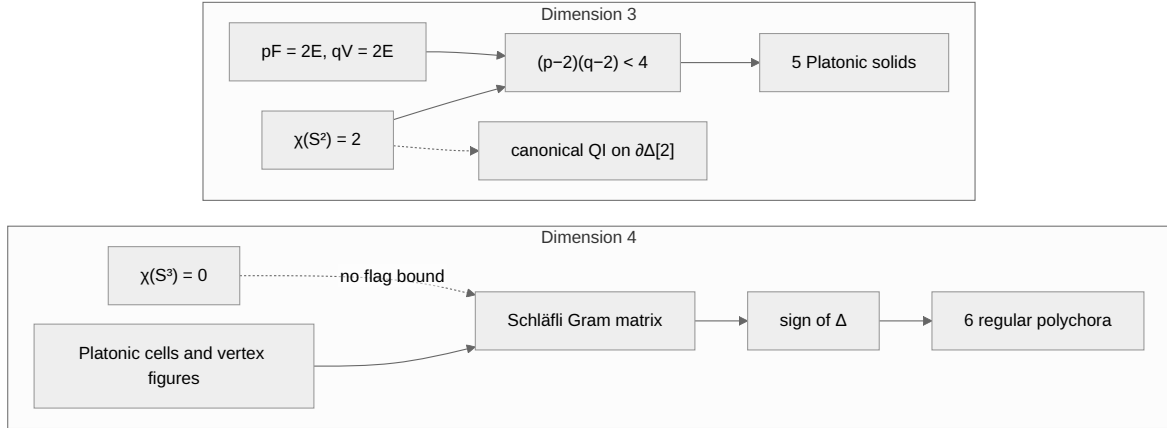


Figure 1: Coarse blueprint of the two classifications. In dimension 3, the compared theorems assume $V + F = E + 2$ and regularity incidence, which force $(p - 2)(q - 2) < 4$ and the five Platonic solids. A fourth compared theorem proves the canonical comparison on $\partial\Delta[2]$; a general cellulation comparison is not claimed. In dimension 4, $\chi(S^3) = 0$ supplies no flag bound; the compared theorem sends Platonic-compatible triples with $\Delta > 0$ to one of the six regular polychora.

2 2. Dimension 3: Topology, Incidence, and the Non-Degeneracy Insight

Figure 2 expands the 3D side of Figure 1 to the Lean lemmas that turn a convex polyhedron into a Platonic pair.

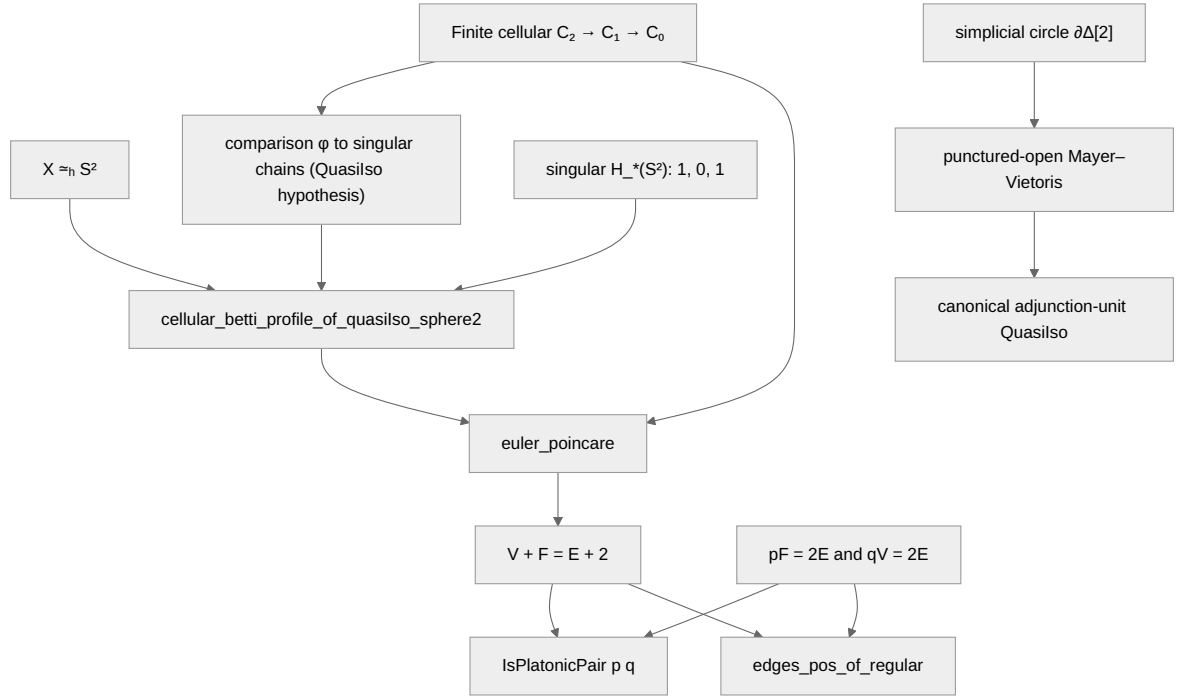


Figure 2: Dimension-3 pipeline. A quasi-isomorphism from a finite cellular complex to singular chains of a model of S^2 yields the Betti profile $(1, 0, 1)$ and hence $V + F = E + 2$. The canonical comparison is now constructed for the specific simplicial circle $\partial\Delta[2]$; arbitrary polyhedral cellulations remain separate. The compared 3D theorem continues to take Euler's formula as a hypothesis.

2.1 2.1. Topological Derivation of Euler's Formula

The Lean gists for §2.1 are the modules under `PlatonicSolids/`: radial projection and Euler–Poincaré (`RadialProjection.lean`, `EulerPoincare.lean`); cellular and singular homology (`CellularHomology.lean`, `SingularHomology.lean`, `RelativeHomology.lean`, `MayerVietoris.lean`); the excision stack (`SingularExcision/`); the simplicial–singular comparison (`SimplicialSingularComparison.lean`, `SimplicialSingularBoundaryTwo.lean`); and the sphere calculations (`SphereSingularHomology.lean`, `Sphere2Homology.lean`, `EulerBetti.lean`). Figure 5 lists every file.

Let $P \subset \mathbb{R}^3$ be a convex polyhedron containing the origin in its interior. Radial projection $\pi : \partial P \rightarrow S^2$ onto the circumscribed 2-sphere is a homeomorphism (`frontier_convex_homeo_sphere`). This projection induces a finite CW-complex structure on S^2 with V 0-cells (vertices), E 1-cells (edges), and F 2-cells (faces).

Mathlib already has the singular homology functor and its homotopy invariance. We package the missing transport lemmas: a homotopy equivalence (or homeomorphism) of spaces induces an isomorphism of singular homology (`singularHomologyIso_of_homotopyEquiv`), a contractible space has vanishing positive homology (`isZero_singularHomology_of_contractible`), and a closed disk is contractible. For the metric sphere itself, path-connectedness in dimension $n \geq 1$ gives

$$H_0(S^n; R) \cong R$$

(`singularHomology0_sphere`). The 0-sphere is the two-point set $\{\pm 1\}$, which is totally disconnected, so $H_0(S^0; R) \cong R \oplus R$ and $H_n(S^0; R) = 0$ for $n > 0$.

A subspace inclusion $A \hookrightarrow X$ is a monomorphism in `TopCat`. The singular chain-complex functor preserves monomorphisms, so

$$0 \rightarrow C_*(A; R) \rightarrow C_*(X; R) \rightarrow C_*(X, A; R) \rightarrow 0$$

is short exact (`pair_shortExact`). Snake lemma supplies the long exact sequence of the pair: connecting $\delta : H_{n+1}(X, A) \rightarrow H_n(A)$ (`pairδ`) and exactness at $H_n(A)$, $H_n(X)$, and $H_n(X, A)$ (`pair_exact_at_A`, `pair_exact_at_X`, `pair_exact_at_rel`).

For a commuting square of spaces $W \rightarrow U, V \rightarrow X$, the same argument on $C_*(W) \rightarrow C_*(U) \oplus C_*(V)$ yields the triad sequence (`triad_shortExact`, `triadδ`). The square induces a comparison from the cover chains to $C_*(X)$; the square is *excisive* when that comparison is a quasi-isomorphism (`IsExcisive`). After excision, the connecting map is Mayer–Vietoris for X (`mayerVietorisδ`).

The stereographic cover of S^n is the standard pair of charts $U = S^n \setminus \{N\}$, $V = S^n \setminus \{S\}$. Each is homeomorphic to a hyperplane (`homeoPuncturedNorth`, `homeoPuncturedSouth`), hence contractible, so $H_k(U) = H_k(V) = 0$ for $k > 0$. They cover S^n (`punctured_cover`) and meet in $S^n \setminus \{N, S\}$. Stereographic projection from the north pole sends the south pole to the origin, so $U \cap V$ is homeomorphic to the punctured hyperplane and therefore homotopy-equivalent to S^{n-1} (`homotopyEquiv_puncturedInter_sphere`).

The open-cover quasi-isomorphism is proved in `PlatonicSolids/SingularExcision/`, not imported from Mathlib. Finite ordered affine chains carry a cone-point barycentric subdivision `subdivide` and prism `prism` with $\partial \text{Sd} = \text{Sd} \partial$ and $\partial T + T \partial = \text{id} - \text{Sd}$ (`boundary_subdivide`, `boundary_prism`). Realization of an affine simplex in Δ^q is functorial on faces (`realize_face`) and on vertex-wise maps (`realize_map`), so the generators assemble to a morphism of singular chain complexes `singularSubdivision` (`ι_singularSubdivisionHom`, `singularSubdivideGenerator_d`). The prism identity is a chain homotopy $\text{id} \simeq \text{Sd}$ (`singularPrismHomotopy`). Cover chains of a two-set open cover are identified with the small-simplex subcomplex (`coverChainsIsoSmall`). Mesh shrinking is proved: each barycentric piece has diameter at most $n/(n+1)$ times its parent (`barycentricMeshRatio`, `diam_range_subdivideStd_le`), so iterated subdivision plus the Lebesgue-number lemma (`exists_subdivideStd_iterate_image_subset_left_`

`or_right`) puts every finite `ModuleCat` chain into the small-chain subgroup (`eventual_small_of_chain`). The algebraic deformation retract on that subgroup is instantiated (`small_cycles_and_boundaries_singular`). The small-subcomplex inclusion is a quasi-isomorphism (`smallι_quasiIso`), so `coverComparison` is a quasi-isomorphism and every two-set open cover is excisive (`IsExcisiveSubspaces_of_openCover`). The stereographic pair is therefore excisive; after the charts vanish in positive degree, Mayer–Vietoris supplies $H_{n+1}(S^{n+1}) \cong H_n(U \cap V) \cong H_n(S^n)$ for $n \geq 1$ (`singularHomology_sphere_succ_iso`). Degree-zero Mayer–Vietoris on S^1 identifies $H_1(S^1; R)$ with $\ker \varepsilon_{S^0} \cong R$ (`singularHomology_sphere1`), so $H_n(S^n; R) \cong R$ for every $n > 0$ (`singularHomology_sphere`) and vanishes in the other positive degrees (`isZero_singularHomology_sphere`).

The compared 3D classification still takes $V + F = E + 2$ as a hypothesis. The optional theorem `cellular_betti_profile_of_quasiIso_sphere2` instead starts with a chain map from a finite 2-complex to the singular chains of a space $X \simeq_h S^2$ and a proof that this map is a quasi-isomorphism. `CellularHomology.lean` identifies categorical homology finranks with `betti0`, `betti1`, and `betti2`; homotopy invariance and the three singular S^2 calculations then derive the Betti profile $(1, 0, 1)$. The corollary `platonic_solids_3d_of_sphere` supplies Euler–Poincaré and the incidence argument. Thus the S^2 bridge no longer assumes the three Betti numbers, although constructing that comparison from a concrete polyhedral cellulation remains an explicit input. The original three Palomar types stay combinatorial.

A fourth compared theorem records the canonical adjunction-unit comparison for the specific simplicial circle $\partial\Delta[2]$ (Palomar name: `simplicialSingularComparison_boundaryTwo_quasiIso`). Figure 3 records the singular-homology dependency stack.

2.1.1 The canonical comparison on the simplicial circle

For a simplicial set X , the chain map `simplicialSingularComparison` (applied at R and X) is obtained by applying simplicial chains to the unit

$$X \longrightarrow \text{Sing}(|X|)$$

of the geometric-realization/singular-complex adjunction. `SimplicialSingularComparison.lean` proves the comparison for standard simplices, the horn $\Lambda[2, 0]$, and the disconnected two-vertex edge–horn intersection.

The boundary geometry is made concrete by the homeomorphism `homeomorph_boundaryTwo_addCircle`,

$$|\partial\Delta[2]| \simeq \text{AddCircle}(3).$$

The obvious decomposition into one closed edge and the complementary two-edge horn gives a short exact sequence on simplicial chains, but the analogous sequence of all singular chains on the two closed arcs is not short exact: a singular simplex in the circle need not lie wholly in either closed arc.

`SimplicialSingularBoundaryTwo.lean` enlarges the arcs to two punctured-circle open neighborhoods. Each neighborhood deformation retracts to its edge or horn, while their intersection deformation retracts to the two shared vertices. These three equivalences give quasi-isomorphisms on singular chains. The source simplicial Mayer–Vietoris sequence is then mapped to the target open-cover short exact sequence; two-out-of-three proves the induced map to cover chains is a quasi-isomorphism, and open-cover excision identifies cover chains with the singular chains of the whole circle. The resulting compared theorem is as follows.

Compared theorem (`simplicialSingularComparison_boundaryTwo_quasiIso`). For any coefficient module R , the chain map induced by the realization–singular adjunction unit

$$C_*^{\text{simp}}(\partial\Delta[2]; R) \longrightarrow C_*^{\text{sing}}(|\partial\Delta[2]|; R)$$

is a quasi-isomorphism.

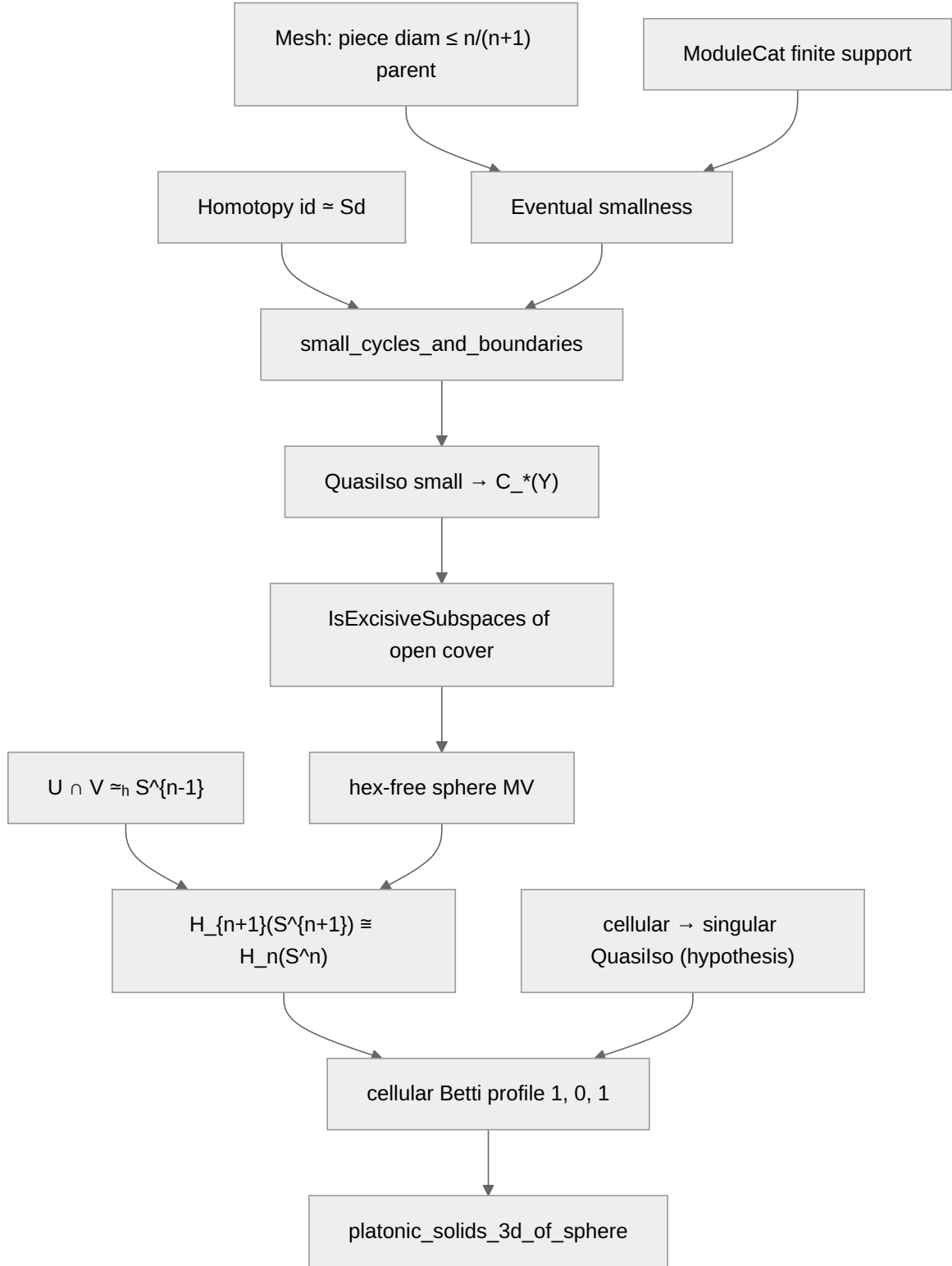


Figure 3: Singular-excision blueprint, finer than Figure 2. Mesh shrinking and the prism homotopy feed the small-chain retract; the open-cover quasi-isomorphism discharges **IsExcisiveSubspaces** and yields $H_{n+1}(S^{n+1}) \cong H_n(S^n)$.

The normalized-chain corollary is supporting material and is not compared. No theorem for arbitrary finite nonsingular simplicial sets or arbitrary polyhedral cellulations is claimed.

The standard simplicial 2-sphere is the surface of a tetrahedron: 4 vertices, 6 edges, 4 triangles. Oriented incidence matrices $d_1 : \mathbb{Q}^6 \rightarrow \mathbb{Q}^4$ and $d_2 : \mathbb{Q}^4 \rightarrow \mathbb{Q}^6$ satisfy $d_1 d_2 = 0$. Each has a 3×3 identity minor, so both ranks are at least 3; the product-zero rank inequality on a 6-dimensional middle term forces both ranks equal to 3. Hence

$$b_0 = 4 - 3 = 1, \quad b_1 = 3 - 3 = 0, \quad b_2 = 4 - 3 = 1$$

(`homology_sphere2`). The Euler characteristic is the alternating sum of Betti numbers:

$$\chi(S^2) = b_0 - b_1 + b_2 = 1 - 0 + 1 = 2$$

Algebraic Euler–Poincaré (`euler_poincare`) is rank-nullity on any 2-complex: cell ranks equal Betti ranks. Specializing to these numbers yields Euler’s formula

$$V - E + F = 2$$

on the tetrahedron (`euler_tetrahedron`) and, for an arbitrary 2-complex with the same Betti numbers, the general identity `euler_formula_of_sphere2`. The tetrahedron incidence structure is then Platonic (`tetrahedron_platonic`). Combining Euler–Poincaré with regularity incidence gives the classification from homology (`platonic_solids_3d_of_homology`).

The two routes are now joined at the correct level of abstraction. Given a quasi-isomorphism from this finite cellular chain complex to singular chains, `cellular_betti_profile_of_quasi_iso_sphere2` transports the independently computed singular homology of S^2 back to the matrix Betti numbers. The canonical comparison has been constructed for $\partial\Delta[2]$ as a first finite gluing case; extending it naturally to general finite simplicial cellulations, and then to the polyhedral cellulations needed here, remains open.

2.2 2.2. Incidence Combinatorics

The Lean gist is `PlatonicSolids/Incidence.lean` (with the pair predicate in `PlatonicSolids/PlatonicPair.lean`). A regular polyhedron has Schläfli symbol $\{p, q\}$, meaning: 1. Every 2D face is an identical regular p -gon ($p \geq 3$). Each face contributes p edge-face incidences. Since every edge is shared by exactly 2 faces:

$$pF = 2E \implies F = \frac{2E}{p}$$

2. Every vertex meets q faces and q edges ($q \geq 3$). Each vertex contributes q edge-vertex incidences. Since every edge connects exactly 2 vertices:

$$qV = 2E \implies V = \frac{2E}{q}$$

2.3 2.3. The Diophantine Reduction

The Lean gist is `PlatonicSolids/PlatonicPair.lean`; the reciprocal form $1/p + 1/q > 1/2$ is `PlatonicSolids/Reciprocal.lean`. Substituting these incidence relations into Euler’s formula:

$$\frac{2E}{q} - E + \frac{2E}{p} = 2$$

Multiplying by pq over $\mathbb{N}$ avoids rational arithmetic:

$$(2p + 2q)E = (pq)E + 2pq$$

Rearranging terms:

$$(2p + 2q - pq)E = 2pq$$

Because $2pq > 0$ for $p, q \geq 3$, the factor $(2p + 2q - pq)$ must be strictly positive, which yields:

$$pq < 2p + 2q \iff (p - 2)(q - 2) < 4$$

Since $p \geq 3$ and $q \geq 3$, both $(p - 2)$ and $(q - 2)$ are strictly positive integers. The only pairs of positive integers with product strictly less than 4 are:

$(p - 2)$	$(q - 2)$	$(p - 2)(q - 2)$	Pair (p, q)	Polyhedron
1	1	1	$(3, 3)$	Tetrahedron
1	2	2	$(3, 4)$	Octahedron
2	1	2	$(4, 3)$	Cube (Hexahedron)
1	3	3	$(3, 5)$	Icosahedron
3	1	3	$(5, 3)$	Dodecahedron

2.4 2.4. Formalization Insight: Elimination of the Non-Degeneracy Axiom

The Lean gist is `edges_pos_of_regular` in `PlatonicSolids/Incidence.lean`. In informal presentations, dividing Euler's equation by $2E$ to obtain:

$$\frac{1}{p} + \frac{1}{q} = \frac{1}{2} + \frac{1}{E} > \frac{1}{2}$$

is typically justified by appealing to physical or geometric intuition: “*a solid polyhedron clearly has edges, so $E > 0$.*”

When formalizing this theorem in Lean 4, one might initially expect to require an explicit non-degeneracy hypothesis (`hE : E > 0`). However, machine verification reveals this hypothesis to be mathematically redundant.

Recall the cleared identity over $\mathbb{N}$:

$$(2p + 2q)E = (pq)E + 2pq$$

If $E = 0$, the equation reduces to:

$$0 = 2pq$$

Because $p \geq 3$ and $q \geq 3$, we have $2pq \geq 18$, which contradicts $0 = 2pq$.

Therefore, **the topological Euler characteristic ($\chi = 2$) together with the local regularity conditions ($p, q \geq 3$) algebraically prohibits an edgeless configuration**. The condition $E \geq 1$ is an algebraic consequence of the system, not an extrinsic geometric requirement.

3 3. Dimension 4: Why Topology Fails and the Gram Matrix Takes Over

Figure 4 is the 4D counterpart of Figure 2: Platonic cells and vertex figures determine a Schläfli Gram matrix, and the sign of $\Delta = \det M$ sorts the eleven triples.

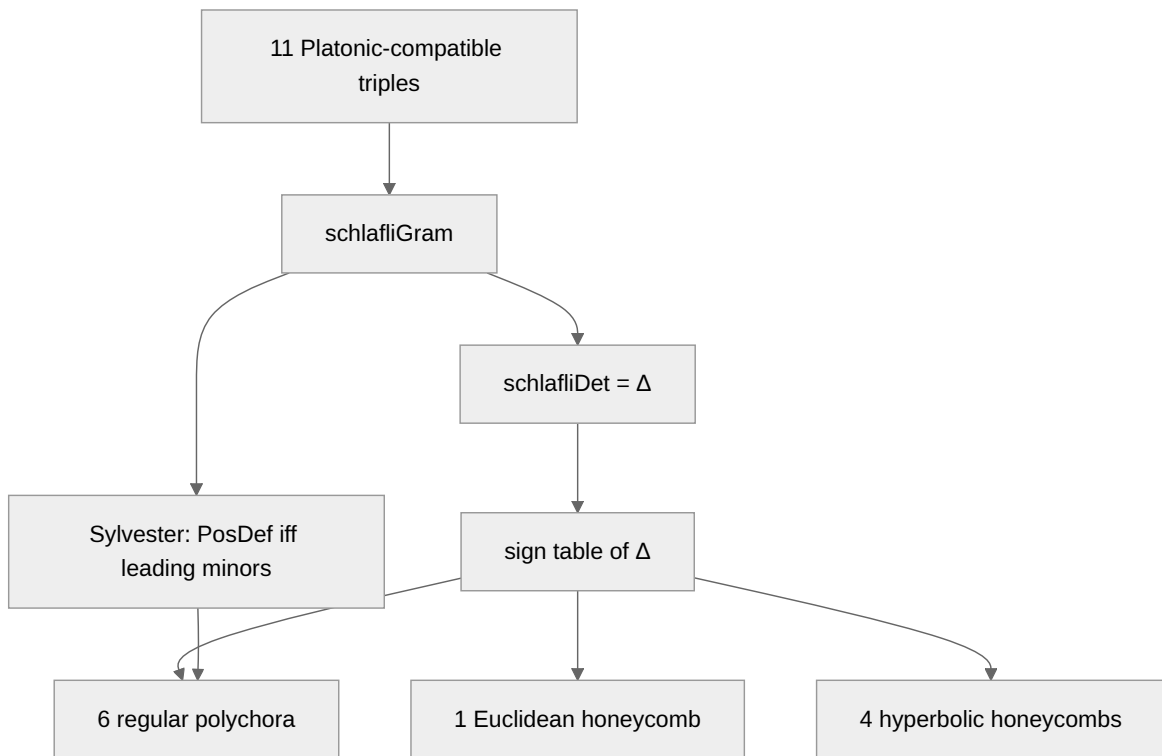


Figure 4: Dimension-4 pipeline. The eleven Platonic-compatible Schläfli triples are classified by the sign of $\Delta = \det(\text{schlafliGram})$. Positive Δ is equivalent, by Sylvester, to positive-definiteness of the Gram matrix, and picks out the six regular convex polychora.

3.1 3.1. The Topological Barrier: $\chi(S^3) = 0$

The Lean gists are `PlatonicSolids/EulerPoincare.lean` (now also the 3-complex identity `euler_poincare3`), `PlatonicSolids/Sphere3Homology.lean` (cellular $H_*(S^3; \mathbb{Q})$ of the 4-simplex boundary), and `eulerChar_sphere3_betti` in `PlatonicSolids/EulerBetti.lean`.

A convex 4-polytope with nonempty interior has frontier homeomorphic to S^3 by the same radial projection as in §2.1. The standard simplicial 3-sphere is the boundary of a 4-simplex: 5 vertices, 10 edges, 10 triangles, 5 tetrahedra. Oriented incidence matrices $d_1 : \mathbb{Q}^{10} \rightarrow \mathbb{Q}^5$, $d_2 : \mathbb{Q}^{10} \rightarrow \mathbb{Q}^{10}$, $d_3 : \mathbb{Q}^5 \rightarrow \mathbb{Q}^{10}$ satisfy $d_1 d_2 = 0$ and $d_2 d_3 = 0$. Identity minors give $\text{rank } d_1 \geq 4$ and $\text{rank } d_3 \geq 4$; a 6×6 minor of d_2 has determinant 1, so $\text{rank } d_2 \geq 6$. The product-zero inequalities on the two middle dimensions of size 10 then force the ranks to be exactly 4, 6, and 4. Hence

$$b_0 = 5 - 4 = 1, \quad b_1 = (10 - 4) - 6 = 0, \quad b_2 = (10 - 6) - 4 = 0, \quad b_3 = 5 - 4 = 1$$

(`homology_sphere3`). The Euler characteristic vanishes:

$$\chi(S^3) = b_0 - b_1 + b_2 - b_3 = 1 - 0 + 0 - 1 = 0$$

Algebraic Euler–Poincaré in dimension 3 (`euler_poincare3`) again equates cell ranks with Betti ranks, so any such cellulation satisfies

$$V - E + F - C = 0$$

(`eulerChar_sphere3` on the 4-simplex, and `euler_formula_of_sphere3` in general). Because the alternating sum is zero, dividing by any flag count yields an identity equal to zero rather than a positive reciprocal like $2/E$. **Topology alone does not impose an upper bound on p, q , or r in 4D.**

3.2 3.2. Coxeter Systems and the Schläfli Gram Matrix

The Lean gist is `PlatonicSolids/GramMatrix.lean`. To obtain an upper bound, informal accounts invoke **Coxeter reflection groups**. The Lean development does not construct those groups; it works with the associated **Schläfli Gram matrix** and its determinant Δ . A regular 4-polytope is described by its Schläfli symbol $\{p, q, r\}$: * The 3D cells are Platonic solids $\{p, q\}$. * The 3D vertex figures are Platonic solids $\{q, r\}$. * r cells meet around every 1D edge.

The Coxeter group W associated with $\{p, q, r\}$ is generated by reflections $\{s_1, s_2, s_3, s_4\}$ across the facet hyperplanes of a spherical fundamental simplex:

$$W = \langle s_1, s_2, s_3, s_4 \mid (s_i s_j)^{m_{ij}} = 1 \rangle$$

where the exponents are $m_{ii} = 1$, $m_{12} = p$, $m_{23} = q$, $m_{34} = r$, and $m_{ij} = 2$ for $|i - j| > 1$.

The canonical bilinear form of this system is represented by the **Schläfli Gram matrix**:

$$M = \begin{pmatrix} 1 & -\cos \frac{\pi}{p} & 0 & 0 \\ -\cos \frac{\pi}{p} & 1 & -\cos \frac{\pi}{q} & 0 \\ 0 & -\cos \frac{\pi}{q} & 1 & -\cos \frac{\pi}{r} \\ 0 & 0 & -\cos \frac{\pi}{r} & 1 \end{pmatrix}$$

3.3 3.3. Sylvester’s Criterion and Principal Minors

The Lean gists are `PlatonicSolids/Angles.lean`, `LeadingMinors.lean`, `TrailingMinor.lean`, `D3IfPlatonic.lean`, `QuadForm.lean`, `CompletingSquare.lean`, and `Sylvester.lean`. By the Cartan–Killing–Coxeter classification: 1. M is **positive definite** ($\det(M) > 0$) $\iff$ W is a **finite reflection group acting on S^3 (a finite regular convex 4-polytope)**. 2. M is

positive semidefinite with nullity 1 ($\det(M) = 0$) $\iff$ W is an affine reflection group (a Euclidean 3D space-filling honeycomb). 3. M has negative eigenvalues ($\det(M) < 0$) $\iff$ W is a hyperbolic reflection group (a honeycomb in hyperbolic space $\mathbb{H}^3$).

By Sylvester's criterion, M is positive definite if and only if all leading principal minors $D_k = \det(M_k)$ are strictly positive: * $D_1 = 1 > 0$. * $D_2 = 1 - \cos^2 \frac{\pi}{p} = \sin^2 \frac{\pi}{p} > 0$ (satisfied for all $p \geq 3$). * $D_3 = \sin^2 \frac{\pi}{p} - \cos^2 \frac{\pi}{q}$.

$$D_3 > 0 \iff \sin \frac{\pi}{p} > \cos \frac{\pi}{q} = \sin \left(\frac{\pi}{2} - \frac{\pi}{q} \right) \iff \frac{1}{p} + \frac{1}{q} > \frac{1}{2}$$

Thus, $D_3 > 0$ requires the cell $\{p, q\}$ to be a Platonic solid. * Symmetrically, the trailing 3×3 minor requires:

$$\frac{1}{q} + \frac{1}{r} > \frac{1}{2}$$

Thus, the vertex figure $\{q, r\}$ must also be a Platonic solid.

3.4 The Critical Dimension-4 Minor: D_4

The Lean gist is `PlatonicSolids/DetFormula.lean`. Expanding $D_4 = \det(M)$ along the fourth row:

$$D_4 = D_3 - \cos^2 \left(\frac{\pi}{r} \right) D_2 = \left(\sin^2 \frac{\pi}{p} - \cos^2 \frac{\pi}{q} \right) - \cos^2 \left(\frac{\pi}{r} \right) \sin^2 \left(\frac{\pi}{p} \right)$$

Factoring $\sin^2 \frac{\pi}{p}$:

$$D_4 = \sin^2 \left(\frac{\pi}{p} \right) \left(1 - \cos^2 \frac{\pi}{r} \right) - \cos^2 \left(\frac{\pi}{q} \right) = \sin^2 \left(\frac{\pi}{p} \right) \sin^2 \left(\frac{\pi}{r} \right) - \cos^2 \left(\frac{\pi}{q} \right)$$

Therefore, the condition that $\{p, q, r\}$ forms a finite regular convex 4-polytope is:

$$\Delta(p, q, r) := \sin^2 \left(\frac{\pi}{p} \right) \sin^2 \left(\frac{\pi}{r} \right) - \cos^2 \left(\frac{\pi}{q} \right) > 0$$

4 Evaluation of the 11 Candidate Triples

The Lean gists are `CompatibleTriples.lean`, `TrigValues.lean`, `SignTable.lean`, `Regular4.lean`, and `Classification.lean` in `PlatonicSolids/`.

Because both $\{p, q\}$ and $\{q, r\}$ must be Platonic, the indices must satisfy $p, q, r \in \{3, 4, 5\}$. Among the $5 \times 5 = 25$ combinations, only **11 triples** have a compatible middle index q .

The exact squared trigonometric values are:

$$\begin{aligned} \cos^2 \frac{\pi}{3} &= \frac{1}{4}, & \sin^2 \frac{\pi}{3} &= \frac{3}{4} \\ \cos^2 \frac{\pi}{4} &= \frac{1}{2}, & \sin^2 \frac{\pi}{4} &= \frac{1}{2} \\ \cos^2 \frac{\pi}{5} &= \frac{3 + \sqrt{5}}{8}, & \sin^2 \frac{\pi}{5} &= \frac{5 - \sqrt{5}}{8} \end{aligned}$$

Evaluating $\Delta(p, q, r)$ across all 11 candidate triples:

4.1 Group I: Spherical Polychora ($\Delta > 0$)

1. $\{3, 3, 3\}$ (5-cell / Simplex):

$$\Delta = \binom{3}{4} \binom{3}{4} - \frac{1}{4} = \frac{9}{16} - \frac{4}{16} = \frac{5}{16} > 0$$

2. $\{3, 3, 4\}$ (16-cell) & $\{4, 3, 3\}$ (8-cell / Tesseract):

$$\Delta = \binom{3}{4} \binom{1}{2} - \frac{1}{4} = \frac{3}{8} - \frac{2}{8} = \frac{1}{8} > 0$$

3. $\{3, 4, 3\}$ (24-cell — Exceptional Self-Dual Polytope):

$$\Delta = \binom{3}{4} \binom{3}{4} - \frac{1}{2} = \frac{9}{16} - \frac{8}{16} = \frac{1}{16} > 0$$

4. $\{3, 3, 5\}$ (600-cell) & $\{5, 3, 3\}$ (120-cell):

$$\Delta = \binom{3}{4} \binom{5 - \sqrt{5}}{8} - \frac{1}{4} = \frac{7 - 3\sqrt{5}}{32}$$

Since $7^2 = 49 > (3\sqrt{5})^2 = 45$, we have $7 - 3\sqrt{5} > 0$, so $\Delta > 0$.

4.2 Group II: Euclidean Honeycomb ($\Delta = 0$)

5. $\{4, 3, 4\}$ (Cubic Honeycomb of $\mathbb{R}^3$):

$$\Delta = \binom{1}{2} \binom{1}{2} - \frac{1}{4} = \frac{1}{4} - \frac{1}{4} = 0 \quad (\text{Affine / flat space-filling})$$

4.3 Group III: Hyperbolic Honeycombs ($\Delta < 0$)

6. $\{3, 5, 3\}$ (Icosacell Honeycomb):

$$\Delta = \binom{3}{4} \binom{3}{4} - \frac{3 + \sqrt{5}}{8} = \frac{3 - 2\sqrt{5}}{16} < 0 \quad (\text{since } 3^2 = 9 < 20)$$

7. $\{4, 3, 5\}$ & $\{5, 3, 4\}$:

$$\Delta = \binom{1}{2} \binom{5 - \sqrt{5}}{8} - \frac{1}{4} = \frac{1 - \sqrt{5}}{16} < 0$$

8. $\{5, 3, 5\}$:

$$\Delta = \left(\frac{5 - \sqrt{5}}{8} \right)^2 - \frac{1}{4} = \frac{14 - 10\sqrt{5}}{64} < 0 \quad (\text{since } 14^2 = 196 < 500)$$

Thus, precisely 6 triples yield positive-definite Gram matrices.

5. Complete Lean 4 Formal Verification

The Lake package compiles against Lean 4 and Mathlib. It contains no project-defined axiom, no `admit`, and no `sorry` outside the deliberate holes in `Challenge.lean`. Compared theorems may still depend on Mathlib’s `propext`, `Classical.choice`, and `Quot.sound`. The development is split into section-sized modules under `PlatonicSolids/`, each meant to be copied as a gist next to the matching paragraph above. `PlatonicSolids.lean` is the root import. The full sources are reproduced in the appendix. Figures 5 and 6 are the module-level blueprints: first the top-level `PlatonicSolids/*.lean` graph, then the `SingularExcision/` subgraph.

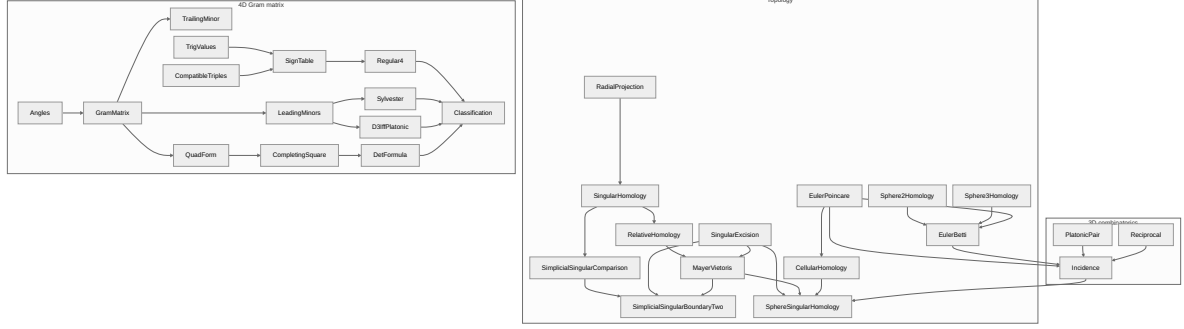


Figure 5: Top-level Lean module graph. `CellularHomology` transports Betti finranks through quasi-isomorphisms; the two simplicial-singular modules prove the canonical comparison for $\partial\Delta[2]$ using open-cover excision. The Gram-matrix stack feeds the 4D classification.

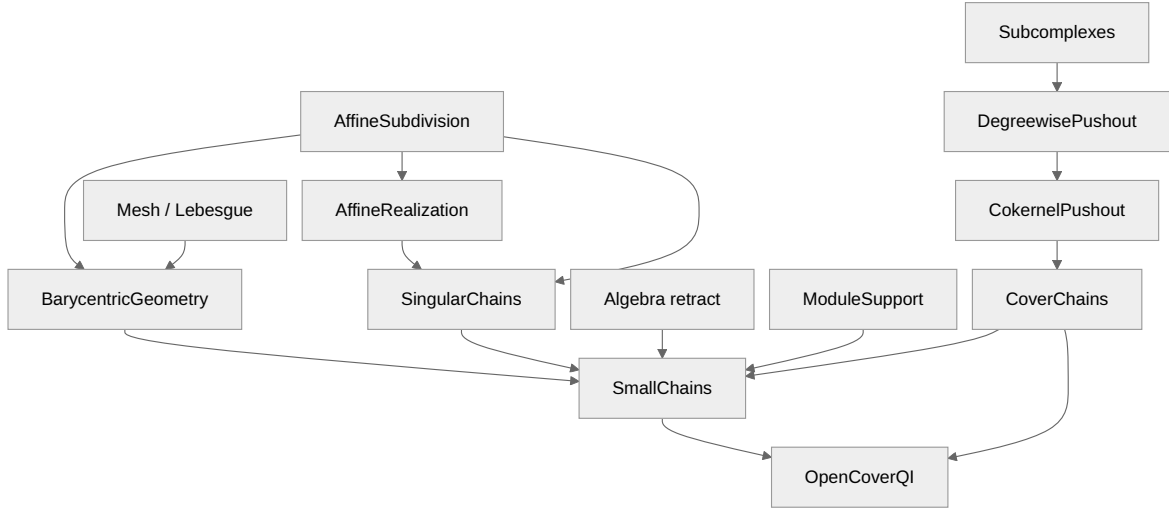


Figure 6: Finer blueprint of `PlatonicSolids/SingularExcision/`. Affine subdivision, mesh geometry, and the prism homotopy assemble the small-chain retract; cover chains are identified with small singular chains; the open-cover quasi-isomorphism discharges excision.

The compared theorems are:

- `platonic_solids_3d` — Euler $V + F = E + 2$ (as a hypothesis) plus regularity incidence $pF = 2E$, $qV = 2E$ force a Platonic pair (p, q) .
- `edges_pos_of_regular` — the same identities algebraically force $E > 0$; non-degeneracy is a theorem, not an axiom.

- `regular_polychora_classification` — Platonic cells and vertex figures with $\Delta(p, q, r) > 0$ force one of the six regular convex 4-polytopes.
- `simplicialSingularComparison_boundaryTwo_quasiIso` — the canonical adjunction-unit chain map from simplicial chains of $\partial\Delta[2]$ to singular chains of its realization is a quasi-isomorphism.

The converse $\Delta > 0$ for each of the six is `schlaflDet_pos_of_regular`. The stronger equivalence `regular_polychora_iff` identifies those six with positive-definiteness of `schlaflGram` for $p, q, r \geq 3$, via Sylvester’s criterion (`PlatonicSolids/Sylvester.lean`). Neither is a Palomar compared declaration.

`Challenge.lean` restates the compared family with deliberate `sorry`s so Palomar Comparator can check types against `Solution.lean`. Supporting definitions include `IsPlatonicPair`, `IsRegular4Polytope`, `schlaflGram`, and `schlaflDet`, together with the boundary-comparison helpers `boundaryTwoSSet`, `singularChainsOfRealization`, and `simplicialSingularComparison`.

File	Paper	Content
<code>PlatonicSolids/RadialProjection.lean</code>	§2.1	Frontier of a convex body $\simeq$ sphere
<code>PlatonicSolids/EulerPoincare.lean</code>	§§2.1, 3.1	Algebraic Euler–Poincaré in dimensions 2 and 3
<code>PlatonicSolids/CellularHomology.lean</code>	§2.1	Categorical finite 2-complex; Betti finranks; quasi-isomorphism transport
<code>PlatonicSolids/SingularHomology.lean</code>	§2.1	Homotopy invariance; $H_0(S^n)$; $H_*(S^0)$
<code>PlatonicSolids/RelativeHomology.lean</code>	§2.1	Pair sequence $0 \rightarrow C_*(A) \rightarrow C_*(X) \rightarrow C_*(X, A) \rightarrow 0$ and LES
<code>PlatonicSolids/MayerVietoris.lean</code>	§2.1	Triad SES, excision predicate, stereographic cover of S^n
<code>PlatonicSolids/SingularExcision.lean</code>	§2.1	Root of the barycentric / small-chain stack
<code>PlatonicSolids/SingularExcision/*.lean</code>	§2.1	Mesh shrinking, prism homotopy, small retract, cover = small, open-cover QI
<code>SimplicialSingularComparison.lean</code>	§2.1	Adjunction-unit comparison; simplex, horn, two-vertex cases
<code>SimplicialSingularBoundaryTwo.lean</code>	§2.1	Open-cover proof for $\partial\Delta[2]$
<code>SphereSingularHomology.lean</code>	§2.1	$H_n(S^n; R) \cong R$; cellular S^2 bridge
<code>Sphere2Homology.lean</code>	§2.1	Cellular $H_*(S^2; \mathbb{Q})$ of tetrahedron surface
<code>Sphere3Homology.lean</code>	§3.1	Cellular $H_*(S^3; \mathbb{Q})$ of 4-simplex boundary
<code>PlatonicSolids/EulerBetti.lean</code>	§§2.1, 3.1	$\chi(S^2) = 2$, $\chi(S^3) = 0$

File	Paper	Content
PlatonicSolids/Platonic Pair.lean	§2.3	Five solutions of $(p-2)(q-2) < 4$
PlatonicSolids/ Reciprocal.lean	§§2.3, 3.3	$1/p + 1/q > 1/2$
PlatonicSolids/Incidence. lean	§§2.2–2.4	platonic_solids_3d, edges_ pos_of_regular
PlatonicSolids/Angles. lean	§3.3	$\pi/n \in (0, \pi/2)$
PlatonicSolids/Gram Matrix.lean	§3.2	schlafliGram, schlafliDet
PlatonicSolids/Leading Minors.lean	§3.3	D_1, D_2, D_3
PlatonicSolids/Trailing Minor.lean	§3.3	Vertex-figure 3×3
PlatonicSolids/D3Iff Platonic.lean	§3.3	$D_3 > 0$ iff Platonic cell
PlatonicSolids/QuadForm. lean	§3.3	Gram quadratic form
PlatonicSolids/Completing Square.lean	§3.3	LDL identity
PlatonicSolids/Det Formula.lean	§3.4	$\det M = \Delta$
PlatonicSolids/Sylvester. lean	§3.3	PosDef iff leading minors > 0
PlatonicSolids/Trig Values.lean	§4	$\sin^2(\pi/n), \cos^2(\pi/n)$ for $n \in \{3, 4, 5\}$
PlatonicSolids/Compatible Triples.lean	§4	Eleven compatible triples
PlatonicSolids/SignTable. lean	§4	Sign of Δ
PlatonicSolids/Regular4. lean	§4	The six regular 4-polytopes
PlatonicSolids/ Classification.lean	§4	Compared 4D theorem and PosDef iff

The remaining named lemmas are the computational steps of those paragraphs: `platonic_pairs_of_inequality` / `platonic_pair_iff` / `platonic_pair_iff_reciprocal` for the Dio-phantine pair; `leadingMinor_one/two/three` and `leadingMinor_three_pos_iff` / `trailing3_pos_iff` for Sylvester’s first three minors; `schlafliGram_quad` / `band_quad_squares` / `schlafliGram_det` for $\det M = \Delta$; `schlafliDet_*` and `schlafliDet_*_pos/neg` for the eleven signs; `regular_polychora_iff` / `posDef_of_regular` / `regular_of_posDef` for the PosDef equivalence.

6 6. Pedagogical and Formalization Significance

1. **Foundational Consistency Across Dimensions:** The mathematical transition between 3D and 4D is made explicit: the compared 3D theorems take Euler $V + F = E + 2$ as

a hypothesis, while the chain-level theorem derives it whenever the cellular complex is quasi-isomorphic to singular chains of an S^2 model. The fourth compared result verifies the canonical comparison in the first noncontractible finite simplicial gluing case, $\partial\Delta[2]$, without presenting it as the still-open general cellulation theorem. The compared 4D theorem is instead bounded by the sign of Δ ; Sylvester’s criterion identifies the six with positive-definiteness of the Gram matrix in the un-compared `regular_polychora_iff`.

2. **Elimination of Redundant Hypotheses:** As demonstrated in `edges_pos_of_regular`, Lean’s linter and tactic engine show that the geometric non-degeneracy condition $E > 0$ is a theorem rather than an independent axiom.
3. **Absence of Ad-Hoc Combinatorial Filters:** Candidate 4D triples are filtered using the **Schläfli determinant** $\Delta(p, q, r) = \sin^2 \frac{\pi}{p} \sin^2 \frac{\pi}{r} - \cos^2 \frac{\pi}{q}$. Every rejection is certified by non-linear arithmetic over real algebraic numbers, with all properties of $\sqrt{5}$ derived from first principles.
4. **An explicit finite comparison, not a general one:** Closed-arc singular chains on $|\partial\Delta[2]|$ do not form a short exact sequence. The compared proof therefore enlarges the last edge and horn to punctured-circle open neighborhoods, uses deformation retractions onto those neighborhoods and onto the two-point intersection, and applies open-cover excision. The result is recorded as a Palomar theorem for $\partial\Delta[2]$ only.

7 Acknowledgements

The author expresses sincere gratitude to Professor Dana Scott (Carnegie Mellon University) for suggesting this topic and for inspiring discussions on the formalization of polyhedral geometry. Any remaining errors, misinterpretations, or omissions are solely the author’s own.

7.1 Declaration of Generative AI and AI-Assisted Technologies

In accordance with the Committee on Publication Ethics (COPE) position statements and guidelines on artificial intelligence in research and formal authorship, the author acknowledges the use of large language model tooling during the preparation of this work. Specifically, **Google Gemini 3.8**, **Cursor Grok 4.6**, and **OpenAI GPT-5.6 Sol (accessed through Cursor)** were employed as interactive coding and drafting assistants. They assisted with exploratory formalization, initial Lean 4 proof tactics, the singular-homology and excision investigation, and the structure of the expository drafts. All mathematical claims, Lean 4 proofs, and final text were subsequently reviewed, verified, refined, and certified by the author, who assumes full responsibility for the integrity and accuracy of the content.

8 References

1. Coxeter, H. S. M. *Regular Polytopes*. 3rd ed., Dover Publications, 1973.
2. Schläfli, L. *Theorie der vielfachen Kontinuität*. Zürcher & Furrer, 1852.
3. Euclid. *The Thirteen Books of the Elements*, Vol. 3 (Books X–XIII). Translated by T. L. Heath, Cambridge University Press, 1908.

4. Humphreys, J. E. *Reflection Groups and Coxeter Groups*. Cambridge Studies in Advanced Mathematics, Cambridge University Press, 1990.
5. The Mathlib Community. *The Lean 4 Mathematical Library*. <https://github.com/leanprover-community/mathlib4>
6. Committee on Publication Ethics (COPE). *Authorship and AI Tools: COPE Position Statement*. <https://publicationethics.org/guidance/guidelines/authorship-and-ai-tools>
7. OpenAI. *GPT-5.6: Frontier Intelligence That Scales with Your Ambition*. 2026. <https://openai.com/index/gpt-5-6/>
8. Honner, P. “There Are Just Five Perfect Shapes in Math—Here’s Why.” *Scientific American*, 2023. <https://www.scientificamerican.com/article/there-are-just-five-perfect-shapes-in-math-heres-why/>
9. Thought Thrill. “The Five Platonic Solids and Why There Can Only Be Five.” YouTube video, 2026.
10. Quick, M. “Regular 4-Polytopes (Polychora) Overview and Interactive Visualizations.” *Quick’s 4D Space*. <https://www.qfbox.info/4d/index>

A Complete Lean source

Checked by lake build against Lean 4 and Mathlib. The development contains no `sorry`, no `admit`, and no project-defined axiom. Palomar Comparator compares four theorems: `platonic_solids_3d`, `edges_pos_of_regular`, `regular_polychora_classification`, and `simplicialSingularComparison_boundaryTwo_quasiIso`.

A.1 PlatonicSolids.lean

Root import of the section-sized modules (41 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/platonic-solids/blob/main/PlatonicSolids.lean
-/
import PlatonicSolids.Angles
import PlatonicSolids.CellularHomology
import PlatonicSolids.Classification
import PlatonicSolids.CompatibleTriples
import PlatonicSolids.CompletingSquare
import PlatonicSolids.D3IffPlatonic
import PlatonicSolids.DetFormula
import PlatonicSolids.EulerBetti
import PlatonicSolids.EulerPoincare
import PlatonicSolids.GramMatrix
import PlatonicSolids.Incidence
import PlatonicSolids.LeadngMinors
import PlatonicSolids.MayerVietoris
import PlatonicSolids.PlatonicPair
import PlatonicSolids.QuadForm
import PlatonicSolids.RadialProjection
import PlatonicSolids.Reciprocal
import PlatonicSolids.Regular4
import PlatonicSolids.RelativeHomology

```

```

import PlatonicSolids.SignTable
import PlatonicSolids.SimplicialSingularBoundaryTwo
import PlatonicSolids.SingularExcision
import PlatonicSolids.SingularHomology
import PlatonicSolids.SphereSingularHomology
import PlatonicSolids.Sphere2Homology
import PlatonicSolids.Sphere3Homology
import PlatonicSolids.Sylvester
import PlatonicSolids.TrailingMinor
import PlatonicSolids.TrigValues

/-!
Root import of the section-sized modules. Each file under `PlatonicSolids/`
is meant to be readable next to the corresponding paragraph of
`PlatonicSolids.md`.
-/

```

A.2 PlatonicSolids/RadialProjection.lean

Frontier of a bounded convex body is homeomorphic to the sphere (25 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Analysis.Convex.GaugeRescale

/-!
Paper §2.1. Radial projection: the frontier of a bounded convex body
with nonempty interior is homeomorphic to the unit sphere. Mathlib's
`gaugeRescaleHomeomorph` is the homeomorphism of the ambient space
that carries the frontier onto `Metric.sphere 0 1`.
-/

open Metric Set

/-- The frontier of a bounded convex set with nonempty interior is a
topological sphere. This is the radial projection of the paper, stated
for a body that need not be centred at the origin. -/
theorem frontier_convex_homeo_sphere
  {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {s : Set E}
  (hc : Convex ℝ s) (hne : (interior s).Nonempty) (hb : Bornology.IsBounded s) :
  ∃ h : E ≃ₜ E, h '' frontier s = sphere (0 : E) 1 :=
  let ⟨h, _, _, hf⟩ :=
    exists_homeomorph_image_interior_closure_frontier_eq_unitBall hc hne hb
  ⟨h, hf⟩

```

A.3 PlatonicSolids/EulerPoincare.lean

Algebraic Euler–Poincaré in dimensions 2 and 3 (112 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.LinearAlgebra.FiniteDimensional.Lemmas

/-!
Paper §2.1. Algebraic Euler–Poincaré for a 2-dimensional chain complex
of finite-dimensional vector spaces (the cellular complex of a polyhedron):

`χ(C) = rank C₀ - rank C₁ + rank C₂` equals `χ(H) = b₀ - b₁ + b₂`.

```

The identity is rank-nullity in dimensions 2 and 3 . Specializing to the Betti numbers of the tetrahedron surface
 (``PlatonicSolids/Sphere2Homology.lean``) yields $V + F = E + 2$.
 Specializing to the 4 -simplex boundary
 (``PlatonicSolids/Sphere3Homology.lean``) yields $V - E + F - C = 0$.
 -/

open Module

```
variable {K : Type*} [Field K]
variable {C2 C1 C0 : Type*}
  [AddCommGroup C2] [Module K C2] [FiniteDimensional K C2]
  [AddCommGroup C1] [Module K C1] [FiniteDimensional K C1]
  [AddCommGroup C0] [Module K C0]

/-- `b0 = rank(C0 / im d1)`. -/
noncomputable def betti0 (d1 : C1 →1[K] C0) : ℤ :=
  finrank K C0 - finrank K (LinearMap.range d1)

/-- `b1 = rank(ker d1 / im d2)`. -/
noncomputable def betti1 (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) : ℤ :=
  finrank K (LinearMap.ker d1) - finrank K (LinearMap.range d2)

/-- `b2 = rank(ker d2)`. -/
noncomputable def betti2 (d2 : C2 →1[K] C1) : ℤ :=
  finrank K (LinearMap.ker d2)

/-- Algebraic Euler-Poincaré: cell Euler characteristic equals homological
Euler characteristic for any 2-complex with `d1.comp d2 = 0`. -/
theorem euler_poincare (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0)
  (hcomp : d1.comp d2 = 0) :
  (finrank K C0 : ℤ) - finrank K C1 + finrank K C2 =
    betti0 d1 - betti1 d2 d1 + betti2 d2 := by
  have _hB : LinearMap.range d2 ≤ LinearMap.ker d1 :=
    LinearMap.range_le_ker_iff.mpr hcomp
  have hr1 : (finrank K (LinearMap.range d1) : ℤ) + finrank K (LinearMap.ker d1) =
    finrank K C1 := by exact_mod_cast d1.finrank_range_add_finrank_ker
  have hr2 : (finrank K (LinearMap.range d2) : ℤ) + finrank K (LinearMap.ker d2) =
    finrank K C2 := by exact_mod_cast d2.finrank_range_add_finrank_ker
  simp only [betti0, betti1, betti2]
  linarith

/-- `V - E + F = 2` is the same counting identity as `V + F = E + 2`. -/
theorem euler_count {V E F : ℕ} (h : (V : ℤ) - E + F = 2) : V + F = E + 2 := by
  omega

/-- Cellular ranks `V, E, F` and the Betti numbers of `S2` give Euler's formula. -/
theorem euler_formula_of_sphere2
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0)
  {V E F : ℕ}
  (hV : finrank K C0 = V) (hE : finrank K C1 = E) (hF : finrank K C2 = F)
  (hcomp : d1.comp d2 = 0)
  (hb0 : betti0 d1 = 1) (hb1 : betti1 d2 d1 = 0) (hb2 : betti2 d2 = 1) :
  V + F = E + 2 := by
  have hχ := euler_poincare d2 d1 hcomp
  rw [hV, hE, hF, hb0, hb1, hb2] at hχ
  exact euler_count hχ

variable {C3 : Type*}
  [AddCommGroup C3] [Module K C3] [FiniteDimensional K C3]

/-- In a 3-complex, `b2 = rank(ker d2 / im d3)`. -/
```

```

noncomputable def betti2Rel (d3 : C3 →1[K] C2) (d2 : C2 →1[K] C1) : ℤ :=
  finrank K (LinearMap.ker d2) - finrank K (LinearMap.range d3)

/-- `b3 = rank(ker d3)` -/
noncomputable def betti3 (d3 : C3 →1[K] C2) : ℤ :=
  finrank K (LinearMap.ker d3)

/-- Algebraic Euler-Poincaré in dimension 3. -/
theorem euler_poincare3 (d3 : C3 →1[K] C2) (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0)
  (h32 : d2.comp d3 = 0) (h21 : d1.comp d2 = 0) :
  (finrank K C0 : ℤ) - finrank K C1 + finrank K C2 - finrank K C3 =
    betti0 d1 - betti1 d2 d1 + betti2Rel d3 d2 - betti3 d3 := by
  have _hB21 : LinearMap.range d2 ≤ LinearMap.ker d1 :=
    LinearMap.range_le_ker_iff.mpr h21
  have _hB32 : LinearMap.range d3 ≤ LinearMap.ker d2 :=
    LinearMap.range_le_ker_iff.mpr h32
  have hr1 : (finrank K (LinearMap.range d1) : ℤ) + finrank K (LinearMap.ker d1) =
    finrank K C1 := by exact_mod_cast d1.finrank_range_add_finrank_ker
  have hr2 : (finrank K (LinearMap.range d2) : ℤ) + finrank K (LinearMap.ker d2) =
    finrank K C2 := by exact_mod_cast d2.finrank_range_add_finrank_ker
  have hr3 : (finrank K (LinearMap.range d3) : ℤ) + finrank K (LinearMap.ker d3) =
    finrank K C3 := by exact_mod_cast d3.finrank_range_add_finrank_ker
  simp only [betti0, betti1, betti2Rel, betti3]
  linarith

/-- Cell counts of an `S3` complex with Betti numbers `1,0,0,1` give `χ = 0`. -/
theorem euler_formula_of_sphere3
  (d3 : C3 →1[K] C2) (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0)
  {V E F C : ℕ}
  (hV : finrank K C0 = V) (hE : finrank K C1 = E)
  (hF : finrank K C2 = F) (hC : finrank K C3 = C)
  (h32 : d2.comp d3 = 0) (h21 : d1.comp d2 = 0)
  (hb0 : betti0 d1 = 1) (hb1 : betti1 d2 d1 = 0)
  (hb2 : betti2Rel d3 d2 = 0) (hb3 : betti3 d3 = 1) :
  (V : ℤ) - E + F - C = 0 := by
  have hχ := euler_poincare3 d3 d2 d1 h32 h21
  rw [hV, hE, hF, hC, hb0, hb1, hb2, hb3] at hχ
  linarith

```

A.4 PlatonicSolids/CellularHomology.lean

Cellular Betti finranks and quasi-isomorphism transport (209 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Homology.QuasiIso
import Mathlib.Algebra.Homology.ShortComplex.ModuleCat
import PlatonicSolids.EulerPoincare

/-!
Categorical infrastructure for a finite-dimensional cellular `2`-complex.

The chain groups are concentrated in degrees `0`, `1`, and `2`. The
categorical homology dimensions are identified with the integer-valued Betti
numbers used by `EulerPoincare.lean`.
-/

open CategoryTheory CategoryTheory.Limits
open HomologicalComplex
open Module

```

```

variable {K : Type} [Field K]
variable {C2 C1 C0 : Type}
  [AddCommGroup C2] [Module K C2]
  [AddCommGroup C1] [Module K C1]
  [AddCommGroup C0] [Module K C0]

/-- The chain complex  $C_2 \rightarrow C_1 \rightarrow C_0$ , extended by zero in degrees at least `3`. -/
noncomputable def cellularChainComplex
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) (hcomp : d1.comp d2 = 0) :
  ChainComplex (ModuleCat.{0} K) ℕ :=
  ChainComplex.mk
    (ModuleCat.of K C0) (ModuleCat.of K C1) (ModuleCat.of K C2)
    (ModuleCat.ofHom d1) (ModuleCat.ofHom d2)
    (by
      apply ModuleCat.hom_ext
      simpa only [ModuleCat.hom_comp, ModuleCat.hom_ofHom] using hcomp)
    (fun S => ⟨ModuleCat.of K (Fin 0 → K), 0, zero_comp⟩)

@[simp]
theorem cellularChainComplex_X_zero
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) (hcomp : d1.comp d2 = 0) :
  (cellularChainComplex d2 d1 hcomp).X 0 = ModuleCat.of K C0 := rfl

@[simp]
theorem cellularChainComplex_X_one
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) (hcomp : d1.comp d2 = 0) :
  (cellularChainComplex d2 d1 hcomp).X 1 = ModuleCat.of K C1 := rfl

@[simp]
theorem cellularChainComplex_X_two
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) (hcomp : d1.comp d2 = 0) :
  (cellularChainComplex d2 d1 hcomp).X 2 = ModuleCat.of K C2 := rfl

@[simp]
theorem cellularChainComplex_d_one_zero
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) (hcomp : d1.comp d2 = 0) :
  (cellularChainComplex d2 d1 hcomp).d 1 0 = ModuleCat.ofHom d1 := by
  simp [cellularChainComplex]

@[simp]
theorem cellularChainComplex_d_two_one
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) (hcomp : d1.comp d2 = 0) :
  (cellularChainComplex d2 d1 hcomp).d 2 1 = ModuleCat.ofHom d2 := by
  simp [cellularChainComplex]

@[simp]
theorem cellularChainComplex_d_high
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) (hcomp : d1.comp d2 = 0) (n : ℕ) :
  (cellularChainComplex d2 d1 hcomp).d (n + 3) (n + 2) = 0 := by
  rw [cellularChainComplex, ChainComplex.mk_d]
  simp

/-- Finrank is invariant under an isomorphism of `ModuleCat` objects. -/
theorem ModuleCat.finrank_eq_of_iso {V W : ModuleCat.{0} K} (e : V ≅ W) :
  finrank K V = finrank K W :=
  e.toLinearEquiv.finrank_eq

/-- Finrank of short-complex homology is the dimension of cycles minus boundaries. -/
theorem ShortComplex.int_finrank_homology_eq
  (S : ShortComplex (ModuleCat.{0} K))
  [FiniteDimensional K S.X1] [FiniteDimensional K S.X2] :
  (finrank K S.homology : ℤ) =
  (finrank K (LinearMap.ker S.g.hom) : ℤ) -

```

```

      finrank K (LinearMap.range S.f.hom) := by
have hrange :
  finrank K (LinearMap.range S.moduleCatToCycles) =
    finrank K (LinearMap.range S.f.hom) := by
  have h1 := S.moduleCatToCycles.finrank_range_add_finrank_ker
  have h2 := S.f.hom.finrank_range_add_finrank_ker
  rw [LinearMap.ker_codRestrict] at h1
  omega
have hiso := S.moduleCatHomologyIso.toLinearEquiv.finrank_eq
change finrank K S.homology =
  finrank K
    (HasQuotient.Quotient (LinearMap.ker S.g.hom)
      (LinearMap.range S.moduleCatToCycles)) at hiso
rw [hiso, Submodule.finrank_quotient]
have hle :
  finrank K (LinearMap.range S.moduleCatToCycles) ≤
    finrank K (LinearMap.ker S.g.hom) :=
  (LinearMap.range S.moduleCatToCycles).finrank_le
omega

/-- Degree-zero categorical homology has the existing integer Betti number `betti0`. -/
theorem cellularChainComplex_homology_finrank_zero
  [FiniteDimensional K C2] [FiniteDimensional K C1] [FiniteDimensional K C0]
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) (hcomp : d1.comp d2 = 0) :
  (finrank K ((cellularChainComplex d2 d1 hcomp).homology 0) : ℤ) = betti0 d1 := by
let C := cellularChainComplex d2 d1 hcomp
letI : FiniteDimensional K (C.sc' 1 0 0).X1 := by
  change FiniteDimensional K C1
  infer_instance
letI : FiniteDimensional K (C.sc' 1 0 0).X2 := by
  change FiniteDimensional K C0
  infer_instance
calc
  (finrank K (C.homology 0) : ℤ) =
    finrank K ((C.sc' 1 0 0).homology) := by
    exact_mod_cast ModuleCat.finrank_eq_of_iso (C.homologyIsoSc' 1 0 0 (by simp) (by simp))
  _ = (finrank K (LinearMap.ker (C.d 0 0).hom) : ℤ) -
    finrank K (LinearMap.range (C.d 1 0).hom) :=
    ShortComplex.int_finrank_homology_eq (C.sc' 1 0 0)
  _ = betti0 d1 := by
    have hd00 : C.d 0 0 = 0 := C.shape 0 0 (by simp)
    have hd10 : C.d 1 0 = ModuleCat.ofHom d1 := by
      simp [C]
    rw [hd00, hd10]
    change (finrank K (LinearMap.ker (0 : C0 →1[K] C0)) : ℤ) -
      finrank K (LinearMap.range d1) = betti0 d1
    rw [LinearMap.ker_zero, finrank_top]
    rfl

/-- Degree-one categorical homology has the existing integer Betti number `betti1`. -/
theorem cellularChainComplex_homology_finrank_one
  [FiniteDimensional K C2] [FiniteDimensional K C1] [FiniteDimensional K C0]
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) (hcomp : d1.comp d2 = 0) :
  (finrank K ((cellularChainComplex d2 d1 hcomp).homology 1) : ℤ) =
    betti1 d2 d1 := by
let C := cellularChainComplex d2 d1 hcomp
letI : FiniteDimensional K (C.sc' 2 1 0).X1 := by
  change FiniteDimensional K C2
  infer_instance
letI : FiniteDimensional K (C.sc' 2 1 0).X2 := by
  change FiniteDimensional K C1
  infer_instance
calc

```



```

(finrank K (C.homology 1) :  $\mathbb{Z}$ ) =
  finrank K ((C.sc' 2 1 0).homology) := by
    exact_mod_cast ModuleCat.finrank_eq_of_iso (C.homologyIsoSc' 2 1 0 (by simp) (by simp))
  _ = (finrank K (LinearMap.ker (C.d 1 0).hom) :  $\mathbb{Z}$ ) -
    finrank K (LinearMap.range (C.d 2 1).hom) :=
      ShortComplex.int_finrank_homology_eq (C.sc' 2 1 0)
  _ = betti1 d2 d1 := by
    have hd10 : C.d 1 0 = ModuleCat.ofHom d1 := by
      simp [C]
    have hd21 : C.d 2 1 = ModuleCat.ofHom d2 := by
      simp [C]
    rw [hd10, hd21]
    rfl

/-- Degree-two categorical homology has the existing integer Betti number `betti2`. -/
theorem cellularChainComplex_homology_finrank_two
  [FiniteDimensional K C2] [FiniteDimensional K C1] [FiniteDimensional K C0]
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) (hcomp : d1.comp d2 = 0) :
  (finrank K ((cellularChainComplex d2 d1 hcomp).homology 2) :  $\mathbb{Z}$ ) = betti2 d2 := by
  let C := cellularChainComplex d2 d1 hcomp
  letI : FiniteDimensional K (C.sc' 3 2 1).X1 := by
    change FiniteDimensional K (Fin 0 → K)
    infer_instance
  letI : FiniteDimensional K (C.sc' 3 2 1).X2 := by
    change FiniteDimensional K C2
    infer_instance
  calc
    (finrank K (C.homology 2) :  $\mathbb{Z}$ ) =
      finrank K ((C.sc' 3 2 1).homology) := by
        exact_mod_cast ModuleCat.finrank_eq_of_iso (C.homologyIsoSc' 3 2 1 (by simp) (by simp))
      _ = (finrank K (LinearMap.ker (C.d 2 1).hom) :  $\mathbb{Z}$ ) -
        finrank K (LinearMap.range (C.d 3 2).hom) :=
          ShortComplex.int_finrank_homology_eq (C.sc' 3 2 1)
      _ = betti2 d2 := by
        have hd21 : C.d 2 1 = ModuleCat.ofHom d2 := by
          simp [C]
        have hd32 : C.d 3 2 = 0 := by
          simp [C]
        rw [hd21, hd32]
        change (finrank K (LinearMap.ker d2) :  $\mathbb{Z}$ ) -
          finrank K (LinearMap.range (0 : (Fin 0 → K) →1[K] C2)) = betti2 d2
        simp [betti2]

/-- An isomorphism between homology objects preserves their finrank. -/
theorem homology_finrank_eq_of_iso
  {A B : ChainComplex (ModuleCat.{0} K)  $\mathbb{N}$ } {n :  $\mathbb{N}$ }
  (e : A.homology n  $\cong$  B.homology n) :
  finrank K (A.homology n) = finrank K (B.homology n) :=
  ModuleCat.finrank_eq_of_iso e

/-- A quasi-isomorphism in one degree preserves homology finrank in that degree. -/
theorem homology_finrank_eq_of_quasiIsoAt
  {A B : ChainComplex (ModuleCat.{0} K)  $\mathbb{N}$ } (f : A → B) (n :  $\mathbb{N}$ )
  [QuasiIsoAt f n] :
  finrank K (A.homology n) = finrank K (B.homology n) :=
  homology_finrank_eq_of_iso (isoOfQuasiIsoAt f n)

/-- A quasi-isomorphism preserves homology finrank in every degree. -/
theorem homology_finrank_eq_of_quasiIso
  {A B : ChainComplex (ModuleCat.{0} K)  $\mathbb{N}$ } (f : A → B) [QuasiIso f] (n :  $\mathbb{N}$ ) :
  finrank K (A.homology n) = finrank K (B.homology n) :=
  homology_finrank_eq_of_quasiIsoAt f n

```

A.5 PlatonicSolids/SingularHomology.lean

Singular $H_0(S^n)$, $H_*(S^0)$, homotopy invariance (238 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.AlgebraicTopology.SingularHomology.HomologyZero
import Mathlib.AlgebraicTopology.SingularHomology.HomotopyInvariance
import Mathlib.Analysis.InnerProductSpace.PiL2
import Mathlib.Analysis.Normed.Module.Connected
import Mathlib.LinearAlgebra.Dimension.StrongRankCondition
import Mathlib.Topology.Homotopy.Contractible
import Mathlib.Topology.Homotopy.Equiv
import Mathlib.Topology.Separation.Lemmas

/-!
Singular homology lemmas that Mathlib does not package: homotopy
equivalences and homeomorphisms induce homology isomorphisms,
contractible spaces have the homology of a point,  $H_0(S^n) \cong \mathbb{R}$  for
 $n \geq 1$ , and the full calculation of  $H_*(S^0)$ .

The pair and Mayer-Vietoris sequences of singular chains are in
`PlatonicSolids/RelativeHomology.lean` and
`PlatonicSolids/MayerVietoris.lean`. Barycentric subdivision and the
open-cover quasi-isomorphism are in `PlatonicSolids/SingularExcision`.
The calculation  $H_n(S^n) \cong \mathbb{R}$  for  $n > 0$  is in
`PlatonicSolids/SphereSingularHomology.lean`. The
paper's  $\chi(S^3) = 0$  is obtained from the cellular calculation in
`PlatonicSolids/Sphere3Homology.lean`.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex
open scoped ContinuousMap Topology

universe v u

variable {C : Type u} [Category.{v} C] [HasCoproducts.{0} C] [Preadditive C]
[CategoryWithHomology C]

/-- Homotopic continuous maps induce equal singular-homology maps. -/
lemma singularHomologyMap_eq_of_homotopic
  {X Y : TopCat.{0}} {f g : X → Y} (h : f.hom.Homotopic g.hom)
  (R : C) (n : ℕ) :
  homologyMap (((singularChainComplexFunctor C).obj R).map f) n =
  homologyMap (((singularChainComplexFunctor C).obj R).map g) n :=
  TopCat.Homotopy.congr_homologyMap_singularChainComplexFunctor h.some R n

/-- A homotopy equivalence of spaces induces a homotopy equivalence of
singular chain complexes. -/
noncomputable def singularChainHomotopyEquiv
  {X Y : Type} [TopologicalSpace X] [TopologicalSpace Y]
  (e : X ≃h Y) (R : C) :
  HomotopyEquiv
  (((singularChainComplexFunctor C).obj R).obj (.of X))
  (((singularChainComplexFunctor C).obj R).obj (.of Y)) where
  hom := ((singularChainComplexFunctor C).obj R).map (TopCat.ofHom e.toFun)
  inv := ((singularChainComplexFunctor C).obj R).map (TopCat.ofHom e.invFun)
  homotopyHomInvId := by
  have H : TopCat.Homotopy
    (TopCat.ofHom e.toFun >> TopCat.ofHom e.invFun) (⌘ _) := by
    simpa [TopCat.ofHom_comp, TopCat.ofHom_id] using e.left_inv.some
    simpa [Functor.map_comp, Functor.map_id] using

```

```

      H.singularChainComplexFunctorObjMap R
homotopyInvHomId := by
  have H : TopCat.Homotopy
    (TopCat.ofHom e.invFun >> TopCat.ofHom e.toFun) (≠ _) := by
      simpa [TopCat.ofHom_comp, TopCat.ofHom_id] using e.right_inv.some
      simpa [Functor.map_comp, Functor.map_id] using
        H.singularChainComplexFunctorObjMap R

/-- Homotopy equivalent spaces have isomorphic singular homology. -/
noncomputable def singularHomologyIso_of_homotopyEquiv
  {X Y : Type} [TopologicalSpace X] [TopologicalSpace Y]
  (e : X ≃h Y) (R : C) (n : ℕ) :
  ((singularHomologyFunctor C n).obj R).obj (.of X) ≅
    ((singularHomologyFunctor C n).obj R).obj (.of Y) :=
  (singularChainHomotopyEquiv e R).toHomologyIso n

/-- Homeomorphic spaces have isomorphic singular homology. -/
noncomputable def singularHomologyIso_of_homeomorph
  {X Y : Type} [TopologicalSpace X] [TopologicalSpace Y]
  (e : X ≃t Y) (R : C) (n : ℕ) :
  ((singularHomologyFunctor C n).obj R).obj (.of X) ≅
    ((singularHomologyFunctor C n).obj R).obj (.of Y) :=
  singularHomologyIso_of_homotopyEquiv e.toHomotopyEquiv R n

/-- A point has vanishing singular homology in positive degree. -/
lemma isZero_singularHomology_unit (R : C) {n : ℕ} (hn : n ≠ 0) :
  IsZero (((singularHomologyFunctor C n).obj R).obj (.of Unit)) :=
  isZero_singularHomologyFunctor_of_totallyDisconnectedSpace C n R (.of Unit) hn

/-- A contractible space has vanishing singular homology in positive degree. -/
lemma isZero_singularHomology_of_contractible
  {X : Type} [TopologicalSpace X] [ContractibleSpace X]
  (R : C) {n : ℕ} (hn : n ≠ 0) :
  IsZero (((singularHomologyFunctor C n).obj R).obj (.of X)) := by
  obtain ⟨e⟩ := ContractibleSpace.hequiv_unit X
  exact (isZero_singularHomology_unit R hn).of_iso
    (singularHomologyIso_of_homotopyEquiv e R n)

/-- Path-connectedness transfers along a homotopy equivalence. -/
lemma pathConnectedSpace_of_homotopyEquiv
  {X Y : Type} [TopologicalSpace X] [TopologicalSpace Y]
  [PathConnectedSpace Y] (e : ContinuousMap.HomotopyEquiv X Y) :
  PathConnectedSpace X :=
  ⟨(e.invFun (Classical.arbitrary Y)), fun x₁ x₂ =>
    (e.left_inv.some.evalAt x₁).symm.trans
      (((PathConnectedSpace.somePath (e.toFun x₁) (e.toFun x₂)).map
        e.invFun.continuous).trans
        (e.left_inv.some.evalAt x₂))⟩⟩

/-- The augmentation  $H_0(X) \rightarrow R$  is natural. -/
lemma singularHomology0ε_natural {X Y : TopCat.{0}} (f : X → Y) (R : C) :
  homologyMap (((singularChainComplexFunctor C).obj R).map f) 0 >>
  Y.singularHomology0ε R =
  X.singularHomology0ε R := by
  let SX := TopCat.toSSet.obj X
  let SY := TopCat.toSSet.obj Y
  let sf := TopCat.toSSet.map f
  let φ := SSet.chainComplexMap sf R
  let KX := SX.chainComplex R
  let KY := SY.chainComplex R
  apply (cancel_epi (KX.homologyπ 0)).1
  apply (cancel_epi (ChainComplex.cycles0Iso KY).inv).1
  apply SSet.chainComplex_hom_ext

```

```

intro x
have hx :
  KX.liftCycles (SX.ιChainComplex x) 0 (by simp) (by simp) =
    SX.ιChainComplex x >> (ChainComplex.cyclesIso KX).inv := by
  rw [← cancel_mono (KX.ιCycles 0)]
  simp [ChainComplex.cyclesIso]
  exact (Category.comp_id _).symm
have hnats := HomologicalComplex.homologyπ_naturality (φ := φ) (i := 0)
have hcycles := HomologicalComplex.liftCycles_comp_cyclesMap
  (k := SX.ιChainComplex x) (j := 0) (hj := by simp) (hk := by simp) φ
have hf : SX.ιChainComplex x >> φ.f 0 = SY.ιChainComplex (sf.app _ x) :=
  SSet.ι_chainComplexMap_f (X := SX) (Y := SY) (f := sf) (R := R) x
have hY := SSet.liftCycles_ιChainComplex_homologyπ_homology0ε SY R (sf.app _ x)
have hX := SSet.liftCycles_ιChainComplex_homologyπ_homology0ε SX R x
have lhs :
  SX.ιChainComplex x >> (ChainComplex.cyclesIso KX).inv >>
    KX.homologyπ 0 >> homologyMap φ 0 >> Y.singularHomology0ε R = ⌞ R := by
  rw [← Category.assoc, ← hx, reassoc_of% hnats, reassoc_of% hcycles]
  exact (congrArg (fun t =>
    KY.liftCycles t 0 (by simp) (by simp) >>
      KY.homologyπ 0 >> Y.singularHomology0ε R) hf).trans hY
have rhs :
  SX.ιChainComplex x >> (ChainComplex.cyclesIso KX).inv >>
    KX.homologyπ 0 >> X.singularHomology0ε R = ⌞ R := by
  rw [← Category.assoc, ← hx]
  exact hX
exact lhs.trans rhs.symm

/-- Path-connected spaces have  $H_0 \cong R$  via the augmentation. -/
noncomputable def singularHomology0Iso_pathConnected
  (X : TopCat.{0}) [PathConnectedSpace X] (R : C) :
  ((singularHomologyFunctor C 0).obj R).obj X  $\cong$  R :=
  asIso (X.singularHomology0ε R)

/-- A map of path-connected spaces induces an isomorphism on  $H_0$ . -/
instance singularHomology0_map_isIso
  {X Y : TopCat.{0}} [PathConnectedSpace X] [PathConnectedSpace Y]
  (f : X  $\longrightarrow$  Y) (R : C) :
  IsIso (homologyMap (((singularChainComplexFunctor C).obj R).map f) 0) := by
  have h := singularHomology0ε_natural f R
  let eX := singularHomology0Iso_pathConnected X R
  let eY := singularHomology0Iso_pathConnected Y R
  refine (eY.hom >> eX.inv, ?_, ?_)
  · erw [← Category.assoc, h, eX.hom_inv_id]; rfl
  · have hf : homologyMap (((singularChainComplexFunctor C).obj R).map f) 0 =
      eX.hom >> eY.inv := by
      refine (cancel_mono eY.hom).mp ?_
      erw [h, Category.assoc, eY.inv_hom_id, Category.comp_id]
      rfl
  calc
    (eY.hom >> eX.inv) >>
      homologyMap (((singularChainComplexFunctor C).obj R).map f) 0 =
      (eY.hom >> eX.inv) >> eX.hom >> eY.inv := by
      rw [hf]; rfl
    _ = eY.hom >> (eX.inv >> eX.hom) >> eY.inv := by
      simp [Category.assoc]
    _ = eY.hom >> (⌞ R) >> eY.inv := by
      rw [eX.inv_hom_id]
    _ = eY.hom >> eY.inv := by
      simp
    _ = ⌞ _ := eY.hom_inv_id

/-- The unit sphere in Euclidean dimension  $n+1 \geq 2$  is path-connected. -/

```

```

instance pathConnectedSpace_sphere_euclidean
  {n : ℕ} [NeZero n] :
    PathConnectedSpace (↑(Metric.sphere (0 : EuclideanSpace ℝ (Fin (n + 1))) 1)) := by
  rw [← isPathConnected_iff_pathConnectedSpace]
  refine isPathConnected_sphere ?_ 0 (by norm_num)
  have : (1 : ℕ) < n + 1 := Nat.lt_add_of_pos_left (NeZero.pos n)
  have hfin : Module.finiteRank ℝ (EuclideanSpace ℝ (Fin (n + 1))) = n + 1 := by
    simp
  rw [← Module.finiteRank_eq_rank, hfin]
  exact_mod_cast this

/--  $H_0(S^n; \mathbb{R}) \cong \mathbb{R}$  for  $n \geq 1$ . -/
noncomputable def singularHomology0_sphere
  {n : ℕ} [NeZero n] (R : C) :
    ((singularHomologyFunctor C 0).obj R).obj
      (.of (↑(Metric.sphere (0 : EuclideanSpace ℝ (Fin (n + 1))) 1))) ≅ R :=
    singularHomologyIso_pathConnected _ R

/--  $S^0 \subset \mathbb{R}$  is the two-point set  $\{\pm 1\}$ . -/
lemma sphere_real_eq : (Metric.sphere (0 : ℝ) 1 : Set ℝ) = {1, -1} := by
  ext x
  simp [mem_sphere_iff_norm, Real.norm_eq_abs, abs_eq]

instance : TotallyDisconnectedSpace (↑(Metric.sphere (0 : ℝ) 1)) := by
  refine totallyDisconnectedSpace_subtype_iff.2
    (Set.Countable.isTotallyDisconnected ?_)
  rw [sphere_real_eq]
  exact ((Set.finite_singleton (-1 : ℝ)).insert 1).countable

/--  $H_n(S^0; \mathbb{R}) = 0$  for  $n > 0$ . -/
lemma isZero_singularHomology_sphere0 (R : C) {n : ℕ} (hn : n ≠ 0) :
  IsZero (((singularHomologyFunctor C n).obj R).obj
    (.of (↑(Metric.sphere (0 : ℝ) 1)))) :=
  isZero_singularHomologyFunctor_of_totallyDisconnectedSpace C n R _ hn

/--  $H_0(S^0; \mathbb{R}) \cong \mathbb{R} \oplus \mathbb{R}$ . -/
noncomputable def singularHomology0_sphere0 (R : C) :
  ((singularHomologyFunctor C 0).obj R).obj (.of (↑(Metric.sphere (0 : ℝ) 1))) ≅
    ∐ fun _ : ↑(Metric.sphere (0 : ℝ) 1) ↦ R :=
  singularHomologyFunctorZeroOfTotallyDisconnectedSpace C R
  (.of (↑(Metric.sphere (0 : ℝ) 1)))

/-- The closed unit disk is contractible, hence has vanishing positive
singular homology. -/
lemma isZero_singularHomology_closedBall
  {E : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
  (R : C) {n : ℕ} (hn : n ≠ 0) :
  IsZero (((singularHomologyFunctor C n).obj R).obj
    (.of (↑(Metric.closedBall (0 : E) 1)))) := by
  have : ContractibleSpace (↑(Metric.closedBall (0 : E) 1)) :=
    Metric.contractibleSpace_closedBall (by norm_num)
  exact isZero_singularHomology_of_contractible R hn

```

A.6 PlatonicSolids/RelativeHomology.lean

Relative singular homology and the LES of a pair (148 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Homology.HomologicalComplexAbelian
import Mathlib.Algebra.Homology.HomologySequence

```

```

import Mathlib.AlgebraicTopology.SingularHomology.Basic
import Mathlib.Topology.Category.TopCat.EpiMono

/-!
Relative singular homology of a pair  $(X, A)$  as the cokernel of the
inclusion of singular chains. Mathlib's singular chain-complex functor
preserves monomorphisms (it is free on simplices), so a subspace
inclusion yields a short exact sequence of chain complexes and therefore
the long exact sequence of the pair.

The pair sequence is the first half of the path to  $H_n(S^n)$ . Excision
of an open cover and the stereographic connecting isomorphism are in
`PlatonicSolids/SingularExcision/` and
`PlatonicSolids/SphereSingularHomology.lean`.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex
open scoped Topology

set_option linter.unusedSectionVars false

universe v u

variable {C : Type u} [Category.{v} C] [Abelian C]
[HasCoproducts.{0} C] [HasPullbacks C]

variable (R : C)

/-- Inclusion of a subspace as a morphism of `TopCat`. -/
def subtypeIncl (X : Type) [TopologicalSpace X] (A : Set X) :
  TopCat.of  $\uparrow A \rightarrow$  TopCat.of X :=
  TopCat.ofHom (Subtype.val, continuous_subtype_val)

instance subtypeIncl_mono (X : Type) [TopologicalSpace X] (A : Set X) :
  Mono (subtypeIncl X A) :=
  (TopCat.mono_iff_injective _).2 Subtype.val_injective

/-- Singular chains of a subspace, included into those of the ambient space. -/
noncomputable def inclChains (X : Type) [TopologicalSpace X] (A : Set X) :
  ((singularChainComplexFunctor C).obj R).obj (.of  $\uparrow A$ )  $\rightarrow$ 
  ((singularChainComplexFunctor C).obj R).obj (.of X) :=
  ((singularChainComplexFunctor C).obj R).map (subtypeIncl X A)

instance inclChains_mono (X : Type) [TopologicalSpace X] (A : Set X) :
  Mono (inclChains (C := C) R X A) :=
  ((singularChainComplexFunctor C).obj R).map_mono (subtypeIncl X A)

/-- Relative singular chains  $C_*(X, A; R) := \text{coker}(C_*(A) \rightarrow C_*(X))$ . -/
noncomputable def relativeSingularChains (X : Type) [TopologicalSpace X] (A : Set X) :
  ChainComplex C  $\mathbb{N}$  :=
  cokernel (inclChains (C := C) R X A)

/-- The short complex  $C_*(A) \rightarrow C_*(X) \rightarrow C_*(X, A)$ . -/
noncomputable def pairShortComplex (X : Type) [TopologicalSpace X] (A : Set X) :
  ShortComplex (ChainComplex C  $\mathbb{N}$ ) :=
  ShortComplex.mk (inclChains (C := C) R X A) (cokernel. $\pi$  _) (cokernel.condition _)

instance pairShortComplex_mono_f (X : Type) [TopologicalSpace X] (A : Set X) :
  Mono (pairShortComplex (C := C) R X A).f :=
  inferInstanceAs (Mono (inclChains (C := C) R X A))

instance pairShortComplex_epi_g (X : Type) [TopologicalSpace X] (A : Set X) :
  Epi (pairShortComplex (C := C) R X A).g :=

```

```

inferInstanceAs (Epi (cokernel.π (inclChains (C := C) R X A)))

/-- The pair sequence of singular chains is short exact. -/
lemma pair_shortExact (X : Type) [TopologicalSpace X] (A : Set X) :
  (pairShortComplex (C := C) R X A).ShortExact :=
  ShortComplex.ShortExact.mk'
  (ShortComplex.exact_of_g_is_cokernel _
    (cokernelIsCokernel (inclChains (C := C) R X A)))
  inferInstance inferInstance

/-- Relative singular homology  $H_n(X, A; R)$ . -/
noncomputable def relativeSingularHomology
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) : C :=
  (relativeSingularChains (C := C) R X A).homology n

/-- The map  $H_n(A) \rightarrow H_n(X)$  induced by the subspace inclusion. -/
noncomputable def pair_i
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
  (((singularChainComplexFunctor C).obj R).obj (.of ↑A)).homology n →
  (((singularChainComplexFunctor C).obj R).obj (.of X)).homology n :=
  homologyMap (inclChains (C := C) R X A) n

/-- The map  $H_n(X) \rightarrow H_n(X, A)$  to relative homology. -/
noncomputable def pair_j
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
  (((singularChainComplexFunctor C).obj R).obj (.of X)).homology n →
  relativeSingularHomology (C := C) R X A n :=
  homologyMap (cokernel.π (inclChains (C := C) R X A)) n

/-- Connecting homomorphism  $\delta : H_{n+1}(X, A) \rightarrow H_n(A)$  of the pair. -/
noncomputable def pair_δ
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
  relativeSingularHomology (C := C) R X A (n + 1) →
  (((singularChainComplexFunctor C).obj R).obj (.of ↑A)).homology n :=
  (pair_shortExact (C := C) R X A).δ (n + 1) n (by simp)

/-- The composition  $H_n(A) \rightarrow H_n(X) \rightarrow H_n(X, A)$  is zero. -/
lemma pair_i_comp_pair_j
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
  pair_i (C := C) R X A n ≫ pair_j (C := C) R X A n = 0 := by
  simpa [pair_i, pair_j] using
    (by
      rw [← homologyMap_comp, cokernel.condition, homologyMap_zero] :
        homologyMap (inclChains (C := C) R X A) n ≫
          homologyMap (cokernel.π (inclChains (C := C) R X A)) n = 0)

/-- The composition  $H_n(X) \rightarrow H_n(X, A) \rightarrow H_{n-1}(A)$  is zero. -/
lemma pair_j_comp_pair_δ
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
  pair_j (C := C) R X A (n + 1) ≫ pair_δ (C := C) R X A n = 0 :=
  (pair_shortExact (C := C) R X A).comp_δ (n + 1) n (by simp)

/-- The composition  $H_{n+1}(X, A) \rightarrow H_n(A) \rightarrow H_n(X)$  is zero. -/
lemma pair_δ_comp_pair_i
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
  pair_δ (C := C) R X A n ≫ pair_i (C := C) R X A n = 0 :=
  (pair_shortExact (C := C) R X A).δ_comp (n + 1) n (by simp)

/-- Exactness of  $H_n(A) \rightarrow H_n(X) \rightarrow H_n(X, A)$ . -/
lemma pair_exact_at_X
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
  (ShortComplex.mk (pair_i (C := C) R X A n) (pair_j (C := C) R X A n)
    (pair_i_comp_pair_j (C := C) R X A n)).Exact := by

```

```

simp [pair_i, pair_j, pairShortComplex] using
  (pair_shortExact (C := C) R X A).homology_exact2 n

/-- Exactness of  $H_n(X) \rightarrow H_n(X, A) \rightarrow H_{n-1}(A)$ . -/
lemma pair_exact_at_rel
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
    (ShortComplex.mk (pair_j (C := C) R X A (n + 1)) (pairδ (C := C) R X A n)
      (pair_j_comp_pairδ (C := C) R X A n)).Exact :=
    (pair_shortExact (C := C) R X A).homology_exact3 (n + 1) n (by simp)

/-- Exactness of  $H_{n+1}(X, A) \rightarrow H_n(A) \rightarrow H_n(X)$ . -/
lemma pair_exact_at_A
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
    (ShortComplex.mk (pairδ (C := C) R X A n) (pair_i (C := C) R X A n)
      (pairδ_comp_pair_i (C := C) R X A n)).Exact :=
    (pair_shortExact (C := C) R X A).homology_exact1 (n + 1) n (by simp)

```

A.7 PlatonicSolids/MayerVietoris.lean

Mayer–Vietoris of an excisive triad; stereographic cover of S^n (303 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Homology.HomologicalComplexBiprod
import Mathlib.Algebra.Homology.QuasiIso
import Mathlib.Analysis.Convex.Contractible
import Mathlib.Analysis.InnerProductSpace.PiL2
import Mathlib.Geometry.Manifold.Instances.Sphere
import Mathlib.Topology.Homeomorph.Lemmas
import PlatonicSolids.RelativeHomology
import PlatonicSolids.SingularHomology

/-!
Mayer-Vietoris for a commuting square of spaces
...
      W --iU--> U
      |         |
    iV         jU
      |         |
      v         v
      V --jV--> X
...

The map  $C_*(W) \rightarrow C_*(U) \oplus C_*(V)$ ,  $\sigma \mapsto (iU_\# \sigma, -iV_\# \sigma)$  is a monomorphism
as soon as  $iU$  is. Its cokernel is the chain complex of the cover; the
commuting square induces a comparison  $coker \rightarrow C_*(X)$ . The square is
*excisive* when that comparison is a quasi-isomorphism. In that case the
homology sequence of the cover is the Mayer-Vietoris sequence of  $X$ .

The stereographic cover of  $S^n$  (complements of the poles) is the
classical excisive pair: each chart is contractible. Barycentric
subdivision, mesh shrinking, and the open-cover quasi-isomorphism live
in PlatonicSolids/SingularExcision/. For ModuleCat coefficients the
pair is excisive (IsExcisiveSubspaces_of_openCover); the hex-free
connecting map and  $H_n(S^n) \cong \mathbb{R}$  for  $n > 0$  are in
PlatonicSolids/SphereSingularHomology.lean.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex Metric Set
open scoped Topology ContinuousMap

```



```

set_option linter.unusedSectionVars false

universe v u

variable {C : Type u} [Category.{v} C] [Abelian C]
  [HasCoproducts.{0} C] [HasPullbacks C]

variable (R : C)

/--! ## Abstract triad -/

variable {W U V X : TopCat.{0}}
variable (iU : W → U) (iV : W → V) (jU : U → X) (jV : V → X)

/-- `C_*(W) → C_*(U) ⊕ C_*(V)`, the Mayer-Vietoris inclusion of the intersection. -/
noncomputable def mvIncl :
  ((singularChainComplexFunctor C).obj R).obj W →
    ((singularChainComplexFunctor C).obj R).obj U ⊕
    ((singularChainComplexFunctor C).obj R).obj V :=
  biprod.lift
    (((singularChainComplexFunctor C).obj R).map iU)
    (-( ((singularChainComplexFunctor C).obj R).map iV))

instance mvIncl_mono [Mono iU] : Mono (mvIncl (C := C) R iU iV) :=
  haveI : Mono (((singularChainComplexFunctor C).obj R).map iU) :=
    ((singularChainComplexFunctor C).obj R).map_mono iU
  mono_of_mono_fac (biprod.lift_fst
    (((singularChainComplexFunctor C).obj R).map iU)
    (-( ((singularChainComplexFunctor C).obj R).map iV))))

/-- Chains of the cover: `coker(C_*(W) → C_*(U) ⊕ C_*(V))`. -/
noncomputable def coverChains : ChainComplex C ℕ :=
  cokernel (mvIncl (C := C) R iU iV)

/-- The short complex of the triad. -/
noncomputable def triadShortComplex [Mono iU] :
  ShortComplex (ChainComplex C ℕ) :=
  ShortComplex.mk (mvIncl (C := C) R iU iV) (cokernel.π _) (cokernel.condition _)

instance triadShortComplex_mono_f [Mono iU] :
  Mono (triadShortComplex (C := C) R iU iV).f :=
  inferInstanceAs (Mono (mvIncl (C := C) R iU iV))

instance triadShortComplex_epi_g [Mono iU] :
  Epi (triadShortComplex (C := C) R iU iV).g :=
  inferInstanceAs (Epi (cokernel.π (mvIncl (C := C) R iU iV)))

/-- The triad sequence of singular chains is short exact. -/
lemma triad_shortExact [Mono iU] :
  (triadShortComplex (C := C) R iU iV).ShortExact :=
  ShortComplex.ShortExact.mk'
    (ShortComplex.exact_of_g_is_cokernel _
      (cokernelIsCokernel (mvIncl (C := C) R iU iV)))
  inferInstance inferInstance

/-- `C_*(U) ⊕ C_*(V) → C_*(X)` induced by the two inclusions into `X`. -/
noncomputable def coverToX (_hcomm : iU ≫ jU = iV ≫ jV) :
  ((singularChainComplexFunctor C).obj R).obj U ⊕
  ((singularChainComplexFunctor C).obj R).obj V →
  ((singularChainComplexFunctor C).obj R).obj X :=
  biprod.desc
    (((singularChainComplexFunctor C).obj R).map jU)

```

```

((singularChainComplexFunctor C).obj R).map jV)

lemma mvIncl_comp_coverToX (hcomm : iU >> jU = iV >> jV) :
  mvIncl (C := C) R iU iV >> coverToX (C := C) R iU iV jU jV hcomm = 0 := by
  dsimp [mvIncl, coverToX]
  ext
  simp [← Functor.map_comp, hcomm]

/-- Comparison of cover chains with ambient chains. -/
noncomputable def coverComparison (hcomm : iU >> jU = iV >> jV) :
  coverChains (C := C) R iU iV →
    ((singularChainComplexFunctor C).obj R).obj X :=
  cokernel.desc _ (coverToX (C := C) R iU iV jU jV hcomm)
  (mvIncl_comp_coverToX (C := C) R iU iV jU jV hcomm)

/-- A commuting square of spaces is excisive when cover chains compute  $H_*(X)$ . -/
def IsExcisive (hcomm : iU >> jU = iV >> jV) : Prop :=
  QuasiIso (coverComparison (C := C) R iU iV jU jV hcomm)

/-- Connecting map of the triad:  $H_{n+1}(U+V) \rightarrow H_n(W)$ . -/
noncomputable def triad $\delta$  [Mono iU] (n :  $\mathbb{N}$ ) :
  (coverChains (C := C) R iU iV).homology (n + 1) →
    ((singularChainComplexFunctor C).obj R).obj W).homology n :=
  (triad_shortExact (C := C) R iU iV). $\delta$  (n + 1) n (by simp)

/-- If the square is excisive, homology of the cover is homology of  $X$ . -/
noncomputable def excisiveHomologyIso
  (hcomm : iU >> jU = iV >> jV)
  (h : IsExcisive (C := C) R iU iV jU jV hcomm) (n :  $\mathbb{N}$ ) :
  (coverChains (C := C) R iU iV).homology n  $\cong$ 
    ((singularChainComplexFunctor C).obj R).obj X).homology n :=
  have : QuasiIso (coverComparison (C := C) R iU iV jU jV hcomm) := h
  asIso (homologyMap (coverComparison (C := C) R iU iV jU jV hcomm) n)

/-- Mayer-Vietoris connecting map, after excision. -/
noncomputable def mayerVietoris $\delta$  [Mono iU]
  (hcomm : iU >> jU = iV >> jV)
  (h : IsExcisive (C := C) R iU iV jU jV hcomm) (n :  $\mathbb{N}$ ) :
  ((singularChainComplexFunctor C).obj R).obj X).homology (n + 1) →
    ((singularChainComplexFunctor C).obj R).obj W).homology n :=
  (excisiveHomologyIso (C := C) R iU iV jU jV hcomm h (n + 1)).inv >>
    triad $\delta$  (C := C) R iU iV n

/-- Exactness of  $H_n(W) \rightarrow H_n(U) \oplus H_n(V) \rightarrow H_n(U+V)$ . -/
lemma triad_exact_at_sum [Mono iU] (n :  $\mathbb{N}$ ) :
  (ShortComplex.mk
    (homologyMap (mvIncl (C := C) R iU iV) n)
    (homologyMap (cokernel. $\pi$  (mvIncl (C := C) R iU iV)) n)
    (by rw [← homologyMap_comp, cokernel.condition, homologyMap_zero])).Exact :=
  (triad_shortExact (C := C) R iU iV).homology_exact $_2$  n

/-- Exactness of  $H_{n+1}(U+V) \rightarrow H_n(W) \rightarrow H_n(U) \oplus H_n(V)$ . -/
lemma triad_exact_at_W [Mono iU] (n :  $\mathbb{N}$ ) :
  (ShortComplex.mk (triad $\delta$  (C := C) R iU iV n)
    (homologyMap (mvIncl (C := C) R iU iV) n)
    (by
      simpa [triad $\delta$ , triadShortComplex] using
        (triad_shortExact (C := C) R iU iV). $\delta$ _comp (n + 1) n (by simp))).Exact :=
  (triad_shortExact (C := C) R iU iV).homology_exact $_1$  (n + 1) n (by simp)

/-! ## Subspace covers -/

variable {Y : Type} [TopologicalSpace Y]

```

```

/-- Inclusion of an intersection as a subspace of the first set. -/
def interInclLeft (A B : Set Y) : TopCat.of  $\uparrow(A \cap B) \longrightarrow$  TopCat.of  $\uparrow A :=$ 
  TopCat.ofHom (ContinuousMap.inclusion (fun _ hx => hx.1))

/-- Inclusion of an intersection as a subspace of the second set. -/
def interInclRight (A B : Set Y) : TopCat.of  $\uparrow(A \cap B) \longrightarrow$  TopCat.of  $\uparrow B :=$ 
  TopCat.ofHom (ContinuousMap.inclusion (fun _ hx => hx.2))

instance interInclLeft_mono (A B : Set Y) : Mono (interInclLeft A B) :=
  (TopCat.mono_iff_injective _).2 fun _ _ h =>
    Subtype.ext (congrArg (Subtype.val :  $\uparrow A \rightarrow Y$ ) h)

instance interInclRight_mono (A B : Set Y) : Mono (interInclRight A B) :=
  (TopCat.mono_iff_injective _).2 fun _ _ h =>
    Subtype.ext (congrArg (Subtype.val :  $\uparrow B \rightarrow Y$ ) h)

lemma subset_square_comm (A B : Set Y) :
  interInclLeft A B  $\gg$  subtypeIncl Y A =
  interInclRight A B  $\gg$  subtypeIncl Y B := by
  ext x
  rfl

/-- A pair of subspaces is excisive when their cover computes  $H_*(Y)$ . -/
def IsExcisiveSubspaces (A B : Set Y) : Prop :=
  IsExcisive (C := C) R (interInclLeft A B) (interInclRight A B)
  (subtypeIncl Y A) (subtypeIncl Y B) (subset_square_comm A B)

/-! ## Stereographic cover of the metric sphere -/

/-- The unit  $n$ -sphere in Euclidean space of dimension  $n+1$ . -/
abbrev MetricSphere (n :  $\mathbb{N}$ ) : Type :=
   $\uparrow$ (sphere (0 : EuclideanSpace  $\mathbb{R}$  (Fin (n + 1))) 1)

/-- North pole  $e_0$  of  $S^n$ . -/
noncomputable def northPole (n :  $\mathbb{N}$ ) : MetricSphere n :=
  (EuclideanSpace.single 0 1, by simp [mem_sphere_iff_norm])

/-- South pole  $-e_0$  of  $S^n$ . -/
noncomputable def southPole (n :  $\mathbb{N}$ ) : MetricSphere n :=
  (-EuclideanSpace.single 0 1, by simp [mem_sphere_iff_norm])

lemma northPole_ne_southPole (n :  $\mathbb{N}$ ) : northPole n  $\neq$  southPole n := by
  intro h
  have h1 : (1 :  $\mathbb{R}$ ) = -1 := by
    simpa [northPole, southPole] using
      congrArg (fun p : MetricSphere n =>
        (p : EuclideanSpace  $\mathbb{R}$  (Fin (n + 1))) 0) h
  norm_num at h1

/-- Open chart  $S^n \setminus \{N\}$ . -/
def puncturedNorth (n :  $\mathbb{N}$ ) : Set (MetricSphere n) :=
  {northPole n}c

/-- Open chart  $S^n \setminus \{S\}$ . -/
def puncturedSouth (n :  $\mathbb{N}$ ) : Set (MetricSphere n) :=
  {southPole n}c

lemma isOpen_puncturedNorth (n :  $\mathbb{N}$ ) : IsOpen (puncturedNorth n) :=
  isOpen_compl_singleton

lemma isOpen_puncturedSouth (n :  $\mathbb{N}$ ) : IsOpen (puncturedSouth n) :=

```

```

isOpen_compl_singleton

lemma punctured_cover (n : ℕ) :
  puncturedNorth n ∪ puncturedSouth n = univ := by
  ext x
  simp [puncturedNorth, puncturedSouth]
  exact or_iff_not_imp_left.2 fun hx => by
    rintro rfl
    exact hx (northPole_ne_southPole n).symm

lemma punctured_inter (n : ℕ) :
  puncturedNorth n ∩ puncturedSouth n = {northPole n, southPole n}c := by
  ext x
  simp [puncturedNorth, puncturedSouth]

lemma norm_northPole (n : ℕ) :
  ||(northPole n : EuclideanSpace ℝ (Fin (n + 1)))|| = 1 := by
  simp [northPole]

lemma norm_southPole (n : ℕ) :
  ||(southPole n : EuclideanSpace ℝ (Fin (n + 1)))|| = 1 := by
  simp [southPole]

/-- Stereographic projection from the north pole is a homeomorphism
`S^n \ {N}` homeomorphic to the orthogonal complement of `ℝ · N`. -/
noncomputable def homeoPuncturedNorth (n : ℕ) :
  ↑(puncturedNorth n) ≃t
  ((ℝ · (northPole n : EuclideanSpace ℝ (Fin (n + 1))))⊥) :=
  have hsrc : (stereographic (norm_northPole n)).source = puncturedNorth n := by
  ext x
  simp [puncturedNorth, northPole, stereographic_source]
  (Homeomorph.setCongr hsrc).trans
  ((stereographic (norm_northPole n)).toHomeomorphSourceTarget.trans
    (Homeomorph.Set.univ _))

instance contractible_puncturedNorth (n : ℕ) :
  ContractibleSpace ↑(puncturedNorth n) :=
  (homeoPuncturedNorth n).contractibleSpace

/-- Stereographic projection from the south pole is a homeomorphism
`S^n \ {S}` homeomorphic to the orthogonal complement of `ℝ · S`. -/
noncomputable def homeoPuncturedSouth (n : ℕ) :
  ↑(puncturedSouth n) ≃t
  ((ℝ · (southPole n : EuclideanSpace ℝ (Fin (n + 1))))⊥) :=
  have hsrc : (stereographic (norm_southPole n)).source = puncturedSouth n := by
  ext x
  simp [puncturedSouth, southPole, stereographic_source]
  (Homeomorph.setCongr hsrc).trans
  ((stereographic (norm_southPole n)).toHomeomorphSourceTarget.trans
    (Homeomorph.Set.univ _))

instance contractible_puncturedSouth (n : ℕ) :
  ContractibleSpace ↑(puncturedSouth n) :=
  (homeoPuncturedSouth n).contractibleSpace

/-- Positive singular homology of the northern stereographic chart vanishes. -/
lemma isZero_singularHomology_puncturedNorth
  (n : ℕ) {k : ℕ} (hk : k ≠ 0) :
  IsZero (((singularHomologyFunctor C k).obj R).obj
    (.of ↑(puncturedNorth n))) :=
  isZero_singularHomology_of_contractible R hk

/-- Positive singular homology of the southern stereographic chart vanishes. -/

```

```

lemma isZero_singularHomology_puncturedSouth
  (n : ℕ) {k : ℕ} (hk : k ≠ 0) :
  IsZero (((singularHomologyFunctor C k).obj R).obj
    (.of ↑(puncturedSouth n))) :=
  isZero_singularHomology_of_contractible R hk

-- The hex-free stereographic connecting map is
-- `singularHomology_sphere_mayerVietorisδ` in
-- `PlatonicSolids/SphereSingularHomology.lean` (ModuleCat coefficients).

```

A.8 PlatonicSolids/SingularExcision.lean

Root import of barycentric subdivision and small-chain excision (22 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.SingularExcision.AffineSubdivision
import PlatonicSolids.SingularExcision.AffineRealization
import PlatonicSolids.SingularExcision.Algebra
import PlatonicSolids.SingularExcision.BarycentricGeometry
import PlatonicSolids.SingularExcision.CokernelPushout
import PlatonicSolids.SingularExcision.CoverChains
import PlatonicSolids.SingularExcision.DegreewisePushout
import PlatonicSolids.SingularExcision.Mesh
import PlatonicSolids.SingularExcision.ModuleSupport
import PlatonicSolids.SingularExcision.OpenCoverQI
import PlatonicSolids.SingularExcision.SingularChains
import PlatonicSolids.SingularExcision.SmallChains
import PlatonicSolids.SingularExcision.Subcomplexes

/-!
Root import for the barycentric-subdivision and small-chain foundations
used by the singular-chain excision proof.
-/

```

A.9 PlatonicSolids/SingularExcision/Algebra.lean

Ungraded subdivision retract on an abelian group (146 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Group.Subgroup.Defs
import Mathlib.Algebra.GroupWithZero.Divisibility
import Mathlib.Logic.Function.Iterate
import Mathlib.Tactic.Abel

/-!
The elementwise algebraic core of the small-simplices proof of singular
excision. It is deliberately independent of singular simplices and is
applied to the direct sum of the chain groups of a chain complex of
modules.
-/

open Function

namespace SingularExcision

/-- Ungraded data underlying subdivision: the differential `d`, subdivision

```

```

`S`, and prism `H`, with `id - S = dH + Hd`. -/
structure SubdivisionData (A : Type*) [AddCommGroup A] where
  d : A →+ A
  S : A →+ A
  H : A →+ A
  S_comm : ∀ x, d (S x) = S (d x)
  homotopy : ∀ x, x - S x = d (H x) + H (d x)

namespace SubdivisionData

variable {A : Type*} [AddCommGroup A] (D : SubdivisionData A)

def Cycle (x : A) : Prop :=
  D.d x = 0

def Homologous (x y : A) : Prop :=
  ∃ w : A, x - y = D.d w

lemma cycle_S {x : A} (hx : D.Cycle x) :
  D.Cycle (D.S x) := by
  rw [Cycle, D.S_comm, hx, map_zero]

lemma cycle_iterate {x : A} (hx : D.Cycle x) (N : ℕ) :
  D.Cycle (D.S^[N] x) := by
  induction N with
  | zero => simp
  | succ N ih =>
    rw [iterate_succ_apply']
    exact D.cycle_S ih

lemma homologous_refl (x : A) :
  D.Homologous x x :=
  ⟨0, by simp⟩

lemma homologous_S_of_cycle {x : A} (hx : D.Cycle x) :
  D.Homologous x (D.S x) := by
  refine ⟨D.H x, ?_⟩
  rw [D.homotopy, show D.d x = 0 from hx, map_zero, add_zero]

lemma homologous_trans {x y z : A}
  (hxy : D.Homologous x y) (hyz : D.Homologous y z) :
  D.Homologous x z := by
  obtain ⟨u, hu⟩ := hxy
  obtain ⟨v, hv⟩ := hyz
  refine ⟨u + v, ?_⟩
  rw [map_add, ← hu, ← hv]
  abel

lemma homologous_iterate_of_cycle {x : A} (hx : D.Cycle x) (N : ℕ) :
  D.Homologous x (D.S^[N] x) := by
  induction N with
  | zero => simp using D.homologous_refl x
  | succ N ih =>
    rw [iterate_succ_apply']
    exact D.homologous_trans ih
    (D.homologous_S_of_cycle (D.cycle_iterate hx N))

variable (B : AddSubgroup A)

/-- The small chains are closed under boundary, subdivision, and prism. -/
structure SmallInvariant : Prop where
  d_mem : ∀ {x}, x ∈ B → D.d x ∈ B
  S_mem : ∀ {x}, x ∈ B → D.S x ∈ B

```

```

H_mem :  $\forall \{x\}, x \in B \rightarrow D.H \ x \in B$ 

lemma iterate_mem (hB : D.SmallInvariant B) {x : A}
  (hx : x  $\in$  B) (N :  $\mathbb{N}$ ) :
  D.S^[N] x  $\in$  B := by
  induction N with
  | zero => simp
  | succ N ih =>
    rw [iterate_succ_apply']
    exact hB.S_mem ih

lemma small_boundary_of_small_homologous
  (hB : D.SmallInvariant B) {x : A}
  (hx : x  $\in$  B) (hxcycle : D.Cycle x) (N :  $\mathbb{N}$ ) :
   $\exists w \in B, x - D.S^[N] x = D.d \ w$  := by
  induction N with
  | zero => exact ⟨0, B.zero_mem, by simp⟩
  | succ N ih =>
    obtain ⟨w, hwB, hw⟩ := ih
    let xN := D.S^[N] x
    refine ⟨w + D.H xN,
      B.add_mem hwB (hB.H_mem (D.iterate_mem B hB hx N)), ?_⟩
    rw [map_add, ← hw]
    have hxNcycle := D.cycle_iterate hxcycle N
    have hstep : xN - D.S xN = D.d (D.H xN) := by
      rw [D.homotopy, show D.d xN = 0 from hxNcycle, map_zero, add_zero]
    rw [← hstep, iterate_succ_apply']
    abel

/-- If subdivision eventually puts every finite chain in the small-chain
subgroup, every homology class has a small representative and every small
boundary bounds a small chain. -/
theorem small_cycles_and_boundaries
  (hB : D.SmallInvariant B)
  (eventual :  $\forall x : A, \exists N, D.S^[N] x \in B$ ) :
  ( $\forall z : A, D.Cycle \ z \rightarrow$ 
     $\exists b \in B, D.Cycle \ b \wedge D.Homologous \ z \ b$ )  $\wedge$ 
  ( $\forall b : A, b \in B \rightarrow D.Cycle \ b \rightarrow$ 
    ( $\exists x : A, b = D.d \ x$ )  $\rightarrow \exists y \in B, b = D.d \ y$ ) := by
  constructor
  · intro z hz
    obtain ⟨N, hN⟩ := eventual z
    exact ⟨D.S^[N] z, hN, D.cycle_iterate hz N,
      D.homologous_iterate_of_cycle hz N⟩
  · intro b hb hbcycle hboundary
    obtain ⟨x, rfl⟩ := hboundary
    obtain ⟨N, hNx⟩ := eventual x
    obtain ⟨w, hwB, hw⟩ :=
      D.small_boundary_of_small_homologous B hB hb hbcycle N
    refine ⟨D.S^[N] x + w, B.add_mem hNx hwB, ?_⟩
    rw [map_add, ← hw]
    have hcomm :  $\forall M : \mathbb{N}, D.S^[M] (D.d \ x) = D.d (D.S^[M] x)$  := by
      intro M
      induction M with
      | zero => simp
      | succ M ih =>
        rw [iterate_succ_apply', iterate_succ_apply', D.S_comm, ih]
    rw [← hcomm N]
    abel

end SubdivisionData
end SingularExcision

```

A.10 PlatonicSolids/SingularExcision/AffineSubdivision.lean

Affine barycentric subdivision and prism (823 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Data.Finsupp.BigOperators
import Mathlib.Data.Finsupp.Ext
import Mathlib.LinearAlgebra.AffineSpace.Simplex.Centroid
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.Module

/-!
# Finite affine chains for singular excision

This file isolates the integral signs in barycentric subdivision from
the categorical singular-chain construction. An `AffineSimplex V n`
is an ordered list of `n + 1` vertices in `V`.

The cone convention is chosen so that
 $\partial (\text{cone } v \ c) = c - \text{cone } v \ (\partial \ c)$ . In dimension one this produces the
usual permutation formula  $[a, m] - [b, m]$  for subdivision of  $[a, b]$ .
-/

open scoped BigOperators

namespace PlatonicSolids.SingularExcision

noncomputable section

set_option maxHeartbeats 800000

/-- An ordered affine `n`-simplex, represented by its vertices. -/
abbrev AffineSimplex (V : Type*) (n : ℕ) := Fin (n + 1) → V

/-- Finite integral chains of ordered affine simplices. -/
abbrev AffineChain (V : Type*) (n : ℕ) := AffineSimplex V n →₀ ℤ

lemma AffineChain.eq_of_single {V : Type*} {n : ℕ} {M : Type*} [AddCommGroup M]
  (f g : AffineChain V n →+ M) (c : AffineChain V n)
  (h : ∀ σ z, f (Finsupp.single σ z) = g (Finsupp.single σ z)) :
  f c = g c :=
  DFunLike.congr_fun (Finsupp.addHom_ext h) c

/-- Delete vertex `i` from an ordered simplex. -/
def AffineSimplex.face {V : Type*} {n : ℕ}
  (σ : AffineSimplex V (n + 1)) (i : Fin (n + 2)) :
  AffineSimplex V n :=
  fun j ↦ σ (i.succAbove j)

/-- The chain consisting of one oriented simplex. -/
def simplex {V : Type*} {n : ℕ} (σ : AffineSimplex V n) :
  AffineChain V n :=
  Finsupp.single σ 1

/-- Boundary of one ordered affine simplex. -/
def simplexBoundary {V : Type*} {n : ℕ}
  (σ : AffineSimplex V (n + 1)) : AffineChain V n :=
  ∑ i : Fin (n + 2), (-1 : ℤ) ^ (i : ℕ) • simplex (σ.face i)

/-- The integral boundary map on finite affine chains. -/
def boundary {V : Type*} {n : ℕ} :
```



```

    AffineChain V (n + 1) →+ AffineChain V n :=
Finsupp.liftAddHom fun σ ↦
    (smulAddHom ℤ (AffineChain V n)).flip (simplexBoundary σ)

@[simp]
theorem boundary_simplex {V : Type*} {n : ℕ} (σ : AffineSimplex V (n + 1)) :
    boundary (simplex σ) = simplexBoundary σ := by
    simp [boundary, simplex]

/-- A vertex as a zero-chain. -/
def vertex {V : Type*} (v : V) : AffineChain V 0 :=
    simplex (fun _ ↦ v)

/-- The ordered edge `[a,b]`. -/
def edge {V : Type*} (a b : V) : AffineChain V 1 :=
    simplex (Fin.cases a fun _ ↦ b)

/-- The ordered triangle `[a,b,c]`. -/
def triangle {V : Type*} (a b c : V) : AffineChain V 2 :=
    simplex (Fin.cases a (Fin.cases b fun _ ↦ c))

/-- The boundary of an edge is its endpoint minus its startpoint. -/
theorem boundary_edge {V : Type*} (a b : V) :
    boundary (edge a b) = vertex b - vertex a := by
    classical
    let σ : AffineSimplex V 1 := Fin.cases a fun _ ↦ b
    have h0 : σ.face (0 : Fin 2) = (fun _ ↦ b) := by
        funext i
        fin_cases i
        rfl
    have h1 : σ.face (1 : Fin 2) = (fun _ ↦ a) := by
        funext i
        fin_cases i
        rfl
    rw [edge, boundary_simplex]
    simp [simplexBoundary, Fin.sum_univ_succ, h0, h1, σ, vertex]
    abel

/-- The low-dimensional oriented boundary formula for a triangle. -/
theorem boundary_triangle {V : Type*} (a b c : V) :
    boundary (triangle a b c) = edge b c - edge a c + edge a b := by
    classical
    let σ : AffineSimplex V 2 := Fin.cases a (Fin.cases b fun _ ↦ c)
    have h0 : σ.face (0 : Fin 3) = (Fin.cases b fun _ ↦ c) := by
        funext i
        fin_cases i <;> rfl
    have h1 : σ.face (1 : Fin 3) = (Fin.cases a fun _ ↦ c) := by
        funext i
        fin_cases i <;> rfl
    have h2 : σ.face (2 : Fin 3) = (Fin.cases a fun _ ↦ b) := by
        funext i
        fin_cases i <;> rfl
    rw [triangle, boundary_simplex]
    simp [simplexBoundary, Fin.sum_univ_succ, h0, h1, h2, σ, edge]
    abel

/-- Append an apex to the ordered vertices of a simplex. -/
def AffineSimplex.append {V : Type*} {n : ℕ}
    (σ : AffineSimplex V n) (v : V) : AffineSimplex V (n + 1) :=
    Fin.lastCases v σ

/-- Deleting the appended last vertex recovers the original simplex. -/
@[simp]

```

```

lemma AffineSimplex.append_face_last {V : Type*} {n : ℕ}
  (σ : AffineSimplex V n) (v : V) :
  (σ.append v).face (Fin.last (n + 1)) = σ := by
  funext i
  simp [AffineSimplex.face, AffineSimplex.append,
    Fin.succAbove_last_apply]

/-- Deleting an original vertex commutes with appending a last vertex. -/
@[simp]
lemma AffineSimplex.append_face_castSucc {V : Type*} {n : ℕ}
  (σ : AffineSimplex V (n + 1)) (v : V) (i : Fin (n + 2)) :
  (σ.append v).face i.castSucc = (σ.face i).append v := by
  funext j
  refine Fin.lastCases ?_ (fun k ↦ ?_) j
  · simp [AffineSimplex.face, AffineSimplex.append]
  · simp [AffineSimplex.face, AffineSimplex.append,
    Fin.castSucc_succAbove_castSucc]

/-- Cone on one simplex, with the dimension-dependent orientation sign. -/
def simplexCone {V : Type*} {n : ℕ} (v : V) (σ : AffineSimplex V n) :
  AffineChain V (n + 1) :=
  (-1 : ℤ) ^ (n + 1) • simplex (σ.append v)

/-- Extend coning linearly to finite chains. -/
def cone {V : Type*} {n : ℕ} (v : V) :
  AffineChain V n →+ AffineChain V (n + 1) :=
  Finsupp.liftAddHom fun σ ↦
    (smulAddHom ℤ (AffineChain V (n + 1))).flip (simplexCone v σ)

@[simp]
theorem cone_simplex {V : Type*} {n : ℕ} (v : V) (σ : AffineSimplex V n) :
  cone v (simplex σ) = simplexCone v σ := by
  simp [cone, simplex]

theorem simplexCone_vertex {V : Type*} (v a : V) :
  simplexCone v (fun _ : Fin 1 ↦ a) = -edge a v := by
  unfold simplexCone edge
  norm_num
  congr 1
  funext i
  fin_cases i <|> rfl

theorem simplexCone_edge {V : Type*} (v a b : V) :
  simplexCone v (Fin.cases a fun _ ↦ b) = triangle a b v := by
  unfold simplexCone triangle
  norm_num
  congr 1
  funext i
  fin_cases i <|> rfl

/-- Boundary of a cone on a positive-dimensional simplex. -/
theorem boundary_simplexCone {V : Type*} {n : ℕ}
  (v : V) (σ : AffineSimplex V (n + 1)) :
  boundary (simplexCone v σ) =
    simplex σ - cone v (simplexBoundary σ) := by
  classical
  rw [simplexCone, map_zsmul, boundary_simplex, simplexBoundary]
  rw [Fin.sum_univ_castSucc]
  simp only [AffineSimplex.append_face_castSucc,
    AffineSimplex.append_face_last, Fin.val_castSucc, Fin.val_last]
  simp only [simplexBoundary]
  rw [map_sum]
  simp_rw [map_zsmul, cone_simplex, simplexCone]

```

```

simp only [Nat.add_assoc, Nat.reduceAdd, smul_add, Finset.smul_sum]
simp_rw [smul_smul]
have hsign :  $(-1 : \mathbb{Z}) \wedge (n + 2) * (-1 : \mathbb{Z}) \wedge (n + 2) = 1$  := by
  rw [← pow_add, show  $n + 2 + (n + 2) = 2 * (n + 2)$  by omega, pow_mul]
  norm_num
rw [hsign, one_smul]
rw [sub_eq_add_neg]
rw [add_comm ( $\sum x : \text{Fin } (n + 2)$ ,
  ( $(-1 : \mathbb{Z}) \wedge (n + 2) * (-1 : \mathbb{Z}) \wedge (x : \mathbb{N})) \bullet$ 
    simplex  $((\sigma.\text{face } x).\text{append } v)$ ) (simplex  $\sigma$ ))]
congr 1
rw [← Finset.sum_neg_distrib]
apply Finset.sum_congr rfl
intro i _
rw [← neg_smul]
congr 1
have hnext :  $(-1 : \mathbb{Z}) \wedge (n + 2) = -(((-1 : \mathbb{Z}) \wedge (n + 1)))$  := by
  rw [show  $n + 2 = \text{Nat.succ } (n + 1)$  by omega, pow_succ]
  ring
rw [hnext]
ring

/-- Boundary of a cone on any positive-dimensional affine chain. -/
theorem boundary_cone {V : Type*} {n : ℕ}
  (v : V) (c : AffineChain V (n + 1)) :
  boundary (cone v c) = c - cone v (boundary c) := by
  classical
  refine AffineChain.eq_of_single
  (boundary.comp (cone v))
  (AddMonoidHom.id _ - (cone v).comp boundary) c ?_
intro  $\sigma$  z
change boundary (cone v (Finsupp.single  $\sigma$  z)) =
  Finsupp.single  $\sigma$  z - cone v (boundary (Finsupp.single  $\sigma$  z))
rw [show Finsupp.single  $\sigma$  z = z • simplex  $\sigma$  by simp [simplex]]
simp [map_zsmul, cone_simplex, boundary_simplexCone, boundary_simplex, smul_sub]

@[simp]
theorem cone_vertex {V : Type*} (v a : V) :
  cone v (vertex a) = -edge a v := by
  rw [vertex, cone_simplex, simplexCone_vertex]

@[simp]
theorem cone_edge {V : Type*} (v a b : V) :
  cone v (edge a b) = triangle a b v := by
  rw [edge, cone_simplex, simplexCone_edge]

/-- The cone identity on zero-simplices. -/
theorem boundary_cone_vertex {V : Type*} (v a : V) :
  boundary (cone v (vertex a)) = vertex a - vertex v := by
  classical
  rw [cone_vertex]
  rw [map_neg, boundary_edge]
  abel

/-- The cone identity on edges. -/
theorem boundary_cone_edge {V : Type*} (v a b : V) :
  boundary (cone v (edge a b)) =
    edge a b - cone v (vertex b - vertex a) := by
  classical
  rw [cone_edge, map_sub, cone_vertex, cone_vertex]
  rw [boundary_triangle]
  abel

```

```

/-- The affine barycenter of the vertices of a simplex in a vector space. -/
def AffineSimplex.barycenter {V : Type*} [AddCommGroup V] [Module ℚ V]
  {n : ℕ} (σ : AffineSimplex V n) : V :=
  (n + 1 : ℚ)-1 • ∑ i, σ i

/-- The midpoint used by one-dimensional barycentric subdivision. -/
def midpoint {V : Type*} [AddCommGroup V] [Module ℚ V] (a b : V) : V :=
  (2 : ℚ)-1 • (a + b)

theorem barycenter_edge {V : Type*} [AddCommGroup V] [Module ℚ V] (a b : V) :
  AffineSimplex.barycenter
    ((Fin.cases a fun _ ↦ b) : AffineSimplex V 1) = midpoint a b := by
  norm_num [AffineSimplex.barycenter, midpoint, Fin.sum_univ_succ]

/-- The first simplicial identity, as an equality of ordered faces. -/
lemma AffineSimplex.face_face {V : Type*} {n : ℕ}
  (σ : AffineSimplex V (n + 2)) {i j : Fin (n + 2)} (hij : i ≤ j) :
  (σ.face j.succ).face i = (σ.face i.castSucc).face j := by
  funext k
  simp only [AffineSimplex.face]
  congr 1
  change j.succ.succAbove (i.succAbove k) =
    i.castSucc.succAbove (j.succAbove k)
  dsimp [Fin.succAbove]
  rcases i with ⟨i, _⟩
  rcases j with ⟨j, _⟩
  rcases k with ⟨k, _⟩
  split_ifs <;> · simp at * <;> lia

/-- The signed sum of coefficients of a finite affine chain. -/
def coeffSum {V : Type*} {n : ℕ} : AffineChain V n →+ ℤ :=
  Finsupp.liftAddHom fun _ => AddMonoidHom.id ℤ

@[simp]
lemma coeffSum_single {V : Type*} {n : ℕ} (σ : AffineSimplex V n) (z : ℤ) :
  coeffSum (Finsupp.single σ z) = z := by
  simp [coeffSum]

@[simp]
lemma coeffSum_simplex {V : Type*} {n : ℕ} (σ : AffineSimplex V n) :
  coeffSum (simplex σ) = 1 := by
  simp [simplex]

@[simp]
lemma coeffSum_vertex {V : Type*} (v : V) :
  coeffSum (vertex v) = 1 := by
  simp [vertex]

lemma coeffSum_simplexBoundary_one {V : Type*} (σ : AffineSimplex V 1) :
  coeffSum (simplexBoundary σ) = 0 := by
  rw [simplexBoundary, Fin.sum_univ_two]
  simp [coeffSum_simplex]

/-- Cone identity in dimension zero: the extra term is the augmentation. -/
theorem boundary_cone_zero {V : Type*} (v : V) (c : AffineChain V 0) :
  boundary (cone v c) = c - coeffSum c • vertex v := by
  classical
  refine AffineChain.eq_of_single
    (boundary.comp (cone v))
    (AddMonoidHom.id _ -
      ((smulAddHom ℤ (AffineChain V 0)).flip (vertex v)).comp coeffSum) c ?_
  intro σ z
  have hσ : σ = fun _ : Fin 1 ↦ σ 0 := by

```

```

    funext i
    fin_cases i
    rfl
change boundary (cone v (Finsupp.single  $\sigma$  z)) =
  Finsupp.single  $\sigma$  z - coeffSum (Finsupp.single  $\sigma$  z) • vertex v
have hsingle : Finsupp.single  $\sigma$  z = z • vertex ( $\sigma$  0) := by
  rw [h $\sigma$ , vertex, simplex, Finsupp.smul_single, smul_eq_mul, mul_one]
  rw [hsingle, map_zsmul, cone_vertex, map_zsmul, map_neg, boundary_edge,
    map_zsmul, coeffSum_vertex]
module

/-- Double faces cancel:  $\partial \circ \partial = 0$  on a generator. -/
theorem boundary_simplexBoundary {V : Type*} {n :  $\mathbb{N}$ }
  ( $\sigma$  : AffineSimplex V (n + 2)) :
  boundary (simplexBoundary  $\sigma$ ) = 0 := by
  classical
  simp only [simplexBoundary, map_sum, map_zsmul, boundary_simplex]
  simp only [Finset.smul_sum, smul_smul]
  rw [Finset.sum_comm, ← Finset.sum_product']
  let P := Fin (n + 2) × Fin (n + 3)
  let S : Finset P := {ij : P | (ij.2 :  $\mathbb{N}$ ) ≤ (ij.1 :  $\mathbb{N}$ )}
  rw [Finset.univ_product_univ, ← Finset.sum_add_sum_compl S,
    ← eq_neg_iff_add_eq_zero, ← Finset.sum_neg_distrib]
  let  $\varphi$  :  $\forall$  ij : P, ij ∈ S → P := fun ij hij =>
    (Fin.castLT ij.2 (lt_of_le_of_lt (Finset.mem_filter.mp hij).right (Fin.is_lt ij.1)),
      ij.1.succ)
  apply Finset.sum_bij  $\varphi$ 
  · intro ij hij
    simp_rw [S,  $\varphi$ , Finset.compl_filter, Finset.mem_filter_univ, Fin.val_succ,
      Fin.val_castLT] at hij ⊢
    omega
  · rintro ⟨j, i⟩ hij ⟨j', i'⟩ hij' h
    rw [Prod.mk_inj]
    exact ⟨by simpa [ $\varphi$ ] using congr_arg Prod.snd h,
      by simpa [ $\varphi$ , Fin.castSucc_castLT] using
        congr_arg Fin.castSucc (congr_arg Prod.fst h)⟩
  · rintro ⟨j', i'⟩ hij'
    simp_rw [S, Finset.compl_filter, Finset.mem_filter_univ, not_le] at hij'
    refine ⟨(i'.pred ?_, Fin.castSucc j'), ?_, ?_⟩
    · exact_mod_cast hij'.ne_zero
    · simpa [S] using Nat.le_sub_one_of_lt hij'
    · simp [ $\varphi$ , Fin.castLT_castSucc, Fin.succ_pred]
  · rintro ⟨j, i⟩ hij
    simp only [← neg_smul]
    congr 1
    · simp [ $\varphi$ , Fin.val_succ, pow_add, pow_one, mul_neg, neg_neg, mul_comm]
    · have hij' : i ≤ j.castSucc := by
        simpa [S, Fin.le_iff_val_le_val] using hij
      have hlt : (i :  $\mathbb{N}$ ) < n + 2 :=
        Nat.lt_of_le_of_lt (Fin.le_iff_val_le_val.mp hij') j.isLt
      have hcast : (i.castLT hlt).castSucc = i := by
        ext
        simp
      have hface :
        ( $\sigma$ .face j.succ).face (i.castLT hlt) = ( $\sigma$ .face i).face j := by
        have := AffineSimplex.face_face ( $\sigma$  :=  $\sigma$ )
          (i := i.castLT hlt) (j := j) (by
            simpa [Fin.le_iff_val_le_val, Fin.val_castLT] using hij')
        rwa [hcast] at this
      congr 1
      simpa [ $\varphi$ ] using hface.symm

theorem boundary_boundary {V : Type*} {n :  $\mathbb{N}$ } (c : AffineChain V (n + 2)) :

```

```

    boundary (boundary c) = 0 := by
classical
refine AffineChain.eq_of_single
  (boundary.comp boundary) 0 c ?_
intro  $\sigma$  z
change boundary (boundary (Finsupp.single  $\sigma$  z)) = 0
rw [show Finsupp.single  $\sigma$  z = z • simplex  $\sigma$  by simp [simplex]]
simp [map_zsmul, boundary_simplex, boundary_simplexBoundary]

/-- Barycentric subdivision of finite affine chains, coning on a
chosen point of each simplex (classically the barycenter). -/
def subdivide {V : Type*} (b : {n : ℕ} → AffineSimplex V n → V) :
  (n : ℕ) → AffineChain V n →+ AffineChain V n
| 0 => AddMonoidHom.id _
| n + 1 =>
  Finsupp.liftAddHom fun  $\sigma$  =>
    (smulAddHom ℤ (AffineChain V (n + 1))).flip
    (cone (b  $\sigma$ ) (subdivide b n (simplexBoundary  $\sigma$ )))

@[simp]
theorem subdivide_zero {V : Type*} (b : {n : ℕ} → AffineSimplex V n → V)
  (c : AffineChain V 0) :
  subdivide b 0 c = c :=
  rfl

theorem subdivide_single {V : Type*} (b : {n : ℕ} → AffineSimplex V n → V)
  {n : ℕ} ( $\sigma$  : AffineSimplex V (n + 1)) (z : ℤ) :
  subdivide b (n + 1) (Finsupp.single  $\sigma$  z) =
  z • cone (b  $\sigma$ ) (subdivide b n (simplexBoundary  $\sigma$ )) := by
  simp [subdivide]

theorem subdivide_simplex {V : Type*} (b : {n : ℕ} → AffineSimplex V n → V)
  {n : ℕ} ( $\sigma$  : AffineSimplex V (n + 1)) :
  subdivide b (n + 1) (simplex  $\sigma$ ) =
  cone (b  $\sigma$ ) (subdivide b n (simplexBoundary  $\sigma$ )) := by
  simpa [simplex] using subdivide_single b  $\sigma$  1

/-- Subdivision commutes with the boundary. -/
theorem boundary_subdivide {V : Type*} (b : {n : ℕ} → AffineSimplex V n → V)
  (n : ℕ) (c : AffineChain V (n + 1)) :
  boundary (subdivide b (n + 1) c) = subdivide b n (boundary c) := by
  induction n with
  | zero =>
    induction c using Finsupp.induction with
    | zero => simp
    | single_add  $\sigma$  z c _ _ ih =>
      rw [map_add, map_add, map_add, ih]
      rw [subdivide_single b, map_zsmul, subdivide_zero]
      rw [boundary_cone_zero, coeffSum_simplexBoundary_one, zero_smul,
        sub_zero]
      rw [show Finsupp.single  $\sigma$  z = z • simplex  $\sigma$  by simp [simplex]]
      simp [map_zsmul, boundary_simplex]
  | succ n ih =>
    induction c using Finsupp.induction with
    | zero => simp
    | single_add  $\sigma$  z c _ _ ihc =>
      rw [map_add, map_add, map_add, ihc]
      rw [subdivide_single b, map_zsmul, boundary_cone, ih,
        boundary_simplexBoundary]
      simp only [map_zero, sub_zero]
      rw [show Finsupp.single  $\sigma$  z = z • simplex  $\sigma$  by simp [simplex]]
      simp [map_zsmul, boundary_simplex]

```

```

/-- The prism operator: a chain homotopy from the identity to subdivision. -/
def prism {V : Type*} (b : {n : ℕ} → AffineSimplex V n → V) :
  (n : ℕ) → AffineChain V n →+ AffineChain V (n + 1)
| 0 => 0
| n + 1 =>
  Finsupp.liftAddHom fun σ =>
    (smulAddHom ℤ (AffineChain V (n + 2))).flip
    (cone (b σ)
      (simplex σ - subdivide b (n + 1) (simplex σ) -
       prism b n (simplexBoundary σ)))

@[simp]
theorem prism_zero {V : Type*} (b : {n : ℕ} → AffineSimplex V n → V)
  (c : AffineChain V 0) :
  prism b 0 c = 0 :=
  rfl

theorem prism_single {V : Type*} (b : {n : ℕ} → AffineSimplex V n → V)
  {n : ℕ} (σ : AffineSimplex V (n + 1)) (z : ℤ) :
  prism b (n + 1) (Finsupp.single σ z) =
    z • cone (b σ)
    (simplex σ - subdivide b (n + 1) (simplex σ) -
     prism b n (simplexBoundary σ)) := by
  simp [prism, smul_sub]

/-- The prism identity  $\partial T + T\partial = \text{id} - Sd$ . -/
theorem boundary_prism {V : Type*} (b : {n : ℕ} → AffineSimplex V n → V)
  (n : ℕ) (c : AffineChain V (n + 1)) :
  boundary (prism b (n + 1) c) + prism b n (boundary c) =
    c - subdivide b (n + 1) c := by
  induction n with
  | zero =>
    induction c using Finsupp.induction with
    | zero => simp
    | single_add σ z c _ _ ih =>
      have hsingle :
        boundary (prism b 1 (Finsupp.single σ z)) +
        prism b 0 (boundary (Finsupp.single σ z)) =
        Finsupp.single σ z - subdivide b 1 (Finsupp.single σ z) := by
        rw [prism_single, map_zsmul]
        simp only [prism_zero, sub_zero, add_zero]
      have hw :
        boundary (simplex σ - subdivide b 1 (simplex σ)) = 0 := by
        rw [map_sub, boundary_subdivide, boundary_simplex,
          subdivide_zero]
      abel
      rw [boundary_cone, hw]
      simp only [map_zero, sub_zero]
      rw [show Finsupp.single σ z = z • simplex σ by
        rw [simplex, Finsupp.smul_single, smul_eq_mul, mul_one]]
      simp [map_zsmul]
      module
      simp only [map_add]
      calc
      _ = (boundary (prism b 1 (Finsupp.single σ z)) +
        prism b 0 (boundary (Finsupp.single σ z))) +
        (boundary (prism b 1 c) + prism b 0 (boundary c)) := by abel
      _ = (Finsupp.single σ z - subdivide b 1 (Finsupp.single σ z)) +
        (c - subdivide b 1 c) := by rw [hsingle, ih]
      _ = _ := by abel
  | succ n ih =>
    induction c using Finsupp.induction with
    | zero => simp

```

```

| single_add  $\sigma$  z c _ _ ihc =>
  have hsingle :
    boundary (prism b (n + 2) (Finsupp.single  $\sigma$  z)) +
    prism b (n + 1) (boundary (Finsupp.single  $\sigma$  z)) =
    Finsupp.single  $\sigma$  z - subdivide b (n + 2) (Finsupp.single  $\sigma$  z) := by
  rw [prism_single, map_zsmul]
  have hw :
    boundary (simplex  $\sigma$  - subdivide b (n + 2) (simplex  $\sigma$ ) -
    prism b (n + 1) (simplexBoundary  $\sigma$ )) = 0 := by
  have hih := ih (simplexBoundary  $\sigma$ )
  have hsd := boundary_subdivide b (n + 1) (simplex  $\sigma$ )
  rw [map_sub, map_sub, hsd, boundary_simplex]
  simp [boundary_simplexBoundary] at hih
  rw [hih]
  abel
  rw [boundary_cone, hw]
  simp only [map_zero, sub_zero]
  rw [show Finsupp.single  $\sigma$  z = z • simplex  $\sigma$  by
    rw [simplex, Finsupp.smul_single, smul_eq_mul, mul_one]]
  simp [map_zsmul, boundary_simplex]
  module
  simp only [map_add]
  calc
    _ = (boundary (prism b (n + 2) (Finsupp.single  $\sigma$  z)) +
    prism b (n + 1) (boundary (Finsupp.single  $\sigma$  z))) +
    (boundary (prism b (n + 2) c) +
    prism b (n + 1) (boundary c)) := by abel
    _ = (Finsupp.single  $\sigma$  z - subdivide b (n + 2) (Finsupp.single  $\sigma$  z)) +
    (c - subdivide b (n + 2) c) := by rw [hsingle, ihc]
    _ = _ := by abel

/-- Barycentric subdivision of an oriented affine interval.

The expression `[a,m] - [b,m]` is the signed-permutation form of
`[a,m] + [m,b]`; no quotient by affine reparametrization is taken here.
-/
def subdivideInterval {V : Type*} [AddCommGroup V] [Module  $\mathbb{Q}$  V]
  (a b : V) : AffineChain V 1 :=
  edge a (midpoint a b) - edge b (midpoint a b)

/-- Subdivision is a chain map on the one-dimensional checkpoint. -/
theorem boundary_subdivideInterval {V : Type*} [AddCommGroup V] [Module  $\mathbb{Q}$  V]
  (a b : V) :
  boundary (subdivideInterval a b) = vertex b - vertex a := by
  simp [subdivideInterval, boundary_edge]

/-- The universal one-dimensional prism between an edge and its subdivision. -/
def intervalPrism {V : Type*} [AddCommGroup V] [Module  $\mathbb{Q}$  V]
  (a b : V) : AffineChain V 2 :=
  cone (midpoint a b) (edge a b - subdivideInterval a b)

/-- The prism identity ` $\partial P = \text{id} - S_d$ ` for an affine interval. -/
theorem boundary_intervalPrism {V : Type*} [AddCommGroup V] [Module  $\mathbb{Q}$  V]
  (a b : V) :
  boundary (intervalPrism a b) = edge a b - subdivideInterval a b := by
  classical
  unfold intervalPrism subdivideInterval
  simp only [map_sub]
  rw [boundary_cone_edge, boundary_cone_edge, boundary_cone_edge]
  simp only [map_sub, cone_vertex]
  abel

/-- Push vertices of an ordered simplex forward along a function. -/

```



```

def AffineSimplex.map {V W : Type*} {n : ℕ} (f : V → W)
  (σ : AffineSimplex V n) : AffineSimplex W n :=
  f ∘ σ

@[simp]
theorem AffineSimplex.map_apply {V W : Type*} {n : ℕ} (f : V → W)
  (σ : AffineSimplex V n) (i : Fin (n + 1)) :
  AffineSimplex.map f σ i = f (σ i) :=
  rfl

@[simp]
theorem AffineSimplex.map_id {V : Type*} {n : ℕ} (σ : AffineSimplex V n) :
  AffineSimplex.map id σ = σ :=
  rfl

@[simp]
theorem AffineSimplex.map_comp {U V W : Type*} {n : ℕ}
  (g : V → W) (f : U → V) (σ : AffineSimplex U n) :
  AffineSimplex.map g (AffineSimplex.map f σ) =
  AffineSimplex.map (g ∘ f) σ :=
  rfl

@[simp]
theorem AffineSimplex.map_face {V W : Type*} {n : ℕ} (f : V → W)
  (σ : AffineSimplex V (n + 1)) (i : Fin (n + 2)) :
  AffineSimplex.map f (σ.face i) = (AffineSimplex.map f σ).face i :=
  rfl

@[simp]
theorem AffineSimplex.map_append {V W : Type*} {n : ℕ} (f : V → W)
  (σ : AffineSimplex V n) (v : V) :
  AffineSimplex.map f (σ.append v) = (AffineSimplex.map f σ).append (f v) := by
  funext i
  refine Fin.lastCases ?_ (fun _ ↦ ?_) i
  · simp [AffineSimplex.map, AffineSimplex.append]
  · simp [AffineSimplex.map, AffineSimplex.append]

/-- Push a finite affine chain forward along a function of vertices. -/
def AffineChain.map {V W : Type*} (f : V → W) {n : ℕ} :
  AffineChain V n →+ AffineChain W n :=
  Finsupp.mapDomain.addMonoidHom (AffineSimplex.map f)

@[simp]
theorem AffineChain.map_single {V W : Type*} (f : V → W) {n : ℕ}
  (σ : AffineSimplex V n) (z : ℤ) :
  AffineChain.map f (Finsupp.single σ z) =
  Finsupp.single (AffineSimplex.map f σ) z := by
  simp [AffineChain.map, Finsupp.mapDomain_single]

@[simp]
theorem AffineChain.map_simplex {V W : Type*} (f : V → W) {n : ℕ}
  (σ : AffineSimplex V n) :
  AffineChain.map f (simplex σ) = simplex (AffineSimplex.map f σ) := by
  simp [simplex]

@[simp]
theorem AffineChain.map_vertex {V W : Type*} (f : V → W) (v : V) :
  AffineChain.map f (vertex v) = vertex (f v) := by
  simp [vertex, AffineSimplex.map]
  rfl

theorem AffineChain.map_simplexBoundary {V W : Type*} (f : V → W) {n : ℕ}
  (σ : AffineSimplex V (n + 1)) :

```

```

    AffineChain.map f (simplexBoundary  $\sigma$ ) =
      simplexBoundary (AffineSimplex.map f  $\sigma$ ) := by
    simp [simplexBoundary, map_sum, map_zsmul]

theorem AffineChain.map_boundary {V W : Type*} (f : V → W) {n : ℕ}
  (c : AffineChain V (n + 1)) :
  AffineChain.map f (boundary c) = boundary (AffineChain.map f c) := by
  refine AffineChain.eq_of_single
    ((AffineChain.map f).comp boundary)
    (boundary.comp (AffineChain.map f)) c ?_
  intro  $\sigma$  z
  change AffineChain.map f (boundary (Finsupp.single  $\sigma$  z)) =
    boundary (AffineChain.map f (Finsupp.single  $\sigma$  z))
  rw [show Finsupp.single  $\sigma$  z = z • simplex  $\sigma$  by simp [simplex]]
  simp [map_zsmul, boundary_simplex, AffineChain.map_simplexBoundary]

theorem AffineChain.map_simplexCone {V W : Type*} (f : V → W) {n : ℕ}
  (v : V) ( $\sigma$  : AffineSimplex V n) :
  AffineChain.map f (simplexCone v  $\sigma$ ) =
    simplexCone (f v) (AffineSimplex.map f  $\sigma$ ) := by
  simp [simplexCone, map_zsmul, AffineSimplex.map_append]

theorem AffineChain.map_cone {V W : Type*} (f : V → W) {n : ℕ}
  (v : V) (c : AffineChain V n) :
  AffineChain.map f (cone v c) = cone (f v) (AffineChain.map f c) := by
  refine AffineChain.eq_of_single
    ((AffineChain.map f).comp (cone v))
    ((cone (f v)).comp (AffineChain.map f)) c ?_
  intro  $\sigma$  z
  change AffineChain.map f (cone v (Finsupp.single  $\sigma$  z)) =
    cone (f v) (AffineChain.map f (Finsupp.single  $\sigma$  z))
  rw [show Finsupp.single  $\sigma$  z = z • simplex  $\sigma$  by simp [simplex]]
  simp [map_zsmul, cone_simplex, AffineChain.map_simplexCone]

/-- Subdivision is natural for maps that send chosen cone points to
chosen cone points. -/
theorem AffineChain.map_subdivide {V W : Type*}
  (bV : {n : ℕ} → AffineSimplex V n → V)
  (bW : {n : ℕ} → AffineSimplex W n → W) (f : V → W)
  (hb : ∀ {n : ℕ} ( $\sigma$  : AffineSimplex V n),
    f (bV  $\sigma$ ) = bW (AffineSimplex.map f  $\sigma$ ))
  (n : ℕ) (c : AffineChain V n) :
  AffineChain.map f (subdivide bV n c) =
    subdivide bW n (AffineChain.map f c) := by
  induction n with
  | zero => simp
  | succ n ih =>
    refine AffineChain.eq_of_single
      ((AffineChain.map f).comp (subdivide bV (n + 1)))
      ((subdivide bW (n + 1)).comp (AffineChain.map f)) c ?_
    intro  $\sigma$  z
    change AffineChain.map f (subdivide bV (n + 1) (Finsupp.single  $\sigma$  z)) =
      subdivide bW (n + 1) (AffineChain.map f (Finsupp.single  $\sigma$  z))
    rw [subdivide_single, AffineChain.map_single, subdivide_single, map_zsmul,
      AffineChain.map_cone, ih, AffineChain.map_simplexBoundary, hb]

/-- The prism is natural for the same cone-point maps. -/
theorem AffineChain.map_prism {V W : Type*}
  (bV : {n : ℕ} → AffineSimplex V n → V)
  (bW : {n : ℕ} → AffineSimplex W n → W) (f : V → W)
  (hb : ∀ {n : ℕ} ( $\sigma$  : AffineSimplex V n),
    f (bV  $\sigma$ ) = bW (AffineSimplex.map f  $\sigma$ ))
  (n : ℕ) (c : AffineChain V n) :

```

```

AffineChain.map f (prism bV n c) =
  prism bW n (AffineChain.map f c) := by
induction n with
| zero => simp
| succ n ih =>
  refine AffineChain.eq_of_single
    ((AffineChain.map f).comp (prism bV (n + 1)))
    ((prism bW (n + 1)).comp (AffineChain.map f)) c ?_
  intro  $\sigma$  z
  change AffineChain.map f (prism bV (n + 1) (Finsupp.single  $\sigma$  z)) =
    prism bW (n + 1) (AffineChain.map f (Finsupp.single  $\sigma$  z))
  rw [prism_single, AffineChain.map_single, prism_single, map_zsmul,
    AffineChain.map_cone]
  simp [map_sub, AffineChain.map_subdivide bV bW f hb,
    ih, AffineChain.map_simplexBoundary, hb]

lemma mem_support_simplexCone {V : Type*} {n : ℕ} (v : V)
  ( $\sigma$  : AffineSimplex V n) { $\tau$  : AffineSimplex V (n + 1)}
  (h $\tau$  :  $\tau \in (\text{simplexCone } v \sigma).\text{support}$ ) :
   $\tau = \sigma.\text{append } v$  := by
  have hz :  $(-1 : \mathbb{Z}) \wedge (n + 1) \neq 0 := \text{pow\_ne\_zero } \_$  (by decide)
  simpa [simplexCone, simplex, Finsupp.support_smul_eq hz,
    Finsupp.support_single_ne_zero _ (one_ne_zero ( $\alpha := \mathbb{Z}$ ))] using h $\tau$ 

lemma cone_single {V : Type*} {n : ℕ} (v : V)
  ( $\sigma$  : AffineSimplex V n) (z : ℤ) :
  cone v (Finsupp.single  $\sigma$  z) = z • simplexCone v  $\sigma$  := by
  rw [show Finsupp.single  $\sigma$  z = z • simplex  $\sigma$  by simp [simplex]]
  simp [map_zsmul, cone_simplex]

lemma mem_support_cone {V : Type*} {n : ℕ} (v : V)
  (c : AffineChain V n) { $\tau$  : AffineSimplex V (n + 1)}
  (h $\tau$  :  $\tau \in (\text{cone } v c).\text{support}$ ) :
   $\exists \sigma \in c.\text{support}, \tau = \sigma.\text{append } v$  := by
  classical
  induction c using Finsupp.induction with
  | zero =>
    simp at h $\tau$ 
  | single_add  $\sigma$  z c hmem hn ih =>
    rw [map_add, cone_single] at h $\tau$ 
    have h $\tau'$  :
       $\tau \in (z \bullet \text{simplexCone } v \sigma).\text{support} \cup (\text{cone } v c).\text{support} :=$ 
      Finset.mem_of_subset Finsupp.support_add h $\tau$ 
    rw [Finset.mem_union] at h $\tau'$ 
    rcases h $\tau'$  with h $\sigma$  | hc
    · refine ⟨ $\sigma$ , ?_,
      mem_support_simplexCone v  $\sigma$  (Finset.mem_of_subset Finsupp.support_smul h $\sigma$ )⟩
      have hc $\sigma$  : c  $\sigma$  = 0 := Finsupp.notMem_support_iff.mp hmem
      simp [Finsupp.mem_support_iff, hc $\sigma$ , hn]
    · obtain ⟨ $\rho$ , h $\rho$ , h $\tau\rho$ ⟩ := ih hc
      refine ⟨ $\rho$ , ?_, h $\tau\rho$ ⟩
      simp [Finsupp.mem_support_iff] at h $\rho$  ⊢
      have h $\rho\sigma$  :  $\rho \neq \sigma := \text{fun } h \Rightarrow \text{hmem } (h \blacktriangleright (\text{Finsupp.mem\_support\_iff.mpr } h\rho))$ 
      simpa [h $\rho\sigma$ ] using h $\rho$ 

lemma mem_support_subdivide_simplexBoundary {V : Type*}
  (b : {n : ℕ} → AffineSimplex V n → V) {n : ℕ}
  ( $\sigma$  : AffineSimplex V (n + 1)) { $\tau$  : AffineSimplex V n}
  (h $\tau$  :  $\tau \in (\text{subdivide } b \ n \ (\text{simplexBoundary } \sigma)).\text{support}$ ) :
   $\exists i : \text{Fin } (n + 2),$ 
   $\tau \in (\text{subdivide } b \ n \ (\text{simplex } (\sigma.\text{face } i))).\text{support} := by
  classical
  simp only [simplexBoundary, map_sum, map_zsmul] at h $\tau$$ 
```

```

have hsum (s : Finset (Fin (n + 2))) :
   $\tau \in (\sum i \in s, (-1 : \mathbb{Z}) \sim (i : \mathbb{N}) \bullet$ 
     $\text{subdivide } b \ n \ (\text{simplex } (\sigma.\text{face } i)))\text{.support} \rightarrow$ 
   $\exists i \in s, \tau \in (\text{subdivide } b \ n \ (\text{simplex } (\sigma.\text{face } i)))\text{.support} := \text{by}$ 
  induction s using Finset.induction with
  | empty => simp
  | insert i s his ih =>
    intro h
    rw [Finset.sum_insert his] at h
    have h' := Finset.mem_of_subset Finsupp.support_add h
    rw [Finset.mem_union] at h'
    rcases h' with hi | hs
    · exact ⟨i, Finset.mem_insert_self _ _,
      Finset.mem_of_subset Finsupp.support_smul hi⟩
    · obtain ⟨j, hj, hj $\tau$ ⟩ := ih hs
      exact ⟨j, Finset.mem_insert_of_mem hj, hj $\tau$ ⟩
obtain ⟨i, _, hi⟩ := hsum _ h $\tau$ 
exact ⟨i, hi⟩

lemma mem_support_subdivide_simplex {V : Type*}
  (b : {n : ℕ} → AffineSimplex V n → V) {n : ℕ}
  ( $\sigma$  : AffineSimplex V (n + 1)) { $\tau$  : AffineSimplex V (n + 1)}
  (h $\tau$  :  $\tau \in (\text{subdivide } b \ (n + 1) \ (\text{simplex } \sigma))\text{.support}$ ) :
   $\exists i : \text{Fin } (n + 2),$ 
   $\exists \rho \in (\text{subdivide } b \ n \ (\text{simplex } (\sigma.\text{face } i)))\text{.support},$ 
   $\tau = \rho.\text{append } (b \ \sigma) := \text{by}$ 
  rw [subdivide_simplex] at h $\tau$ 
  obtain ⟨ $\rho$ , h $\rho$ , h $\tau\rho$ ⟩ := mem_support_cone (b  $\sigma$ ) _ h $\tau$ 
  obtain ⟨i, hi⟩ := mem_support_subdivide_simplexBoundary b  $\sigma$  h $\rho$ 
  exact ⟨i,  $\rho$ , hi, h $\tau\rho$ ⟩

/-- Every simplex in the support of a subdivided chain comes from
subdividing one of the original generators. -/
lemma mem_support_subdivide {V : Type*}
  (b : {n : ℕ} → AffineSimplex V n → V) (n : ℕ)
  {c : AffineChain V n} { $\tau$  : AffineSimplex V n}
  (h $\tau$  :  $\tau \in (\text{subdivide } b \ n \ c)\text{.support}$ ) :
   $\exists \sigma \in c.\text{support}, \tau \in (\text{subdivide } b \ n \ (\text{simplex } \sigma))\text{.support} := \text{by}$ 
  classical
  induction c using Finsupp.induction with
  | zero =>
    simp at h $\tau$ 
  | single_add  $\sigma$  z c hmem hn ih =>
    rw [map_add] at h $\tau$ 
    have h $\tau'$  :
       $\tau \in (\text{subdivide } b \ n \ (\text{Finsupp.single } \sigma \ z))\text{.support} \cup$ 
       $(\text{subdivide } b \ n \ c)\text{.support} :=$ 
      Finset.mem_of_subset Finsupp.support_add h $\tau$ 
    rw [Finset.mem_union] at h $\tau'$ 
    rcases h $\tau'$  with h $\sigma$  | hc
    · refine ⟨ $\sigma$ , ?_, ?_⟩
      · have hc $\sigma$  : c  $\sigma$  = 0 := Finsupp.notMem_support_iff.mp hmem
        simp [Finsupp.mem_support_iff, hc $\sigma$ , hn]
      · have hsmul :
           $\text{subdivide } b \ n \ (\text{Finsupp.single } \sigma \ z) =$ 
           $z \bullet \text{subdivide } b \ n \ (\text{simplex } \sigma) := \text{by}$ 
          rw [show Finsupp.single  $\sigma$  z = z • simplex  $\sigma$  by simp [simplex]]
          simp
          rw [hsmul] at h $\sigma$ 
          exact Finset.mem_of_subset Finsupp.support_smul h $\sigma$ 
    · obtain ⟨ $\rho$ , h $\rho$ , h $\tau\rho$ ⟩ := ih hc
      refine ⟨ $\rho$ , ?_, h $\tau\rho$ ⟩

```

```

      simp [Finsupp.mem_support_iff] at hρ ⊢
      have hρσ : ρ ≠ σ := fun h => hmem (h ▶ (Finsupp.mem_support_iff.mpr hρ))
      simpa [hρσ] using hρ

end

end PlatonicSolids.SingularExcision

```

A.11 PlatonicSolids/SingularExcision/AffineRealization.lean

Realization of affine simplices in a space (322 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.AlgebraicTopology.SingularSet
import Mathlib.AlgebraicTopology.SimplicialSet.TopAdj
import Mathlib.LinearAlgebra.Finsupp.Pi
import PlatonicSolids.SingularExcision.AffineSubdivision

/-!
# Realization of finite affine simplices

Finite affine chains become singular chains by realizing a list of vertices
in a standard simplex as its barycentric affine map.
-/

open CategoryTheory
open scoped BigOperators Simplicial

namespace PlatonicSolids.SingularExcision

noncomputable section

/-- Barycentric realization of ordered vertices in a standard simplex. -/
def AffineSimplex.realize {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  C(stdSimplex ℝ (Fin (p + 1)), stdSimplex ℝ (Fin (q + 1))) where
toFun x :=
  ⟨fun j ↦ ∑ i, x.1 i * (σ i).1 j, by
    constructor
    · intro j
      exact Finset.sum_nonneg fun i _ ↦
        mul_nonneg (x.2.1 i) ((σ i).2.1 j)
    · rw [Finset.sum_comm]
      calc
        ∑ i, ∑ j, x.1 i * (σ i).1 j =
          ∑ i, x.1 i * ∑ j, (σ i).1 j := by
            congr with i
            rw [Finset.mul_sum]
          _ = ∑ i, x.1 i := by
            congr with i
            rw [(σ i).2.2, mul_one]
          _ = 1 := x.2.2)
continuous_toFun := by
  apply Continuous.subtype_mk
  apply continuous_pi
  intro j
  apply continuous_finsetSum
  intro i _
  exact ((continuous_apply i).comp continuous_subtype_val).mul continuous_const

```

```

@[simp]
theorem AffineSimplex.realize_apply {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p)
  (x : stdSimplex ℝ (Fin (p + 1))) (j : Fin (q + 1)) :
  (σ.realize x : Fin (q + 1) → ℝ) j =
    ∑ i, x.1 i * (σ i).1 j :=
  rfl

/-- Realization sends each standard vertex to the corresponding listed vertex. -/
@[simp]
theorem AffineSimplex.realize_vertex {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p)
  (i : Fin (p + 1)) :
  σ.realize (stdSimplex.vertex i) = σ i := by
  classical
  apply Subtype.ext
  funext j
  change ∑ k, (Pi.single i (1 : ℝ) : Fin (p + 1) → ℝ) k * (σ k).1 j =
    (σ i).1 j
  rw [Finset.sum_eq_single i]
  · simp
  · intro k _ hki
    simp [Pi.single_eq_of_ne hki]
  · simp

/-- Pairing against a `FunOnFinite` pushforward is the original pairing. -/
lemma sum_linearMap_mul {α β : Type*} [Fintype α] [Fintype β] [DecidableEq β]
  (f : α → β) (s : α → ℝ) (t : β → ℝ) :
  ∑ y : β, FunOnFinite.linearMap ℝ ℝ f s y * t y = ∑ x : α, s x * t (f x) := by
  simp_rw [FunOnFinite.linearMap_apply_apply]
  calc
    ∑ y : β, (∑ x ∈ Finset.univ.filter (fun x ↦ f x = y), s x) * t y =
      ∑ y : β, ∑ x ∈ Finset.univ.filter (fun x ↦ f x = y), s x * t y := by
      simp_rw [Finset.sum_mul]
    _ = ∑ y : β, ∑ x ∈ Finset.univ.filter (fun x ↦ f x = y), s x * t (f x) := by
      refine Finset.sum_congr rfl fun y _ ↦ Finset.sum_congr rfl fun x hx ↦ ?_
      rw [(Finset.mem_filter.mp hx).2]
    _ = ∑ x : α, s x * t (f x) :=
      Finset.sum_fiberwise Finset.univ f (fun x ↦ s x * t (f x))

/-- Realization commutes with deleting a vertex. -/
theorem AffineSimplex.realize_face {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) (p + 1))
  (i : Fin (p + 2)) (x : stdSimplex ℝ (Fin (p + 1))) :
  σ.realize (stdSimplex.map i.succAbove x) = (σ.face i).realize x := by
  classical
  apply Subtype.ext
  funext j
  simp [stdSimplex.map_coe] using
    sum_linearMap_mul i.succAbove x.1 (fun k ↦ (σ k).1 j)

/-- Reparametrize a singular simplex by a realized finite affine simplex. -/
noncomputable def reparametrizeSingularSimplex {X : TopCat.{0}} {p q : ℕ}
  (s : (TopCat.toSSet.obj X) _[q])
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  (TopCat.toSSet.obj X) _[p] :=
  (X.toSSetObjEquiv _).symm ((X.toSSetObjEquiv _ s).comp σ.realize)

@[simp]
theorem reparametrizeSingularSimplex_toContinuousMap
  {X : TopCat.{0}} {p q : ℕ}
  (s : (TopCat.toSSet.obj X) _[q])
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :

```

```

X.toSSetObjEquiv _ (reparametrizeSingularSimplex s  $\sigma$ ) =
  (X.toSSetObjEquiv _ s).comp  $\sigma$ .realize := by
simp [reparametrizeSingularSimplex]

/-- Faces of a reparametrized simplex are reparametrizations of the faces. -/
theorem reparametrizeSingularSimplex_face {X : TopCat.{0}} {p q : ℕ}
  (s : (TopCat.toSSet.obj X) _[[q]])
  ( $\sigma$  : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) (p + 1))
  (i : Fin (p + 2)) :
  reparametrizeSingularSimplex s ( $\sigma$ .face i) =
    (TopCat.toSSet.obj X). $\delta$  i (reparametrizeSingularSimplex s  $\sigma$ ) := by
  apply (X.toSSetObjEquiv _).injective
  ext x
  rw [reparametrizeSingularSimplex_toContinuousMap,
    TopCat.toSSetObjEquiv_ $\delta$ _apply,
    reparametrizeSingularSimplex_toContinuousMap]
  simp [AffineSimplex.realize_face]

/-- The identity affine simplex on a standard simplex. -/
def idSimplex (n : ℕ) :
  AffineSimplex (stdSimplex ℝ (Fin (n + 1))) n :=
  stdSimplex.vertex

/-- The identity affine simplex realizes as the identity map. -/
@[simp]
theorem realize_idSimplex (n : ℕ) :
  (idSimplex n).realize = ContinuousMap.id _ := by
  classical
  ext x j
  change  $\sum i, x.1 i * (stdSimplex.vertex i : Fin (n + 1) \rightarrow \mathbb{R}) j = x.1 j$ 
  simp_rw [stdSimplex.vertex_coe]
  rw [Finset.sum_eq_single j]
  · simp
  · intro i _ hij
    rw [Pi.single_eq_of_ne hij.symm, mul_zero]
  · simp

/-- Reparametrizing by the identity simplex is the identity. -/
@[simp]
theorem reparametrizeSingularSimplex_id {X : TopCat.{0}} {n : ℕ}
  (s : (TopCat.toSSet.obj X) _[[n]]) :
  reparametrizeSingularSimplex s (idSimplex n) = s := by
  apply (X.toSSetObjEquiv _).injective
  rw [reparametrizeSingularSimplex_toContinuousMap, realize_idSimplex]
  ext x
  simp

/-- Reparametrizing by a face of the identity simplex is an ordinary face. -/
theorem reparametrizeSingularSimplex_id_face {X : TopCat.{0}} {n : ℕ}
  (s : (TopCat.toSSet.obj X) _[[n + 1]]) (i : Fin (n + 2)) :
  reparametrizeSingularSimplex s ((idSimplex (n + 1)).face i) =
    (TopCat.toSSet.obj X). $\delta$  i s := by
  rw [reparametrizeSingularSimplex_face, reparametrizeSingularSimplex_id]

/-- Geometric barycenter of an affine simplex inside a standard simplex. -/
def AffineSimplex.stdBarycenter {p q : ℕ}
  ( $\sigma$  : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  stdSimplex ℝ (Fin (q + 1)) :=
   $\sigma$ .realize stdSimplex.barycenter

/-- Realization is functorial for vertex-wise maps coming from an
ambient realized simplex. -/
theorem AffineSimplex.realize_map {p q k : ℕ}

```

```

    (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p)
    (ρ : AffineSimplex (stdSimplex ℝ (Fin (p + 1))) k) :
    (AffineSimplex.map σ.realize ρ).realize = σ.realize.comp ρ.realize := by
classical
apply ContinuousMap.ext
intro x
apply Subtype.ext
ext j
change (∑ i, x.1 i * ∑ m, (ρ i).1 m * (σ m).1 j) =
  ∑ m, (∑ i, x.1 i * (ρ i).1 m) * (σ m).1 j
simp_rw [Finset.mul_sum]
rw [Finset.sum_comm]
refine Finset.sum_congr rfl fun m _ => ?_
rw [Finset.sum_mul]
refine Finset.sum_congr rfl fun i _ => ?_
ring

theorem AffineSimplex.stdBarycenter_map {p q k : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p)
  (ρ : AffineSimplex (stdSimplex ℝ (Fin (p + 1))) k) :
  AffineSimplex.stdBarycenter (AffineSimplex.map σ.realize ρ) =
    σ.realize (AffineSimplex.stdBarycenter ρ) := by
simp [AffineSimplex.stdBarycenter, AffineSimplex.realize_map]

@[simp]
theorem AffineSimplex.map_realize_idSimplex {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  AffineSimplex.map σ.realize (idSimplex p) = σ := by
funext i
simp [AffineSimplex.map, idSimplex]

theorem reparametrizeSingularSimplex_map {X : TopCat.{0}} {p q k : ℕ}
  (s : (TopCat.toSSet.obj X) _[q])
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p)
  (ρ : AffineSimplex (stdSimplex ℝ (Fin (p + 1))) k) :
  reparametrizeSingularSimplex s (AffineSimplex.map σ.realize ρ) =
    reparametrizeSingularSimplex (reparametrizeSingularSimplex s σ) ρ := by
apply (X.toSSetObjEquiv _).injective
rw [reparametrizeSingularSimplex_toContinuousMap,
  reparametrizeSingularSimplex_toContinuousMap,
  reparametrizeSingularSimplex_toContinuousMap,
  AffineSimplex.realize_map]
ext x
simp [ContinuousMap.comp_assoc]

/-- Barycentric subdivision of affine chains in a standard simplex. -/
def subdivideStd {q : ℕ} :
  (n : ℕ) → AffineChain (stdSimplex ℝ (Fin (q + 1))) n →+
    AffineChain (stdSimplex ℝ (Fin (q + 1))) n :=
  subdivide AffineSimplex.stdBarycenter

/-- Prism operator for affine chains in a standard simplex. -/
def prismStd {q : ℕ} :
  (n : ℕ) → AffineChain (stdSimplex ℝ (Fin (q + 1))) n →+
    AffineChain (stdSimplex ℝ (Fin (q + 1))) (n + 1) :=
  prism AffineSimplex.stdBarycenter

theorem boundary_subdivideStd {q n : ℕ}
  (c : AffineChain (stdSimplex ℝ (Fin (q + 1))) (n + 1)) :
  boundary (subdivideStd (q := q) (n + 1) c) =
    subdivideStd (q := q) n (boundary c) :=
  boundary_subdivide AffineSimplex.stdBarycenter n c

```



```

theorem boundary_prismStd {q n : ℕ}
  (c : AffineChain (stdSimplex ℝ (Fin (q + 1))) (n + 1)) :
  boundary (prismStd (q := q) (n + 1) c) +
    prismStd (q := q) n (boundary c) =
    c - subdivideStd (q := q) (n + 1) c :=
  boundary_prism AffineSimplex.stdBarycenter n c

/-- Subdivision of the identity simplex. -/
def standardSubdivision (n : ℕ) :
  AffineChain (stdSimplex ℝ (Fin (n + 1))) n :=
  subdivideStd (q := n) n (simplex (idSimplex n))

/-- Prism of the identity simplex. -/
def standardPrism (n : ℕ) :
  AffineChain (stdSimplex ℝ (Fin (n + 1))) (n + 1) :=
  prismStd (q := n) n (simplex (idSimplex n))

theorem AffineChain.map_subdivideStd {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p)
  (n : ℕ) (c : AffineChain (stdSimplex ℝ (Fin (p + 1))) n) :
  AffineChain.map σ.realize (subdivideStd (q := p) n c) =
    subdivideStd (q := q) n (AffineChain.map σ.realize c) :=
  AffineChain.map_subdivide
  AffineSimplex.stdBarycenter AffineSimplex.stdBarycenter σ.realize
  (fun ρ ↦ (AffineSimplex.stdBarycenter_map σ ρ).symm) n c

theorem AffineChain.map_prismStd {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p)
  (n : ℕ) (c : AffineChain (stdSimplex ℝ (Fin (p + 1))) n) :
  AffineChain.map σ.realize (prismStd (q := p) n c) =
    prismStd (q := q) n (AffineChain.map σ.realize c) :=
  AffineChain.map_prism
  AffineSimplex.stdBarycenter AffineSimplex.stdBarycenter σ.realize
  (fun ρ ↦ (AffineSimplex.stdBarycenter_map σ ρ).symm) n c

theorem AffineChain.map_standardSubdivision {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  AffineChain.map σ.realize (standardSubdivision p) =
    subdivideStd (q := q) p (simplex σ) := by
  simp [standardSubdivision, AffineChain.map_subdivideStd]

theorem AffineChain.map_standardPrism {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  AffineChain.map σ.realize (standardPrism p) =
    prismStd (q := q) p (simplex σ) := by
  simp [standardPrism, AffineChain.map_prismStd]

/-- The signed barycentric subdivision chain of the standard interval. -/
def standardIntervalSubdivision :
  AffineChain (stdSimplex ℝ (Fin 2)) 1 :=
  edge (stdSimplex.vertex 0) stdSimplex.barycenter -
  edge (stdSimplex.vertex 1) stdSimplex.barycenter

/-- The standard interval subdivision has the original endpoint boundary. -/
theorem boundary_standardIntervalSubdivision :
  boundary standardIntervalSubdivision =
    vertex (stdSimplex.vertex (1 : Fin 2)) -
    vertex (stdSimplex.vertex (0 : Fin 2)) := by
  simp [standardIntervalSubdivision, boundary_edge]

/-- Universal affine prism for subdivision of the standard interval. -/
def standardIntervalPrism :
  AffineChain (stdSimplex ℝ (Fin 2)) 2 :=

```

```

cone stdSimplex.barycenter
  (edge (stdSimplex.vertex 0) (stdSimplex.vertex 1) -
   standardIntervalSubdivision)

/-- The standard one-dimensional prism identity. -/
theorem boundary_standardIntervalPrism :
  boundary standardIntervalPrism =
    edge (stdSimplex.vertex 0) (stdSimplex.vertex 1) -
      standardIntervalSubdivision := by
classical
unfold standardIntervalPrism standardIntervalSubdivision
simp only [map_sub]
rw [boundary_cone_edge, boundary_cone_edge, boundary_cone_edge]
simp only [map_sub, cone_vertex]
abel

end

end PlatonicSolids.SingularExcision

```

A.12 PlatonicSolids/SingularExcision/Mesh.lean

Lebesgue number and mesh-ratio $n/(n+1)$ (117 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Analysis.Convex.StdSimplex
import Mathlib.Topology.MetricSpace.Pseudo.Lemmas

/-!
Metric input for the small-simplices theorem.

The Lebesgue-number lemma is most useful for subdivision in the following
form: every nonempty subset of the compact domain whose diameter is smaller
than one fixed positive number lies in one member of the open cover.
-/

open Metric Set

universe u v

variable {α : Type u} {ι : Sort v} [PseudoMetricSpace α]

/-- The classical mesh contraction factor for barycentric subdivision
of an `n`-simplex. -/
noncomputable def barycentricMeshRatio (n : ℕ) : ℝ :=
  n / (n + 1)

lemma barycentricMeshRatio_nonneg (n : ℕ) :
  0 ≤ barycentricMeshRatio n := by
  rw [barycentricMeshRatio]
  exact div_nonneg (Nat.cast_nonneg n) (by positivity)

lemma barycentricMeshRatio_lt_one (n : ℕ) :
  barycentricMeshRatio n < 1 := by
  rw [barycentricMeshRatio, div_lt_one]
  · exact_mod_cast Nat.lt_succ_self n
  · positivity

/-- Iterated barycentric subdivision eventually has mesh below every
positive threshold. -/

```

```

lemma exists_barycentricMeshRatio_pow_lt (n : ℕ) {ε : ℝ} (hε : 0 < ε) :
  ∃ k : ℕ, barycentricMeshRatio n ^ k < ε :=
  exists_pow_lt_of_lt_one hε (barycentricMeshRatio_lt_one n)

/-- Diameter form of the Lebesgue-number lemma. -/
theorem exists_diam_lt_forall_subset_openCover
  {K : Set α} {U : ι → Set α}
  (hK : IsCompact K) (hU : ∀ i, IsOpen (U i))
  (hcover : K ⊆ ⋃ i, U i) :
  ∃ δ > 0, ∀ {s : Set α}, s ⊆ K → s.Nonempty →
    Metric.diam s < δ → ∃ i, s ⊆ U i := by
  obtain ⟨δ, hδ, hball⟩ :=
    lebesgue_number_lemma_of_metric hK hU hcover
  refine ⟨δ, hδ, ?_⟩
  intro s hsK hsne hsdiam
  obtain ⟨x, hxs⟩ := hsne
  obtain ⟨i, hi⟩ := hball x (hsK hxs)
  refine ⟨i, fun y hys ↦ hi ?_⟩
  rw [mem_ball, dist_comm]
  exact (Metric.dist_le_diam_of_mem (hK.isBounded.subset hsK) hxs hys).trans_lt hsdiam

/-- A continuous simplex pulls a two-set open cover back to an open cover
of its compact standard-simplex domain, with a uniform diameter bound. -/
theorem exists_diam_lt_forall_image_subset_left_or_right
  {n : ℕ} {Y : Type u} [TopologicalSpace Y]
  (σ : C(stdSimplex ℝ (Fin (n + 1)), Y))
  {A B : Set Y} (hA : IsOpen A) (hB : IsOpen B)
  (hAB : A ∪ B = univ) :
  ∃ δ > 0, ∀ {s : Set (stdSimplex ℝ (Fin (n + 1)))},
    s.Nonempty → Metric.diam s < δ →
      σ '' s ⊆ A ∨ σ '' s ⊆ B := by
  let U : Bool → Set (stdSimplex ℝ (Fin (n + 1))) :=
    fun b ↦ if b then σ ⁻¹' A else σ ⁻¹' B
  have hU : ∀ b, IsOpen (U b) := by
    intro b
    cases b
    · simp [U] using hB.preimage σ.continuous
    · simp [U] using hA.preimage σ.continuous
  have hcover : (univ : Set (stdSimplex ℝ (Fin (n + 1)))) ⊆ ⋃ b, U b := by
    intro x hx
    have hxAB : σ x ∈ A ∪ B := by rw [hAB]; exact mem_univ _
    rcases hxAB with hxA | hxB
    · exact mem_iUnion.2 ⟨true, by simp [U]⟩
    · exact mem_iUnion.2 ⟨false, by simp [U]⟩
  obtain ⟨δ, hδ, hsmall⟩ :=
    exists_diam_lt_forall_subset_openCover
      (U := U) isCompact_univ hU hcover
  refine ⟨δ, hδ, ?_⟩
  intro s hsne hsdiam
  obtain ⟨b, hb⟩ := hsmall (subset_univ s) hsne hsdiam
  cases b
  · right
    rintro _ ⟨x, hx, rfl⟩
    exact hb hx
  · left
    rintro _ ⟨x, hx, rfl⟩
    exact hb hx

/-- A geometric mesh bound times an iterated contraction factor
eventually falls below every positive threshold. The remaining
geometric input is that each barycentric piece of an `n`-simplex
has diameter at most `barycentricMeshRatio n` times the parent. -/
lemma exists_barycentricMeshRatio_pow_mul_lt (n : ℕ) (C : ℝ) {ε : ℝ}

```

```

    (hε : 0 < ε) :
    ∃ k : ℕ, barycentricMeshRatio n ^ k * C < ε := by
rcases le_or_gt C 0 with hC | hC
· exact ⟨0, by simp; linarith⟩
· obtain ⟨k, hk⟩ :=
    exists_barycentricMeshRatio_pow_lt n (div_pos hε hC)
    exact ⟨k, (lt_div_iff₀ hC).mp hk⟩

lemma exists_barycentricMeshRatio_pow_mul_diam_stdSimplex_lt
  (n : ℕ) {ε : ℝ} (hε : 0 < ε) :
  ∃ k : ℕ,
    barycentricMeshRatio n ^ k *
      Metric.diam
        (Set.univ : Set (stdSimplex ℝ (Fin (n + 1)))) < ε :=
exists_barycentricMeshRatio_pow_mul_lt n _ hε

```

A.13 PlatonicSolids/SingularExcision/BarycentricGeometry.lean

Diameter of barycentric pieces and eventual smallness (380 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Analysis.Normed.Module.Convex
import PlatonicSolids.SingularExcision.AffineRealization
import PlatonicSolids.SingularExcision.Mesh

/-!
Diameter of barycentric pieces. Each cone-on-face piece of an
`n`-simplex has diameter at most `n/(n+1)` times the parent, so
iterated subdivision has mesh going to zero.
-/

open Metric Set Bornology
open scoped BigOperators

namespace PlatonicSolids.SingularExcision

noncomputable section

lemma barycentricMeshRatio_le_succ (n : ℕ) :
  barycentricMeshRatio n ≤ barycentricMeshRatio (n + 1) := by
simp only [barycentricMeshRatio]
rw [div_le_div_iff₀ (by positivity) (by positivity)]
norm_cast
nlinarith

lemma AffineSimplex.range_face_subset {V : Type*} {n : ℕ}
  (σ : AffineSimplex V (n + 1)) (i : Fin (n + 2)) :
  Set.range (σ.face i) ⊆ Set.range σ :=
fun _ ⟨j, hj⟩ => ⟨i.succAbove j, hj⟩

lemma AffineSimplex.range_append {V : Type*} {n : ℕ}
  (ρ : AffineSimplex V n) (v : V) :
  Set.range (ρ.append v) = insert v (Set.range ρ) := by
ext x
constructor
· rintro ⟨i, rfl⟩
  induction i using Fin.lastCases with
  | last =>
    left
    simp [AffineSimplex.append]

```

```

| cast j =>
  right
  exact ⟨j, by simp [AffineSimplex.append]⟩
· rintro (rfl | ⟨j, rfl⟩)
· exact ⟨Fin.last _, by simp [AffineSimplex.append]⟩
· exact ⟨j.castSucc, by simp [AffineSimplex.append]⟩

lemma AffineSimplex.realize_val {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p)
  (x : stdSimplex ℝ (Fin (p + 1))) :
  (σ.realize x : Fin (q + 1) → ℝ) =
    ∑ i, x.1 i • (σ i : Fin (q + 1) → ℝ) := by
  funext j
  rw [realize_apply, Finset.sum_apply]
  refine Finset.sum_congr rfl fun i _ => ?_
  rw [Pi.smul_apply, smul_eq_mul]
  rfl

lemma AffineSimplex.realize_mem_convexHull {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p)
  (x : stdSimplex ℝ (Fin (p + 1))) :
  (σ.realize x : Fin (q + 1) → ℝ) ∈
    convexHull ℝ ((↑) '' Set.range σ) := by
  rw [AffineSimplex.realize_val]
  exact (convex_convexHull ℝ _).sum_mem
  (fun i _ => x.2.1 i) x.2.2
  (fun i _ => subset_convexHull ℝ _ ⟨σ i, ⟨i, rfl⟩, rfl⟩)

lemma AffineSimplex.stdBarycenter_mem_convexHull {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  (σ.stdBarycenter : Fin (q + 1) → ℝ) ∈
    convexHull ℝ ((↑) '' Set.range σ) :=
  σ.realize_mem_convexHull stdSimplex.barycenter

lemma AffineSimplex.range_subset_range_realize {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  Set.range σ ⊆ Set.range σ.realize := by
  rintro _ ⟨i, rfl⟩
  exact ⟨stdSimplex.vertex i, σ.realize_vertex i⟩

lemma isBounded_range_affineSimplex {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  IsBounded (Set.range σ) :=
  (Set.finite_range σ).isBounded

lemma isBounded_range_realize {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  IsBounded (Set.range σ.realize) :=
  (isCompact_range σ.realize.continuous).isBounded

lemma diam_coe_image_of_bounded {ι : Type*} [Fintype ι]
  {s : Set (stdSimplex ℝ ι)} (hs : IsBounded s)
  (hs' : IsBounded ((↑) '' s : Set (ι → ℝ))) :
  diam ((↑) '' s : Set (ι → ℝ)) = diam s := by
  refine le_antisymm ?_ ?_
· refine diam_le_of_forall_dist_le diam_nonneg ?_
  rintro _ ⟨x, hx, rfl⟩ _ ⟨y, hy, rfl⟩
  simpa [Subtype.dist_eq] using dist_le_diam_of_mem hs hx hy
· refine diam_le_of_forall_dist_le diam_nonneg ?_
  intro x hx y hy
  simpa [Subtype.dist_eq] using
    dist_le_diam_of_mem hs' ⟨x, hx, rfl⟩ ⟨y, hy, rfl⟩

```

```

lemma diam_coe_image_finite {ι : Type*} [Fintype ι]
  (s : Set (stdSimplex ℝ ι)) (hs : s.Finite) :
  diam ((↑) ' s : Set (ι → ℝ)) = diam s :=
  diam_coe_image_of_bounded hs.isBounded (hs.image _).isBounded

/-- Realization cannot increase diameter: it is the convex hull of the
listed vertices in the ambient coordinate space. -/
lemma diam_range_realize_le {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  diam (Set.range σ.realize) ≤ diam (Set.range σ) := by
  refine diam_le_of_forall_dist_le diam_nonneg ?_
  intro x hx y hy
  obtain ⟨a, rfl⟩ := hx
  obtain ⟨b, rfl⟩ := hy
  have ha := σ.realize_mem_convexHull a
  have hb := σ.realize_mem_convexHull b
  have hB : IsBounded ((↑) ' Set.range σ : Set (Fin (q + 1) → ℝ)) :=
    ((Set.finite_range σ).image _).isBounded
  have hle : dist (σ.realize a : Fin (q + 1) → ℝ)
    (σ.realize b : Fin (q + 1) → ℝ) ≤
    diam ((↑) ' Set.range σ : Set (Fin (q + 1) → ℝ)) :=
    (dist_le_diam_of_mem (isBounded_convexHull.mpr hB) ha hb).trans
    (by rw [convexHull_diam])
  simpa [Subtype.dist_eq, diam_coe_image_finite _ (Set.finite_range σ)] using hle

lemma diam_range_realize_eq {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  diam (Set.range σ.realize) = diam (Set.range σ) :=
  le_antisymm (diam_range_realize_le σ)
  (diam_mono σ.range_subset_range_realize (isBounded_range_realize σ))

lemma diam_range_affineSimplex_zero {q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) 0) :
  diam (Set.range σ) = 0 := by
  refine diam_subsingleton ?_
  intro x hx y hy
  obtain ⟨i, rfl⟩ := hx
  obtain ⟨j, rfl⟩ := hy
  rw [Fin.eq_zero i, Fin.eq_zero j]

/-- Distance from the barycenter to a vertex is at most the mesh ratio
times the vertex diameter. -/
lemma dist_stdBarycenter_vertex {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p)
  (i : Fin (p + 1)) :
  dist σ.stdBarycenter (σ i) ≤
    barycentricMeshRatio p * diam (Set.range σ) := by
  classical
  have hne : (p + 1 : ℝ) ≠ 0 := by positivity
  have hval : (σ.stdBarycenter : Fin (q + 1) → ℝ) =
    ∑ j : Fin (p + 1), (p + 1 : ℝ)-1 • (σ j : Fin (q + 1) → ℝ) := by
    rw [AffineSimplex.stdBarycenter, AffineSimplex.realize_val]
    refine Finset.sum_congr rfl fun j _ => ?_
    simp [stdSimplex.barycenter, Fintype.card_fin]
  have hsum : ∑ _j : Fin (p + 1), (p + 1 : ℝ)-1 = (1 : ℝ) := by
    simp [Finset.card_univ, Fintype.card_fin, nsmul_eq_mul,
      mul_inv_cancel₀ hne]
  have hi : (σ i : Fin (q + 1) → ℝ) =
    ∑ j : Fin (p + 1), (p + 1 : ℝ)-1 • (σ i : Fin (q + 1) → ℝ) := by
    rw [← Finset.sum_smul, hsum, one_smul]
  have hdiff :
    (σ.stdBarycenter : Fin (q + 1) → ℝ) - (σ i : Fin (q + 1) → ℝ) =
    ∑ j : Fin (p + 1),

```

```

      (p + 1 : ℝ)⁻¹ •
      ((σ j : Fin (q + 1) → ℝ) - (σ i : Fin (q + 1) → ℝ)) := by
    rw [hval]
    conv_lhs => rw [hi]
    rw [← Finset.sum_sub_distrib]
    exact Finset.sum_congr rfl fun j _ => (smul_sub _ _ _).symm
  have hsub :
    (σ.stdBarycenter : Fin (q + 1) → ℝ) - (σ i : Fin (q + 1) → ℝ) =
      ∑ j ∈ Finset.univ.erase i,
        (p + 1 : ℝ)⁻¹ •
          ((σ j : Fin (q + 1) → ℝ) - (σ i : Fin (q + 1) → ℝ)) := by
    rw [hdiff, ← Finset.sum_erase_add (f := fun j =>
      (p + 1 : ℝ)⁻¹ •
        ((σ j : Fin (q + 1) → ℝ) - (σ i : Fin (q + 1) → ℝ)))
      (h := Finset.mem_univ i)]
    simp [sub_self]
  rw [Subtype.dist_eq, dist_eq_norm]
  change ||(σ.stdBarycenter : Fin (q + 1) → ℝ) - (σ i : Fin (q + 1) → ℝ)|| ≤ _
  rw [hsub]
  refine (norm_sum_le _ _).trans ?_
  simp only [norm_smul, Real.norm_eq_abs,
    abs_of_nonneg (inv_nonneg.2 (by positivity : (0 : ℝ) ≤ p + 1))]
  have hterm (j : Fin (p + 1)) :
    (p + 1 : ℝ)⁻¹ *
      ||(σ j : Fin (q + 1) → ℝ) - (σ i : Fin (q + 1) → ℝ)|| ≤
      (p + 1 : ℝ)⁻¹ * diam (Set.range σ) := by
    refine mul_le_mul_of_nonneg_left ?_ (inv_nonneg.2 (by positivity))
    simp [dist_eq_norm, Subtype.dist_eq] using
      dist_le_diam_of_mem (isBounded_range_affineSimplex σ)
      ⟨j, rfl⟩ ⟨i, rfl⟩
  refine (Finset.sum_le_sum fun j _ => hterm j).trans ?_
  simp only [Finset.sum_const, nsmul_eq_mul,
    Finset.card_erase_of_mem (Finset.mem_univ i), Finset.card_univ,
    Fintype.card_fin]
  have hcard : ((p + 1 : ℕ) - 1 : ℕ) : ℝ = (p : ℝ) := by
    norm_cast
  rw [hcard, barycentricMeshRatio, div_eq_mul_inv, mul_assoc]

/-- Distance from the barycenter to any point of the convex hull of the
vertices is controlled by the same mesh ratio. -/
lemma dist_stdBarycenter_mem_convexHull {p q : ℕ}
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1)))) p
  {x : Fin (q + 1) → ℝ}
  (hx : x ∈ convexHull ℝ ((↑) '' Set.range σ)) :
  dist (σ.stdBarycenter : Fin (q + 1) → ℝ) x ≤
    barycentricMeshRatio p * diam (Set.range σ) := by
  obtain ⟨y, hy, hle⟩ :=
    convexHull_exists_dist_ge hx (σ.stdBarycenter : Fin (q + 1) → ℝ)
  obtain ⟨v, ⟨i, rfl⟩, rfl⟩ := hy
  rw [dist_comm]
  exact hle.trans (by
    simp [dist_comm, Subtype.dist_eq] using dist_stdBarycenter_vertex σ i)

/-- Vertices of every barycentric piece lie in the convex hull of the
parent vertices. -/
lemma vertices_subdivideStd_mem_convexHull {q : ℕ} (n : ℕ) :
  ∀ (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1)))) n
  {τ : AffineSimplex (stdSimplex ℝ (Fin (q + 1)))) n,
  τ ∈ (subdivideStd (q := q) n (simplex σ)).support →
  ∀ i : Fin (n + 1),
    (τ i : Fin (q + 1) → ℝ) ∈ convexHull ℝ ((↑) '' Set.range σ) := by
  induction n with
  | zero =>

```

```

intro  $\sigma$   $\tau$   $h\tau$  i
classical
simp [subdivideStd, subdivide_zero, simplex,
  Finsupp.mem_support_single] at  $h\tau$ 
rcases  $h\tau$  with  $\langle rfl, \_ \rangle$ 
exact subset_convexHull  $\mathbb{R} \_ \langle \sigma$  i,  $\langle i, rfl \rangle$ ,  $rfl \rangle$ 
| succ n ih =>
  intro  $\sigma$   $\tau$   $h\tau$  i
  obtain  $\langle j, \rho, h\rho, rfl \rangle :=$ 
    mem_support_subdivide_simplex AffineSimplex.stdBarycenter  $\sigma$   $h\tau$ 
  induction i using Fin.lastCases with
  | last =>
    simp [AffineSimplex.append] using  $\sigma$ .stdBarycenter_mem_convexHull
  | cast k =>
    exact convexHull_mono (image_mono ( $\sigma$ .range_face_subset j))
      (by simp [AffineSimplex.append] using ih ( $\sigma$ .face j)  $h\rho$  k)

/-- Vertex-diameter of a barycentric piece is at most the mesh ratio
times the parent vertex-diameter. -/
lemma diam_range_subdivideStd_le {q :  $\mathbb{N}$ } (n :  $\mathbb{N}$ ) :
   $\forall (\sigma : \text{AffineSimplex } (\text{stdSimplex } \mathbb{R} (\text{Fin } (q + 1))) n)$ 
  { $\tau : \text{AffineSimplex } (\text{stdSimplex } \mathbb{R} (\text{Fin } (q + 1))) n$ },
   $\tau \in (\text{subdivideStd } (q := q) n (\text{simplex } \sigma)).\text{support} \rightarrow$ 
   $\text{diam } (\text{Set.range } \tau) \leq$ 
    barycentricMeshRatio n *  $\text{diam } (\text{Set.range } \sigma) :=$  by
  induction n with
  | zero =>
    intro  $\sigma$   $\tau$   $h\tau$ 
    classical
    simp [subdivideStd, subdivide_zero, simplex,
      Finsupp.mem_support_single] at  $h\tau$ 
    rcases  $h\tau$  with  $\langle rfl, \_ \rangle$ 
    simp [diam_range_affineSimplex_zero]
  | succ n ih =>
    intro  $\sigma$   $\tau$   $h\tau$ 
    obtain  $\langle j, \rho, h\rho, rfl \rangle :=$ 
      mem_support_subdivide_simplex AffineSimplex.stdBarycenter  $\sigma$   $h\tau$ 
    set C := barycentricMeshRatio (n + 1) *  $\text{diam } (\text{Set.range } \sigma)$ 
    have hC :  $0 \leq C :=$ 
      mul_nonneg (barycentricMeshRatio_nonneg _) diam_nonneg
    have h $\rho$ diam :  $\text{diam } (\text{Set.range } \rho) \leq C :=$  by
      have h1 := ih ( $\sigma$ .face j) ( $\tau := \rho$ )  $h\rho$ 
      have h2 :  $\text{diam } (\text{Set.range } (\sigma.\text{face } j)) \leq \text{diam } (\text{Set.range } \sigma) :=$ 
        diam_mono ( $\sigma$ .range_face_subset j) (isBounded_range_affineSimplex  $\sigma$ )
      have h3 : barycentricMeshRatio n *  $\text{diam } (\text{Set.range } \sigma) \leq C :=$ 
        mul_le_mul_of_nonneg_right (barycentricMeshRatio_le_succ n)
          diam_nonneg
    exact (h1.trans (mul_le_mul_of_nonneg_left h2
      (barycentricMeshRatio_nonneg n))).trans h3
    have hdist {x : stdSimplex  $\mathbb{R} (\text{Fin } (q + 1))$ }
      (hx : x  $\in$  Set.range  $\rho$ ) :
      dist  $\sigma$ .stdBarycenter x  $\leq C :=$  by
      obtain  $\langle k, rfl \rangle :=$  hx
      have hx' : ( $\rho$  k : Fin (q + 1)  $\rightarrow \mathbb{R}$ )  $\in$ 
        convexHull  $\mathbb{R} ((\uparrow) '' \text{Set.range } \sigma) :=$ 
          convexHull_mono (image_mono ( $\sigma$ .range_face_subset j))
            (vertices_subdivideStd_mem_convexHull n ( $\sigma$ .face j)  $h\rho$  k)
      simp [C, Subtype.dist_eq] using
        dist_stdBarycenter_mem_convexHull  $\sigma$  hx'
    rw [AffineSimplex.range_append]
    refine diam_le_of_forall_dist_le hC ?_
    intro x hx y hy
    simp only [mem_insert_iff] at hx hy

```



```

    rcases hx with rfl | hx <=> rcases hy with rfl | hy
    · simp [dist_self] using hc
    · exact hdist hy
    · rw [dist_comm]
      exact hdist hx
    · exact (dist_le_diam_of_mem (Set.finite_range  $\rho$ ).isBounded hx hy).trans
      h $\rho$ diam

lemma diam_range_idSimplex_le (n :  $\mathbb{N}$ ) :
  diam (Set.range (idSimplex n))  $\leq$ 
    diam (univ : Set (stdSimplex  $\mathbb{R}$  (Fin (n + 1)))) :=
  diam_mono (subset_univ _) isBounded_of_compactSpace

/-- After `k` barycentric subdivisions of the identity `n`-simplex,
every remaining piece has vertex-diameter at most the contracted mesh. -/
lemma diam_range_subdivideStd_iterate_le (n k :  $\mathbb{N}$ )
  { $\tau$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (n + 1))) n}
  (h $\tau$  :  $\tau \in ((\text{subdivideStd } (q := n) n)^{[k]}$ 
    (simplex (idSimplex n))).support) :
  diam (Set.range  $\tau$ )  $\leq$ 
    barycentricMeshRatio n  $\wedge$  k * diam (Set.range (idSimplex n)) := by
  induction k generalizing  $\tau$  with
  | zero =>
    classical
    simp [Function.iterate_zero, simplex, Finsupp.mem_support_single] at h $\tau$ 
    rcases h $\tau$  with (rfl, _)
    simp
  | succ k ih =>
    rw [Function.iterate_succ_apply'] at h $\tau$ 
    obtain  $\langle \rho, h\rho, h\tau\rho \rangle :=$ 
      mem_support_subdivide AffineSimplex.stdBarycenter n h $\tau$ 
    have h1 := diam_range_subdivideStd_le (q := n) n  $\rho$  h $\tau\rho$ 
    have h2 := ih h $\rho$ 
    have h3 :
      barycentricMeshRatio n *
        (barycentricMeshRatio n  $\wedge$  k * diam (Set.range (idSimplex n))) =
        barycentricMeshRatio n  $\wedge$  (k + 1) *
        diam (Set.range (idSimplex n)) := by
      rw [ $\leftarrow$  mul_assoc, mul_comm (barycentricMeshRatio n),  $\leftarrow$  pow_succ]
    exact (h1.trans (mul_le_mul_of_nonneg_left h2
      (barycentricMeshRatio_nonneg n))).trans_eq h3

lemma diam_range_realize_subdivideStd_iterate_le (n k :  $\mathbb{N}$ )
  { $\tau$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (n + 1))) n}
  (h $\tau$  :  $\tau \in ((\text{subdivideStd } (q := n) n)^{[k]}$ 
    (simplex (idSimplex n))).support) :
  diam (Set.range  $\tau$ .realize)  $\leq$ 
    barycentricMeshRatio n  $\wedge$  k *
    diam (univ : Set (stdSimplex  $\mathbb{R}$  (Fin (n + 1)))) :=
  (diam_range_realize_le  $\tau$ ).trans <|
  (diam_range_subdivideStd_iterate_le n k h $\tau$ ).trans <|
  mul_le_mul_of_nonneg_left (diam_range_idSimplex_le n)
  (pow_nonneg (barycentricMeshRatio_nonneg n) k)

/-- Iterated barycentric pieces of  $\Delta^n$  become arbitrarily small. -/
lemma exists_subdivideStd_iterate_diam_lt (n :  $\mathbb{N}$ ) { $\varepsilon$  :  $\mathbb{R}$ } (h $\varepsilon$  : 0 <  $\varepsilon$ ) :
   $\exists$  k :  $\mathbb{N}$ ,  $\forall \tau \in ((\text{subdivideStd } (q := n) n)^{[k]}$ 
    (simplex (idSimplex n))).support,
    diam (Set.range  $\tau$ .realize) <  $\varepsilon$  := by
  obtain (k, hk) :=
    exists_barycentricMeshRatio_pow_mul_diam_stdSimplex_lt n h $\varepsilon$ 
  refine (k, fun  $\tau$  h $\tau$  => ?_)
  exact (diam_range_realize_subdivideStd_iterate_le n k h $\tau$ ).trans_lt hk

```

```

/-- Every piece of a sufficiently iterated subdivision of a singular
simplex lands in one member of a two-set open cover. -/
lemma exists_subdivideStd_iterate_image_subset_left_or_right
  {n : ℕ} {Y : Type*} [TopologicalSpace Y]
  (σ : C(stdSimplex ℝ (Fin (n + 1)), Y))
  {A B : Set Y} (hA : IsOpen A) (hB : IsOpen B)
  (hAB : A ∪ B = univ) :
  ∃ N : ℕ, ∀ τ ∈ ((subdivideStd (q := n) n)^[N]
    (simplex (idSimplex n))).support,
    σ '' Set.range τ.realize ⊆ A ∨
    σ '' Set.range τ.realize ⊆ B := by
obtain ⟨δ, hδ, hsmall⟩ :=
  exists_diam_lt_forall_image_subset_left_or_right σ hA hB hAB
obtain ⟨N, hN⟩ := exists_subdivideStd_iterate_diam_lt n hδ
refine ⟨N, fun τ hτ => ?_⟩
exact hsmall ⟨τ.realize (stdSimplex.vertex 0), ⟨stdSimplex.vertex 0, rfl⟩⟩
  (hN τ hτ)

end

end PlatonicSolids.SingularExcision

```

A.14 PlatonicSolids/SingularExcision/SingularChains.lean

Singular Sd as a chain map and the prism homotopy (472 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Homology.Homotopy
import Mathlib.AlgebraicTopology.SingularHomology.Basic
import PlatonicSolids.SingularExcision.AffineRealization

/-!
# Interpreting finite affine chains as singular chains

A finite integral affine chain in the standard `q`-simplex is composed
with a singular `q`-simplex, then interpreted in Mathlib's coproduct of
copies of the coefficient object. Face-compatibility of realization
makes the barycentric generators assemble to a chain map, with the
prism as a chain homotopy to the identity.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex
open scoped Simplicial

namespace PlatonicSolids.SingularExcision

noncomputable section

universe v u

variable {C : Type u} [Category.{v} C] [HasCoproducts.{0} C] [Preadditive C]

/-- The singular-chain generator obtained by affine reparametrization. -/
noncomputable def realizeAffineSimplex
  (R : C) {X : TopCat.{0}} {p q : ℕ}
  (s : (TopCat.toSSet.obj X) _[q])
  (σ : AffineSimplex (stdSimplex ℝ (Fin (q + 1))) p) :
  R → ((singularChainComplexFunctor C).obj R).obj X).X p :=
  (TopCat.toSSet.obj X).ιChainComplex

```

```

(reparametrizeSingularSimplex s  $\sigma$ )

/-- Interpret a finite integral affine chain as a morphism into singular chains. -/
noncomputable def realizeAffineChain
  (R : C) {X : TopCat.{0}} {p q : ℕ}
  (s : (TopCat.toSSet.obj X)  $\_$ [q]) :
  AffineChain (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) p  $\rightarrow$ +
  (R  $\rightarrow$  (((singularChainComplexFunctor C).obj R).obj X).X p) :=
  Finsupp.liftAddHom fun  $\sigma$  =>
    (smulAddHom  $\mathbb{Z}$ 
      (R  $\rightarrow$  (((singularChainComplexFunctor C).obj R).obj X).X p)).flip
      (realizeAffineSimplex R s  $\sigma$ )

@[simp]
theorem realizeAffineChain_zero
  (R : C) {X : TopCat.{0}} {p q : ℕ}
  (s : (TopCat.toSSet.obj X)  $\_$ [q]) :
  realizeAffineChain R s
  (0 : AffineChain (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) p) = 0 := by
  simp

@[simp]
theorem realizeAffineChain_single
  (R : C) {X : TopCat.{0}} {p q : ℕ}
  (s : (TopCat.toSSet.obj X)  $\_$ [q])
  ( $\sigma$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) p) (z :  $\mathbb{Z}$ ) :
  realizeAffineChain R s (Finsupp.single  $\sigma$  z) =
  z • realizeAffineSimplex R s  $\sigma$  := by
  simp [realizeAffineChain]

@[simp]
theorem realizeAffineChain_simplex
  (R : C) {X : TopCat.{0}} {p q : ℕ}
  (s : (TopCat.toSSet.obj X)  $\_$ [q])
  ( $\sigma$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) p) :
  realizeAffineChain R s (simplex  $\sigma$ ) = realizeAffineSimplex R s  $\sigma$  := by
  simp [simplex]

theorem realizeAffineSimplex_map
  (R : C) {X : TopCat.{0}} {p q k : ℕ}
  (s : (TopCat.toSSet.obj X)  $\_$ [q])
  ( $\sigma$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) p)
  ( $\rho$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (p + 1))) k) :
  realizeAffineSimplex R s (AffineSimplex.map  $\sigma$ .realize  $\rho$ ) =
  realizeAffineSimplex R (reparametrizeSingularSimplex s  $\sigma$ )  $\rho$  := by
  simp [realizeAffineSimplex, reparametrizeSingularSimplex_map]

theorem realizeAffineChain_map
  (R : C) {X : TopCat.{0}} {p q k : ℕ}
  (s : (TopCat.toSSet.obj X)  $\_$ [q])
  ( $\sigma$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) p)
  (c : AffineChain (stdSimplex  $\mathbb{R}$  (Fin (p + 1))) k) :
  realizeAffineChain R s (AffineChain.map  $\sigma$ .realize c) =
  realizeAffineChain R (reparametrizeSingularSimplex s  $\sigma$ ) c := by
  induction c using Finsupp.induction with
  | zero => simp
  | single_add  $\tau$  z c  $\_$  ih =>
    simp [map_add, ih, AffineChain.map_single, realizeAffineSimplex_map]

theorem realizeAffineSimplex_d
  (R : C) {X : TopCat.{0}} {p q : ℕ}
  (s : (TopCat.toSSet.obj X)  $\_$ [q])
  ( $\sigma$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) (p + 1)) :

```

```

    realizeAffineSimplex R s  $\sigma$  >>
      (((singularChainComplexFunctor C).obj R).obj X).d (p + 1) p =
         $\sum i : \text{Fin } (p + 2),$ 
         $(-1 : \mathbb{Z}) \sim (i : \mathbb{N}) \bullet \text{realizeAffineSimplex } R \text{ s } (\sigma.\text{face } i) := \text{by}$ 
    simp only [realizeAffineSimplex]
    change _ >> ((TopCat.toSSet.obj X).chainComplex R).d (p + 1) p = _
    rw [SSet.ChainComplex_d]
    refine Finset.sum_congr rfl fun i _ => ?_
    congr 1
    rw [reparametrizeSingularSimplex_face]

theorem realizeAffineChain_simplexBoundary
  (R : C) {X : TopCat.{0}} {p q :  $\mathbb{N}$ }
  (s : (TopCat.toSSet.obj X)  $\llbracket q \rrbracket$ )
  ( $\sigma$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) (p + 1)) :
  realizeAffineChain R s (simplexBoundary  $\sigma$ ) =
     $\sum i : \text{Fin } (p + 2),$ 
     $(-1 : \mathbb{Z}) \sim (i : \mathbb{N}) \bullet \text{realizeAffineSimplex } R \text{ s } (\sigma.\text{face } i) := \text{by}$ 
  simp [simplexBoundary, map_sum, map_zsmul]

theorem realizeAffineChain_boundary
  (R : C) {X : TopCat.{0}} {p q :  $\mathbb{N}$ }
  (s : (TopCat.toSSet.obj X)  $\llbracket q \rrbracket$ )
  (c : AffineChain (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) (p + 1)) :
  realizeAffineChain R s (boundary c) =
    realizeAffineChain R s c >>
      (((singularChainComplexFunctor C).obj R).obj X).d (p + 1) p := by
  induction c using Finsupp.induction with
  | zero => simp
  | single_add  $\sigma$  z c _ _ ih =>
    simp [map_add, ih, Preadditive.add_comp]
    rw [show Finsupp.single  $\sigma$  z = z  $\bullet$  simplex  $\sigma$  by simp [simplex]]
    simp [map_zsmul, boundary_simplex,
      realizeAffineChain_simplexBoundary, realizeAffineSimplex_d]

theorem realizeAffineChain_subdivideStd_simplex
  (R : C) {X : TopCat.{0}} {p q :  $\mathbb{N}$ }
  (s : (TopCat.toSSet.obj X)  $\llbracket q \rrbracket$ )
  ( $\sigma$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) p) :
  realizeAffineChain R s (subdivideStd (q := q) p (simplex  $\sigma$ )) =
    realizeAffineChain R (reparametrizeSingularSimplex s  $\sigma$ )
    (standardSubdivision p) := by
  rw [← AffineChain.map_standardSubdivision, realizeAffineChain_map]

theorem realizeAffineChain_prismStd_simplex
  (R : C) {X : TopCat.{0}} {p q :  $\mathbb{N}$ }
  (s : (TopCat.toSSet.obj X)  $\llbracket q \rrbracket$ )
  ( $\sigma$  : AffineSimplex (stdSimplex  $\mathbb{R}$  (Fin (q + 1))) p) :
  realizeAffineChain R s (prismStd (q := q) p (simplex  $\sigma$ )) =
    realizeAffineChain R (reparametrizeSingularSimplex s  $\sigma$ )
    (standardPrism p) := by
  rw [← AffineChain.map_standardPrism, realizeAffineChain_map]

@[simp]
theorem realizeAffineSimplex_id
  (R : C) {X : TopCat.{0}} {q :  $\mathbb{N}$ }
  (s : (TopCat.toSSet.obj X)  $\llbracket q \rrbracket$ ) :
  realizeAffineSimplex R s (idSimplex q) =
    (TopCat.toSSet.obj X).ChainComplex s := by
  simp [realizeAffineSimplex]

/-- Subdivision of one singular simplex, on its coproduct generator. -/
noncomputable def singularSubdivideGenerator

```

```

(R : C) {X : TopCat.{0}} {n : ℕ}
(s : (TopCat.toSSet.obj X) _[n]) :
R → ((singularChainComplexFunctor C).obj R).obj X.X n :=
realizeAffineChain R s (standardSubdivision n)

/-- Prism of one singular simplex, on its coproduct generator. -/
noncomputable def singularPrismGenerator
(R : C) {X : TopCat.{0}} {n : ℕ}
(s : (TopCat.toSSet.obj X) _[n]) :
R → ((singularChainComplexFunctor C).obj R).obj X.X (n + 1) :=
realizeAffineChain R s (standardPrism n)

theorem singularPrismGenerator_homotopy
(R : C) {X : TopCat.{0}} {n : ℕ}
(s : (TopCat.toSSet.obj X) _[n + 1]) :
singularPrismGenerator R s >>
(((singularChainComplexFunctor C).obj R).obj X).d (n + 2) (n + 1) +
realizeAffineChain R s
(prismStd (q := n + 1) n (simplexBoundary (idSimplex (n + 1)))) =
(TopCat.toSSet.obj X).ιChainComplex s -
singularSubdivideGenerator R s := by
have hprism :=
boundary_prismStd (q := n + 1) (simplex (idSimplex (n + 1)))
simp only [singularPrismGenerator, singularSubdivideGenerator,
standardPrism, standardSubdivision]
rw [← realizeAffineChain_boundary]
have hreal := congrArg (realizeAffineChain R s) hprism
simp [map_add, map_sub, boundary_simplex, realizeAffineSimplex_id] at hreal
exact hreal

/-- Subdivision of one singular one-simplex, on its coproduct generator. -/
noncomputable def singularIntervalSubdivision
(R : C) {X : TopCat.{0}}
(s : (TopCat.toSSet.obj X) _[1]) :
R → ((singularChainComplexFunctor C).obj R).obj X.X 1 :=
realizeAffineChain R s standardIntervalSubdivision

/-- The one-dimensional prism, interpreted as a singular two-chain. -/
noncomputable def singularIntervalPrism
(R : C) {X : TopCat.{0}}
(s : (TopCat.toSSet.obj X) _[1]) :
R → ((singularChainComplexFunctor C).obj R).obj X.X 2 :=
realizeAffineChain R s standardIntervalPrism

/-- Degreewise barycentric subdivision of singular chains. -/
noncomputable def singularSubdivisionHom
(R : C) (X : TopCat.{0}) (n : ℕ) :
(((singularChainComplexFunctor C).obj R).obj X).X n →
(((singularChainComplexFunctor C).obj R).obj X).X n :=
((TopCat.toSSet.obj X).isColimitChainComplexXCofan R n).desc
(Cofan.mk _ fun s => singularSubdivideGenerator R s)

@[reassoc (attr := simp)]
theorem ι_singularSubdivisionHom
(R : C) (X : TopCat.{0}) {n : ℕ}
(s : (TopCat.toSSet.obj X) _[n]) :
(TopCat.toSSet.obj X).ιChainComplex s >> singularSubdivisionHom R X n =
singularSubdivideGenerator R s :=
((TopCat.toSSet.obj X).isColimitChainComplexXCofan R n).fac _ ⟨s⟩

/-- Degreewise prism operator on singular chains. -/
noncomputable def singularPrismHom
(R : C) (X : TopCat.{0}) (n : ℕ) :

```

```

    (((singularChainComplexFunctor C).obj R).obj X).X n →
    (((singularChainComplexFunctor C).obj R).obj X).X (n + 1) :=
    ((TopCat.toSSet.obj X).isColimitChainComplexXCofan R n).desc
    (Cofan.mk _ fun s => singularPrismGenerator R s)

@[reassoc (attr := simp)]
theorem  $\iota$ _singularPrismHom
  (R : C) (X : TopCat.{0}) {n : ℕ}
  (s : (TopCat.toSSet.obj X) _[[n]]) :
  (TopCat.toSSet.obj X). $\iota$ ChainComplex s  $\gg$  singularPrismHom R X n =
  singularPrismGenerator R s :=
  ((TopCat.toSSet.obj X).isColimitChainComplexXCofan R n).fac _ ⟨s⟩

theorem singularSubdivideGenerator_d
  (R : C) {X : TopCat.{0}} {n : ℕ}
  (s : (TopCat.toSSet.obj X) _[[n + 1]]) :
  singularSubdivideGenerator R s  $\gg$ 
  (((singularChainComplexFunctor C).obj R).obj X).d (n + 1) n =
   $\sum i : \text{Fin } (n + 2),$ 
   $(-1 : \mathbb{Z}) \sim (i : \mathbb{N}) \bullet$ 
  singularSubdivideGenerator R ((TopCat.toSSet.obj X). $\delta$  i s) := by
  have hbound :
    boundary (standardSubdivision (n + 1)) =
    subdivideStd (q := n + 1) n
    (simplexBoundary (idSimplex (n + 1))) := by
  simp [standardSubdivision, boundary_simplex] using
    boundary_subdivideStd (q := n + 1) (simplex (idSimplex (n + 1)))
  simp only [singularSubdivideGenerator]
  rw [← realizeAffineChain_boundary, hbound]
  simp only [simplexBoundary, map_sum, map_zsmul]
  refine Finset.sum_congr rfl fun i _ => ?_
  rw [realizeAffineChain_subdivideStd_simplex, reparametrizeSingularSimplex_id_face]

/-- Barycentric subdivision of singular chains, as a chain map. -/
noncomputable def singularSubdivision
  (R : C) (X : TopCat.{0}) :
  (((singularChainComplexFunctor C).obj R).obj X →
  ((singularChainComplexFunctor C).obj R).obj X where
  f := singularSubdivisionHom R X
  comm' i j hij := by
  rw [ComplexShape.down_Rel] at hij
  subst hij
  apply (TopCat.toSSet.obj X).chainComplex_hom_ext
  intro s
  calc
    (TopCat.toSSet.obj X). $\iota$ ChainComplex s  $\gg$ 
    singularSubdivisionHom R X (j + 1)  $\gg$ 
    (((singularChainComplexFunctor C).obj R).obj X).d (j + 1) j =
    singularSubdivideGenerator R s  $\gg$ 
    (((singularChainComplexFunctor C).obj R).obj X).d (j + 1) j := by
  rw [← Category.assoc,  $\iota$ _singularSubdivisionHom]
  _ =  $\sum i : \text{Fin } (j + 2),$ 
   $(-1 : \mathbb{Z}) \sim (i : \mathbb{N}) \bullet$ 
  singularSubdivideGenerator R ((TopCat.toSSet.obj X). $\delta$  i s) :=
  singularSubdivideGenerator_d R s
  _ = (TopCat.toSSet.obj X). $\iota$ ChainComplex s  $\gg$ 
  (((singularChainComplexFunctor C).obj R).obj X).d (j + 1) j  $\gg$ 
  singularSubdivisionHom R X j := by
  have hd :
    (TopCat.toSSet.obj X). $\iota$ ChainComplex s  $\gg$ 
    (((singularChainComplexFunctor C).obj R).obj X).d (j + 1) j =
     $\sum i : \text{Fin } (j + 2),$ 
     $(-1 : \mathbb{Z}) \sim (i : \mathbb{N}) \bullet$ 

```

```

      (TopCat.toSSet.obj X).ιChainComplex
      ((TopCat.toSSet.obj X).δ i s) := by
    change _ >> ((TopCat.toSSet.obj X).chainComplex R).d (j + 1) j = _
    rw [SSet.ιChainComplex_d]
  rw [← Category.assoc, hd]
  refine Eq.trans ?_
    (Preadditive.sum_comp (s := Finset.univ)
      (fun i : Fin (j + 2) =>
        (-1 : ℤ) ^ (i : ℕ) •
          (TopCat.toSSet.obj X).ιChainComplex
            ((TopCat.toSSet.obj X).δ i s))
        (singularSubdivisionHom R X j)).symm
    refine Finset.sum_congr rfl fun i _ => ?_
  rw [← ι_singularSubdivisionHom, ← Preadditive.zsmul_comp]
  rfl

lemma singularSubdivisionHom_zero
  (R : C) (X : TopCat.{0}) :
  singularSubdivisionHom R X 0 = 0 := by
  apply (TopCat.toSSet.obj X).chainComplex_hom_ext
  intro s
  rw [ι_singularSubdivisionHom]
  simp [singularSubdivideGenerator, standardSubdivision, subdivideStd,
    realizeAffineSimplex_id]
  exact (Category.comp_id _).symm

lemma singularPrismHom_zero
  (R : C) (X : TopCat.{0}) :
  singularPrismHom R X 0 = 0 := by
  apply (TopCat.toSSet.obj X).chainComplex_hom_ext
  intro s
  rw [ι_singularPrismHom]
  simp [singularPrismGenerator, standardPrism, prismStd]
  exact (Limits.comp_zero (f := (TopCat.toSSet.obj X).ιChainComplex s)).symm

theorem singularPrismGenerator_d
  (R : C) {X : TopCat.{0}} {n : ℕ}
  (s : (TopCat.toSSet.obj X) _[n + 1]) :
  realizeAffineChain R s
    (prismStd (q := n + 1) n (simplexBoundary (idSimplex (n + 1)))) =
  (TopCat.toSSet.obj X).ιChainComplex s >>
    (((singularChainComplexFunctor C).obj R).obj X).d (n + 1) n >>
      singularPrismHom R X n := by
  have hd :
    (TopCat.toSSet.obj X).ιChainComplex s >>
      (((singularChainComplexFunctor C).obj R).obj X).d (n + 1) n =
      ∑ i : Fin (n + 2),
        (-1 : ℤ) ^ (i : ℕ) •
          (TopCat.toSSet.obj X).ιChainComplex
            ((TopCat.toSSet.obj X).δ i s) := by
    change _ >> ((TopCat.toSSet.obj X).chainComplex R).d (n + 1) n = _
    rw [SSet.ιChainComplex_d]
  simp only [simplexBoundary, map_sum, map_zsmul]
  rw [← Category.assoc, hd]
  refine Eq.trans ?_
    (Preadditive.sum_comp (s := Finset.univ)
      (fun i : Fin (n + 2) =>
        (-1 : ℤ) ^ (i : ℕ) •
          (TopCat.toSSet.obj X).ιChainComplex
            ((TopCat.toSSet.obj X).δ i s))
        (singularPrismHom R X n)).symm
  refine Finset.sum_congr rfl fun i _ => ?_
  have hpiece :

```

```

    realizeAffineChain R s
      (prismStd (q := n + 1) n (simplex ((idSimplex (n + 1)).face i))) =
      (TopCat.toSSet.obj X).ιChainComplex
      ((TopCat.toSSet.obj X).δ i s) >>
      singularPrismHom R X n := by
    rw [realizeAffineChain_prismStd_simplex, reparametrizeSingularSimplex_id_face]
    exact (ι_singularPrismHom R X _).symm
  rw [hpiece]
  exact (Preadditive.zsmul_comp
    ((TopCat.toSSet.obj X).ιChainComplex
      ((TopCat.toSSet.obj X).δ i s))
    (singularPrismHom R X n)
    ((-1 : ℤ) ^ (i : ℕ))).symm

/-- The prism operator is a chain homotopy from the identity to
barycentric subdivision. -/
noncomputable def singularPrismHomotopy
  (R : C) (X : TopCat.{0}) :
  Homotopy
    (⊢ (((singularChainComplexFunctor C).obj R).obj X))
    (singularSubdivision R X) where
  hom i j :=
    if h : i + 1 = j then
      singularPrismHom R X i >> eqToHom (by rw [h])
    else
      0
  zero i j hij := by
    refine dif_neg ?_
    intro h
    exact hij (by rw [ComplexShape.down_Rel, h])
  comm i := by
    rw [Homotopy.prevD_chainComplex]
    cases i with
    | zero =>
      rw [Homotopy.dNext_zero_chainComplex, zero_add]
      simp [dif_pos, singularPrismHom_zero]
      exact (singularSubdivisionHom_zero R X).symm
    | succ n =>
      rw [Homotopy.dNext_succ_chainComplex]
      apply (TopCat.toSSet.obj X).chainComplex_hom_ext
      intro s
      simp only [dif_pos, eqToHom_refl]
      have h := singularPrismGenerator_homotopy R s
      have hd := singularPrismGenerator_d R s
      change
        (TopCat.toSSet.obj X).ιChainComplex s >>
        ⊢ (((TopCat.toSSet.obj X).chainComplex R).X (n + 1)) =
        -
      simp [Category.comp_id, eqToHom_refl]
      have hadd :=
        (Preadditive.comp_add _ _ _
          ((TopCat.toSSet.obj X).ιChainComplex s)
          (((singularChainComplexFunctor C).obj R).obj X).d
            (n + 1) n >>
            singularPrismHom R X n +
            singularPrismHom R X (n + 1) >>
          (((singularChainComplexFunctor C).obj R).obj X).d
            (n + 2) (n + 1))
          ((singularSubdivision R X).f (n + 1))).trans
      (congrArg
        (fun t => t +
          (TopCat.toSSet.obj X).ιChainComplex s >>
          (singularSubdivision R X).f (n + 1))

```



```

(Preadditive.comp_add _ _ _
  ((TopCat.toSSet.obj X).ιChainComplex s)
  (((singularChainComplexFunctor C).obj R).obj X).d
    (n + 1) n >>
    singularPrismHom R X n)
  (singularPrismHom R X (n + 1) >>
    ((singularChainComplexFunctor C).obj R).obj X).d
    (n + 2) (n + 1))))
erw [hadd]
rw [← Category.assoc
  ((TopCat.toSSet.obj X).ιChainComplex s)
  (singularPrismHom R X (n + 1)),
  ι_singularPrismHom]
change _ =
  (TopCat.toSSet.obj X).ιChainComplex s >>
    (((singularChainComplexFunctor C).obj R).obj X).d
      (n + 1) n >>
      singularPrismHom R X n +
      singularPrismGenerator R s >>
      (((singularChainComplexFunctor C).obj R).obj X).d
        (n + 2) (n + 1) +
      (TopCat.toSSet.obj X).ιChainComplex s >>
        singularSubdivisionHom R X (n + 1)
  rw [ι_singularSubdivisionHom, ← hd,
    add_comm (realizeAffineChain R s _), h]
  exact (sub_add_cancel
    ((TopCat.toSSet.obj X).ιChainComplex s)
    (singularSubdivideGenerator R s)).syymm

/-- Iterated barycentric subdivision, as a chain map. -/
noncomputable def singularSubdivisionIterate
  (R : C) (X : TopCat.{0}) (N : ℕ) :
  ((singularChainComplexFunctor C).obj R).obj X →
  ((singularChainComplexFunctor C).obj R).obj X :=
  ((fun f => f >> singularSubdivision R X)^[N]) (λ _ _)

lemma singularSubdivisionIterate_zero
  (R : C) (X : TopCat.{0}) :
  singularSubdivisionIterate R X 0 = λ _ _ :=
  rfl

lemma singularSubdivisionIterate_succ
  (R : C) (X : TopCat.{0}) (n : ℕ) :
  singularSubdivisionIterate R X (n + 1) =
  singularSubdivisionIterate R X n >> singularSubdivision R X :=
  Function.iterate_succ_apply' _ n _

lemma singularSubdivisionIterate_f_succ
  (R : C) (X : TopCat.{0}) (N n : ℕ) :
  (singularSubdivisionIterate R X (N + 1)).f n =
  (singularSubdivisionIterate R X N).f n >>
  singularSubdivisionHom R X n := by
  rw [singularSubdivisionIterate_succ]
  rfl

end

end PlatonicSolids.SingularExcision

```

A.15 PlatonicSolids/SingularExcision/ModuleSupport.lean

Finite coproduct support in ModuleCat (46 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Category.ModuleCat.Products
import Mathlib.Algebra.Category.ModuleCat.EpiMono
import Mathlib.Algebra.DirectSum.Module

/-!
Finite-support access for categorical coproducts in `ModuleCat`.

This is why the elementwise small-simplices proof is specialized to module
coefficients: every element of the categorical coproduct is, through
`ModuleCat.coprodIsoDirectSum`, a genuinely finite direct sum.
-/

open CategoryTheory.Limits

universe u v w

namespace ModuleCat

variable {R : Type u} [Ring R] {ι : Type v}
variable (Z : ι → ModuleCat.{max v w} R) [DecidableEq ι] [HasCoproduct Z]

/-- The finite set of summands on which an element of a categorical
coproduct of modules is nonzero. -/
noncomputable def coprodSupport
  (x : Limits.sigmaObj (C := ModuleCat.{max v w} R) Z) : Finset ι :=
  by
    classical
    exact (coprodIsoDirectSum Z).hom x |>.support

/-- Every element of a categorical coproduct of modules is the finite sum
of its components included into the coproduct. -/
lemma eq_sum_ι_apply_coprodSupport
  (x : Limits.sigmaObj (C := ModuleCat.{max v w} R) Z) :
  x =  $\sum i \in \text{coprodSupport } Z, x,$ 
  Sigma.ι Z i ((coprodIsoDirectSum Z).hom x i) := by
  classical
  apply_fun (coprodIsoDirectSum Z).hom using
    (ModuleCat.mono_iff_injective (coprodIsoDirectSum Z).hom).mp inferInstance
  simp only [map_sum, ι_coprodIsoDirectSum_hom_apply]
  exact (DirectSum.sum_support_of ((coprodIsoDirectSum Z).hom x)).symm

end ModuleCat

```

A.16 PlatonicSolids/SingularExcision/Subcomplexes.lean

Lands-in and small singular subcomplexes (174 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.AlgebraicTopology.SimplicialSet.SubcomplexColimits
import Mathlib.AlgebraicTopology.SingularHomology.Basic
import PlatonicSolids.RelativeHomology

/-!
# Singular subcomplexes subordinate to a pair of subsets

The singular simplices which land in a subset form the range subcomplex

```

```

of the singular set of the corresponding subtype. The simplices small
with respect to a two-set cover form the supremum of the two ranges.
-/

open AlgebraicTopology CategoryTheory Limits Set
open scoped Topology

namespace SingularExcision

variable {Y : Type} [TopologicalSpace Y]

/-- The map of singular sets induced by the inclusion of a subtype. -/
noncomputable def subtypeSingularSetMap (A : Set Y) :
  TopCat.toSSet.obj (.of ↑A) → TopCat.toSSet.obj (.of Y) :=
  TopCat.toSSet.map (subtypeIncl Y A)

/-- The singular subcomplex consisting of simplices which factor through `A`. -/
noncomputable abbrev landsIn (A : Set Y) :
  (TopCat.toSSet.obj (.of Y)).Subcomplex :=
  SSet.Subcomplex.range (subtypeSingularSetMap A)

/-- A simplex is small for the pair `(A, B)` if it lands in one member. -/
noncomputable abbrev small (A B : Set Y) :
  (TopCat.toSSet.obj (.of Y)).Subcomplex :=
  landsIn A ⊔ landsIn B

/-- The singular set of a subtype maps onto the corresponding range subcomplex. -/
noncomputable def subtypeToLandsIn (A : Set Y) :
  TopCat.toSSet.obj (.of ↑A) → (landsIn A : SSet) :=
  SSet.Subcomplex.toRange (subtypeSingularSetMap A)

instance subtypeSingularSetMap_mono (A : Set Y) :
  Mono (subtypeSingularSetMap A) := by
  exact TopCat.toSSet.map_mono (subtypeIncl Y A)

instance subtypeToLandsIn_isIso (A : Set Y) :
  IsIso (subtypeToLandsIn A) := by
  change IsIso (SSet.Subcomplex.toRange (subtypeSingularSetMap A))
  infer_instance

/-- The singular set of `A` is canonically isomorphic to the subcomplex
of ambient singular simplices which land in `A`. -/
noncomputable def subtypeSingularSetIso (A : Set Y) :
  TopCat.toSSet.obj (.of ↑A) ≅ (landsIn A : SSet) :=
  asIso (subtypeToLandsIn A)

@[simp, reassoc]
lemma subtypeToLandsIn_ι (A : Set Y) :
  subtypeToLandsIn A ≫ (landsIn A).ι = subtypeSingularSetMap A :=
  SSet.Subcomplex.toRange_ι _

/-- Under the concrete description of singular simplices, the singular-set
map of a subtype is postcomposition with `Subtype.val`. -/
lemma subtypeSingularSetMap_toSSetObjEquiv (A : Set Y)
  (n : SimplexCategoryop)
  (σ : (TopCat.toSSet.obj (.of ↑A)).obj n) :
  (TopCat.of Y).toSSetObjEquiv n
  ((subtypeSingularSetMap A).app n σ) =
  (subtypeIncl Y A).hom.comp
  ((TopCat.of ↑A).toSSetObjEquiv n σ) :=
  rfl

/-- The two evident maps out of an intersection subtype. -/

```

```

def interToLeft (A B : Set Y) : TopCat.of  $\uparrow(A \cap B) \longrightarrow$  TopCat.of  $\uparrow A :=$ 
  TopCat.ofHom (ContinuousMap.inclusion fun _ h => h.1)

def interToRight (A B : Set Y) : TopCat.of  $\uparrow(A \cap B) \longrightarrow$  TopCat.of  $\uparrow B :=$ 
  TopCat.ofHom (ContinuousMap.inclusion fun _ h => h.2)

lemma interToLeft_comp_subtypeIncl (A B : Set Y) :
  interToLeft A B  $\gg$  subtypeIncl Y A = subtypeIncl Y (A  $\cap$  B) := by
  rfl

lemma interToRight_comp_subtypeIncl (A B : Set Y) :
  interToRight A B  $\gg$  subtypeIncl Y B = subtypeIncl Y (A  $\cap$  B) := by
  rfl

/-- Factoring through the intersection is equivalent to factoring through
both subsets. -/
lemma landsIn_inter (A B : Set Y) :
  landsIn (A  $\cap$  B) = landsIn A  $\sqcap$  landsIn B := by
  ext n  $\sigma$ 
  constructor
  · rintro  $\langle \tau, \text{rfl} \rangle$ 
    constructor
    · refine  $\langle \langle \text{TopCat.toSSet.map (interToLeft A B) \rangle \rangle \text{.app } n \tau, ?_ \rangle$ 
      simp [subtypeSingularSetMap,  $\leftarrow$  Functor.map_comp] using
      congrArg (fun q => q.app n  $\tau$ )
      (congrArg TopCat.toSSet.map (interToLeft_comp_subtypeIncl A B))
    · refine  $\langle \langle \text{TopCat.toSSet.map (interToRight A B) \rangle \rangle \text{.app } n \tau, ?_ \rangle$ 
      simp [subtypeSingularSetMap,  $\leftarrow$  Functor.map_comp] using
      congrArg (fun q => q.app n  $\tau$ )
      (congrArg TopCat.toSSet.map (interToRight_comp_subtypeIncl A B))
  · rintro  $\langle \langle \tau A, hA \rangle, \langle \tau B, hB \rangle \rangle$ 
    let fA := (TopCat.of  $\uparrow A$ ).toSSetObjEquiv n  $\tau A$ 
    let fB := (TopCat.of  $\uparrow B$ ).toSSetObjEquiv n  $\tau B$ 
    have hval :  $\forall x, (fA \ x : Y) = fB \ x :=$  by
      intro x
      have h := congrArg
        (fun q => (TopCat.of Y).toSSetObjEquiv n q x)
        (hA.trans hB.symm)
      simp [fA, fB, subtypeSingularSetMap_toSSetObjEquiv] using h
    let fI : C(TopCat.of (stdSimplex  $\mathbb{R}$  (Fin (n.unop.len + 1)))),
      TopCat.of  $\uparrow(A \cap B)$ ) :=
       $\langle$ fun x =>  $\langle fA \ x, (fA \ x) \text{.property},$  by
        rw [hval x]
        exact (fB x).property $\rangle,$ 
      (continuous_subtype_val.comp fA.continuous).subtype_mk  $_ \rangle$ 
    refine  $\langle ((\text{TopCat.of } \uparrow(A \cap B)).\text{toSSetObjEquiv } n).\text{symm } fI, ?_ \rangle$ 
    apply (TopCat.of Y).toSSetObjEquiv n  $|> \text{.injective}$ 
  ext x
  simp [fI, fA, subtypeSingularSetMap_toSSetObjEquiv] using
  congrArg (fun q => (TopCat.of Y).toSSetObjEquiv n q x) hA

/-- The corresponding simplicial sets are canonically isomorphic. -/
noncomputable def landsInInterIso (A B : Set Y) :
  (landsIn (A  $\cap$  B) : SSet)  $\cong$ 
  ((landsIn A  $\sqcap$  landsIn B :
    (TopCat.toSSet.obj (.of Y)).Subcomplex) : SSet) :=
  SSet.Subcomplex.eqToIso (landsIn_inter A B)

@[simp, reassoc]
lemma landsInInterIso_hom_ $\iota$  (A B : Set Y) :
  (landsInInterIso A B).hom  $\gg$  (landsIn A  $\sqcap$  landsIn B). $\iota$  =
  (landsIn (A  $\cap$  B)). $\iota$  :=
  SSet.Subcomplex.homOfLE_ $\iota$  (landsIn_inter A B).le

```

```

/-- The two lands-in subcomplexes and their infimum/supremum form a
pushout square of simplicial sets. -/
noncomputable def smallIsPushout (A B : Set Y) :
  IsPushout
    (SSet.Subcomplex.homOfLE (inf_le_left :
      landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn A))
    (SSet.Subcomplex.homOfLE (inf_le_right :
      landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn B))
    (SSet.Subcomplex.homOfLE (le_sup_left :
      landsIn A  $\leq$  small A B))
    (SSet.Subcomplex.homOfLE (le_sup_right :
      landsIn B  $\leq$  small A B)) :=
(show SSet.Subcomplex.BicartSq
  (landsIn A  $\sqcap$  landsIn B) (landsIn A) (landsIn B) (small A B) from
  { sup_eq := rfl
    inf_eq := rfl }).isPushout

/-- Degreewise membership in `landsIn A` is exactly factorization through
the singular set of the subtype. -/
lemma mem_landsIn_iff {A : Set Y} {n : SimplexCategoryop}
  ( $\sigma$  : (TopCat.toSSet.obj (.of Y)).obj n) :
   $\sigma \in (\text{landsIn } A).obj\ n \leftrightarrow$ 
   $\exists \tau : (\text{TopCat.toSSet.obj (.of } \uparrow A)).obj\ n,$ 
  ( $\text{subtypeSingularSetMap } A$ ).app n  $\tau = \sigma :=$ 
  Iff.rfl

/-- Degreewise membership in the small subcomplex has the expected
disjunctive description. -/
lemma mem_small_iff {A B : Set Y} {n : SimplexCategoryop}
  ( $\sigma$  : (TopCat.toSSet.obj (.of Y)).obj n) :
   $\sigma \in (\text{small } A\ B).obj\ n \leftrightarrow$ 
   $\sigma \in (\text{landsIn } A).obj\ n \vee \sigma \in (\text{landsIn } B).obj\ n :=$ 
  Iff.rfl

end SingularExcision

```

A.17 PlatonicSolids/SingularExcision/DegreewisePushout.lean

Degreewise pushout of small chains (109 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.CategoryTheory.Limits.Preserves.SigmaConst
import PlatonicSolids.SingularExcision.Subcomplexes

/-!
# Degreewise pushouts for small singular chains

The lattice pushout of the two lands-in subcomplexes remains a pushout
after evaluation in each simplicial degree and after applying the free
`R`-object functor `sigmaConst.obj R`.
-/

open AlgebraicTopology CategoryTheory Limits
open scoped Simplicial

namespace SingularExcision

variable {Y : Type} [TopologicalSpace Y]
variable {C : Type*} [Category C] [Preadditive C] [HasCoproducts.{0} C]

```

```

/-- The degreewise pushout square of sets of small singular simplices. -/
noncomputable def smallSimplexSetIsPushout (A B : Set Y)
  (n : SimplexCategoryop) :
  IsPushout
    ((SSet.Subcomplex.homOfLE (inf_le_left :
      landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn A)).app n)
    ((SSet.Subcomplex.homOfLE (inf_le_right :
      landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn B)).app n)
    ((SSet.Subcomplex.homOfLE (le_sup_left :
      landsIn A  $\leq$  small A B)).app n)
    ((SSet.Subcomplex.homOfLE (le_sup_right :
      landsIn B  $\leq$  small A B)).app n) :=
  (smallIsPushout A B).map ((evaluation SimplexCategoryop (Type)).obj n)

/-- Applying `sigmaConst.obj R` to the degreewise set pushout gives a
pushout in any coefficient category with the required coproducts. -/
noncomputable def smallSigmaConstIsPushout (R : C) (A B : Set Y)
  (n : SimplexCategoryop) :
  IsPushout
    ((sigmaConst.obj R).map
      ((SSet.Subcomplex.homOfLE (inf_le_left :
        landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn A)).app n))
    ((sigmaConst.obj R).map
      ((SSet.Subcomplex.homOfLE (inf_le_right :
        landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn B)).app n))
    ((sigmaConst.obj R).map
      ((SSet.Subcomplex.homOfLE (le_sup_left :
        landsIn A  $\leq$  small A B)).app n))
    ((sigmaConst.obj R).map
      ((SSet.Subcomplex.homOfLE (le_sup_right :
        landsIn B  $\leq$  small A B)).app n)) :=
  (smallSimplexSetIsPushout A B n).map (sigmaConst.obj R)

/-- The chain complex of the singular set of a subtype is isomorphic to
the chain complex on the corresponding lands-in subcomplex. -/
noncomputable def subtypeChainComplexIso (R : C) (A : Set Y) :
  (TopCat.toSSet.obj (.of  $\uparrow$ A)).chainComplex R  $\cong$ 
  (landsIn A : SSet).chainComplex R :=
  ((SSet.chainComplexFunctor C).obj R).mapIso (subtypeSingularSetIso A)

/-- In each degree, the maps between chain objects of the two lands-in
subcomplexes and their supremum form a pushout square. -/
noncomputable def smallChainDegreeIsPushout (R : C) (A B : Set Y) (n :  $\mathbb{N}$ ) :
  IsPushout
    ((SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE (inf_le_left :
        landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn A)) R).f n)
    ((SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE (inf_le_right :
        landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn B)) R).f n)
    ((SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE (le_sup_left :
        landsIn A  $\leq$  small A B)) R).f n)
    ((SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE (le_sup_right :
        landsIn B  $\leq$  small A B)) R).f n) := by
  simpa [SSet.chainComplexMap, SSet.chainComplexFunctor,
    AlgebraicTopology.alternatingFaceMapComplex] using
    smallSigmaConstIsPushout R A B (Opposite.op  $\llbracket$ n $\rrbracket$ )

/-- The degreewise pushout assembles to a pushout of chain complexes. -/
noncomputable def smallChainIsPushout (R : C) (A B : Set Y) :

```

```

IsPushout
  (SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (inf_le_left :
      landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn A)) R)
  (SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (inf_le_right :
      landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn B)) R)
  (SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (le_sup_left :
      landsIn A  $\leq$  small A B)) R)
  (SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (le_sup_right :
      landsIn B  $\leq$  small A B)) R) := by
refine
  { w := by
    rw [← Functor.map_comp, ← Functor.map_comp]
    rfl
    isColimit' := ⟨?_⟩ }
apply HomologicalComplex.isColimitOfEval
intro n
exact (PushoutCocone.isColimitMapCoconeEquiv _ _).symm
  (smallChainDegreeIsPushout R A B n).isColimit

end SingularExcision

```

A.18 PlatonicSolids/SingularExcision/CokernelPushout.lean

Cokernel model of a preadditive pushout (111 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.CategoryTheory.Limits.Shapes.Pullback.IsPullback.Basic
import Mathlib.CategoryTheory.Limits.Shapes.Kernels
import PlatonicSolids.SingularExcision.DegreewisePushout

/-!
# A cokernel model for a preadditive pushout

The pushout of `f : I → A` and `g : I → B` is the cokernel of
`(f, -g) : I → A  $\boxplus$  B`.
-/

open CategoryTheory Limits

universe v u

namespace SingularExcision

variable {C : Type u} [Category.{v} C] [Preadditive C]
[HasBinaryBiproducts C] [HasCokernels C]
variable {I A B P : C} (f : I → A) (g : I → B)

/-- The relation map `(f, -g)` presenting a pushout as a cokernel. -/
noncomputable def pushoutRelation : I → A  $\boxplus$  B :=
  biprod.lift f (-g)

omit [HasCokernels C] in
lemma pushoutRelation_eq :
  pushoutRelation f g = f  $\gg$  biprod.inl + (-g)  $\gg$  biprod.inr := by
  ext
  · simp [pushoutRelation, Preadditive.add_comp]

```

```

· simp [pushoutRelation, Preadditive.add_comp]

/-- The two canonical maps into the cokernel of `(f, -g)` -/
noncomputable def cokernelPushoutInl :
  A → cokernel (pushoutRelation f g) :=
  biprod.inl >> cokernel.π (pushoutRelation f g)

noncomputable def cokernelPushoutInr :
  B → cokernel (pushoutRelation f g) :=
  biprod.inr >> cokernel.π (pushoutRelation f g)

lemma cokernelPushout_comm :
  f >> cokernelPushoutInl f g = g >> cokernelPushoutInr f g := by
  have h := cokernel.condition (pushoutRelation f g)
  rw [pushoutRelation_eq, Preadditive.add_comp] at h
  rw [← pushoutRelation_eq] at h
  apply sub_eq_zero.mp
  simp only [cokernelPushoutInl, cokernelPushoutInr, Category.assoc,
    Preadditive.neg_comp, sub_eq_add_neg] using h

/-- The cokernel of `(f, -g)` satisfies the pushout universal property. -/
noncomputable def cokernelIsPushout :
  IsPushout f g (cokernelPushoutInl f g) (cokernelPushoutInr f g) := by
  refine
    { w := cokernelPushout_comm f g
      isColimit' := (PushoutCocone.IsColimit.mk _ ?_ ?_ ?_ ?_) }
  · intro s
    refine cokernel.desc (pushoutRelation f g) (biprod.desc s.inl s.inr) ?_
    rw [pushoutRelation_eq, Preadditive.add_comp]
    simp only [Category.assoc, biprod.inl_desc, biprod.inr_desc]
    simp only [Preadditive.neg_comp, sub_eq_add_neg] using
      sub_eq_zero.mpr s.condition
  · intro s
    simp [cokernelPushoutInl]
  · intro s
    simp [cokernelPushoutInr]
  · intro s m hmA hmB
    apply (cancel_epi (cokernel.π (pushoutRelation f g))).1
    rw [cokernel.π_desc]
  ext
  · simp [cokernelPushoutInl, Category.assoc] using hmA
  · simp [cokernelPushoutInr, Category.assoc] using hmB

/-- Any chosen pushout is canonically isomorphic to the cokernel
presentation. -/
noncomputable def pushoutIsoCokernel {h : A → P} {k : B → P}
  (sq : IsPushout f g h k) :
  P ≅ cokernel (pushoutRelation f g) :=
  sq.isoIsPushout _ _ (cokernelIsPushout f g)

section SmallChains

open AlgebraicTopology
open scoped Simplicial

variable {Y : Type} [TopologicalSpace Y]
variable {D : Type*} [Category D] [Abelian D] [HasCoproducts.{0} D]

/-- In every degree, the small-chain object is the cokernel of the
signed map from the infimum of the two lands-in subcomplexes. -/
noncomputable def smallChainDegreeIsoCokernel (R : D)
  (A B : Set Y) (n : ℕ) :
  ((small A B : SSet).chainComplex R).X n ≅

```



```

    cokernel
      (pushoutRelation
        ((SSet.chainComplexMap
          (SSet.Subcomplex.homOfLE (inf_le_left :
            landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn A)) R).f n)
          ((SSet.chainComplexMap
            (SSet.Subcomplex.homOfLE (inf_le_right :
              landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn B)) R).f n)) :=
        pushoutIsoCokernel _ _
          (smallChainDegreeIsPushout R A B n)

end SmallChains

end SingularExcision

```

A.19 PlatonicSolids/SingularExcision/CoverChains.lean

Cover chains isomorphic to small singular chains (133 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.MayerVietoris
import PlatonicSolids.SingularExcision.CokernelPushout

/-!
# Cover chains as small singular chains

The singular sets of the three subtypes in a two-set cover are transported
to the two lands-in subcomplexes and their infimum. The resulting pushout
identifies the existing `coverChains` cokernel with the chain complex on
small singular simplices.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex
open scoped Topology

universe v u

namespace SingularExcision

variable {Y : Type} [TopologicalSpace Y]

/-- The singular set of the intersection, expressed as the infimum of the
two ambient lands-in subcomplexes. -/
noncomputable def intersectionSingularSetIso (A B : Set Y) :
  TopCat.toSSet.obj (.of  $\uparrow(A \cap B)$ )  $\cong$ 
  ((landsIn A  $\sqcap$  landsIn B :
    (TopCat.toSSet.obj (.of Y)).Subcomplex) : SSet) :=
  subtypeSingularSetIso (A  $\cap$  B)  $\ll \gg$  landsInInterIso A B

lemma intersectionSingularSetIso_hom_comp_left (A B : Set Y) :
  (intersectionSingularSetIso A B).hom  $\gg$ 
  SSet.Subcomplex.homOfLE (inf_le_left :
    landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn A) =
  TopCat.toSSet.map (interInclLeft A B)  $\gg$ 
  (subtypeSingularSetIso A).hom := by
  rw [← cancel_mono (landsIn A). $\iota$ ]
  simp [intersectionSingularSetIso, subtypeSingularSetIso,
    subtypeSingularSetMap, interInclLeft,
    ← Functor.map_comp]
  simpa [interInclLeft, interToLeft] using

```

```

congrArg TopCat.toSSet.map (interToLeft_comp_subtypeIncl A B).symm

lemma intersectionSingularSetIso_hom_comp_right (A B : Set Y) :
  (intersectionSingularSetIso A B).hom >>
    SSet.Subcomplex.homOfLE (inf_le_right :
      landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn B) =
    TopCat.toSSet.map (interInclRight A B) >>
      (subtypeSingularSetIso B).hom := by
rw [← cancel_mono (landsIn B).ι]
simp [intersectionSingularSetIso, subtypeSingularSetIso,
  subtypeSingularSetMap, interInclRight,
  ← Functor.map_comp]
simpa [interInclRight, interToRight] using
  congrArg TopCat.toSSet.map (interToRight_comp_subtypeIncl A B).symm

variable {C : Type u} [Category.{v} C] [Abelian C]
[HasCoproducts.{0} C] [HasPullbacks C]

/-- Chain-level form of `intersectionSingularSetIso`. -/
noncomputable def intersectionChainComplexIso (R : C) (A B : Set Y) :
  ((singularChainComplexFunctor C).obj R).obj (.of  $\uparrow$ (A  $\cap$  B))  $\cong$ 
  ((landsIn A  $\sqcap$  landsIn B :
    (TopCat.toSSet.obj (.of Y)).Subcomplex) : SSet).chainComplex R :=
  ((SSet.chainComplexFunctor C).obj R).mapIso
  (intersectionSingularSetIso A B)

omit [HasPullbacks C] in
lemma intersectionChainComplexIso_hom_comp_left (R : C) (A B : Set Y) :
  (intersectionChainComplexIso R A B).hom >>
    SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE (inf_le_left :
        landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn A)) R =
    ((singularChainComplexFunctor C).obj R).map (interInclLeft A B) >>
      (subtypeChainComplexIso R A).hom := by
simpa [intersectionChainComplexIso, subtypeChainComplexIso,
  singularChainComplexFunctor, SSet.chainComplexMap, Functor.map_comp] using
  congrArg (fun q => ((SSet.chainComplexFunctor C).obj R).map q)
  (intersectionSingularSetIso_hom_comp_left A B)

omit [HasPullbacks C] in
lemma intersectionChainComplexIso_hom_comp_right (R : C) (A B : Set Y) :
  (intersectionChainComplexIso R A B).hom >>
    SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE (inf_le_right :
        landsIn A  $\sqcap$  landsIn B  $\leq$  landsIn B)) R =
    ((singularChainComplexFunctor C).obj R).map (interInclRight A B) >>
      (subtypeChainComplexIso R B).hom := by
simpa [intersectionChainComplexIso, subtypeChainComplexIso,
  singularChainComplexFunctor, SSet.chainComplexMap, Functor.map_comp] using
  congrArg (fun q => ((SSet.chainComplexFunctor C).obj R).map q)
  (intersectionSingularSetIso_hom_comp_right A B)

/-- The square of actual subtype singular chains is a pushout onto the
small singular chain complex. -/
noncomputable def subtypeChainsSmallIsPushout (R : C) (A B : Set Y) :
  IsPushout
    (((singularChainComplexFunctor C).obj R).map (interInclLeft A B))
    (((singularChainComplexFunctor C).obj R).map (interInclRight A B))
    ((subtypeChainComplexIso R A).hom >>
      SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE (le_sup_left :
          landsIn A  $\leq$  small A B)) R)
    ((subtypeChainComplexIso R B).hom >>

```

```

      SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE (le_sup_right :
        landsIn B ≤ small A B)) R) := by
    apply (smallChainIsPushout R A B).of_iso'
      (intersectionChainComplexIso R A B)
      (subtypeChainComplexIso R A)
      (subtypeChainComplexIso R B)
      (Iso.refl _)
    · exact intersectionChainComplexIso_hom_comp_left R A B
    · exact intersectionChainComplexIso_hom_comp_right R A B
    · simp only [Iso.refl_hom, Category.comp_id]
      congr
    · simp only [Iso.refl_hom, Category.comp_id]
      congr

/-- The existing Mayer--Vietoris cover-chain cokernel is canonically
isomorphic to the chain complex generated by small singular simplices. -/
noncomputable def coverChainsIsoSmall (R : C) (A B : Set Y) :
  coverChains (C := C) R (interInclLeft A B) (interInclRight A B) ≅
  (small A B : SSet).chainComplex R :=
  (pushoutIsoCokernel
    (((singularChainComplexFunctor C).obj R).map (interInclLeft A B))
    (((singularChainComplexFunctor C).obj R).map (interInclRight A B))
    (subtypeChainsSmallIsPushout R A B)).symm

end SingularExcision

```

A.20 PlatonicSolids/SingularExcision/SmallChains.lean

Small-chain subgroup, invariance, and the retract (1078 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Category.ModuleCat.Colimits
import Mathlib.Algebra.DirectSum.Basic
import PlatonicSolids.SingularExcision.Algebra
import PlatonicSolids.SingularExcision.BarycentricGeometry
import PlatonicSolids.SingularExcision.CoverChains
import PlatonicSolids.SingularExcision.ModuleSupport
import PlatonicSolids.SingularExcision.SingularChains

/-!
Small singular chains for a two-set open cover.

Coefficients stay in `ModuleCat` so that finite coproduct support is
available; the geometric smallness lemmas are coefficient-independent.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex Set
open scoped DirectSum Simplicial Topology

namespace SingularExcision

noncomputable section

variable {Y : Type} [TopologicalSpace Y]

/-- If a singular simplex has image in `A`, it is a simplex of `landsIn A`. -/
lemma mem_landsIn_of_image_subset {n : SimplexCategoryop}
  (s : (TopCat.toSSet.obj (.of Y)).obj n) {A : Set Y}
  (h : Set.range ((TopCat.of Y).toSSetObjEquiv n s) ⊆ A) :

```

```

    ∃ τ, (subtypeSingularSetMap A).app n τ = s := by
  let fY := (TopCat.of Y).toSSetObjEquiv n s
  let f : C(TopCat.of (stdSimplex ℝ (Fin (n.unop.len + 1))),
    TopCat.of ↑A) :=
    ⟨fun x => ⟨fY x, h ⟨x, rfl⟩⟩, fY.continuous.subtype_mk _⟩
  refine ⟨((TopCat.of ↑A).toSSetObjEquiv n).symm f, ?_⟩
  apply ((TopCat.of Y).toSSetObjEquiv n).injective
  ext x
  rw [subtypeSingularSetMap_toSSetObjEquiv, Equiv.apply_symm_apply]
  rfl

/-- A simplex of `landsIn A` has image contained in `A`. -/
lemma image_subset_of_mem_landsIn {n : SimplexCategoryop}
  {s : (TopCat.toSSet.obj (.of Y)).obj n} {A : Set Y}
  (h : s ∈ (landsIn A).obj n) :
  Set.range ((TopCat.of Y).toSSetObjEquiv n s) ⊆ A := by
  obtain ⟨τ, rfl⟩ := h
  intro y hy
  obtain ⟨x, rfl⟩ := hy
  rw [subtypeSingularSetMap_toSSetObjEquiv]
  exact ((TopCat.of ↑A).toSSetObjEquiv n τ x).property

/-- Image in `A` or `B` means the simplex is small for the pair. -/
lemma mem_small_of_image_subset_left_or_right {n : SimplexCategoryop}
  {s : (TopCat.toSSet.obj (.of Y)).obj n} {A B : Set Y}
  (h : Set.range ((TopCat.of Y).toSSetObjEquiv n s) ⊆ A ∨
    Set.range ((TopCat.of Y).toSSetObjEquiv n s) ⊆ B) :
  s ∈ (small A B).obj n := by
  change s ∈ (landsIn A).obj n ∪ (landsIn B).obj n
  rcases h with hA | hB
  · obtain ⟨τ, hτ⟩ := mem_landsIn_of_image_subset s hA
    exact Or.inl ⟨τ, hτ⟩
  · obtain ⟨τ, hτ⟩ := mem_landsIn_of_image_subset s hB
    exact Or.inr ⟨τ, hτ⟩

/-- A small simplex has image in `A` or in `B`. -/
lemma image_subset_left_or_right_of_mem_small {n : SimplexCategoryop}
  {s : (TopCat.toSSet.obj (.of Y)).obj n} {A B : Set Y}
  (h : s ∈ (small A B).obj n) :
  Set.range ((TopCat.of Y).toSSetObjEquiv n s) ⊆ A ∨
  Set.range ((TopCat.of Y).toSSetObjEquiv n s) ⊆ B := by
  rcases (show s ∈ (landsIn A).obj n ∪ (landsIn B).obj n by
    simp [small, Subfunctor.max_obj] using h) with hA | hB
  · exact Or.inl (image_subset_of_mem_landsIn hA)
  · exact Or.inr (image_subset_of_mem_landsIn hB)

/-- After enough barycentric subdivisions, every affine piece of a
singular simplex is small for a two-set open cover. -/
lemma eventual_small_pieces_of_openCover
  {n : ℕ} (σ : C(stdSimplex ℝ (Fin (n + 1)), Y))
  {A B : Set Y} (hA : IsOpen A) (hB : IsOpen B)
  (hAB : A ∪ B = univ) :
  ∃ N : ℕ, ∀ τ ∈ ((PlatonicSolids.SingularExcision.subdivideStd
    (q := n) n)[N]
    (PlatonicSolids.SingularExcision.simplex
      (PlatonicSolids.SingularExcision.idSimplex n))).support,
    σ '' Set.range τ.realize ⊆ A ∨
    σ '' Set.range τ.realize ⊆ B :=
  PlatonicSolids.SingularExcision.exists_subdivideStd_iterate_image_subset_left_or_right
  σ hA hB hAB

/-- Reparametrized pieces of a sufficiently subdivided singular simplex
are small simplices. -/

```

```

lemma eventual_small_reparametrize_of_openCover
  {n : ℕ} (s : (TopCat.toSSet.obj (.of Y)) _[[n]])
  {A B : Set Y} (hA : IsOpen A) (hB : IsOpen B)
  (hAB : A ∪ B = univ) :
  ∃ N : ℕ, ∀ τ ∈ ((PlatonicSolids.SingularExcision.subdivideStd
    (q := n) n) ^ [N]
    (PlatonicSolids.SingularExcision.simplex
      (PlatonicSolids.SingularExcision.idSimplex n))).support,
    PlatonicSolids.SingularExcision.reparametrizeSingularSimplex s τ ∈
      (small A B).obj (.op (.mk n)) := by
obtain ⟨N, hN⟩ :=
  eventual_small_pieces_of_openCover
    ((TopCat.of Y).toSSetObjEquiv _ s) hA hB hAB
refine ⟨N, fun τ hτ => ?_⟩
refine mem_small_of_image_subset_left_or_right _ ?_
have himg :
  Set.range ((TopCat.of Y).toSSetObjEquiv _
    (PlatonicSolids.SingularExcision.reparametrizeSingularSimplex s τ)) =
    ((TopCat.of Y).toSSetObjEquiv _ s) ' Set.range τ.realize := by
  rw [PlatonicSolids.SingularExcision.reparametrizeSingularSimplex_toContinuousMap]
  simp [ContinuousMap.coe_comp, Set.range_comp]
  simpa [himg] using hN τ hτ

/-- Reparametrizing a small simplex keeps it small: the image of a piece
is contained in the image of the parent. -/
lemma reparametrize_mem_small_of_mem_small {p q : ℕ}
  {A B : Set Y}
  (s : (TopCat.toSSet.obj (.of Y)) _[[q]])
  (σ : PlatonicSolids.SingularExcision.AffineSimplex
    (stdSimplex ℝ (Fin (q + 1))) p)
  (hs : s ∈ (small A B).obj (.op (.mk q))) :
  PlatonicSolids.SingularExcision.reparametrizeSingularSimplex s σ ∈
    (small A B).obj (.op (.mk p)) := by
  refine mem_small_of_image_subset_left_or_right _ ?_
  have himg :
    Set.range ((TopCat.of Y).toSSetObjEquiv _
      (PlatonicSolids.SingularExcision.reparametrizeSingularSimplex s σ)) =
      ((TopCat.of Y).toSSetObjEquiv _ s) ' Set.range σ.realize := by
    rw [PlatonicSolids.SingularExcision.reparametrizeSingularSimplex_toContinuousMap]
    simp [ContinuousMap.coe_comp, Set.range_comp]
  rw [himg]
  rcases image_subset_left_or_right_of_mem_small hs with hA | hB
  · exact Or.inl ((Set.image_subset_range _ _).trans hA)
  · exact Or.inr ((Set.image_subset_range _ _).trans hB)

end

/-! ## Module coefficients: total chains and the small retract -/

noncomputable section

open Classical

variable {k : Type} [Ring k] (R : ModuleCat.{0} k)
variable [HasCoproducts.{0} (ModuleCat.{0} k)]

/-- Underlying abelian group of singular `n`-chains. -/
abbrev singularChainType (X : TopCat.{0}) (n : ℕ) : Type :=
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj X).X n : Type)

instance (X : TopCat.{0}) (n : ℕ) : AddCommGroup (singularChainType R X n) :=
  inferInstance

```

```

/-- Direct sum of all singular chain groups. -/
abbrev TotalSingularChains (X : TopCat.{0}) : Type :=
  DirectSum ℕ (singularChainType R X)

instance (X : TopCat.{0}) : AddCommGroup (TotalSingularChains R X) :=
  inferInstance

noncomputable def dApply (X : TopCat.{0}) (n : ℕ) :
  singularChainType R X (n + 1) →+ singularChainType R X n :=
  AddMonoidHom.mk'
    (fun x =>
      (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj X).d
        (n + 1) n).hom x)
    (by intro x y; simp [map_add])

noncomputable def SApply (X : TopCat.{0}) (n : ℕ) :
  singularChainType R X n →+ singularChainType R X n :=
  AddMonoidHom.mk'
    (fun x =>
      (PlatonicSolids.SingularExcision.singularSubdivisionHom R X n).hom x)
    (by intro x y; simp [map_add])

noncomputable def HApply (X : TopCat.{0}) (n : ℕ) :
  singularChainType R X n →+ singularChainType R X (n + 1) :=
  AddMonoidHom.mk'
    (fun x =>
      (PlatonicSolids.SingularExcision.singularPrismHom R X n).hom x)
    (by intro x y; simp [map_add])

noncomputable def dTotal (X : TopCat.{0}) :
  TotalSingularChains R X →+ TotalSingularChains R X :=
  DirectSum.toAddMonoid fun n =>
    match n with
    | 0 => 0
    | n + 1 => (DirectSum.of (singularChainType R X) n).comp (dApply R X n)

noncomputable def STotal (X : TopCat.{0}) :
  TotalSingularChains R X →+ TotalSingularChains R X :=
  DirectSum.toAddMonoid fun n =>
    (DirectSum.of (singularChainType R X) n).comp (SApply R X n)

noncomputable def HTotal (X : TopCat.{0}) :
  TotalSingularChains R X →+ TotalSingularChains R X :=
  DirectSum.toAddMonoid fun n =>
    (DirectSum.of (singularChainType R X) (n + 1)).comp (HApply R X n)

lemma dTotal_of_zero (X : TopCat.{0})
  (y : singularChainType R X 0) :
  dTotal R X (DirectSum.of (singularChainType R X) 0 y) = 0 :=
  DirectSum.toAddMonoid_of _ 0 y

lemma dTotal_of_succ (X : TopCat.{0}) (n : ℕ)
  (y : singularChainType R X (n + 1)) :
  dTotal R X (DirectSum.of (singularChainType R X) (n + 1) y) =
  DirectSum.of (singularChainType R X) n (dApply R X n y) :=
  DirectSum.toAddMonoid_of _ (n + 1) y

lemma STotal_of (X : TopCat.{0}) (n : ℕ)
  (y : singularChainType R X n) :
  STotal R X (DirectSum.of (singularChainType R X) n y) =
  DirectSum.of (singularChainType R X) n (SApply R X n y) :=
  DirectSum.toAddMonoid_of _ n y

```

```

lemma HTotal_of (X : TopCat.{0}) (n : ℕ)
  (y : singularChainType R X n) :
  HTotal R X (DirectSum.of (singularChainType R X) n y) =
    DirectSum.of (singularChainType R X) (n + 1) (HApply R X n y) :=
  DirectSum.toAddMonoid_of _ n y

lemma SApply_dApply (X : TopCat.{0}) (n : ℕ)
  (y : singularChainType R X (n + 1)) :
  SApply R X n (dApply R X n y) = dApply R X n (SApply R X (n + 1) y) := by
  have h :=
    (PlatonicSolids.SingularExcision.singularSubdivision R X).comm (n + 1) n
  calc
    SApply R X n (dApply R X n y) =
      ModuleCat.Hom.hom
        ((PlatonicSolids.SingularExcision.singularSubdivision R X).f n)
        (ModuleCat.Hom.hom
          (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj X).d
            (n + 1) n) y) :=
      rfl
    _ = ModuleCat.Hom.hom
      (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj X).d
        (n + 1) n >>
        (PlatonicSolids.SingularExcision.singularSubdivision R X).f n) y :=
      (ModuleCat.comp_apply _ _ _).symm
    _ = ModuleCat.Hom.hom
      ((PlatonicSolids.SingularExcision.singularSubdivision R X).f (n + 1) >>
        (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj X).d
          (n + 1) n)) y := by
      rw [← h]
    _ = dApply R X n (SApply R X (n + 1) y) := by
      rw [ModuleCat.comp_apply]; rfl

lemma homotopyApply_zero (X : TopCat.{0})
  (y : singularChainType R X 0) :
  y - SApply R X 0 y = dApply R X 0 (HApply R X 0 y) := by
  have hS := PlatonicSolids.SingularExcision.singularSubdivisionHom_zero R X
  have hT := PlatonicSolids.SingularExcision.singularPrismHom_zero R X
  simp [SApply, HApply, dApply, hS, hT]

lemma homotopyApply_succ (X : TopCat.{0}) (n : ℕ)
  (y : singularChainType R X (n + 1)) :
  y - SApply R X (n + 1) y =
    HApply R X n (dApply R X n y) +
    dApply R X (n + 1) (HApply R X (n + 1) y) := by
  have hcomm :=
    (PlatonicSolids.SingularExcision.singularPrismHomotopy R X).comm (n + 1)
  have hdNext :
    dNext (n + 1)
      (PlatonicSolids.SingularExcision.singularPrismHomotopy R X).hom =
      (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj X).d
        (n + 1) n) >>
      PlatonicSolids.SingularExcision.singularPrismHom R X n := by
    rw [Homotopy.dNext_succ_chainComplex]
    simp [PlatonicSolids.SingularExcision.singularPrismHomotopy]
  have hprevD :
    prevD (n + 1)
      (PlatonicSolids.SingularExcision.singularPrismHomotopy R X).hom =
      PlatonicSolids.SingularExcision.singularPrismHom R X (n + 1) >>
      (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj X).d
        (n + 2) (n + 1)) := by
    rw [Homotopy.prevD_chainComplex]
    simp [PlatonicSolids.SingularExcision.singularPrismHomotopy]
  have hfun :=

```

```

    congrArg (fun f : _ → _ => ModuleCat.Hom.hom f y) hcomm
  rw [hdNext, hprevD] at hfun
  simp [SApply, HApply, dApply, ModuleCat.hom_add, ModuleCat.comp_apply] at hfun
  rw [sub_eq_iff_eq_add]
  simp [add_assoc, add_left_comm, add_comm] using hfun

lemma homotopyApply (X : TopCat.{0}) (n : ℕ)
  (y : singularChainType R X n) :
  y - SApply R X n y =
    (match n with
    | 0 => 0
    | m + 1 => HApply R X m (dApply R X m y)) +
    dApply R X n (HApply R X n y) := by
cases n with
| zero =>
  simp using homotopyApply_zero R X y
| succ m =>
  simp using homotopyApply_succ R X m y

/-- Ungraded subdivision data on the total singular chain group. -/
noncomputable def subdivisionData_singular (X : TopCat.{0}) :
  SubdivisionData (TotalSingularChains R X) where
  d := dTotal R X
  S := STotal R X
  H := HTotal R X
  S_comm := by
    intro x
    refine DirectSum.induction_on x ?_ ?_ ?_
    · simp
    · intro n y
      match n with
      | 0 =>
        simp [dTotal_of_zero, STotal_of]
      | n + 1 =>
        simp [dTotal_of_succ, STotal_of, SApply_dApply]
    · intro a b ha hb
      simp [map_add, ha, hb]
  homotopy := by
    intro x
    refine DirectSum.induction_on x ?_ ?_ ?_
    · simp
    · intro n y
      have h := homotopyApply R X n y
      match n with
      | 0 =>
        simp [dTotal_of_zero, STotal_of, HTotal_of,
          PlatonicSolids.SingularExcision.singularPrismHom_zero,
          PlatonicSolids.SingularExcision.singularSubdivisionHom_zero,
          HApply, SApply]
      | n + 1 =>
        simp [dTotal_of_succ, STotal_of, HTotal_of]
        have h' := congrArg (DirectSum.of (singularChainType R X) (n + 1)) h
        simp [map_add, map_sub, add_comm] using h'
    · intro a b ha hb
      simp only [map_add]
      have hsum : a + b - (STotal R X a + STotal R X b) =
        (a - STotal R X a) + (b - STotal R X b) := by abel
      rw [hsum, ha, hb]
      abel

variable {Y : Type} [TopologicalSpace Y]

/-- A degree-`n` chain is small when it is in the image of the

```



```

small-subcomplex inclusion. -/
def isSmallChain {A B : Set Y} {n : ℕ}
  (x : singularChainType R (.of Y) n) : Prop :=
  ∃ y : ((small A B : SSet).chainComplex R).X n,
  ModuleCat.Hom.hom ((SSet.chainComplexMap (small A B).ι R).f n) y = x

lemma isSmallChain_zero {A B : Set Y} {n : ℕ} :
  isSmallChain (R := R) (A := A) (B := B) (n := n) 0 :=
  ⟨0, map_zero _⟩

lemma isSmallChain_add {A B : Set Y} {n : ℕ}
  {x₁ x₂ : singularChainType R (.of Y) n}
  (h₁ : isSmallChain (R := R) (A := A) (B := B) x₁)
  (h₂ : isSmallChain (R := R) (A := A) (B := B) x₂) :
  isSmallChain (R := R) (A := A) (B := B) (x₁ + x₂) := by
  obtain ⟨y₁, hy₁⟩ := h₁
  obtain ⟨y₂, hy₂⟩ := h₂
  refine ⟨y₁ + y₂, ?_⟩
  rw [map_add, hy₁, hy₂]
  exact rfl

lemma isSmallChain_neg {A B : Set Y} {n : ℕ}
  {x : singularChainType R (.of Y) n}
  (h : isSmallChain (R := R) (A := A) (B := B) x) :
  isSmallChain (R := R) (A := A) (B := B) (-x) := by
  obtain ⟨y, hy⟩ := h
  refine ⟨-y, ?_⟩
  rw [map_neg, hy]
  exact rfl

/-- The small-chain subgroup of the total singular chain group. -/
def smallChainsSubgroup (A B : Set Y) :
  AddSubgroup (TotalSingularChains R (.of Y)) where
  carrier := { x | ∀ n, isSmallChain (R := R) (A := A) (B := B) (x n) }
  zero_mem' := fun n => by
    convert isSmallChain_zero (R := R) (A := A) (B := B) (n := n)
  add_mem' hx hy n := isSmallChain_add (R := R) (hx n) (hy n)
  neg_mem' hx n := isSmallChain_neg (R := R) (hx n)

lemma isSmallChain_d {A B : Set Y} {n : ℕ}
  {x : singularChainType R (.of Y) (n + 1)}
  (hx : isSmallChain (R := R) (A := A) (B := B) x) :
  isSmallChain (R := R) (A := A) (B := B) (dApply R (.of Y) n x) := by
  obtain ⟨y, hy⟩ := hx
  refine ⟨((small A B : SSet).chainComplex R).d (n + 1) n y, ?_⟩
  have hcomm := (SSet.chainComplexMap (small A B).ι R).comm (n + 1) n
  calc
    ModuleCat.Hom.hom ((SSet.chainComplexMap (small A B).ι R).f n)
      (((small A B : SSet).chainComplex R).d (n + 1) n y) =
      ModuleCat.Hom.hom
        (((small A B : SSet).chainComplex R).d (n + 1) n) >>
        (SSet.chainComplexMap (small A B).ι R).f n y :=
      (ModuleCat.comp_apply _ _ _).symm
  _ = ModuleCat.Hom.hom
    ((SSet.chainComplexMap (small A B).ι R).f (n + 1) >>
      ((TopCat.toSSet.obj (.of Y)).chainComplex R).d (n + 1) n) y := by
  rw [← hcomm]
  _ = dApply R (.of Y) n x := by
  rw [ModuleCat.comp_apply, hy]
  rfl

/-- The inclusion of a small simplex, as an ambient chain, is small. -/
lemma isSmallChain_ι {A B : Set Y} {n : ℕ}

```

```

(s : (TopCat.toSSet.obj (.of Y)) _[n])
(hs : s ∈ (small A B).obj (.op (.mk n))) (r : R) :
isSmallChain (R := R) (A := A) (B := B)
  (ModuleCat.Hom.hom
    ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R) s) r) := by
let s' : (small A B : SSet) _[n] := ⟨s, hs⟩
refine ⟨ModuleCat.Hom.hom ((small A B : SSet).ιChainComplex (R := R) s') r, ?_⟩
have hι :=
  SSet.ι_chainComplexMap_f (X := (small A B : SSet))
    (Y := TopCat.toSSet.obj (.of Y)) (f := (small A B).ι) (R := R) s'
have happ := congrArg (fun f : _ → _ => ModuleCat.Hom.hom f r) hι
have hcomp :
  ModuleCat.Hom.hom
    (((small A B : SSet).ιChainComplex (R := R) s') >>
      (SSet.chainComplexMap (small A B).ι R).f n) r =
    ModuleCat.Hom.hom ((SSet.chainComplexMap (small A B).ι R).f n)
      (ModuleCat.Hom.hom ((small A B : SSet).ιChainComplex (R := R) s') r) := by
  rw [ModuleCat.hom_comp, LinearMap.comp_apply]
calc
ModuleCat.Hom.hom ((SSet.chainComplexMap (small A B).ι R).f n)
  (ModuleCat.Hom.hom ((small A B : SSet).ιChainComplex (R := R) s') r) =
ModuleCat.Hom.hom
  (((small A B : SSet).ιChainComplex (R := R) s') >>
    (SSet.chainComplexMap (small A B).ι R).f n) r :=
  hcomp.symm
_ = ModuleCat.Hom.hom
  ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R)
    ((small A B).ι.app (.op (.mk n)) s')) r :=
  happ
_ = ModuleCat.Hom.hom
  ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R) s) r := by
  congr 2

lemma SApply_ι {X : TopCat.{0}} {n : ℕ}
(s : (TopCat.toSSet.obj X) _[n]) (r : R) :
SApply R X n
  (ModuleCat.Hom.hom ((TopCat.toSSet.obj X).ιChainComplex (R := R) s) r) =
ModuleCat.Hom.hom
  (PlatonicSolids.SingularExcision.singularSubdivideGenerator R s) r := by
have h :=
  PlatonicSolids.SingularExcision.ι_singularSubdivisionHom (R := R) (X := X) s
have happ := congrArg (fun f : _ → _ => ModuleCat.Hom.hom f r) h
have hcomp :
  ModuleCat.Hom.hom
    (((TopCat.toSSet.obj X).ιChainComplex (R := R) s) >>
      PlatonicSolids.SingularExcision.singularSubdivisionHom R X n) r =
SApply R X n
  (ModuleCat.Hom.hom
    ((TopCat.toSSet.obj X).ιChainComplex (R := R) s) r) := by
  rw [ModuleCat.hom_comp, LinearMap.comp_apply]; rfl
exact hcomp.symm.trans happ

lemma HApply_ι {X : TopCat.{0}} {n : ℕ}
(s : (TopCat.toSSet.obj X) _[n]) (r : R) :
HApply R X n
  (ModuleCat.Hom.hom ((TopCat.toSSet.obj X).ιChainComplex (R := R) s) r) =
ModuleCat.Hom.hom
  (PlatonicSolids.SingularExcision.singularPrismGenerator R s) r := by
have h :=
  PlatonicSolids.SingularExcision.ι_singularPrismHom (R := R) (X := X) s
have happ := congrArg (fun f : _ → _ => ModuleCat.Hom.hom f r) h
have hcomp :
  ModuleCat.Hom.hom

```

```

      (((TopCat.toSSet.obj X).!ChainComplex (R := R) s) >>
        PlatonicSolids.SingularExcision.singularPrismHom R X n) r =
    HApply R X n
    (ModuleCat.Hom.hom
      ((TopCat.toSSet.obj X).!ChainComplex (R := R) s) r) := by
    rw [ModuleCat.hom_comp, LinearMap.comp_apply]; rfl
  exact hcomp.symm.trans happ

lemma isSmallChain_nsmul {A B : Set Y} {n : ℕ}
  {x : singularChainType R (.of Y) n} (m : ℕ)
  (hx : isSmallChain (R := R) (A := A) (B := B) x) :
  isSmallChain (R := R) (A := A) (B := B) (m • x) := by
  induction m with
  | zero =>
    simpa using isSmallChain_zero (R := R) (A := A) (B := B) (n := n)
  | succ m ih =>
    rw [succ_nsmul]
    exact isSmallChain_add (R := R) ih hx

lemma isSmallChain_zsmul {A B : Set Y} {n : ℕ}
  {x : singularChainType R (.of Y) n} (z : ℤ)
  (hx : isSmallChain (R := R) (A := A) (B := B) x) :
  isSmallChain (R := R) (A := A) (B := B) (z • x) := by
  cases z with
  | ofNat m =>
    simpa using isSmallChain_nsmul (R := R) m hx
  | negSucc m =>
    simpa [negSucc_zsmul] using
      isSmallChain_neg (R := R)
      (isSmallChain_nsmul (R := R) (m + 1) hx)

lemma isSmallChain_realizeAffine {A B : Set Y} {p q : ℕ}
  (s : (TopCat.toSSet.obj (.of Y)) _[q])
  (c : PlatonicSolids.SingularExcision.AffineChain
    (stdSimplex ℝ (Fin (q + 1))) p)
  (hc : ∀ τ ∈ c.support,
    PlatonicSolids.SingularExcision.reparametrizeSingularSimplex s τ ∈
    (small A B).obj (.op (.mk p)))
  (r : R) :
  isSmallChain (R := R) (A := A) (B := B)
    (ModuleCat.Hom.hom
      (PlatonicSolids.SingularExcision.realizeAffineChain R s c) r) := by
  induction c using Finsupp.induction with
  | zero =>
    simpa [PlatonicSolids.SingularExcision.realizeAffineChain_zero] using
      isSmallChain_zero (R := R) (A := A) (B := B) (n := p)
  | single_add τ z c hr hz ih =>
    have hx :
      ModuleCat.Hom.hom
        (PlatonicSolids.SingularExcision.realizeAffineChain R s
          (Finsupp.single τ z + c)) r =
      z • ModuleCat.Hom.hom
        (PlatonicSolids.SingularExcision.realizeAffineSimplex R s τ) r +
      ModuleCat.Hom.hom
        (PlatonicSolids.SingularExcision.realizeAffineChain R s c) r := by
    rw [map_add, PlatonicSolids.SingularExcision.realizeAffineChain_single]
    simp [ModuleCat.hom_add, LinearMap.map_smul]
    rw [hx]
  refine isSmallChain_add (R := R) ?_
  (ih fun ρ hρ => hc ρ (by
    have hne : ρ ≠ τ := fun e => hτ (e ► hρ)
    simp [Finsupp.mem_support_iff, Finsupp.add_apply, Finsupp.single_apply, hne] at hρ ⊢
    exact hρ))

```

```

have hrs := hc  $\tau$  (by
  simp [Finsupp.mem_support_iff, Finsupp.add_apply,
    Finsupp.notMem_support_iff.mp hrs] using hz)
have hgen := isSmallChain_ℓ (R := R) (A := A) (B := B)
  (PlatonicSolids.SingularExcision.reparametrizeSingularSimplex s  $\tau$ )
hrs r
simp [PlatonicSolids.SingularExcision.realizeAffineSimplex] using
  isSmallChain_zsmul (R := R) z hgen

lemma isSmallChain_S_generator {A B : Set Y} {n : ℕ}
  {s : (TopCat.toSSet.obj (.of Y)) _[n]}
  (hs : s ∈ (small A B).obj (.op (.mk n))) (r : R) :
  isSmallChain (R := R) (A := A) (B := B)
    (SApply R (.of Y) n
      (ModuleCat.Hom.hom
        ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R) s) r)) := by
rw [SApply_ℓ]
exact isSmallChain_realizeAffine (R := R) (A := A) (B := B) s
  (PlatonicSolids.SingularExcision.standardSubdivision n)
  (fun  $\tau$  _ => reparametrize_mem_small_of_mem_small s  $\tau$  hs) r

lemma isSmallChain_H_generator {A B : Set Y} {n : ℕ}
  {s : (TopCat.toSSet.obj (.of Y)) _[n]}
  (hs : s ∈ (small A B).obj (.op (.mk n))) (r : R) :
  isSmallChain (R := R) (A := A) (B := B)
    (HApply R (.of Y) n
      (ModuleCat.Hom.hom
        ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R) s) r)) := by
rw [HApply_ℓ]
exact isSmallChain_realizeAffine (R := R) (A := A) (B := B) s
  (PlatonicSolids.SingularExcision.standardPrism n)
  (fun  $\tau$  _ => reparametrize_mem_small_of_mem_small s  $\tau$  hs) r

lemma realizeAffineChain_comp_singularSubdivisionHom
  {X : TopCat.{0}} {p q : ℕ}
  (s : (TopCat.toSSet.obj X) _[q])
  (c : PlatonicSolids.SingularExcision.AffineChain
    (stdSimplex ℝ (Fin (q + 1))) p) :
  PlatonicSolids.SingularExcision.realizeAffineChain R s c >>
    PlatonicSolids.SingularExcision.singularSubdivisionHom R X p =
    PlatonicSolids.SingularExcision.realizeAffineChain R s
      (PlatonicSolids.SingularExcision.subdivideStd (q := q) p c) := by
induction c using Finsupp.induction with
| zero =>
  simp [PlatonicSolids.SingularExcision.realizeAffineChain_zero]
| single_add  $\tau$  z c hr hz ih =>
  simp [map_add, Preadditive.add_comp, ih,
    PlatonicSolids.SingularExcision.realizeAffineChain_single,
    Preadditive.zsmul_comp]
have hσ :
  PlatonicSolids.SingularExcision.realizeAffineSimplex R s  $\tau$  >>
    PlatonicSolids.SingularExcision.singularSubdivisionHom R X p =
    PlatonicSolids.SingularExcision.realizeAffineChain R s
      (PlatonicSolids.SingularExcision.subdivideStd (q := q) p
        (PlatonicSolids.SingularExcision.simplex  $\tau$ )) := by
rw [PlatonicSolids.SingularExcision.realizeAffineSimplex]
erw [PlatonicSolids.SingularExcision.ι_singularSubdivisionHom]
rw [PlatonicSolids.SingularExcision.singularSubdivideGenerator,
  ← PlatonicSolids.SingularExcision.realizeAffineChain_subdivideStd_simplex]
rw [hσ]
rw [show Finsupp.single  $\tau$  z = z • PlatonicSolids.SingularExcision.simplex  $\tau$ 
  by simp [PlatonicSolids.SingularExcision.simplex]]
simp [map_zsmul]

```

```

lemma  $\iota$ _singularSubdivisionIterate
  {X : TopCat.{0}} {n : ℕ}
  (s : (TopCat.toSSet.obj X)  $\iota$ [[n]]) (N : ℕ) :
  (TopCat.toSSet.obj X)  $\iota$ ChainComplex (R := R) s  $\gg$ 
    (PlatonicSolids.SingularExcision.singularSubdivisionIterate R X N).f n =
  PlatonicSolids.SingularExcision.realizeAffineChain R s
    ((PlatonicSolids.SingularExcision.subdivideStd (q := n) n)[N]
      (PlatonicSolids.SingularExcision.simplex
        (PlatonicSolids.SingularExcision.idSimplex n))) := by
  induction N with
  | zero =>
    simp [PlatonicSolids.SingularExcision.singularSubdivisionIterate_zero,
      PlatonicSolids.SingularExcision.realizeAffineChain_simplex,
      PlatonicSolids.SingularExcision.realizeAffineSimplex_id]
    exact Category.comp_id _
  | succ N ih =>
    rw [PlatonicSolids.SingularExcision.singularSubdivisionIterate_f_succ]
    erw [← Category.assoc, ih, realizeAffineChain_comp_singularSubdivisionHom]
    rw [Function.iterate_succ_apply']

lemma eventual_small_generator_of_openCover
  {n : ℕ} (s : (TopCat.toSSet.obj (.of Y))  $\iota$ [[n]])
  {A B : Set Y} (hA : IsOpen A) (hB : IsOpen B)
  (hAB : A  $\cup$  B = univ) (r : R) :
   $\exists$  N : ℕ, isSmallChain (R := R) (A := A) (B := B)
  (ModuleCat.Hom.hom
    ((PlatonicSolids.SingularExcision.singularSubdivisionIterate
      R (.of Y) N).f n)
    (ModuleCat.Hom.hom
      ((TopCat.toSSet.obj (.of Y))  $\iota$ ChainComplex (R := R) s) r)) := by
  obtain ⟨N, hN⟩ :=
    eventual_small_reparametrize_of_openCover s hA hB hAB
  refine ⟨N, ?_⟩
  have hiter :=  $\iota$ _singularSubdivisionIterate (R := R) (X := .of Y) s N
  have hcomp :
    ModuleCat.Hom.hom
      (((TopCat.toSSet.obj (.of Y))  $\iota$ ChainComplex (R := R) s)  $\gg$ 
        (PlatonicSolids.SingularExcision.singularSubdivisionIterate
          R (.of Y) N).f n) r =
    ModuleCat.Hom.hom
      ((PlatonicSolids.SingularExcision.singularSubdivisionIterate
        R (.of Y) N).f n)
      (ModuleCat.Hom.hom
        ((TopCat.toSSet.obj (.of Y))  $\iota$ ChainComplex (R := R) s) r) := by
    rw [ModuleCat.hom_comp, LinearMap.comp_apply]
    rfl
  have happ := congrArg (fun f : _  $\rightarrow$  _ => ModuleCat.Hom.hom f r) hiter
  have hval :
    ModuleCat.Hom.hom
      ((PlatonicSolids.SingularExcision.singularSubdivisionIterate
        R (.of Y) N).f n)
      (ModuleCat.Hom.hom
        ((TopCat.toSSet.obj (.of Y))  $\iota$ ChainComplex (R := R) s) r) =
    ModuleCat.Hom.hom
      (PlatonicSolids.SingularExcision.realizeAffineChain R s
        ((PlatonicSolids.SingularExcision.subdivideStd (q := n) n)[N]
          (PlatonicSolids.SingularExcision.simplex
            (PlatonicSolids.SingularExcision.idSimplex n)))) r :=
    hcomp.symm.trans happ
  rw [hval]
  exact isSmallChain_realizeAffine (R := R) (A := A) (B := B) s _ hN r

```

```

/-- Index type of singular `n`-simplices. -/
abbrev SingularSimplex (X : TopCat.{0}) (n : ℕ) : Type :=
  (TopCat.toSSet.obj X) _[n]

/-- Degree-`n` singular chains are the coproduct of copies of `R`. -/
lemma singularChain_X_eq (X : TopCat.{0}) (n : ℕ) :
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj X).X n =
    ∐ fun _ : SingularSimplex X n ↦ R :=
  rfl

/-- Small `n`-chains are the coproduct of copies of `R` indexed by small
simplices. -/
lemma smallChain_X_eq (A B : Set Y) (n : ℕ) :
  (((small A B : SSet).chainComplex R).X n) =
    ∐ fun _ : (small A B : SSet) _[n] ↦ R :=
  rfl

/-- Every singular `n`-chain is a finite linear combination of generators. -/
lemma singularChain_eq_sum {X : TopCat.{0}} {n : ℕ}
  (x : singularChainType R X n) :
  x = ∑ s ∈ ModuleCat.coprodSupport (fun _ : SingularSimplex X n ↦ R) x,
    ModuleCat.Hom.hom
      ((TopCat.toSSet.obj X).ιChainComplex (R := R) s)
      ((ModuleCat.coprodIsoDirectSum
        (fun _ : SingularSimplex X n ↦ R)).hom x s) := by
  classical
  simpa [SSet.ιChainComplex] using
    ModuleCat.eq_sum_ι_apply_coprodSupport
      (fun _ : SingularSimplex X n ↦ R) x

/-- Every small `n`-chain is a finite linear combination of small generators. -/
lemma smallChain_eq_sum {A B : Set Y} {n : ℕ}
  (y : ((small A B : SSet).chainComplex R).X n) :
  y = ∑ s ∈ ModuleCat.coprodSupport
    (fun _ : (small A B : SSet) _[n] ↦ R) y,
    ModuleCat.Hom.hom
      ((small A B : SSet).ιChainComplex (R := R) s)
      ((ModuleCat.coprodIsoDirectSum
        (fun _ : (small A B : SSet) _[n] ↦ R)).hom y s) := by
  classical
  simpa [SSet.ιChainComplex] using
    ModuleCat.eq_sum_ι_apply_coprodSupport
      (fun _ : (small A B : SSet) _[n] ↦ R) y

lemma SApply_sum {X : TopCat.{0}} {n : ℕ} {ι : Type}
  (s : Finset ι) (f : ι → singularChainType R X n) :
  SApply R X n (∑ i ∈ s, f i) = ∑ i ∈ s, SApply R X n (f i) :=
  map_sum (SApply R X n) f s

lemma HApply_sum {X : TopCat.{0}} {n : ℕ} {ι : Type}
  (s : Finset ι) (f : ι → singularChainType R X n) :
  HApply R X n (∑ i ∈ s, f i) = ∑ i ∈ s, HApply R X n (f i) :=
  map_sum (HApply R X n) f s

lemma SApply_iterate_sum {X : TopCat.{0}} {n N : ℕ} {ι : Type}
  (s : Finset ι) (f : ι → singularChainType R X n) :
  (SApply R X n)^[N] (∑ i ∈ s, f i) = ∑ i ∈ s, (SApply R X n)^[N] (f i) := by
  induction N with
  | zero => simp
  | succ N ih =>
    rw [Function.iterate_succ_apply', ih, SApply_sum]
    simp [Function.iterate_succ_apply']

```

```

lemma isSmallChain_sum {A B : Set Y} {n : ℕ} {ι : Type}
  (s : Finset ι) (f : ι → singularChainType R (.of Y) n)
  (hf : ∀ i ∈ s, isSmallChain (R := R) (A := A) (B := B) (f i)) :
  isSmallChain (R := R) (A := A) (B := B) (∑ i ∈ s, f i) := by
  classical
  revert hf
  refine Finset.induction_on s ?_ ?_
  · intro _
    simp using isSmallChain_zero (R := R) (A := A) (B := B) (n := n)
  · intro i s his ih hf
    rw [Finset.sum_insert his]
    exact isSmallChain_add (R := R)
      (hf i (Finset.mem_insert_self i s))
      (ih fun j hj => hf j (Finset.mem_insert_of_mem hj))

lemma SApply_iterate {X : TopCat.{0}} (n N : ℕ)
  (y : singularChainType R X n) :
  (SApply R X n)^[N] y =
    ModuleCat.Hom.hom
      ((PlatonicSolids.SingularExcision.singularSubdivisionIterate R X N).f n)
      y := by
  induction N with
  | zero =>
    simp [PlatonicSolids.SingularExcision.singularSubdivisionIterate_zero]
  | succ N ih =>
    rw [Function.iterate_succ_apply', ih,
      PlatonicSolids.SingularExcision.singularSubdivisionIterate_f_succ]
    rfl

lemma STotal_apply (X : TopCat.{0}) (n : ℕ)
  (x : TotalSingularChains R X) :
  STotal R X x n = SApply R X n (x n) := by
  refine DirectSum.induction_on x ?_ ?_ ?_
  · simp
  · intro m y
    rw [STotal_of]
    by_cases h : m = n
    · subst h
      simp [DirectSum.of_eq_same]
    · rw [DirectSum.of_eq_of_ne m n _ (Ne.symm h),
      DirectSum.of_eq_of_ne m n y (Ne.symm h)]
      simp
  · intro a b ha hb
    simp [map_add, ha, hb]

lemma STotal_iterate_apply (X : TopCat.{0}) (n N : ℕ)
  (x : TotalSingularChains R X) :
  ((STotal R X)^[N] x) n = (SApply R X n)^[N] (x n) := by
  induction N with
  | zero => simp
  | succ N ih =>
    rw [Function.iterate_succ_apply', STotal_apply, ih,
      Function.iterate_succ_apply']

lemma dTotal_apply (X : TopCat.{0}) (n : ℕ)
  (x : TotalSingularChains R X) :
  dTotal R X x n = dApply R X n (x (n + 1)) := by
  refine DirectSum.induction_on x ?_ ?_ ?_
  · simp
  · intro m y
    match m with
    | 0 =>
      rw [dTotal_of_zero,

```

```

      DirectSum.of_eq_of_ne 0 (n + 1) y (Nat.succ_ne_zero n)]
    simp
  | m + 1 =>
    rw [dTotal_of_succ]
    by_cases h : m = n
    · subst h
      simp [DirectSum.of_eq_same]
    · rw [DirectSum.of_eq_of_ne m n _ (Ne.symm h),
        DirectSum.of_eq_of_ne (m + 1) (n + 1) y (mt Nat.succ.inj (Ne.symm h))]
      simp
  · intro a b ha hb
    simp [map_add, ha, hb]

lemma HTotal_apply_zero (X : TopCat.{0})
  (x : TotalSingularChains R X) :
  HTotal R X x 0 = 0 := by
  refine DirectSum.induction_on x ?_ ?_ ?_
  · simp
  · intro m y
    rw [HTotal_of]
    exact DirectSum.of_eq_of_ne (m + 1) 0 _ (Nat.succ_ne_zero m).symm
  · intro a b ha hb
    simp [map_add, ha, hb]

lemma HTotal_apply_succ (X : TopCat.{0}) (n : ℕ)
  (x : TotalSingularChains R X) :
  HTotal R X x (n + 1) = HApply R X n (x n) := by
  refine DirectSum.induction_on x ?_ ?_ ?_
  · simp
  · intro m y
    rw [HTotal_of]
    by_cases h : m = n
    · subst h
      simp [DirectSum.of_eq_same]
    · rw [DirectSum.of_eq_of_ne (m + 1) (n + 1) _ (mt Nat.succ.inj (Ne.symm h)),
        DirectSum.of_eq_of_ne m n y (Ne.symm h)]
      simp
  · intro a b ha hb
    simp [map_add, ha, hb]

/-- Subdivision preserves smallness of an arbitrary small chain. -/
lemma isSmallChain_S {A B : Set Y} {n : ℕ}
  {x : singularChainType R (.of Y) n}
  (hx : isSmallChain (R := R) (A := A) (B := B) x) :
  isSmallChain (R := R) (A := A) (B := B) (SApply R (.of Y) n x) := by
  classical
  obtain ⟨y, hy⟩ := hx
  have hy_sum := smallChain_eq_sum (R := R) (A := A) (B := B) y
  have hx_sum :
    x = ∑ s ∈ ModuleCat.coprodSupport
      (fun _ : (small A B : SSet) _[n] ↦ R) y,
      ModuleCat.Hom.hom
        ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R)
          ((small A B).ι.app _ s))
        ((ModuleCat.coprodIsoDirectSum
          (fun _ : (small A B : SSet) _[n] ↦ R)).hom y s) := by
  have hmap := congrArg
    (ModuleCat.Hom.hom ((SSet.chainComplexMap (small A B).ι R).f n)) hy_sum
  rw [hy] at hmap
  rw [hmap, map_sum]
  refine Finset.sum_congr rfl fun s _ => ?_
  have hι :=
    SSet.ι_chainComplexMap_f (X := (small A B : SSet))

```



```

(Y := TopCat.toSSet.obj (.of Y)) (f := (small A B).ι) (R := R) s
have happ :=
  congrArg (fun f : _ → _ =>
    ModuleCat.Hom.hom f
    ((ModuleCat.coprodIsoDirectSum
      (fun _ : (small A B : SSet) _[n] ↦ R)).hom y s)) hι
have hcomp :
  ModuleCat.Hom.hom
    (((small A B : SSet).ιChainComplex (R := R) s) >>
      (SSet.chainComplexMap (small A B).ι R).f n)
    ((ModuleCat.coprodIsoDirectSum
      (fun _ : (small A B : SSet) _[n] ↦ R)).hom y s) =
  ModuleCat.Hom.hom ((SSet.chainComplexMap (small A B).ι R).f n)
    (ModuleCat.Hom.hom ((small A B : SSet).ιChainComplex (R := R) s)
      ((ModuleCat.coprodIsoDirectSum
        (fun _ : (small A B : SSet) _[n] ↦ R)).hom y s)) := by
    rw [ModuleCat.hom_comp, LinearMap.comp_apply]
  exact hcomp.symm.trans happ
rw [hx_sum]
refine (SApply_sum (R := R) (X := .of Y)
  (ModuleCat.coprodSupport (fun _ : (small A B : SSet) _[n] ↦ R) y)
  (fun s => ModuleCat.Hom.hom
    ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R)
      ((small A B).ι.app _ s))
    ((ModuleCat.coprodIsoDirectSum
      (fun _ : (small A B : SSet) _[n] ↦ R)).hom y s))).symm ▸ ?_
refine isSmallChain_sum (R := R) _ _ fun s _ => ?_
simp [SApply_ι] using
  isSmallChain_S_generator (R := R) (A := A) (B := B) s.property
  ((ModuleCat.coprodIsoDirectSum
    (fun _ : (small A B : SSet) _[n] ↦ R)).hom y s)

/-- The prism operator preserves smallness of an arbitrary small chain. -/
lemma isSmallChain_H {A B : Set Y} {n : ℕ}
  {x : singularChainType R (.of Y) n}
  (hx : isSmallChain (R := R) (A := A) (B := B) x) :
  isSmallChain (R := R) (A := A) (B := B) (HApply R (.of Y) n x) := by
  classical
  obtain ⟨y, hy⟩ := hx
  have hy_sum := smallChain_eq_sum (R := R) (A := A) (B := B) y
  have hx_sum :
    x = ∑ s ∈ ModuleCat.coprodSupport
      (fun _ : (small A B : SSet) _[n] ↦ R) y,
    ModuleCat.Hom.hom
      ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R)
        ((small A B).ι.app _ s))
      ((ModuleCat.coprodIsoDirectSum
        (fun _ : (small A B : SSet) _[n] ↦ R)).hom y s) := by
  have hmap := congrArg
    (ModuleCat.Hom.hom ((SSet.chainComplexMap (small A B).ι R).f n)) hy_sum
  rw [hy] at hmap
  rw [hmap, map_sum]
  refine Finset.sum_congr rfl fun s _ => ?_
  have hι :=
    SSet.ι_chainComplexMap_f (X := (small A B : SSet))
    (Y := TopCat.toSSet.obj (.of Y)) (f := (small A B).ι) (R := R) s
  have happ :=
    congrArg (fun f : _ → _ =>
      ModuleCat.Hom.hom f
      ((ModuleCat.coprodIsoDirectSum
        (fun _ : (small A B : SSet) _[n] ↦ R)).hom y s)) hι
  have hcomp :
    ModuleCat.Hom.hom

```

```

      (((small A B : SSet).ιChainComplex (R := R) s) >>
        (SSet.chainComplexMap (small A B).ι R).f n)
      ((ModuleCat.coprodIsoDirectSum
        (fun _ : (small A B : SSet) _[[n]] ↦ R)).hom y s) =
      ModuleCat.Hom.hom ((SSet.chainComplexMap (small A B).ι R).f n)
        (ModuleCat.Hom.hom ((small A B : SSet).ιChainComplex (R := R) s)
          ((ModuleCat.coprodIsoDirectSum
            (fun _ : (small A B : SSet) _[[n]] ↦ R)).hom y s))) := by
      rw [ModuleCat.hom_comp, LinearMap.comp_apply]
      exact hcomp.symm.trans happ
    rw [hx_sum]
  refine (HApply_sum (R := R) (X := .of Y)
    (ModuleCat.coprodSupport (fun _ : (small A B : SSet) _[[n]] ↦ R) y)
    (fun s => ModuleCat.Hom.hom
      ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R)
        ((small A B).ι.app _ s))
      ((ModuleCat.coprodIsoDirectSum
        (fun _ : (small A B : SSet) _[[n]] ↦ R)).hom y s))).symm ► ?_
  refine isSmallChain_sum (R := R) _ _ fun s _ => ?_
  simpa [HApply_ι] using
    isSmallChain_H_generator (R := R) (A := A) (B := B) s.property
    ((ModuleCat.coprodIsoDirectSum
      (fun _ : (small A B : SSet) _[[n]] ↦ R)).hom y s)

/-- The small-chain subgroup is invariant under `d`, `S`, and `H`. -/
lemma smallInvariant_of_openCover (A B : Set Y) :
  (subdivisionData_singular R (.of Y)).SmallInvariant
    (smallChainsSubgroup R A B) where
  d_mem := by
    intro x hx n
    change isSmallChain (R := R) (A := A) (B := B) (dTotal R (.of Y) x n)
    rw [dTotal_apply]
    exact isSmallChain_d (R := R) (hx (n + 1))
  S_mem := by
    intro x hx n
    change isSmallChain (R := R) (A := A) (B := B) (STotal R (.of Y) x n)
    rw [STotal_apply]
    exact isSmallChain_S (R := R) (hx n)
  H_mem := by
    intro x hx n
    match n with
    | 0 =>
      change isSmallChain (R := R) (A := A) (B := B)
        (HTotal R (.of Y) x 0)
      rw [HTotal_apply_zero]
      exact isSmallChain_zero (R := R) (A := A) (B := B) (n := 0)
    | n + 1 =>
      change isSmallChain (R := R) (A := A) (B := B)
        (HTotal R (.of Y) x (n + 1))
      rw [HTotal_apply_succ]
      exact isSmallChain_H (R := R) (hx n)

lemma isSmallChain_S_iterate {A B : Set Y} {n : ℕ}
  {x : singularChainType R (.of Y) n} (N : ℕ)
  (hx : isSmallChain (R := R) (A := A) (B := B) x) :
  isSmallChain (R := R) (A := A) (B := B) ((SApply R (.of Y) n)^[N] x) := by
  induction N with
  | zero => simpa
  | succ N ih =>
    rw [Function.iterate_succ_apply']
    exact isSmallChain_S (R := R) ih

lemma eventual_small_generator_iterate_of_openCover

```

```

{n : ℕ} (s : (TopCat.toSSet.obj (.of Y)) _[[n]])
{A B : Set Y} (hA : IsOpen A) (hB : IsOpen B)
(hAB : A ∪ B = univ) (r : R) :
∃ N : ℕ, ∀ N' ≥ N,
  isSmallChain (R := R) (A := A) (B := B)
    ((SApply R (.of Y) n)^[N'])
      (ModuleCat.Hom.hom
        ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R) s) r)) := by
obtain ⟨N, hN⟩ :=
  eventual_small_generator_of_openCover (R := R) s hA hB hAB r
refine ⟨N, fun N' hN' => ?_⟩
obtain ⟨k, hk⟩ := Nat.exists_eq_add_of_le hN'
rw [hk, Nat.add_comm, Function.iterate_add_apply]
exact isSmallChain_S_iterate (R := R) k (by
  simpa [SApply_iterate] using hN)

/-- Every finite singular chain becomes small after enough subdivisions. -/
lemma eventual_small_of_degree
  {n : ℕ} (x : singularChainType R (.of Y) n)
  {A B : Set Y} (hA : IsOpen A) (hB : IsOpen B)
  (hAB : A ∪ B = univ) :
  ∃ N : ℕ, isSmallChain (R := R) (A := A) (B := B)
    ((SApply R (.of Y) n)^[N] x) := by
classical
have hx := singularChain_eq_sum (R := R) x
let supp := ModuleCat.coprodSupport (fun _ : SingularSimplex (.of Y) n ↦ R) x
have hgen : ∀ s ∈ supp, ∃ N : ℕ, ∀ N' ≥ N,
  isSmallChain (R := R) (A := A) (B := B)
    ((SApply R (.of Y) n)^[N'])
      (ModuleCat.Hom.hom
        ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R) s)
        ((ModuleCat.coprodIsoDirectSum
          (fun _ : SingularSimplex (.of Y) n ↦ R)).hom x s))) :=
  fun s _ =>
    eventual_small_generator_iterate_of_openCover (R := R) s hA hB hAB _
choose! N hN using hgen
refine ⟨supp.sup N, ?_⟩
rw [hx]
refine (SApply_iterate_sum (R := R) (X := .of Y) supp
  (fun s => ModuleCat.Hom.hom
    ((TopCat.toSSet.obj (.of Y)).ιChainComplex (R := R) s)
    ((ModuleCat.coprodIsoDirectSum
      (fun _ : SingularSimplex (.of Y) n ↦ R)).hom x s)))) .symm ▸ ?_
refine isSmallChain_sum (R := R) _ _ fun s hs =>
  hN s hs _ (Finset.le_sup hs)

lemma eventual_small_of_degree_le
  {n : ℕ} (x : singularChainType R (.of Y) n)
  {A B : Set Y} (hA : IsOpen A) (hB : IsOpen B)
  (hAB : A ∪ B = univ) :
  ∃ N : ℕ, ∀ N' ≥ N, isSmallChain (R := R) (A := A) (B := B)
    ((SApply R (.of Y) n)^[N'] x) := by
obtain ⟨N, hN⟩ := eventual_small_of_degree (R := R) x hA hB hAB
refine ⟨N, fun N' hN' => ?_⟩
obtain ⟨k, hk⟩ := Nat.exists_eq_add_of_le hN'
rw [hk, Nat.add_comm, Function.iterate_add_apply]
exact isSmallChain_S_iterate (R := R) k hN

lemma eventual_small_of_chain
  (x : TotalSingularChains R (.of Y))
  {A B : Set Y} (hA : IsOpen A) (hB : IsOpen B)
  (hAB : A ∪ B = univ) :
  ∃ N : ℕ, (STotal R (.of Y))^[N] x ∈ smallChainsSubgroup R A B := by

```

```

classical
have hdeg :  $\forall n \in x.\text{support}, \exists N : \mathbb{N}, \forall N' \geq N,$ 
  isSmallChain (R := R) (A := A) (B := B)
  ((SApply R (.of Y) n)[N'] (x n)) :=
  fun n _ => eventual_small_of_degree_le (R := R) (x n) hA hB hAB
choose! N hN using hdeg
refine ⟨x.support.sup N, fun n => ?_⟩
rw [STotal_iterate_apply]
by_cases hn : n ∈ x.support
· exact hN n hn _ (Finset.le_sup hn)
· have hx0 : x n = 0 := DFinsupp.notMem_support_iff.mp hn
  simpa [hx0] using isSmallChain_zero (R := R) (A := A) (B := B) (n := n)

/-- Every homology class of `Y` has a small cycle representative, and
every small boundary is the boundary of a small chain. -/
theorem small_cycles_and_boundaries_singular
  {A B : Set Y} (hA : IsOpen A) (hB : IsOpen B)
  (hAB : A ∪ B = univ) :
  (∀ z : TotalSingularChains R (.of Y),
    (subdivisionData_singular R (.of Y)).Cycle z →
    ∃ b ∈ smallChainsSubgroup R A B,
      (subdivisionData_singular R (.of Y)).Cycle b ∧
      (subdivisionData_singular R (.of Y)).Homologous z b) ∧
  (∀ b : TotalSingularChains R (.of Y),
    b ∈ smallChainsSubgroup R A B →
    (subdivisionData_singular R (.of Y)).Cycle b →
    (∃ x, b = (subdivisionData_singular R (.of Y)).d x) →
    ∃ y ∈ smallChainsSubgroup R A B,
      b = (subdivisionData_singular R (.of Y)).d y) :=
  (subdivisionData_singular R (.of Y)).small_cycles_and_boundaries
  (smallChainsSubgroup R A B)
  (smallInvariant_of_openCover (R := R) A B)
  (fun x => eventual_small_of_chain (R := R) x hA hB hAB)

end

end SingularExcision

```

A.21 PlatonicSolids/SingularExcision/OpenCoverQI.lean

Open-cover quasi-isomorphism and IsExcisiveSubspaces (523 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Category.ModuleCat.Abelian
import Mathlib.Algebra.Category.ModuleCat.EpiMono
import Mathlib.Algebra.Homology.ConcreteCategory
import Mathlib.Algebra.Homology.QuasiIso
import Mathlib.Algebra.Homology.ShortComplex.ModuleCat
import Mathlib.CategoryTheory.Limits.MonoCoproduct
import PlatonicSolids.SingularExcision.CoverChains
import PlatonicSolids.SingularExcision.SmallChains

/-!
Open-cover excision: the small-subcomplex inclusion is a
quasi-isomorphism, so an open cover is excisive. The geometric retract
is in `SmallChains`; the triangle `coverComparison = iso.hom >> smallℓ`
identifies the cover comparison with that inclusion.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex Set

```

```

open scoped Topology

set_option linter.unusedSectionVars false

namespace SingularExcision

noncomputable section

variable {k : Type} [Ring k] (R : ModuleCat.{0} k)
variable [HasCoproducts.{0} (ModuleCat.{0} k)]
variable [HasPullbacks (ModuleCat.{0} k)]
variable {Y : Type} [TopologicalSpace Y]

open Classical

/-- Ambient singular chains of `Y`. -/
abbrev ambientChains : ChainComplex (ModuleCat.{0} k)  $\mathbb{N}$  :=
  ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj (.of Y)

/-- Small-subcomplex inclusion on singular chains. -/
def small $\iota$  (A B : Set Y) :
  (small A B : SSet).chainComplex R  $\longrightarrow$ 
  ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj (.of Y) :=
  SSet.chainComplexMap (small A B). $\iota$  R

lemma toTotal_cycle_zero (z : singularChainType R (.of Y) 0) :
  (subdivisionData_singular R (.of Y)).Cycle
  (DirectSum.of (singularChainType R (.of Y)) 0 z) :=
  dTotal_of_zero R (.of Y) z

lemma toTotal_cycle_succ {n :  $\mathbb{N}$ } {z : singularChainType R (.of Y) (n + 1)}
  (hz : dApply R (.of Y) n z = 0) :
  (subdivisionData_singular R (.of Y)).Cycle
  (DirectSum.of (singularChainType R (.of Y)) (n + 1) z) := by
  change dTotal R (.of Y) (DirectSum.of _ (n + 1) z) = 0
  rw [dTotal_of_succ, hz, map_zero]

lemma inclChains_eq_subtypeIso_comp_landsIn $\iota$  (A : Set Y) :
  ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
  (subtypeIncl Y A) =
  (subtypeChainComplexIso (C := ModuleCat.{0} k) R A).hom  $\gg$ 
  SSet.chainComplexMap (landsIn A). $\iota$  R := by
  dsimp [subtypeChainComplexIso, singularChainComplexFunctor,
  SSet.chainComplexMap, subtypeSingularSetIso]
  rw [← Functor.map_comp, subtypeToLandsIn_ $\iota$ , subtypeSingularSetMap]

lemma landsIn $\iota$ _eq_to_small $\iota$  (A B : Set Y) :
  SSet.chainComplexMap (landsIn A). $\iota$  R =
  SSet.chainComplexMap
  (SSet.Subcomplex.homOfLE (le_sup_left : landsIn A  $\leq$  small A B)) R  $\gg$ 
  small $\iota$  R A B := by
  change
  ((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map (landsIn A). $\iota$  =
  ((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map
  (SSet.Subcomplex.homOfLE (le_sup_left : landsIn A  $\leq$  small A B))  $\gg$ 
  ((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map (small A B). $\iota$ 
  rw [← Functor.map_comp]
  rfl

lemma landsIn $\iota$ _eq_to_small $\iota$ _right (A B : Set Y) :
  SSet.chainComplexMap (landsIn B). $\iota$  R =
  SSet.chainComplexMap
  (SSet.Subcomplex.homOfLE (le_sup_right : landsIn B  $\leq$  small A B)) R  $\gg$ 

```

```

      small $\iota$  R A B := by
change
  ((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map (landsIn B). $\iota$  =
    ((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map
      (SSet.Subcomplex.homOfLE (le_sup_right : landsIn B  $\leq$  small A B))  $\gg$ 
    ((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map (small A B). $\iota$ 
rw [← Functor.map_comp]
rfl

/-- The two inclusions of `A` and `B` into `Y`, after transport to
lands-in subcomplexes, factor through `small $\iota$ `. -/
lemma coverToX_eq_desc_comp_small $\iota$  (A B : Set Y) :
  coverToX (C := ModuleCat.{0} k) R
    (interInclLeft A B) (interInclRight A B)
    (subtypeIncl Y A) (subtypeIncl Y B) (subset_square_comm A B) =
  biprod.desc
    ((subtypeChainComplexIso (C := ModuleCat.{0} k) R A).hom  $\gg$ 
      SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE (le_sup_left :
          landsIn A  $\leq$  small A B)) R)
    ((subtypeChainComplexIso (C := ModuleCat.{0} k) R B).hom  $\gg$ 
      SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE (le_sup_right :
          landsIn B  $\leq$  small A B)) R)  $\gg$ 
  small $\iota$  R A B := by
rw [coverToX]
rw [inclChains_eq_subtypeIso_comp_landsIn $\iota$  (R := R) A]
rw [inclChains_eq_subtypeIso_comp_landsIn $\iota$  (R := R) B]
rw [landsIn $\iota$ _eq_to_small $\iota$  (R := R) A B]
rw [landsIn $\iota$ _eq_to_small $\iota$ _right (R := R) A B]
refine biprod.hom_ext' _ _ ?_ ?_
· rw [biprod.inl_desc]
  exact (biprod.inl_desc_assoc
    ((subtypeChainComplexIso (C := ModuleCat.{0} k) R A).hom  $\gg$ 
      SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE (le_sup_left : landsIn A  $\leq$  small A B)) R)
    ((subtypeChainComplexIso (C := ModuleCat.{0} k) R B).hom  $\gg$ 
      SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE (le_sup_right : landsIn B  $\leq$  small A B)) R)
    (small $\iota$  R A B)).symm
· rw [biprod.inr_desc]
  exact (biprod.inr_desc_assoc
    ((subtypeChainComplexIso (C := ModuleCat.{0} k) R A).hom  $\gg$ 
      SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE (le_sup_left : landsIn A  $\leq$  small A B)) R)
    ((subtypeChainComplexIso (C := ModuleCat.{0} k) R B).hom  $\gg$ 
      SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE (le_sup_right : landsIn B  $\leq$  small A B)) R)
    (small $\iota$  R A B)).symm

/-- Abbreviation for the small-pushout legs. -/
abbrev smallInl (A B : Set Y) :=
  (subtypeChainComplexIso (C := ModuleCat.{0} k) R A).hom  $\gg$ 
  SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (le_sup_left : landsIn A  $\leq$  small A B)) R

abbrev smallInr (A B : Set Y) :=
  (subtypeChainComplexIso (C := ModuleCat.{0} k) R B).hom  $\gg$ 
  SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (le_sup_right : landsIn B  $\leq$  small A B)) R

lemma mvIncl_eq_pushoutRelation (A B : Set Y) :
  mvIncl (C := ModuleCat.{0} k) R (interInclLeft A B) (interInclRight A B) =

```

```

pushoutRelation
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
    (interInclLeft A B))
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
    (interInclRight A B)) :=
rfl

lemma  $\pi\_comp\_coverChainsIsoSmall\_hom$  (A B : Set Y) :
  cokernel. $\pi$  (mvIncl (C := ModuleCat.{0} k) R
    (interInclLeft A B) (interInclRight A B))  $\gg$ 
    (coverChainsIsoSmall (C := ModuleCat.{0} k) R A B).hom =
  biprod.desc (smallInl (R := R) A B) (smallInr (R := R) A B) := by
  let f := ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
    (interInclLeft A B)
  let g := ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
    (interInclRight A B)
  let sq := subtypeChainsSmallIsPushout (C := ModuleCat.{0} k) R A B
  have hinl := sq.inl_isoIsPushout_inv _ _ (cokernelIsPushout f g)
  have hinr := sq.inr_isoIsPushout_inv _ _ (cokernelIsPushout f g)
  refine biprod.hom_ext' _ _ ?_ ?_
  · refine Eq.trans (Eq.trans ?_ hinl)
    (biprod.inl_desc (smallInl (R := R) A B)
      (smallInr (R := R) A B)).symm
  exact (Category.assoc
    (biprod.inl : _  $\rightarrow$ 
      ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj (.of  $\uparrow$ A)  $\boxplus$ 
      ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj (.of  $\uparrow$ B))
      (cokernel. $\pi$  (mvIncl (C := ModuleCat.{0} k) R
        (interInclLeft A B) (interInclRight A B)))
      (coverChainsIsoSmall (C := ModuleCat.{0} k) R A B).hom).symm
  · refine Eq.trans (Eq.trans ?_ hinr)
    (biprod.inr_desc (smallInl (R := R) A B)
      (smallInr (R := R) A B)).symm
  exact (Category.assoc
    (biprod.inr : _  $\rightarrow$ 
      ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj (.of  $\uparrow$ A)  $\boxplus$ 
      ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj (.of  $\uparrow$ B))
      (cokernel. $\pi$  (mvIncl (C := ModuleCat.{0} k) R
        (interInclLeft A B) (interInclRight A B)))
      (coverChainsIsoSmall (C := ModuleCat.{0} k) R A B).hom).symm

lemma coverComparison_eq_iso_comp_small $\iota$  (A B : Set Y) :
  coverComparison (C := ModuleCat.{0} k) R
    (interInclLeft A B) (interInclRight A B)
    (subtypeIncl Y A) (subtypeIncl Y B) (subset_square_comm A B) =
  (coverChainsIsoSmall (C := ModuleCat.{0} k) R A B).hom  $\gg$ 
    small $\iota$  R A B := by
  refine (cancel_epi (cokernel. $\pi$ 
    (mvIncl (C := ModuleCat.{0} k) R (interInclLeft A B)
      (interInclRight A B)))) $\cdot$ 1 ?_
  simp only [coverComparison, cokernel. $\pi\_desc$ ]
  rw [coverToX_eq_desc_comp_small $\iota$ ,  $\leftarrow$   $\pi\_comp\_coverChainsIsoSmall\_hom$ ]
  exact Category.assoc
    (cokernel. $\pi$  (mvIncl (C := ModuleCat.{0} k) R
      (interInclLeft A B) (interInclRight A B)))
    (coverChainsIsoSmall (C := ModuleCat.{0} k) R A B).hom
    (small $\iota$  R A B)

instance small $\iota\_mono\_f$  (A B : Set Y) (n :  $\mathbb{N}$ ) : Mono ((small $\iota$  R A B).f n) := by
  dsimp [small $\iota$ , SSet.chainComplexMap, SSet.chainComplexFunctor]
  exact MonoCoproduct.mono_map'_of_injective
    (fun _ : (TopCat.toSSet.obj (.of Y)).obj (.op (.mk n))  $\mapsto$  R)
    (fun s : (small A B).obj (.op (.mk n))  $\mapsto$  (s : _))

```

```

Subtype.val_injective

instance smallℓ_mono (A B : Set Y) : Mono (smallℓ R A B) :=
  HomologicalComplex.mono_of_mono_f _ fun n => smallℓ_mono_f (R := R) A B n

lemma smallℓ_hom_injective (A B : Set Y) (n : ℕ) :
  Function.Injective (ModuleCat.Hom.hom ((smallℓ R A B).f n)) :=
  (ModuleCat.mono_iff_injective _).1 inferInstance

lemma ambient_d_apply {n : ℕ} (z : singularChainType R (.of Y) (n + 1)) :
  ModuleCat.Hom.hom
    (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj (.of Y)).d
      (n + 1) n) z =
  dApply R (.of Y) n z :=
  rfl

lemma of_mem_smallChainsSubgroup {A B : Set Y} {n : ℕ}
  {z : singularChainType R (.of Y) n}
  (hz : isSmallChain (R := R) (A := A) (B := B) z) :
  DirectSum.of (singularChainType R (.of Y)) n z ∈ smallChainsSubgroup R A B := by
  intro m
  by_cases h : m = n
  · subst h
    simp [DirectSum.of_eq_same] using hz
  · have hm : (DirectSum.of (singularChainType R (.of Y)) n z) m = 0 :=
      DirectSum.of_eq_of_ne n m z h
    simp [hm] using isSmallChain_zero (R := R) (A := A) (B := B) (n := m)

lemma cycle_component {n : ℕ} {z : TotalSingularChains R (.of Y)}
  (hz : (subdivisionData_singular R (.of Y)).Cycle z) :
  dApply R (.of Y) n (z (n + 1)) = 0 := by
  have h : dTotal R (.of Y) z = 0 := hz
  rw [← dTotal_apply, h]
  simp

lemma homologyπ_eq_zero_iff_boundary
  (K : ChainComplex (ModuleCat.{0} k) ℕ) (n : ℕ)
  (z : K.cycles n) :
  K.homologyπ n z = 0 ↔
  ∃ x : K.X (n + 1), K.toCycles (n + 1) n x = z := by
  have hex : (ShortComplex.mk (K.toCycles (n + 1) n) (K.homologyπ n)
    (K.toCycles_comp_homologyπ (n + 1) n)).Exact :=
    ShortComplex.exact_of_g_is_cokernel _
    (K.homologyIsCokernel (i := n + 1) (j := n) (ChainComplex.prev ℕ n))
  rw [ShortComplex.moduleCat_exact_iff] at hex
  constructor
  · exact hex z
  · rintro ⟨x, rfl⟩
    exact congrArg (fun f => f x) (K.toCycles_comp_homologyπ (n + 1) n)

lemma iCycles_apply_toCycles
  (K : ChainComplex (ModuleCat.{0} k) ℕ) (n : ℕ) (x : K.X (n + 1)) :
  ModuleCat.Hom.hom (K.iCycles n) (K.toCycles (n + 1) n x) =
  ModuleCat.Hom.hom (K.d (n + 1) n) x := by
  have hcomp : K.toCycles (n + 1) n >> K.iCycles n = K.d (n + 1) n :=
  K.liftCycles_i (K.d (n + 1) n) ((ComplexShape.down ℕ).next n) rfl
  (K.d_comp_d _ _ _)
  rw [← LinearMap.comp_apply, ← ModuleCat.hom_comp, hcomp]

lemma iCycles_apply_cyclesMap (A B : Set Y) (n : ℕ)
  (za : ((small A B : SSet).chainComplex R).cycles n) :
  ModuleCat.Hom.hom ((ambientChains (R := R)).iCycles n)
  (cyclesMap (smallℓ R A B) n za) =

```



```

ModuleCat.Hom.hom ((smallℓ R A B).f n)
  (ModuleCat.Hom.hom
    (((small A B : SSet).chainComplex R).iCycles n) za) := by
have hcomp := cyclesMap_i (smallℓ R A B) n
rw [← LinearMap.comp_apply, ← ModuleCat.hom_comp, hcomp,
  ModuleCat.hom_comp, LinearMap.comp_apply]

lemma iCycles_d_apply (K : ChainComplex (ModuleCat.{0} k) ℕ) (n : ℕ)
  (z : K.cycles n) :
  ModuleCat.Hom.hom (K.d n ((ComplexShape.down ℕ).next n))
    (ModuleCat.Hom.hom (K.iCycles n) z) = 0 := by
have hcomp := K.iCycles_d n ((ComplexShape.down ℕ).next n)
rw [← LinearMap.comp_apply, ← ModuleCat.hom_comp, hcomp]
simp

lemma iCycles_hom_injective (K : ChainComplex (ModuleCat.{0} k) ℕ) (n : ℕ) :
  Function.Injective (ModuleCat.Hom.hom (K.iCycles n)) :=
  (ModuleCat.mono_iff_injective _).1 inferInstance

lemma homologous_degree {z b w : TotalSingularChains R (.of Y)} {n : ℕ}
  (h : z - b = dTotal R (.of Y) w) :
  z n - b n = dApply R (.of Y) n (w (n + 1)) := by
have hn := congrArg (fun x : TotalSingularChains R (.of Y) => x n) h
change (z - b) n = dTotal R (.of Y) w n at hn
rw [dTotal_apply] at hn
exact hn

lemma cycle_of_iCycles {n : ℕ}
  (z : ((ambientChains (R := R)).cycles n)) :
  (subdivisionData_singular R (.of Y)).Cycle
    (DirectSum.of (singularChainType R (.of Y)) n
      (ModuleCat.Hom.hom ((ambientChains (R := R)).iCycles n) z)) := by
cases n with
| zero => exact toTotal_cycle_zero (R := R) _
| succ n =>
  refine toTotal_cycle_succ (R := R) ?_
  have hz := iCycles_d_apply (ambientChains (R := R)) (n + 1) z
  rw [ChainComplex.next_nat_succ] at hz
  simpa [dApply] using hz

lemma small_cycle_of_iCycles {A B : Set Y} {n : ℕ}
  (z : ((small A B : SSet).chainComplex R).cycles n) :
  (subdivisionData_singular R (.of Y)).Cycle
    (DirectSum.of (singularChainType R (.of Y)) n
      (ModuleCat.Hom.hom ((smallℓ R A B).f n)
        (ModuleCat.Hom.hom
          (((small A B : SSet).chainComplex R).iCycles n) z))) := by
cases n with
| zero => exact toTotal_cycle_zero (R := R) _
| succ n =>
  refine toTotal_cycle_succ (R := R) ?_
  have hz := iCycles_d_apply ((small A B : SSet).chainComplex R) (n + 1) z
  rw [ChainComplex.next_nat_succ] at hz
  have hcomp := (smallℓ R A B).comm (n + 1) n
  change ModuleCat.Hom.hom
    (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj (.of Y)).d
      (n + 1) n)
    (ModuleCat.Hom.hom ((smallℓ R A B).f (n + 1))
      (ModuleCat.Hom.hom
        (((small A B : SSet).chainComplex R).iCycles (n + 1)) z)) = 0
  rw [← LinearMap.comp_apply, ← ModuleCat.hom_comp, hcomp,
    ModuleCat.hom_comp, LinearMap.comp_apply, hz, map_zero]

```

```

lemma homologyMap_apply_homologyπ (A B : Set Y) (n : ℕ)
  (za : ((small A B : SSet).chainComplex R).cycles n) :
  ModuleCat.Hom.hom (homologyMap (smallℓ R A B) n)
    (ModuleCat.Hom.hom
      (((small A B : SSet).chainComplex R).homologyπ n) za) =
  ModuleCat.Hom.hom ((ambientChains (R := R)).homologyπ n)
    (cyclesMap (smallℓ R A B) n za) := by
have h := congrArg (fun f => ModuleCat.Hom.hom f za)
  (homologyπ_naturality (smallℓ R A B) (i := n))
convert h using 1 <|> rw [ModuleCat.hom_comp, LinearMap.comp_apply]

lemma d_small_of_d_image {A B : Set Y} {n : ℕ}
  (y : ((small A B : SSet).chainComplex R).X (n + 1))
  {z : ((small A B : SSet).chainComplex R).X n}
  (h : ModuleCat.Hom.hom ((smallℓ R A B).f n)
    (((small A B : SSet).chainComplex R).d (n + 1) n y) =
  ModuleCat.Hom.hom ((smallℓ R A B).f n) z) :
  ((small A B : SSet).chainComplex R).d (n + 1) n y = z :=
smallℓ_hom_injective (R := R) A B n h

lemma smallℓ_quasiIso {A B : Set Y}
  (hA : IsOpen A) (hB : IsOpen B) (hAB : A ∪ B = univ) :
  QuasiIso (smallℓ R A B) := by
refine (quasiIso_iff (smallℓ R A B)).2 fun n =>
  (quasiIsoAt_iff_isIso_homologyMap (smallℓ R A B) n).2 ?_
refine (ConcreteCategory.isIso_iff_bijective
  (homologyMap (smallℓ R A B) n)).2 ⟨?inj, ?surj⟩
· intro a b hab
  apply sub_eq_zero.mp
  set c := a - b with hcdef
  have hc : ModuleCat.Hom.hom (homologyMap (smallℓ R A B) n) c = 0 := by
    simp [c, hab]
  obtain ⟨za, hza⟩ := (ModuleCat.epi_iff_surjective
    (((small A B : SSet).chainComplex R).homologyπ n)).1 inferInstance c
  have hbound :
    ModuleCat.Hom.hom ((ambientChains (R := R)).homologyπ n)
      (cyclesMap (smallℓ R A B) n za) = 0 := by
    rw [← homologyMap_apply_homologyπ, hza, hc]
  obtain ⟨x, hx⟩ :=
    (homologyπ_eq_zero_iff_boundary (ambientChains (R := R)) n _).1 hbound
  set zsmall :=
    ModuleCat.Hom.hom
      (((small A B : SSet).chainComplex R).iCycles n) za
  set btot := DirectSum.of (singularChainType R (.of Y)) n
    (ModuleCat.Hom.hom ((smallℓ R A B).f n) zsmall)
  have hbmemb : btot ∈ smallChainsSubgroup R A B :=
    of_mem_smallChainsSubgroup (R := R) ⟨zsmall, rfl⟩
  have hbcycle := small_cycle_of_iCycles (R := R) (A := A) (B := B) za
  have hdx :
    ModuleCat.Hom.hom ((ambientChains (R := R)).d (n + 1) n) x =
    ModuleCat.Hom.hom ((smallℓ R A B).f n) zsmall := by
  have h1 := iCycles_apply_toCycles (ambientChains (R := R)) n x
  have h2 := iCycles_apply_cyclesMap (R := R) A B n za
  rw [hx] at h1
  exact h1.symm.trans h2
have hbound : ∃ w, btot = (subdivisionData_singular R (.of Y)).d w :=
  ⟨DirectSum.of (singularChainType R (.of Y)) (n + 1) x, by
    change btot = dTotal R (.of Y)
    (DirectSum.of (singularChainType R (.of Y)) (n + 1) x)
    rw [dTotal_of_succ]
    simp [btot, dApply, hdx]⟩
obtain ⟨ytot, hymemb, hpeq⟩ :=
  (small_cycles_and_boundaries_singular (R := R) hA hB hAB).2

```

```

    btot hbmemb hbcycle hbbound
obtain ⟨ypred, hypred⟩ := hymemb (n + 1)
have hdy :
  ModuleCat.Hom.hom ((smallℓ R A B).f n)
    (((small A B : SSet).chainComplex R).d (n + 1) n ypred) =
    ModuleCat.Hom.hom ((smallℓ R A B).f n) zsmall := by
have hb_n : btot n = ModuleCat.Hom.hom ((smallℓ R A B).f n) zsmall := by
  simp [btot, DirectSum.of_eq_same]
have hdeg := congrArg (fun w : TotalSingularChains R (.of Y) => w n) hyeq
have hytot : dApply R (.of Y) n (ytot (n + 1)) =
  ModuleCat.Hom.hom ((smallℓ R A B).f n) zsmall := by
  have : btot n =
    (subdivisionData_singular R (.of Y)).d ytot n := hdeg
  rw [show (subdivisionData_singular R (.of Y)).d ytot =
    dTotal R (.of Y) ytot from rfl] at this
  rw [dTotal_apply, hb_n] at this
  exact this.symm
have hcomm := (smallℓ R A B).comm (n + 1) n
rw [← LinearMap.comp_apply, ← ModuleCat.hom_comp, ← hcomm,
  ModuleCat.hom_comp, LinearMap.comp_apply]
have hypred' :
  ModuleCat.Hom.hom ((smallℓ R A B).f (n + 1)) ypred = ytot (n + 1) :=
  hypred
rw [hypred']
exact hytot
have hza_eq :
  za = ((small A B : SSet).chainComplex R).toCycles (n + 1) n ypred := by
  refine (iCycles_hom_injective ((small A B : SSet).chainComplex R) n) ?_
  have := d_small_of_d_image (R := R) (A := A) (B := B) ypred hdy
  rw [iCycles_apply_toCycles, this]
rw [← hza, hza_eq]
exact congrArg (fun f => ModuleCat.Hom.hom f ypred)
(((small A B : SSet).chainComplex R).toCycles_comp_homologyπ (n + 1) n)
intro h
obtain ⟨zcyc, hzcyc⟩ := (ModuleCat.epi_iff_surjective
  ((ambientChains (R := R)).homologyπ n)).1 inferInstance h
set z := ModuleCat.Hom.hom ((ambientChains (R := R)).iCycles n) zcyc
have hzcycle := cycle_of_iCycles (R := R) zcyc
obtain ⟨btot, hbmemb, hbcycle, hhom⟩ :=
  (small_cycles_and_boundaries_singular (R := R) hA hB hAB).1 _ hzcycle
obtain ⟨w, hw⟩ := hhom
obtain ⟨yb, hyb⟩ := hbmemb n
have hycycle :
  ModuleCat.Hom.hom
    (((small A B : SSet).chainComplex R).d n
      ((ComplexShape.down ℕ).next n)) yb = 0 := by
  refine smallℓ_hom_injective (R := R) A B
    ((ComplexShape.down ℕ).next n) ?_
  rw [map_zero]
  cases n with
  | zero =>
    rw [ChainComplex.next_nat_zero]
    have hcomm := (smallℓ R A B).comm 0 0
    rw [← LinearMap.comp_apply, ← ModuleCat.hom_comp, ← hcomm,
      ModuleCat.hom_comp, LinearMap.comp_apply]
    simp
  | succ m =>
    rw [ChainComplex.next_nat_succ]
    have hcomm := (smallℓ R A B).comm (m + 1) m
    rw [← LinearMap.comp_apply, ← ModuleCat.hom_comp, ← hcomm,
      ModuleCat.hom_comp, LinearMap.comp_apply]
    have hyb' :
      ModuleCat.Hom.hom ((smallℓ R A B).f (m + 1)) yb = btot (m + 1) :=

```

```

      hyb
      rw [hyb']
      exact cycle_component (R := R) (n := m) hbcycle
let ycy := ((small A B : SSet).chainComplex R).cyclesMk
  (x := yb) (j := (ComplexShape.down N).next n) (hj := rfl)
  (hx := by
    change (forget₂ (ModuleCat.{0} k) Ab).map _ yb = 0
    simp using hycycle)
refine ⟨((small A B : SSet).chainComplex R).homologyπ n ycy, ?_⟩
rw [← hzy, homologyMap_apply_homologyπ]
have hzy :
  ModuleCat.Hom.hom ((ambientChains (R := R)).iCycles n)
    (cyclesMap (smallℓ R A B) n ycy) =
  ModuleCat.Hom.hom ((smallℓ R A B).f n) yb := by
have hmap := iCycles_apply_cyclesMap (R := R) A B n ycy
have hy : ModuleCat.Hom.hom
  ((small A B : SSet).chainComplex R).iCycles n ycy = yb := by
  simp using i_cyclesMk ((small A B : SSet).chainComplex R) yb
  ((ComplexShape.down N).next n) rfl (by
    change (forget₂ (ModuleCat.{0} k) Ab).map _ yb = 0
    simp using hycycle)
rw [hmap, hy]
have hex :
  (ambientChains (R := R)).toCycles (n + 1) n (w (n + 1)) =
  zcy - cyclesMap (smallℓ R A B) n ycy := by
refine (iCycles_hom_injective (ambientChains (R := R)) n) ?_
have hdeg := homologous_degree (R := R) (n := n) hw
rw [iCycles_apply_toCycles, map_sub, hzy]
have hyb' : ModuleCat.Hom.hom ((smallℓ R A B).f n) yb = btot n := hyb
rw [hyb']
simp [z, DirectSum.of_eq_same, dApply] using hdeg.symm
have hπ0 :
  (ambientChains (R := R)).homologyπ n
  ((ambientChains (R := R)).toCycles (n + 1) n (w (n + 1))) = 0 :=
  (homologyπ_eq_zero_iff_boundary (ambientChains (R := R)) n _).2
  ⟨w (n + 1), rfl⟩
have hzero :
  (ambientChains (R := R)).homologyπ n
  (zcy - cyclesMap (smallℓ R A B) n ycy) = 0 := by
  rw [← hex]
  exact hπ0
rw [map_sub] at hzero
exact (eq_of_sub_eq_zero hzero).symm

lemma IsExcisiveSubspaces_of_openCover {A B : Set Y}
  (hA : IsOpen A) (hB : IsOpen B) (hAB : A ∪ B = univ) :
  IsExcisiveSubspaces (C := ModuleCat.{0} k) R A B := by
  dsimp [IsExcisiveSubspaces, IsExcisive]
  rw [coverComparison_eq_iso_comp_smallℓ]
  haveI : QuasiIso (smallℓ R A B) := smallℓ_quasiIso (R := R) hA hB hAB
  infer_instance

lemma IsExcisiveSubspaces_puncturedSphere (n : ℕ) :
  IsExcisiveSubspaces (C := ModuleCat.{0} k) R
  (puncturedNorth n) (puncturedSouth n) :=
  IsExcisiveSubspaces_of_openCover (R := R)
  (isOpen_puncturedNorth n) (isOpen_puncturedSouth n) (punctured_cover n)

end

end SingularExcision

```

A.22 PlatonicSolids/SimplicialSingularComparison.lean

Canonical realization–singular comparison; simplex, horn, and two-vertex cases (3054 lines).

```
/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Category.ModuleCat.Abelian
import Mathlib.Algebra.Category.ModuleCat.Colimits
import Mathlib.Algebra.Category.ModuleCat.EpiMono
import Mathlib.Algebra.Homology.CommSq
import Mathlib.Algebra.Homology.HomologicalComplexAbelian
import Mathlib.Algebra.Homology.HomologicalComplexBiprod
import Mathlib.Algebra.Homology.ShortComplex.Abelian
import Mathlib.Algebra.Homology.SingleHomology
import Mathlib.Algebra.Homology.HomologySequenceLemmas
import Mathlib.Algebra.Homology.HomologicalComplexLimits
import Mathlib.AlgebraicTopology.ExtraDegeneracy
import Mathlib.AlgebraicTopology.SimplicialSet.HornColimits
import Mathlib.AlgebraicTopology.SimplicialSet.Boundary
import Mathlib.AlgebraicTopology.SimplicialSet.Dimension
import Mathlib.AlgebraicTopology.SimplicialSet.Horn
import Mathlib.AlgebraicTopology.SimplicialSet.Homology.HomologyZero
import Mathlib.AlgebraicTopology.SimplicialSet.Monoidal
import Mathlib.AlgebraicTopology.SimplicialSet.Homology.Nondegenerate
import Mathlib.AlgebraicTopology.SimplicialSet.SubcomplexColimits
import Mathlib.AlgebraicTopology.SimplicialSet.TopAdj
import Mathlib.AlgebraicTopology.SingularHomology.Basic
import Mathlib.Analysis.Convex.Contractible
import Mathlib.CategoryTheory.Adjunction.Limits
import Mathlib.CategoryTheory.Limits.Shapes.ConcreteCategory
import Mathlib.CategoryTheory.Limits.Preserves.SigmaConst
import Mathlib.Topology.Instances.AddCircle.Real
import PlatonicSolids.SingularHomology

/-!
The canonical chain map from the simplicial chains of a simplicial set to
the singular chains of its geometric realization.

The intended comparison theorem says that this map is a quasi-isomorphism
for finite nonsingular simplicial sets. This file begins by fixing the
canonical map, its normalized form, and the naturality and reduction lemmas
needed by an attachment proof.
-/

noncomputable section

open AlgebraicTopology CategoryTheory CategoryTheory.Limits HomologicalComplex Module
open scoped Simplicial Topology

variable {k : Type} [Ring k]

universe u

/-- Singular chains on the geometric realization of `X`. -/
abbrev singularChainsOfRealization (R : ModuleCat.{0} k) (X : SSet.{0}) :
  ChainComplex (ModuleCat.{0} k)  $\mathbb{N}$  :=
  ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj |X|

/-- The canonical comparison from simplicial chains to singular chains of
the geometric realization, induced by the realization--singular adjunction
unit. -/
def simplicialSingularComparison (R : ModuleCat.{0} k) (X : SSet.{0}) :
```

```

X.chainComplex R → singularChainsOfRealization R X :=
SSet.chainComplexMap (sSetTopAdj.unit.app X) R

/-- The normalized canonical comparison. -/
def normalizedSimplicialSingularComparison
  (R : ModuleCat.{0} k) (X : SSet.{0}) :
  X.normalizedChainComplex R → singularChainsOfRealization R X :=
  X.fromNormalizedChainComplex R >> simplicialSingularComparison R X

/-- On a simplex generator, the comparison is the singular simplex given by
the canonical realization of that abstract simplex. -/
@[reassoc (attr := simp)]
lemma  $\iota$ _simplicialSingularComparison
  (R : ModuleCat.{0} k) (X : SSet.{0}) {n : ℕ}
  (x : X  $\llbracket$ n $\rrbracket$ ) :
  X. $\iota$ ChainComplex x >> (simplicialSingularComparison R X).f n =
  (TopCat.toSSet.obj |X|). $\iota$ ChainComplex
  ((sSetTopAdj.unit.app X).app  $\_$  x) := by
exact SSet. $\iota$ _chainComplexMap_f X (TopCat.toSSet.obj |X|)
(sSetTopAdj.unit.app X) R x

/-- The canonical comparison is natural in the simplicial set. -/
lemma simplicialSingularComparison_naturality
  (R : ModuleCat.{0} k) {X Y : SSet.{0}} (f : X → Y) :
  SSet.chainComplexMap f R >> simplicialSingularComparison R Y =
  simplicialSingularComparison R X >>
  SSet.chainComplexMap
  (TopCat.toSSet.map (SSet.toTop.map f)) R := by
change
  ((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map f >>
  ((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map
  (sSetTopAdj.unit.app Y) =
  ((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map
  (sSetTopAdj.unit.app X) >>
  ((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map
  (TopCat.toSSet.map (SSet.toTop.map f))
rw [← Functor.map_comp, ← Functor.map_comp]
congr 1
simp

/-- The normalized comparison is a quasi-isomorphism exactly when the
unnormalized comparison is. -/
lemma normalizedSimplicialSingularComparison_quasiIso_iff
  (R : ModuleCat.{0} k) (X : SSet.{0}) :
  QuasiIso (normalizedSimplicialSingularComparison R X) ↔
  QuasiIso (simplicialSingularComparison R X) := by
exact quasiIso_iff_comp_left
(X.fromNormalizedChainComplex R) (simplicialSingularComparison R X)

/-! ## Chain-level gluing for subcomplexes -/

/-- The infimum and supremum of two subcomplexes form a pushout square
after evaluation in each simplicial degree. -/
noncomputable def subcomplexSimplexSetIsPushout
  {X : SSet.{0}} (A B : X.Subcomplex) (n : SimplexCategoryop) :
  IsPushout
  ((SSet.Subcomplex.homOfLE (inf_le_left : A  $\sqcap$  B ≤ A)).app n)
  ((SSet.Subcomplex.homOfLE (inf_le_right : A  $\sqcap$  B ≤ B)).app n)
  ((SSet.Subcomplex.homOfLE (le_sup_left : A ≤ A  $\sqcup$  B)).app n)
  ((SSet.Subcomplex.homOfLE (le_sup_right : B ≤ A  $\sqcup$  B)).app n) :=
(show SSet.Subcomplex.BicartSq (A  $\sqcap$  B) A B (A  $\sqcup$  B) from
{ sup_eq := rfl
  inf_eq := rfl }).isPushout.map

```

```

      ((evaluation SimplexCategoryop (Type)).obj n)

/-- Applying the free `R`-module functor preserves the degreewise
subcomplex pushout. -/
noncomputable def subcomplexSigmaConstIsPushout
  (R : ModuleCat.{0} k) {X : SSet.{0}}
  (A B : X.Subcomplex) (n : SimplexCategoryop) :
  IsPushout
  ((sigmaConst.obj R).map
    ((SSet.Subcomplex.homOfLE (inf_le_left : A  $\sqcap$  B  $\leq$  A)).app n))
  ((sigmaConst.obj R).map
    ((SSet.Subcomplex.homOfLE (inf_le_right : A  $\sqcap$  B  $\leq$  B)).app n))
  ((sigmaConst.obj R).map
    ((SSet.Subcomplex.homOfLE (le_sup_left : A  $\leq$  A  $\sqcup$  B)).app n))
  ((sigmaConst.obj R).map
    ((SSet.Subcomplex.homOfLE (le_sup_right : B  $\leq$  A  $\sqcup$  B)).app n)) :=
  (subcomplexSimplexSetIsPushout A B n).map (sigmaConst.obj R)

/-- In every chain degree, simplicial chains carry the union of two
subcomplexes to the pushout of their chain objects. -/
noncomputable def subcomplexChainDegreeIsPushout
  (R : ModuleCat.{0} k) {X : SSet.{0}}
  (A B : X.Subcomplex) (n :  $\mathbb{N}$ ) :
  IsPushout
  ((SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (inf_le_left : A  $\sqcap$  B  $\leq$  A)) R).f n)
  ((SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (inf_le_right : A  $\sqcap$  B  $\leq$  B)) R).f n)
  ((SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (le_sup_left : A  $\leq$  A  $\sqcup$  B)) R).f n)
  ((SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (le_sup_right : B  $\leq$  A  $\sqcup$  B)) R).f n) := by
  simpa [SSet.chainComplexMap, SSet.chainComplexFunctor,
    AlgebraicTopology.alternatingFaceMapComplex] using
    subcomplexSigmaConstIsPushout R A B (Opposite.op  $\llbracket n \rrbracket$ )

/-- Simplicial chain complexes turn a union of two subcomplexes into a
pushout square. This is the source-side gluing theorem needed by an
attachment induction. -/
noncomputable def subcomplexChainIsPushout
  (R : ModuleCat.{0} k) {X : SSet.{0}} (A B : X.Subcomplex) :
  IsPushout
  (SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (inf_le_left : A  $\sqcap$  B  $\leq$  A)) R)
  (SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (inf_le_right : A  $\sqcap$  B  $\leq$  B)) R)
  (SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (le_sup_left : A  $\leq$  A  $\sqcup$  B)) R)
  (SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE (le_sup_right : B  $\leq$  A  $\sqcup$  B)) R) := by
  refine
  { w := by
    rw [← Functor.map_comp, ← Functor.map_comp]
    rfl
    isColimit' := {?_} }
  apply HomologicalComplex.isColimitOfEval
  intro n
  exact (PushoutCocone.isColimitMapCoconeEquiv _ _).symm
    (subcomplexChainDegreeIsPushout R A B n).isColimit

/-! ### The first two-dimensional boundary attachment -/

/-- One edge of the boundary of the standard two-simplex. -/

```

```

abbrev boundaryTwoFirstFace :
  (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex :=
  SSet.stdSimplex.face ({0}^c : Finset (Fin 3))

/-- The union of the other two edges of the boundary of the standard
two-simplex. -/
abbrev boundaryTwoRemainingFaces :
  (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex :=
  SSet.stdSimplex.face ({1}^c : Finset (Fin 3))  $\sqcup$ 
  SSet.stdSimplex.face ({2}^c : Finset (Fin 3))

/-- The boundary of the two-simplex is the union of one edge and the
remaining two-edge path. -/
lemma boundary_two_eq_face_sup :
  (SSet.boundary 2 : (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) =
    boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces := by
  rw [SSet.boundary_eq_iSup]
  apply le_antisymm
  · rw [iSup_le_iff]
    intro i
    fin_cases i
    · exact le_sup_left
    · exact le_sup_left.trans le_sup_right
    · exact le_sup_right.trans le_sup_right
  · rw [← SSet.boundary_eq_iSup]
    exact sup_le
      (SSet.face_singleton_compl_le_boundary (n := 2) 0)
      (sup_le
        (SSet.face_singleton_compl_le_boundary (n := 2) 1)
        (SSet.face_singleton_compl_le_boundary (n := 2) 2))

/-- On simplicial chains, the decomposition of  $\partial\Delta[2]$  into an edge
and the other two edges is a pushout square. -/
noncomputable def boundaryTwoChainIsPushout (R : ModuleCat.{0} k) :
  IsPushout
    (SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE
        (inf_le_left :
          boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
          boundaryTwoFirstFace)) R)
    (SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE
        (inf_le_right :
          boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
          boundaryTwoRemainingFaces)) R)
    (SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE
        (le_sup_left :
          boundaryTwoFirstFace  $\leq$ 
          boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces)) R)
    (SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE
        (le_sup_right :
          boundaryTwoRemainingFaces  $\leq$ 
          boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces)) R) :=
  subcomplexChainIsPushout R
  boundaryTwoFirstFace boundaryTwoRemainingFaces

/-! ### Source facts for  $\partial\Delta[2]$  -/

/-- The simplicial set underlying the boundary of the standard two-simplex. -/
abbrev boundaryTwoSSet : SSet.{0} :=
  (SSet.boundary 2 : (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex)

```



```

instance : boundaryTwoSSet.Nonempty :=
  ⟨⟨SSet.stdSimplex.objEquiv.symm 0, by
    simp [SSet.boundary_obj_eq_univ]⟩⟩

/-- Connectedness of simplicial sets is invariant under isomorphism. -/
lemma isConnected_of_sset_iso {X Y : SSet.{0}} (e : X ≅ Y) [X.isConnected] :
  Y.isConnected := by
  rw [SSet.isConnected_iff]
  refine ⟨?, ⟨e.hom.app _ (Classical.arbitrary (X _[0]))⟩⟩
  have hsub : Subsingleton (SSet.π₀ X) :=
    ((SSet.isConnected_iff (X := X)).mp inferInstance).1
  haveI : IsIso (SSet.π₀Functor.map e.hom) := inferInstance
  have eπ : SSet.π₀ X ≅ SSet.π₀ Y :=
    (asIso (SSet.π₀Functor.map e.hom)).toEquiv
  exact @Equiv.subsingleton (SSet.π₀ Y) (SSet.π₀ X) eπ.symm hsub

/-- The union of two connected subcomplexes that share a vertex is connected. -/
lemma subcomplex_sup_isConnected {X : SSet.{0}} (A B : X.Subcomplex)
  [hA : (A : SSet.{0}).IsConnected] [hB : (B : SSet.{0}).IsConnected]
  (hAB : ((A ⊔ B : X.Subcomplex) : SSet.{0}).Nonempty) :
  ((A ⊔ B : X.Subcomplex) : SSet.{0}).IsConnected := by
  rw [SSet.isConnected_iff]
  constructor
  · constructor
    intro a b
    induction a using SSet.π₀.rec with
    | mk x =>
      induction b using SSet.π₀.rec with
      | mk y =>
        obtain ⟨z₀⟩ := hAB
        let z : ((A ⊔ B : X.Subcomplex) : SSet.{0}) _[0] :=
          ⟨z₀.val, (inf_le_left : A ⊔ B ≤ A).trans le_sup_left _ z₀.property⟩
        let inclA : (A : SSet.{0}) → (A ⊔ B : X.Subcomplex) :=
          SSet.Subcomplex.homOfLE le_sup_left
        let inclB : (B : SSet.{0}) → (A ⊔ B : X.Subcomplex) :=
          SSet.Subcomplex.homOfLE le_sup_right
        have mem_sup {s : ((A ⊔ B : X.Subcomplex) : SSet.{0}) _[0]} :
          s.val ∈ A.obj _ ∨ s.val ∈ B.obj _ := by
            rw [← Set.mem_union, ← Subfunctor.max_obj]
            exact s.property
        have eq_of_mem_left {s : ((A ⊔ B : X.Subcomplex) : SSet.{0}) _[0]}
          (hs : s.val ∈ A.obj _) :
          SSet.π₀.mk s = SSet.π₀.mk z := by
            let sA : (A : SSet.{0}) _[0] := ⟨s.val, hs⟩
            let zA : (A : SSet.{0}) _[0] :=
              ⟨z₀.val, (inf_le_left : A ⊔ B ≤ A) _ z₀.property⟩
            have hπ : SSet.π₀.mk sA = SSet.π₀.mk zA := Subsingleton.elim _ _
            have hsA : inclA.app _ sA = s := Subtype.ext rfl
            have hzA : inclA.app _ zA = z := Subtype.ext rfl
            simpa [SSet.mapπ₀_mk, hsA, hzA] using congrArg (SSet.mapπ₀ inclA) hπ
        have eq_of_mem_right {s : ((A ⊔ B : X.Subcomplex) : SSet.{0}) _[0]}
          (hs : s.val ∈ B.obj _) :
          SSet.π₀.mk s = SSet.π₀.mk z := by
            let sB : (B : SSet.{0}) _[0] := ⟨s.val, hs⟩
            let zB : (B : SSet.{0}) _[0] :=
              ⟨z₀.val, (inf_le_right : A ⊔ B ≤ B) _ z₀.property⟩
            have hπ : SSet.π₀.mk sB = SSet.π₀.mk zB := Subsingleton.elim _ _
            have hsB : inclB.app _ sB = s := Subtype.ext rfl
            have hzB : inclB.app _ zB = z := Subtype.ext rfl
            simpa [SSet.mapπ₀_mk, hsB, hzB] using congrArg (SSet.mapπ₀ inclB) hπ
        have hx := mem_sup (s := x)
        have hy := mem_sup (s := y)

```

```

cases hx with
| inl hx =>
  cases hy with
  | inl hy => exact (eq_of_mem_left hx).trans (eq_of_mem_left hy).symm
  | inr hy => exact (eq_of_mem_left hx).trans (eq_of_mem_right hy).symm
| inr hx =>
  cases hy with
  | inl hy => exact (eq_of_mem_right hx).trans (eq_of_mem_left hy).symm
  | inr hy => exact (eq_of_mem_right hx).trans (eq_of_mem_right hy).symm
· obtain ⟨z₀⟩ := hAB
  exact ⟨⟨z₀.val, (inf_le_left : A ⊔ B ≤ A).trans le_sup_left _ z₀.property⟩⟩

/-- The two remaining faces of `∂Δ[2]` are the horn `Λ[2, 0]`. -/
lemma boundaryTwoRemainingFaces_eq_horn :
  boundaryTwoRemainingFaces = Λ[2, 0] := by
  rw [SSet.horn_eq_iSup]
  refine le_antisymm ?_ ?_
· exact sup_le
  (le_iSup_of_le ⟨(1 : Fin 3), by decide⟩ le_rfl)
  (le_iSup_of_le ⟨(2 : Fin 3), by decide⟩ le_rfl)
· rw [iSup_le_iff]
  intro ⟨j, hj⟩
  fin_cases j
  · exact (hj (by decide)).elim
  · exact le_sup_left
  · exact le_sup_right

/-- The boundary of the standard two-simplex is connected. -/
instance boundary_two_isConnected : boundaryTwoSSet.IsConnected := by
  rw [SSet.isConnected_iff]
  constructor
· constructor
  intro a b
  induction a using SSet.π₀.rec with
  | mk x =>
    induction b using SSet.π₀.rec with
    | mk y =>
      let z : boundaryTwoSSet _[[0]] :=
        (SSet.stdSimplex.obj₀Equiv.symm 0, by
          simp [SSet.boundary_obj_eq_univ])
      have connected_to_z (w : boundaryTwoSSet _[[0]]) :
        SSet.π₀.mk z = SSet.π₀.mk w := by
      let s : boundaryTwoSSet _[[1]] :=
        (SSet.stdSimplex.edge 2 0
          (SSet.stdSimplex.obj₀Equiv w.val) (Fin.zero_le _), by
          simp [SSet.boundary_obj_eq_univ])
      have hsrc : boundaryTwoSSet.δ 1 s = z := by
        apply Subtype.ext
        apply SSet.stdSimplex.obj₀Equiv.injective
        rfl
      have htgt : boundaryTwoSSet.δ 0 s = w := by
        apply Subtype.ext
        apply SSet.stdSimplex.obj₀Equiv.injective
        rfl
      rw [← hsrc, ← htgt]
      exact SSet.π₀.sound (SSet.Edge.mk' s)
      exact (connected_to_z x).symm.trans (connected_to_z y)
· infer_instance

/-- Simplicial homology of `∂Δ[2]` vanishes in degrees at least two. -/
lemma isZero_boundary_two_homology_of_ge_two
  (R : ModuleCat.{0} k) (n : ℕ) (hn : 2 ≤ n) :
  IsZero (boundaryTwoSSet.homology R n) :=

```

```

SSet.isZero_homology_of_hasDimensionLT _ R n 2 hn

/-! ## The representable-simplex case -/

/-- Naturality of the canonical augmentation from simplicial homology in
degree zero to the coefficient object. -/
lemma sset_homology0ε_natural
  {C : Type u} [Category.{0} C] [HasCoproducts.{0} C]
  [Preadditive C] [CategoryWithHomology C]
  {X Y : SSet.{0}} (f : X → Y) (R : C) :
  homologyMap (SSet.chainComplexMap f R) 0 >> Y.homology0ε R =
    X.homology0ε R := by
let φ := SSet.chainComplexMap f R
let KX := X.chainComplex R
let KY := Y.chainComplex R
apply (cancel_epi (KX.homologyπ 0)).1
apply (cancel_epi (ChainComplex.cycles0Iso KX).inv).1
apply SSet.chainComplex_hom_ext
intro x
have hx :
  KX.liftCycles (X.ιChainComplex x) 0 (by simp) (by simp) =
    X.ιChainComplex x >> (ChainComplex.cycles0Iso KX).inv := by
  rw [← cancel_mono (KX.ιCycles 0)]
  simp [ChainComplex.cycles0Iso]
  exact (Category.comp_id _).symm
have hnat := HomologicalComplex.homologyπ_naturality (φ := φ) (i := 0)
have hcycle := HomologicalComplex.liftCycles_comp_cyclesMap
  (k := X.ιChainComplex x) (j := 0) (hj := by simp) (hk := by simp) φ
have hf : X.ιChainComplex x >> φ.f 0 = Y.ιChainComplex (f.app _ x) :=
  SSet.ι_chainComplexMap_f (X := X) (Y := Y) (f := f) (R := R) x
have hY := SSet.liftCycles_ιChainComplex_homologyπ_homology0ε Y R (f.app _ x)
have hX := SSet.liftCycles_ιChainComplex_homologyπ_homology0ε X R x
have lhs :
  X.ιChainComplex x >> (ChainComplex.cycles0Iso KX).inv >>
    KX.homologyπ 0 >> homologyMap φ 0 >> Y.homology0ε R =  $\mathbb{K}$  R := by
  rw [← Category.assoc, ← hx, reassoc_of% hnat, reassoc_of% hcycle]
  exact (congrArg (fun t =>
    KY.liftCycles t 0 (by simp) (by simp) >>
    KY.homologyπ 0 >> Y.homology0ε R) hf).trans hY
have rhs :
  X.ιChainComplex x >> (ChainComplex.cycles0Iso KX).inv >>
    KX.homologyπ 0 >> X.homology0ε R =  $\mathbb{K}$  R := by
  rw [← Category.assoc, ← hx]
  exact hX
exact lhs.trans rhs.symm

namespace SSet

open stdSimplex

/-- Every representable standard simplex is connected as a simplicial set. -/
instance stdSimplex_isConnected (n : ℕ) : (Δ[n] : SSet.{0}).IsConnected := by
  rw [isConnected_iff]
  constructor
  · constructor
    intro a b
    induction a using SSet.π0.rec with
    | mk x =>
      induction b using SSet.π0.rec with
      | mk y =>
        let z : Δ[n] _[0] := obj0Equiv.symm 0
        have hx : SSet.π0.mk z = SSet.π0.mk x := by
          let s := edge n 0 (obj0Equiv x) (Fin.zero_le _)

```

```

    have hsrc : ( $\Delta[n]$  : SSet.{0}). $\delta$  1 s = z := by
      apply objEquiv.injective
      rfl
    have htgt : ( $\Delta[n]$  : SSet.{0}). $\delta$  0 s = x := by
      apply objEquiv.injective
      rfl
    simp [hsrc, htgt] using SSet. $\pi_0$ .sound (SSet.Edge.mk' s)
    have hy : SSet. $\pi_0$ .mk z = SSet. $\pi_0$ .mk y := by
      let s := edge n 0 (objEquiv y) (Fin.zero_le _)
      have hsrc : ( $\Delta[n]$  : SSet.{0}). $\delta$  1 s = z := by
        apply objEquiv.injective
        rfl
      have htgt : ( $\Delta[n]$  : SSet.{0}). $\delta$  0 s = y := by
        apply objEquiv.injective
        rfl
      simp [hsrc, htgt] using SSet. $\pi_0$ .sound (SSet.Edge.mk' s)
    exact hx.symm.trans hy
  · infer_instance

/-- A vertex face of  $\Delta[n]$  is connected. -/
instance stdSimplex_faceSingleton_isConnected {n :  $\mathbb{N}$ } (i : Fin (n + 1)) :
  (stdSimplex.face ({i} : Finset (Fin (n + 1))) : SSet.{0}).IsConnected :=
  isConnected_of_sset_iso (stdSimplex.faceSingletonIso i)

/-- An edge face of  $\Delta[n]$  is connected. -/
instance stdSimplex_facePair_isConnected {n :  $\mathbb{N}$ } (i j : Fin (n + 1)) (hij : i < j) :
  (stdSimplex.face ({i, j} : Finset (Fin (n + 1))) : SSet.{0}).IsConnected :=
  isConnected_of_sset_iso (stdSimplex.facePairIso i j hij)

/-- Each complementary edge of  $\Delta[2]$  is connected. -/
instance stdSimplex_faceSingletonCompl_isConnected (i : Fin 3) :
  (stdSimplex.face ({i}^c : Finset (Fin 3)) : SSet.{0}).IsConnected :=
  isConnected_of_sset_iso (stdSimplex.faceSingletonComplIso (n := 1) i)

end SSet

/-- The two-edge horn  $\Lambda[2, 0]$  is connected. -/
instance horn_two_zero_isConnected : ( $\Lambda[2, 0]$  : SSet.{0}).IsConnected := by
  rw [← boundaryTwoRemainingFaces_eq_horn]
  refine subcomplex_sup_isConnected
    (SSet.stdSimplex.face ({1}^c : Finset (Fin 3)))
    (SSet.stdSimplex.face ({2}^c : Finset (Fin 3))) ?_
  rw [SSet.stdSimplex.face_inter_face]
  exact ⟨(SSet.stdSimplex.objEquiv.symm 0, by
    rw [SSet.stdSimplex.objEquiv.symm_mem_face_iff]
    decide)⟩

/-- Reindexing the coproduct of copies of  $R$  along an injection is monic. -/
lemma sigmaConst_map_mono_of_injective
  (R : ModuleCat.{0} k) { $\alpha$   $\beta$  : Type} (f :  $\alpha \rightarrow \beta$ ) (hf : Function.Injective f) :
  Mono ((sigmaConst.obj R).map f) := by
  let e := Equiv.ofInjective f hf
  let reindex :  $\coprod$  (fun _ :  $\alpha \mapsto R$ )  $\cong$   $\coprod$  (fun _ : Set.range f  $\mapsto R$ ) :=
    Sigma.whiskerEquiv e (fun _  $\mapsto$  Iso.refl R)
  let incl :  $\coprod$  (fun _ : Set.range f  $\mapsto R$ )  $\rightarrow$   $\coprod$  (fun _ :  $\beta \mapsto R$ ) :=
    Sigma.map' (Subtype.val : Set.range f  $\rightarrow \beta$ ) fun _  $\mapsto$   $\mathbb{K}$  R
  classical
  let retr :  $\coprod$  (fun _ :  $\beta \mapsto R$ )  $\rightarrow$   $\coprod$  (fun _ : Set.range f  $\mapsto R$ ) :=
    Sigma.desc fun b  $\mapsto$ 
      if h : b  $\in$  Set.range f then
        Sigma.ι (fun _ : Set.range f  $\mapsto R$ ) ⟨b, h⟩
      else
        0

```

```

have hretr : incl  $\gg$  retr =  $\llcorner$  _ := by
  apply colimit.hom_ext
  rintro ⟨b, hb⟩
  dsimp [incl, retr]
  rw [Sigma.ι_comp_map'_assoc, Category.id_comp, Sigma.ι_desc, dif_pos hb,
    Category.comp_id]
haveI : Mono incl := mono_of_mono_fac hretr
have hmap : (sigmaConst.obj R).map f = reindex.hom  $\gg$  incl := by
  dsimp [reindex, incl, sigmaConst]
  rw [Sigma.map'_comp_map']
  refine Sigma.map'_eq ?_ fun _  $\mapsto$  Category.id_comp _
  ext a
  rfl
rw [hmap]
exact mono_comp reindex.hom incl

/-- A monomorphism of simplicial sets remains monic on simplicial chains. -/
lemma chainComplexMap_mono_of_mono {X Y : SSet.{0}} (f : X  $\longrightarrow$  Y) [Mono f]
  (R : ModuleCat.{0} k) :
  Mono (SSet.chainComplexMap f R) := by
  refine HomologicalComplex.mono_of_mono_f _ fun n  $\mapsto$  ?_
  have : Mono (f.app (Opposite.op  $\llbracket$ n $\rrbracket$ )) := inferInstance
  have hf : Function.Injective (f.app (Opposite.op  $\llbracket$ n $\rrbracket$ )) :=
    (ConcreteCategory.mono_iff_injective_of_preservesPullback _).1 this
  change Mono ((sigmaConst.obj R).map (f.app (Opposite.op  $\llbracket$ n $\rrbracket$ )))
  exact sigmaConst_map_mono_of_injective R (f.app (Opposite.op  $\llbracket$ n $\rrbracket$ )) hf

/-- The Mayer-Vietoris short exact sequence of simplicial chains for a
union of two subcomplexes. -/
noncomputable def subcomplexChainShortComplex
  (R : ModuleCat.{0} k) {X : SSet.{0}} (A B : X.Subcomplex) :
  ShortComplex (ChainComplex (ModuleCat.{0} k)  $\mathbb{N}$ ) :=
  (subcomplexChainIsPushout R A B).shortComplex

lemma subcomplexChainShortComplex_shortExact
  (R : ModuleCat.{0} k) {X : SSet.{0}} (A B : X.Subcomplex) :
  (subcomplexChainShortComplex R A B).ShortExact := by
  let h := subcomplexChainIsPushout R A B
  let S := subcomplexChainShortComplex R A B
  let iA := SSet.Subcomplex.homOfLE (inf_le_left : A  $\sqcap$  B  $\leq$  A)
  haveI : Mono iA := inferInstance
  let iB := SSet.Subcomplex.homOfLE (inf_le_right : A  $\sqcap$  B  $\leq$  B)
  have hmonoIA : Mono (SSet.chainComplexMap iA R) :=
    chainComplexMap_mono_of_mono iA R
  have hfst : S.f  $\gg$  biprod.fst = SSet.chainComplexMap iA R :=
    biprod.lift_fst (SSet.chainComplexMap iA R) (SSet.chainComplexMap iB R)
  haveI : Mono S.f :=
    ⟨fun {Z} a b eq  $\mapsto$  by
      haveI := hmonoIA
      apply (cancel_mono (SSet.chainComplexMap iA R)).1
      simpa [← hfst, ← Category.assoc] using
        congrArg (fun k  $\Rightarrow$  k  $\gg$  biprod.fst) eq⟩
  haveI : Epi S.g := h.epi_shortComplex_g
  letI : CategoryWithHomology (ChainComplex (ModuleCat.{0} k)  $\mathbb{N}$ ) :=
    CategoryTheory.categoryWithHomology_of_abelian
  exact ShortComplex.ShortExact.mk' (S.exact_of_g_is_cokernel h.isColimitCokernelCofork)
  inferInstance inferInstance

/-- Simplicial homology of  $\Lambda[2, 0]$  vanishes in degrees at least two. -/
lemma isZero_horn_two_zero_homology_of_ge_two
  (R : ModuleCat.{0} k) (n :  $\mathbb{N}$ ) (hn : 2  $\leq$  n) :
  IsZero (( $\Lambda[2, 0]$  : SSet.{0}).homology R n) :=
  SSet.isZero_homology_of_hasDimensionLT _ R n 2 hn

```

```

/-- The realization of every representable standard simplex is contractible. -/
instance realization_stdSimplex_contractible (n : ℕ) :
  ContractibleSpace |(SSet.stdSimplex.obj [n] : SSet.{0})| := by
  letI : ContractibleSpace (_root_.stdSimplex ℝ (Fin (n + 1))) :=
    (convex_stdSimplex ℝ (Fin (n + 1))).contractibleSpace
  ⟨(_root_.stdSimplex.vertex 0).1, (_root_.stdSimplex.vertex 0).2⟩
  exact (SimplexCategory.toTopHomeo [n]).contractibleSpace

/-- A standard simplex has zero simplicial homology in every positive degree.
This is the chain-level contraction supplied by its extra degeneracy. -/
lemma isZero_stdSimplex_homology
  (R : ModuleCat.{0} k) (n i : ℕ) (hi : i ≠ 0) :
  IsZero ((SSet.stdSimplex.obj [n]).homology R i) := by
  let ed := (SSet.Augmented.StandardSimplex.extraDegeneracy [n]).map
    (sigmaConst.obj R)
  let he := ed.homotopyEquiv
  letI : QuasiIso he.hom := he.quasiIso_hom
  have hz := HomologicalComplex.isZero_single_obj_homology
    (ComplexShape.down N) 0
    (SimplicialObject.Augmented.point.obj
      (((SimplicialObject.Augmented.whiskering Type (ModuleCat.{0} k)).obj
        (sigmaConst.obj R)).obj (SSet.Augmented.stdSimplex.obj [n]))) i hi
  exact hz.of_iso (isoOfQuasiIsoAt he.hom i)

/-- An isomorphism with a standard simplex identifies positive-degree
simplicial homology. -/
lemma isZero_homology_of_stdSimplex_iso
  (R : ModuleCat.{0} k) {X : SSet.{0}} {n : ℕ}
  (e : SSet.stdSimplex.obj [n] ≅ X) (i : ℕ) (hi : i ≠ 0) :
  IsZero (X.homology R i) := by
  haveI : IsIso (SSet.chainComplexMap e.hom R) :=
    Functor.map_isIso _ _
  exact (isZero_stdSimplex_homology R n i hi).of_iso
    (asIso (homologyMap (SSet.chainComplexMap e.hom R) i)).symm

/-- The two remaining faces of `∂Δ[2]` meet at the vertex `0`. -/
lemma boundaryTwoRemainingFaces_inf :
  SSet.stdSimplex.face ({1}^c : Finset (Fin 3)) ⊓
  SSet.stdSimplex.face ({2}^c : Finset (Fin 3)) =
  SSet.stdSimplex.face ({0} : Finset (Fin 3)) := by
  rw [SSet.stdSimplex.face_inter_face]
  congr 1

/-- The last edge of `∂Δ[2]` meets the two-edge horn at the two endpoints. -/
lemma boundaryTwoFirstFace_inf :
  boundaryTwoFirstFace ⊓ boundaryTwoRemainingFaces =
  SSet.stdSimplex.face ({1} : Finset (Fin 3)) ⊓
  SSet.stdSimplex.face ({2} : Finset (Fin 3)) := by
  unfold boundaryTwoFirstFace boundaryTwoRemainingFaces
  rw [inf_comm, Subfunctor.max_min, inf_comm,
    inf_comm (a := SSet.stdSimplex.face ({2}^c : Finset (Fin 3)))]
  rw [SSet.stdSimplex.face_inter_face, SSet.stdSimplex.face_inter_face]
  have h2 : ({0}^c ⊓ {1}^c : Finset (Fin 3)) = {2} := by decide
  have h1 : ({0}^c ⊓ {2}^c : Finset (Fin 3)) = {1} := by decide
  rw [h2, h1, sup_comm]

/-- The two endpoints of the last edge are disjoint as faces. -/
lemma face_one_inf_face_two :
  SSet.stdSimplex.face ({1} : Finset (Fin 3)) ⊓
  SSet.stdSimplex.face ({2} : Finset (Fin 3)) = ⊥ := by
  rw [SSet.stdSimplex.face_inter_face]
  have : ({1} ⊓ {2} : Finset (Fin 3)) = ∅ := by decide

```

```

rw [this, SSet.stdSimplex.face_empty]

/-- The two endpoints form a 0-dimensional subcomplex. -/
instance hasDimensionLT_one_twoVertices :
  SSet.HasDimensionLT
  ((SSet.stdSimplex.face ({1} : Finset (Fin 3))  $\sqcup$ 
    SSet.stdSimplex.face ({2} : Finset (Fin 3)) :
    (SSet.stdSimplex.obj  $\llbracket$ 2 $\rrbracket$  : SSet.{0}).Subcomplex) : SSet.{0}) 1 := by
let A : Bool  $\rightarrow$  (SSet.stdSimplex.obj  $\llbracket$ 2 $\rrbracket$  : SSet.{0}).Subcomplex :=
  fun b => if b then SSet.stdSimplex.face ({1} : Finset (Fin 3))
    else SSet.stdSimplex.face ({2} : Finset (Fin 3))
have hA :  $\bigcup$  b, A b =
  SSet.stdSimplex.face ({1} : Finset (Fin 3))  $\sqcup$ 
  SSet.stdSimplex.face ({2} : Finset (Fin 3)) := by
  apply le_antisymm
  · rw [iSup_le_iff]
    intro b
    cases b <;> simp [A]
  · exact sup_le (le_iSup A true) (le_iSup A false)
rw [ $\leftarrow$  hA]
exact (SSet.hasDimensionLT_iSup_iff A 1).2 fun b => by
  cases b
  · exact SSet.stdSimplex.hasDimensionLT_face _ 1 (by decide)
  · exact SSet.stdSimplex.hasDimensionLT_face _ 1 (by decide)

/-- The last-edge  $\cap$  horn intersection has vanishing positive simplicial homology. -/
lemma isZero_boundaryTwoFirstFace_inf_homology
  (R : ModuleCat.{0} k) (n :  $\mathbb{N}$ ) (hn : n  $\neq$  0) :
  IsZero
  (((boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces :
    (SSet.stdSimplex.obj  $\llbracket$ 2 $\rrbracket$  : SSet.{0}).Subcomplex) : SSet.{0}).homology R n) := by
  rw [boundaryTwoFirstFace_inf]
  exact SSet.isZero_homology_of_hasDimensionLT _ R n 1 (by omega)

/-- As simplicial sets,  $\partial\Delta[2]$  is the union of the last edge and the horn. -/
lemma boundaryTwoSSet_eq_sup :
  boundaryTwoSSet =
    ((boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces :
      (SSet.stdSimplex.obj  $\llbracket$ 2 $\rrbracket$  : SSet.{0}).Subcomplex) : SSet.{0}) :=
  congrArg
    (fun C : (SSet.stdSimplex.obj  $\llbracket$ 2 $\rrbracket$  : SSet.{0}).Subcomplex => (C : SSet.{0}))
    boundary_two_eq_face_sup

/-- The last-edge decomposition of  $\partial\Delta[2]$  is a pushout of simplicial sets. -/
lemma boundaryTwo_isPushout :
  IsPushout
    (SSet.Subcomplex.homOfLE
      (inf_le_left :
        boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
        boundaryTwoFirstFace))
    (SSet.Subcomplex.homOfLE
      (inf_le_right :
        boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
        boundaryTwoRemainingFaces))
    (SSet.Subcomplex.homOfLE
      (le_sup_left :
        boundaryTwoFirstFace  $\leq$ 
        boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces))
    (SSet.Subcomplex.homOfLE
      (le_sup_right :
        boundaryTwoRemainingFaces  $\leq$ 
        boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces)) :=
  SSet.Subcomplex.BicartSq.isPushout

```

```

    (⟨rfl, rfl⟩ : SSet.Subcomplex.BicartSq
      (boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces)
      boundaryTwoFirstFace boundaryTwoRemainingFaces
      (boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces))

/-- Realization preserves the last-edge pushout of  $\partial\Delta[2]$ . -/
lemma toTop_boundaryTwo_isPushout :
  IsPushout
    (SSet.toTop.map
      (SSet.Subcomplex.homOfLE
        (inf_le_left :
          boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
            boundaryTwoFirstFace)))
    (SSet.toTop.map
      (SSet.Subcomplex.homOfLE
        (inf_le_right :
          boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
            boundaryTwoRemainingFaces)))
    (SSet.toTop.map
      (SSet.Subcomplex.homOfLE
        (le_sup_left :
          boundaryTwoFirstFace  $\leq$ 
            boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces)))
    (SSet.toTop.map
      (SSet.Subcomplex.homOfLE
        (le_sup_right :
          boundaryTwoRemainingFaces  $\leq$ 
            boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces)))) :=
  boundaryTwo_isPushout.map SSet.toTop

/-- Mayer-Vietoris short exact sequence of simplicial chains for the
last-edge decomposition of  $\partial\Delta[2]$ . -/
lemma boundaryTwoChainShortExact (R : ModuleCat.{0} k) :
  (subcomplexChainShortComplex R boundaryTwoFirstFace
    boundaryTwoRemainingFaces).ShortExact :=
  subcomplexChainShortComplex_shortExact R _ _

/-- A map into a zero object is an epimorphism. -/
lemma epi_of_isZero_target {C : Type*} [Category* C] [HasZeroMorphisms C]
  {X Y : C} (f : X  $\longrightarrow$  Y) (hY : IsZero Y) : Epi f :=
  ⟨fun _ _ => hY.eq_of_src _ _⟩

/-- Homology in a fixed degree preserves binary biproducts of chain complexes. -/
instance preservesBinaryBiproduct_sset_homologyFunctor (n :  $\mathbb{N}$ )
  (K L : ChainComplex (ModuleCat.{0} k)  $\mathbb{N}$ ) :
  PreservesBinaryBiproduct K L
  (homologyFunctor (ModuleCat.{0} k) (ComplexShape.down  $\mathbb{N}$ ) n) :=
  preservesBinaryBiproduct_of_preservesBiproduct _ K L

/-- The biproduct of two chain complexes with vanishing homology in degree `n`
again has vanishing homology in that degree. -/
lemma isZero_homology_sset_biprod
  {K L : ChainComplex (ModuleCat.{0} k)  $\mathbb{N}$ } {n :  $\mathbb{N}$ }
  (hK : IsZero (K.homology n)) (hL : IsZero (L.homology n)) :
  IsZero ((K  $\boxplus$  L).homology n) := by
  rw [IsZero.iff_id_eq_zero,  $\leftarrow$  homologyMap_id (K  $\boxplus$  L) n,  $\leftarrow$  biprod.total]
  rw [homologyMap_add, homologyMap_comp, homologyMap_comp]
  have hf : homologyMap (biprod.fst : K  $\boxplus$  L  $\longrightarrow$  K) n = 0 := hK.eq_of_tgt _ _
  have hs : homologyMap (biprod.snd : K  $\boxplus$  L  $\longrightarrow$  L) n = 0 := hL.eq_of_tgt _ _
  simp [hf, hs]

/--  $(1, -1) : X \longrightarrow X \boxplus X$  is a monomorphism. -/
lemma mono_biprod_lift_id_neg (X : ModuleCat.{0} k) :

```



```

Mono (biprod.lift ( $\mathbb{K}$  X) ( $-\mathbb{K}$  X)) :=
⟨fun {Z} a b eq  $\mapsto$  by
  simpa using congrArg (fun t  $\Rightarrow$  t  $\gg$  biprod.fst) eq⟩

/-- Complementary edges of  $\Delta[2]$  have vanishing positive simplicial homology. -/
lemma isZero_boundaryTwoFace_homology
  (R : ModuleCat.{0} k) (i : Fin 3) (n :  $\mathbb{N}$ ) (hn : n  $\neq$  0) :
  IsZero
  ((SSet.stdSimplex.face ({i}c : Finset (Fin 3)) : SSet.{0}).homology R n) :=
  isZero_homology_of_stdSimplex_iso R
  (SSet.stdSimplex.faceSingletonComplIso (n := 1) i) n hn

/-- The shared vertex of the two remaining faces has vanishing positive
simplicial homology. -/
lemma isZero_boundaryTwoRemainingFaces_inf_homology
  (R : ModuleCat.{0} k) (n :  $\mathbb{N}$ ) (hn : n  $\neq$  0) :
  IsZero
  (((SSet.stdSimplex.face ({1}c : Finset (Fin 3))  $\sqcap$ 
    SSet.stdSimplex.face ({2}c : Finset (Fin 3)) :
    (SSet.stdSimplex.obj  $\llbracket$ 2 $\rrbracket$  : SSet.{0}).Subcomplex) :
    SSet.{0}).homology R n) := by
  rw [boundaryTwoRemainingFaces_inf]
  exact isZero_homology_of_stdSimplex_iso R
  (SSet.stdSimplex.faceSingletonIso 0) n hn

/-- The Mayer-Vietoris inclusion on  $H_0$  for a union of connected subcomplexes
is monic: after augmentation it is  $(1, -1) : R \rightarrow R \oplus R$ . -/
lemma mono_homologyMap_subcomplex_incl_zero
  (R : ModuleCat.{0} k) {X : SSet.{0}} (A B : X.Subcomplex)
  [((A  $\sqcap$  B : X.Subcomplex) : SSet.{0}).IsConnected]
  [(A : SSet.{0}).IsConnected] [(B : SSet.{0}).IsConnected] :
  Mono (homologyMap (subcomplexChainShortComplex R A B).f 0) := by
  let S := subcomplexChainShortComplex R A B
  let iA := SSet.Subcomplex.homOfLE (inf_le_left : A  $\sqcap$  B  $\leq$  A)
  let iB := SSet.Subcomplex.homOfLE (inf_le_right : A  $\sqcap$  B  $\leq$  B)
  let  $\varepsilon_A$  := (A : SSet.{0}).homology0 $\varepsilon$  R
  let  $\varepsilon_B$  := (B : SSet.{0}).homology0 $\varepsilon$  R
  let  $\varepsilon_W$  := ((A  $\sqcap$  B : X.Subcomplex) : SSet.{0}).homology0 $\varepsilon$  R
  let F := homologyFunctor (ModuleCat.{0} k) (ComplexShape.down  $\mathbb{N}$ ) 0
  let e := F.mapBiprod ((A : SSet.{0}).chainComplex R) ((B : SSet.{0}).chainComplex R)
  have hnata := sset_homology0 $\varepsilon$ _natural iA R
  have hnataB := sset_homology0 $\varepsilon$ _natural iB R
  have hlift :
    homologyMap S.f 0  $\gg$  e.hom  $\gg$  biprod.map  $\varepsilon_A$   $\varepsilon_B$  =
     $\varepsilon_W$   $\gg$  biprod.lift ( $\mathbb{K}$  R) ( $-\mathbb{K}$  R) := by
  change F.map
    (biprod.lift (SSet.chainComplexMap iA R) ( $-\mathbb{K}$  SSet.chainComplexMap iB R))  $\gg$ 
    (F.mapBiprod ((A : SSet.{0}).chainComplex R)
      ((B : SSet.{0}).chainComplex R)).hom  $\gg$ 
    biprod.map  $\varepsilon_A$   $\varepsilon_B$  =
     $\varepsilon_W$   $\gg$  biprod.lift ( $\mathbb{K}$  R) ( $-\mathbb{K}$  R)
  rw [← Category.assoc, biprod.map_lift_mapBiprod, F.map_neg]
  have hfst : ( $\varepsilon_W$   $\gg$  biprod.lift ( $\mathbb{K}$  R) ( $-\mathbb{K}$  R))  $\gg$  biprod.fst =  $\varepsilon_W$  := by
  rw [Category.assoc, biprod.lift_fst, Category.comp_id]
  have hsnd : ( $\varepsilon_W$   $\gg$  biprod.lift ( $\mathbb{K}$  R) ( $-\mathbb{K}$  R))  $\gg$  biprod.snd =  $-\varepsilon_W$  := by
  rw [Category.assoc, biprod.lift_snd, Preadditive.comp_neg, Category.comp_id]
  apply biprod.hom_ext
  · simp [biprod.map_fst]
    exact hnata.trans hfst.symm
  · simp [biprod.map_snd, Preadditive.neg_comp]
    exact (congrArg Neg.neg hnataB).trans hsnd.symm
  refine ⟨fun {Z} a b eq  $\mapsto$  ?_⟩
  haveI : IsIso  $\varepsilon_W$  := inferInstance

```

```

haveI : Mono (biprod.lift ( $\mathbb{K}$  R) ( $-\mathbb{K}$  R)) := mono_biprod_lift_id_neg R
haveI : Mono ( $\varepsilon W \gg$  biprod.lift ( $\mathbb{K}$  R) ( $-\mathbb{K}$  R)) := inferInstance
apply (cancel_mono ( $\varepsilon W \gg$  biprod.lift ( $\mathbb{K}$  R) ( $-\mathbb{K}$  R))).1
simpa [← hlift, ← Category.assoc, S] using
  congrArg (fun t => t  $\gg$  e.hom  $\gg$  biprod.map  $\varepsilon A \varepsilon B$ ) eq

/--  $H_1(\Lambda[2, 0]) = 0$ : the two-edge horn is a tree. -/
lemma isZero_horn_two_zero_homology_one (R : ModuleCat.{0} k) :
  IsZero (( $\Lambda[2, 0]$  : SSet.{0}).homology R 1) := by
  rw [← boundaryTwoRemainingFaces_eq_horn]
  let A := SSet.stdSimplex.face ({1}c : Finset (Fin 3))
  let B := SSet.stdSimplex.face ({2}c : Finset (Fin 3))
  let S := subcomplexChainShortComplex R A B
  have hS := subcomplexChainShortComplex_shortExact R A B
  have hA := isZero_boundaryTwoFace_homology R 1 1 (by decide)
  have hB := isZero_boundaryTwoFace_homology R 2 1 (by decide)
  have hInf := isZero_boundaryTwoRemainingFaces_inf_homology R 1 (by decide)
  have hX2 : IsZero (S.X2.homology 1) :=
    isZero_homology_sset_biprod hA hB
  haveI : ((A  $\sqcap$  B : (SSet.stdSimplex.obj  $\llbracket 2 \rrbracket$  : SSet.{0}).Subcomplex) :
    SSet.{0}).IsConnected := by
    rw [boundaryTwoRemainingFaces_inf]
    infer_instance
  have hmono0 : Mono (homologyMap S.f 0) :=
    mono_homologyMap_subcomplex_incl_zero R A B
  have hX3 : S.X3.ExactAt 1 :=
    hS.exactAt_X3 (i := 1) (h1 := epi_of_isZero_target _ hX2)
    (h2 := fun j hij => by
      have : j = 0 := by
        simp [ComplexShape.down_Rel] at hij
        exact hij
      subst j
      exact hmono0)
  rwa [exactAt_iff_isZero_homology] at hX3

/-- Simplicial homology of  $\Lambda[2, 0]$  vanishes in every positive degree. -/
lemma isZero_horn_two_zero_homology
  (R : ModuleCat.{0} k) (n : ℕ) (hn : n ≠ 0) :
  IsZero (( $\Lambda[2, 0]$  : SSet.{0}).homology R n) := by
  by_cases h2 : 2 ≤ n
  · exact isZero_horn_two_zero_homology_of_ge_two R n h2
  · have : n = 1 := by omega
    subst n
    exact isZero_horn_two_zero_homology_one R

/-- The face inclusion  $\delta_1 : \Delta[0] \rightarrow \Delta[1]$  realises as the endpoint  $0$  of  $I$ . -/
lemma toTop_map_δ_one_comp_toTopObjIsoI :
  SSet.toTop.map (SSet.stdSimplex.δ (1 : Fin 2))  $\gg$ 
  SSet.stdSimplex.toTopObjIsoI.hom =
  TopCat.const (0 : TopCat.I.{0}) := by
  apply (sSetTopAdj.homEquiv ( $\Delta[0]$  : SSet.{0}) TopCat.I.{0}).injective
  rw [Adjunction.homEquiv_naturality_left]
  change SSet.stdSimplex.δ (1 : Fin 2)  $\gg$  SSet.stdSimplex.toSSetObjI = _
  rw [SSet.stdSimplex.δ_one_toSSetObjI, sSetTopAdj_homEquiv_stdSimplex_zero]
  simp [TopCat.const_apply]

/-- The face inclusion  $\delta_0 : \Delta[0] \rightarrow \Delta[1]$  realises as the endpoint  $1$  of  $I$ . -/
lemma toTop_map_δ_zero_comp_toTopObjIsoI :
  SSet.toTop.map (SSet.stdSimplex.δ (0 : Fin 2))  $\gg$ 
  SSet.stdSimplex.toTopObjIsoI.hom =
  TopCat.const (1 : TopCat.I.{0}) := by
  apply (sSetTopAdj.homEquiv ( $\Delta[0]$  : SSet.{0}) TopCat.I.{0}).injective
  rw [Adjunction.homEquiv_naturality_left]

```



```

lemma hornIcc_inl_comp_δ :
  SSet.toTop.map (SSet.stdSimplex.δ (1 : Fin 2)) >> hornIcc_inl =
    SSet.toTop.map (SSet.stdSimplex.δ (1 : Fin 2)) >> hornIcc_inr := by
  rw [hornIcc_inl_δ_one, hornIcc_inr_δ_one]

/-- The comparison  $|\Lambda[2, 0]| \rightarrow [-1, 1]$  induced by the two endpoint-glued intervals. -/
noncomputable def horn_two_zero_toIcc :
  |( $\Lambda[2, 0]$  : SSet.{0})|  $\rightarrow$  hornIcc :=
  toTop_horn20_isPushout.desc hornIcc_inl hornIcc_inr hornIcc_inl_comp_δ

lemma toTopObjIsoI_inv_zero :
  SSet.stdSimplex.toTopObjIsoI.inv (0 : TopCat.I.{0}) =
    SSet.toTop.map (SSet.stdSimplex.δ (1 : Fin 2))
      (default : |( $\Delta[0]$  : SSet.{0})|) := by
  have h : SSet.stdSimplex.toTopObjIsoI.hom
    (SSet.toTop.map (SSet.stdSimplex.δ (1 : Fin 2)) default) = 0 := by
    simpa using congrArg (fun f :  $\_ \rightarrow$  TopCat.I.{0} => f default)
    toTop_map_δ_one_comp_toTopObjIsoI
  simpa using (congrArg SSet.stdSimplex.toTopObjIsoI.inv h).symm

lemma horn_two_zero_glue_zero :
  SSet.toTop.map SSet.horn20.ℓ01
    (SSet.stdSimplex.toTopObjIsoI.inv (0 : TopCat.I.{0})) =
    SSet.toTop.map SSet.horn20.ℓ02
      (SSet.stdSimplex.toTopObjIsoI.inv (0 : TopCat.I.{0})) := by
  rw [toTopObjIsoI_inv_zero]
  have hw := congrArg (fun f => f (default : |( $\Delta[0]$  : SSet.{0})|))
    toTop_horn20_isPushout.w
  simpa using hw

/-- The inverse  $[-1, 1] \rightarrow |\Lambda[2, 0]|$ , sending the two halves back to the two edges. -/
noncomputable def hornIcc_to_horn_two_zero (t : Set.Icc (-1 : ℝ) 1) :
  |( $\Lambda[2, 0]$  : SSet.{0})| :=
  if (0 : ℝ) ≤ t.1 then
    SSet.toTop.map SSet.horn20.ℓ01
      (SSet.stdSimplex.toTopObjIsoI.inv (Set.projIcc (0 : ℝ) 1 zero_le_one t.1))
  else
    SSet.toTop.map SSet.horn20.ℓ02
      (SSet.stdSimplex.toTopObjIsoI.inv (Set.projIcc (0 : ℝ) 1 zero_le_one (-t.1)))

lemma continuous_hornIcc_to_horn_two_zero :
  Continuous hornIcc_to_horn_two_zero := by
  refine Continuous.if_le
    (f' := fun t : Set.Icc (-1 : ℝ) 1 =>
      SSet.toTop.map SSet.horn20.ℓ01
        (SSet.stdSimplex.toTopObjIsoI.inv (Set.projIcc (0 : ℝ) 1 zero_le_one t.1)))
    (g' := fun t : Set.Icc (-1 : ℝ) 1 =>
      SSet.toTop.map SSet.horn20.ℓ02
        (SSet.stdSimplex.toTopObjIsoI.inv (Set.projIcc (0 : ℝ) 1 zero_le_one (-t.1))))
    (f := fun  $\_$  : Set.Icc (-1 : ℝ) 1 => (0 : ℝ))
    (g := fun t => t.1) ?_ ?_ (by fun_prop) continuous_subtype_val ?_
  · fun_prop
  · fun_prop
  · intro t ht
    have ht0 : t.1 = 0 := ht.symm
    have h0 : Set.projIcc (0 : ℝ) 1 zero_le_one t.1 = ⟨0, by norm_num⟩ := by
      rw [ht0, Set.projIcc_left]
    have h0' : Set.projIcc (0 : ℝ) 1 zero_le_one (-t.1) = ⟨0, by norm_num⟩ := by
      rw [ht0, neg_zero, Set.projIcc_left]
    change SSet.toTop.map SSet.horn20.ℓ01
      (SSet.stdSimplex.toTopObjIsoI.inv (Set.projIcc (0 : ℝ) 1 zero_le_one t.1)) =
      SSet.toTop.map SSet.horn20.ℓ02
        (SSet.stdSimplex.toTopObjIsoI.inv (Set.projIcc (0 : ℝ) 1 zero_le_one (-t.1)))

```

```

    rw [h0, h0']
    exact horn_two_zero_glue_zero

noncomputable def hornIcc_from :
  hornIcc  $\longrightarrow$   $|\Delta[2, 0] : \text{SSet}\{0\}| :=$ 
  TopCat.ofHom (hornIcc_to_horn_two_zero, continuous_hornIcc_to_horn_two_zero)

lemma toTopObjIsoI_inv_homeomorph
  (x :  $|\Delta[1] : \text{SSet}\{0\}|$ ) :
  SSet.stdSimplex.toTopObjIsoI.inv
    (TopCat.I.homeomorph.symm
      (TopCat.I.homeomorph (SSet.stdSimplex.toTopObjIsoI.hom x))) = x := by
  rw [Homeomorph.symm_apply_apply]
  exact congrArg (fun f :  $\_ \longrightarrow |\Delta[1] : \text{SSet}\{0\}| \Rightarrow f\ x$ )
    SSet.stdSimplex.toTopObjIsoI.hom_inv_id

lemma hornIcc_from_inl (x :  $|\Delta[1] : \text{SSet}\{0\}|$ ) :
  hornIcc_from (hornIcc_inl x) = SSet.toTop.map SSet.horn20. $\iota_{01}$  x := by
  let t := TopCat.I.homeomorph (SSet.stdSimplex.toTopObjIsoI.hom x)
  have ht :  $(0 : \mathbb{R}) \leq (\text{hornIcc\_inl } x).1 := t.2.1$ 
  have hproj : Set.projIcc  $(0 : \mathbb{R})$  1 zero_le_one (hornIcc_inl x).1 = t :=
    Set.projIcc_of_mem _ t.2
  change hornIcc_to_horn_two_zero (hornIcc_inl x) = SSet.toTop.map SSet.horn20. $\iota_{01}$  x
  rw [hornIcc_to_horn_two_zero, if_pos ht, hproj]
  exact congrArg (SSet.toTop.map SSet.horn20. $\iota_{01}$ ) (toTopObjIsoI_inv_homeomorph x)

lemma hornIcc_from_inr (x :  $|\Delta[1] : \text{SSet}\{0\}|$ ) :
  hornIcc_from (hornIcc_inr x) = SSet.toTop.map SSet.horn20. $\iota_{02}$  x := by
  let t := TopCat.I.homeomorph (SSet.stdSimplex.toTopObjIsoI.hom x)
  change hornIcc_to_horn_two_zero (hornIcc_inr x) = SSet.toTop.map SSet.horn20. $\iota_{02}$  x
  by_cases h0 :  $(t : \mathbb{R}) = 0$ 
  · have ht :  $(0 : \mathbb{R}) \leq (\text{hornIcc\_inr } x).1 := by
      change  $(0 : \mathbb{R}) \leq -(t : \mathbb{R})$ 
      simp [h0]
    have hproj0 : Set.projIcc  $(0 : \mathbb{R})$  1 zero_le_one (hornIcc_inr x).1 =
       $\langle 0, \text{by norm\_num} \rangle := by
      change Set.projIcc  $(0 : \mathbb{R})$  1 zero_le_one  $-(t : \mathbb{R}) = \_$ 
      simp [h0, Set.projIcc_left]
    have hx0 : SSet.stdSimplex.toTopObjIsoI.hom x = 0 :=
      TopCat.I.homeomorph.injective (by
        simp [TopCat.I.homeomorph_zero] using h0)
    rw [hornIcc_to_horn_two_zero, if_pos ht, hproj0]
    have hx := toTopObjIsoI_inv_homeomorph x
    rw [hx0] at hx
    have hx' : SSet.stdSimplex.toTopObjIsoI.inv  $(0 : \text{TopCat.I}\{0\}) = x := by
      simp [TopCat.I.homeomorph_zero] using hx
    have hI0 :  $\langle \langle 0, \text{by norm\_num} \rangle : \text{unitInterval} \rangle : \text{TopCat.I}\{0\} = 0 :=$ 
      ULift.ext _ (Subtype.ext rfl)
    rw [hI0]
    exact horn_two_zero_glue_zero.trans
      (congrArg (SSet.toTop.map SSet.horn20. $\iota_{02}$ ) hx')
  · have hneg :  $\neg (0 : \mathbb{R}) \leq (\text{hornIcc\_inr } x).1 := by
      change  $\neg (0 : \mathbb{R}) \leq -(t : \mathbb{R})$ 
      have :  $0 < (t : \mathbb{R}) := \text{lt\_of\_le\_of\_ne } t.2.1 (\text{Ne.symm } h0)$ 
      exact not_le.mpr (neg_neg_iff_pos.mpr this)
    have hproj : Set.projIcc  $(0 : \mathbb{R})$  1 zero_le_one  $-(\text{hornIcc\_inr } x).1 = t := by
      change Set.projIcc  $(0 : \mathbb{R})$  1 zero_le_one  $-(- (t : \mathbb{R})) = t$ 
      rw [neg_neg]
      exact Set.projIcc_of_mem _ t.2
    rw [hornIcc_to_horn_two_zero, if_neg hneg, hproj]
    exact congrArg (SSet.toTop.map SSet.horn20. $\iota_{02}$ ) (toTopObjIsoI_inv_homeomorph x)

lemma horn_two_zero_toIcc_from :$$$$$ 
```

```

    horn_two_zero_toIcc >> hornIcc_from =  $\mathbb{K}$  _ := by
  apply toTop_horn20_isPushout.hom_ext
  · have h := toTop_horn20_isPushout.inl_desc hornIcc_inl hornIcc_inr
    hornIcc_inl_comp_δ
    change (SSet.toTop.map SSet.horn20.ℓ01 >> horn_two_zero_toIcc) >> hornIcc_from =
      SSet.toTop.map SSet.horn20.ℓ01 >>  $\mathbb{K}$  _
    rw [horn_two_zero_toIcc, h, Category.comp_id]
  ext x
  exact hornIcc_from_inl x
  · have h := toTop_horn20_isPushout.inr_desc hornIcc_inl hornIcc_inr
    hornIcc_inl_comp_δ
    change (SSet.toTop.map SSet.horn20.ℓ02 >> horn_two_zero_toIcc) >> hornIcc_from =
      SSet.toTop.map SSet.horn20.ℓ02 >>  $\mathbb{K}$  _
    rw [horn_two_zero_toIcc, h, Category.comp_id]
  ext x
  exact hornIcc_from_inr x

lemma horn_two_zero_toIcc_inl (x : |( $\Delta$ [1] : SSet.{0})|) :
  horn_two_zero_toIcc (SSet.toTop.map SSet.horn20.ℓ01 x) = hornIcc_inl x := by
  have h := toTop_horn20_isPushout.inl_desc hornIcc_inl hornIcc_inr
  hornIcc_inl_comp_δ
  exact congrArg (fun f : _ → hornIcc => f x) (by
    change SSet.toTop.map SSet.horn20.ℓ01 >> horn_two_zero_toIcc = hornIcc_inl
    rwa [horn_two_zero_toIcc])

lemma horn_two_zero_toIcc_inr (x : |( $\Delta$ [1] : SSet.{0})|) :
  horn_two_zero_toIcc (SSet.toTop.map SSet.horn20.ℓ02 x) = hornIcc_inr x := by
  have h := toTop_horn20_isPushout.inr_desc hornIcc_inl hornIcc_inr
  hornIcc_inl_comp_δ
  exact congrArg (fun f : _ → hornIcc => f x) (by
    change SSet.toTop.map SSet.horn20.ℓ02 >> horn_two_zero_toIcc = hornIcc_inr
    rwa [horn_two_zero_toIcc])

lemma toTopObjIsoI_hom_inv (y : TopCat.I.{0}) :
  SSet.stdSimplex.toTopObjIsoI.hom (SSet.stdSimplex.toTopObjIsoI.inv y) = y :=
  congrArg (fun f : _ → TopCat.I.{0} => f y)
  SSet.stdSimplex.toTopObjIsoI.inv_hom_id

lemma hornIcc_from_toIcc :
  hornIcc_from >> horn_two_zero_toIcc =  $\mathbb{K}$  _ := by
  ext t
  change (horn_two_zero_toIcc (hornIcc_to_horn_two_zero t)).1 = t.1
  by_cases ht : (0 : ℝ) ≤ t.1
  · have hproj : Set.projIcc (0 : ℝ) 1 zero_le_one t.1 = ⟨t.1, ht, t.2.2⟩ :=
    Set.projIcc_of_mem _ ⟨ht, t.2.2⟩
    rw [hornIcc_to_horn_two_zero, if_pos ht, horn_two_zero_toIcc_inl, hproj]
    simp [hornIcc_inl]
    rfl
  · have hneg : ¬ (0 : ℝ) ≤ t.1 := ht
    have htmem : (-t.1) ∈ Set.Icc (0 : ℝ) 1 :=
      ⟨le_of_lt (neg_pos.mpr (not_le.mp hneg)),
        (neg_le_neg t.2.1).trans (by norm_num)⟩
    have hproj : Set.projIcc (0 : ℝ) 1 zero_le_one (-t.1) = ⟨-t.1, htmem⟩ :=
      Set.projIcc_of_mem _ htmem
    rw [hornIcc_to_horn_two_zero, if_neg hneg, horn_two_zero_toIcc_inr, hproj]
    simp [hornIcc_inr, TopCat.I.homeomorph]
    exact neg_neg t.1

/-- Geometric realization of the two-edge horn is homeomorphic to  $[-1, 1]$ . -/
noncomputable def homeomorph_horn_two_zero_Icc :
  |( $\Delta$ [2, 0] : SSet.{0})|  $\simeq_t$  Set.Icc (-1 : ℝ) 1 :=
  TopCat.homeoOfIso
  { hom := horn_two_zero_toIcc

```

```

    inv := hornIcc_from
    hom_inv_id := horn_two_zero_toIcc_from
    inv_hom_id := hornIcc_from_toIcc }

/-- The realization of  $\Delta[2, 0]$  is contractible: it is two intervals glued
at a common endpoint, hence homeomorphic to a closed interval. -/
instance realization_horn_two_zero_contractible :
  ContractibleSpace  $(\Delta[2, 0] : \text{SSet}\{0\}) :=$  by
  let I : ContractibleSpace (Set.Icc  $(-1 : \mathbb{R})$  1) :=
    (convex_Icc  $(-1 : \mathbb{R})$  1).contractibleSpace (0, by norm_num)
  exact homeomorph_horn_two_zero_Icc.contractibleSpace

/-- The canonical adjunction-unit chain map is a quasi-isomorphism on every
representable simplex. -/
instance simplicialSingularComparison_stdSimplex_quasiIso
  (R : ModuleCat. $\{0\}$  k) (n :  $\mathbb{N}$ ) :
  QuasiIso (simplicialSingularComparison R (SSet.stdSimplex.obj  $\llbracket n \rrbracket$ )) := by
  rw [quasiIso_iff]
  intro i
  rw [quasiIsoAt_iff_isIso_homologyMap]
  by_cases hi : i = 0
  · subst i
    let unit := sSetTopAdj.unit.app (SSet.stdSimplex.obj  $\llbracket n \rrbracket$ )
    have hnata := sset_homology $_{0 \in}$ _natural unit R
    change IsIso (homologyMap (SSet.chainComplexMap unit R) 0)
    let I : IsIso (((SSet.toTop  $\ggg$  TopCat.toSSet).obj
      (SSet.stdSimplex.obj  $\llbracket n \rrbracket$ )).homology $_{0 \in}$  R) := by
      change IsIso ((TopCat.toSSet.obj |SSet.stdSimplex.obj  $\llbracket n \rrbracket$  |).homology $_{0 \in}$  R)
    infer_instance
    have I : IsIso (homologyMap (SSet.chainComplexMap unit R) 0  $\ggg$ 
      ((SSet.toTop  $\ggg$  TopCat.toSSet).obj
        (SSet.stdSimplex.obj  $\llbracket n \rrbracket$ )).homology $_{0 \in}$  R) := by
      rw [hnata]
      change IsIso ((SSet.stdSimplex.obj  $\llbracket n \rrbracket$ )).homology $_{0 \in}$  R
    infer_instance
    exact IsIso.of_isIso_comp_right _
      (((SSet.toTop  $\ggg$  TopCat.toSSet).obj
        (SSet.stdSimplex.obj  $\llbracket n \rrbracket$ )).homology $_{0 \in}$  R)
  · apply IsZero.isIso
    (isZero_stdSimplex_homology R n i hi)
    exact isZero_singularHomology_of_contractible R hi

/-- If  $X$  is connected,  $|X|$  is contractible, and the simplicial homology
of  $X$  vanishes in positive degree, then the canonical comparison is a
quasi-isomorphism. -/
lemma simplicialSingularComparison_quasiIso_of_contractible
  (R : ModuleCat. $\{0\}$  k) (X : SSet. $\{0\}$ )
  [X.IsConnected] [ContractibleSpace  $|X|$ ]
  (hpos :  $\forall n, n \neq 0 \rightarrow \text{IsZero } (X.\text{homology } R \ n)$ ) :
  QuasiIso (simplicialSingularComparison R X) := by
  rw [quasiIso_iff]
  intro i
  rw [quasiIsoAt_iff_isIso_homologyMap]
  by_cases hi : i = 0
  · subst i
    let unit := sSetTopAdj.unit.app X
    have hnata := sset_homology $_{0 \in}$ _natural unit R
    change IsIso (homologyMap (SSet.chainComplexMap unit R) 0)
    let I : IsIso (((SSet.toTop  $\ggg$  TopCat.toSSet).obj X).homology $_{0 \in}$  R) := by
      change IsIso ((TopCat.toSSet.obj  $|X|$ ).homology $_{0 \in}$  R)
    infer_instance
    have I : IsIso (homologyMap (SSet.chainComplexMap unit R) 0  $\ggg$ 
      ((SSet.toTop  $\ggg$  TopCat.toSSet).obj X).homology $_{0 \in}$  R) := by

```

```

    rw [hnat]
    change IsIso (X.homology0 ∈ R)
    infer_instance
    exact IsIso.of_isIso_comp_right _
      (((SSet.toTop >>> TopCat.toSSet).obj X).homology0 ∈ R)
  · apply IsZero.isIso (hpos i hi)
    exact isZero_singularHomology_of_contractible R hi

/-- The canonical comparison is a quasi-isomorphism on the two-edge horn. -/
instance simplicialSingularComparison_horn_two_zero_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso (simplicialSingularComparison R (Λ[2, 0] : SSet.{0})) :=
  simplicialSingularComparison_quasiIso_of_contractible R _
  (isZero_horn_two_zero_homology R)

instance normalizedSimplicialSingularComparison_horn_two_zero_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso
    (normalizedSimplicialSingularComparison R (Λ[2, 0] : SSet.{0})) :=
  (normalizedSimplicialSingularComparison_quasiIso_iff R _).2 inferInstance

/-- The normalized comparison is consequently a quasi-isomorphism on every
representable simplex. -/
instance normalizedSimplicialSingularComparison_stdSimplex_quasiIso
  (R : ModuleCat.{0} k) (n : ℕ) :
  QuasiIso
    (normalizedSimplicialSingularComparison R (SSet.stdSimplex.obj [n])) :=
  (normalizedSimplicialSingularComparison_quasiIso_iff R _).2 inferInstance

/-- The comparison is a quasi-isomorphism for a simplicial set isomorphic
to one on which it is already a quasi-isomorphism. -/
lemma simplicialSingularComparison_quasiIso_of_iso
  (R : ModuleCat.{0} k) {X Y : SSet.{0}} (f : X → Y) [IsIso f]
  [QuasiIso (simplicialSingularComparison R X)] :
  QuasiIso (simplicialSingularComparison R Y) := by
  let F := (SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R
  haveI : IsIso (SSet.toTop.map f) := inferInstance
  haveI : IsIso (TopCat.toSSet.map (SSet.toTop.map f)) := inferInstance
  refine quasiIso_of_arrow_mk_iso
    (simplicialSingularComparison R X)
    (simplicialSingularComparison R Y)
    (Arrow.isoMk (F.mapIso (asIso f))
      (F.mapIso (asIso (TopCat.toSSet.map (SSet.toTop.map f)))) ?_)
  simpa [F, SSet.chainComplexMap] using
    simplicialSingularComparison_naturality R f

/-- Each closed face of  $\Delta[2]$  is isomorphic to  $\Delta[1]$ , so the comparison
is a quasi-isomorphism on every edge of  $\partial\Delta[2]$ . -/
instance simplicialSingularComparison_boundaryTwoFace_quasiIso
  (R : ModuleCat.{0} k) (i : Fin 3) :
  QuasiIso
    (simplicialSingularComparison R
      (SSet.stdSimplex.face ({i}^c : Finset (Fin 3)) : SSet.{0})) :=
  simplicialSingularComparison_quasiIso_of_iso R
    (SSet.stdSimplex.faceSingletonComplIso (n := 1) i).hom

/-- The two remaining faces are the horn, so the comparison is a
quasi-isomorphism there as well. -/
instance simplicialSingularComparison_boundaryTwoRemaining_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso
    (simplicialSingularComparison R
      (boundaryTwoRemainingFaces : SSet.{0})) := by

```



```

rw [boundaryTwoRemainingFaces_eq_horn]
infer_instance

/-- Each vertex face of  $\Delta[n]$  is isomorphic to  $\Delta[0]$ , so the comparison
is a quasi-isomorphism there. -/
instance simplicialSingularComparison_faceSingleton_quasiIso
  (R : ModuleCat.{0} k) {n : ℕ} (i : Fin (n + 1)) :
  QuasiIso
  (simplicialSingularComparison R
    (SSet.stdSimplex.face ({i} : Finset (Fin (n + 1))) : SSet.{0})) :=
  simplicialSingularComparison_quasiIso_of_iso R
  (SSet.stdSimplex.faceSingletonIso i).hom

/-- The two endpoints of the last edge of  $\partial\Delta[2]$ . -/
abbrev boundaryTwoVertices :
  (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex :=
  SSet.stdSimplex.face ({1} : Finset (Fin 3)) ⊔
  SSet.stdSimplex.face ({2} : Finset (Fin 3))

lemma face_one_le_boundaryTwoFirstFace :
  SSet.stdSimplex.face ({1} : Finset (Fin 3)) ≤ boundaryTwoFirstFace := by
  rw [SSet.stdSimplex.face_le_face_iff]
  decide

lemma face_two_le_boundaryTwoFirstFace :
  SSet.stdSimplex.face ({2} : Finset (Fin 3)) ≤ boundaryTwoFirstFace := by
  rw [SSet.stdSimplex.face_le_face_iff]
  decide

/-- The inclusion of a vertex face into  $\Delta[n]$  is the constant simplex at
that vertex. -/
lemma faceSingletonIso_hom_comp_ι {n : ℕ} (i : Fin (n + 1)) :
  (SSet.stdSimplex.faceSingletonIso i).hom >>
  (SSet.stdSimplex.face ({i} : Finset (Fin (n + 1)))) .ι =
  SSet.const (SSet.stdSimplex.objEquiv.symm i) := by
  apply (SSet.yonedaEquiv (X := SSet.stdSimplex.obj [n]) (n := [0])).injective
  rw [SSet.yonedaEquiv_comp, SSet.yonedaEquiv_const]
  set x := SSet.yonedaEquiv (SSet.stdSimplex.faceSingletonIso i).hom
  have hx : x.val = SSet.stdSimplex.objEquiv.symm i := by
    apply SSet.stdSimplex.objEquiv.injective
    have mem := x.property
    rw [SSet.stdSimplex.mem_face_iff] at mem
    have : x.val 0 = i := by
      simpa using mem 0
    simpa [SSet.stdSimplex.objEquiv] using this
  exact hx

/--  $\delta_1 \gg \delta_0 : \Delta[0] \rightarrow \Delta[2]$  is the vertex  $`1`$ . -/
lemma δ_one_comp_δ_zero_eq_const_one :
  SSet.stdSimplex.δ (1 : Fin 2) >> SSet.stdSimplex.δ (0 : Fin 3) =
  SSet.const (SSet.stdSimplex.objEquiv.symm 1) := by
  rw [SSet.stdSimplex.δ_one_eq_const, SSet.const_comp]
  congr 1

/--  $\delta_0 \gg \delta_0 : \Delta[0] \rightarrow \Delta[2]$  is the vertex  $`2`$ . -/
lemma δ_zero_comp_δ_zero_eq_const_two :
  SSet.stdSimplex.δ (0 : Fin 2) >> SSet.stdSimplex.δ (0 : Fin 3) =
  SSet.const (SSet.stdSimplex.objEquiv.symm 2) := by
  rw [SSet.stdSimplex.δ_zero_eq_const, SSet.const_comp]
  congr 1

/--  $\delta_0 \gg \delta_2 : \Delta[0] \rightarrow \Delta[2]$  is the vertex  $`1`$ . -/
lemma δ_zero_comp_δ_two_eq_const_one :

```

```

    SSet.stdSimplex.δ (0 : Fin 2) >> SSet.stdSimplex.δ (2 : Fin 3) =
      SSet.const (SSet.stdSimplex.obj0Equiv.symm 1) := by
rw [SSet.stdSimplex.δ_zero_eq_const, SSet.const_comp]
congr 1

/-- `δ₀ >> δ₁ : Δ[0] → Δ[2]` is the vertex `2`. -/
lemma δ_zero_comp_δ_one_eq_const_two :
  SSet.stdSimplex.δ (0 : Fin 2) >> SSet.stdSimplex.δ (1 : Fin 3) =
    SSet.const (SSet.stdSimplex.obj0Equiv.symm 2) := by
rw [SSet.stdSimplex.δ_zero_eq_const, SSet.const_comp]
congr 1

/-- Vertex 1 of the last edge is the `δ₁` endpoint of `Δ[1]`. -/
lemma lastEdge_vertex_one :
  (SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
    SSet.Subcomplex.homOfLE face_one_le_boundaryTwoFirstFace =
  SSet.stdSimplex.δ (1 : Fin 2) >>
    (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).hom := by
rw [← cancel_mono (boundaryTwoFirstFace.ι)]
have hι :
  SSet.Subcomplex.homOfLE face_one_le_boundaryTwoFirstFace >>
    boundaryTwoFirstFace.ι =
  (SSet.stdSimplex.face ({1} : Finset (Fin 3))).ι := rfl
rw [Category.assoc, hι, Category.assoc,
  SSet.stdSimplex.faceSingletonComplIso_hom_ι,
  faceSingletonIso_hom_comp_ι, δ_one_comp_δ_zero_eq_const_one]

/-- Vertex 2 of the last edge is the `δ₀` endpoint of `Δ[1]`. -/
lemma lastEdge_vertex_two :
  (SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
    SSet.Subcomplex.homOfLE face_two_le_boundaryTwoFirstFace =
  SSet.stdSimplex.δ (0 : Fin 2) >>
    (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).hom := by
rw [← cancel_mono (boundaryTwoFirstFace.ι)]
have hι :
  SSet.Subcomplex.homOfLE face_two_le_boundaryTwoFirstFace >>
    boundaryTwoFirstFace.ι =
  (SSet.stdSimplex.face ({2} : Finset (Fin 3))).ι := rfl
rw [Category.assoc, hι, Category.assoc,
  SSet.stdSimplex.faceSingletonComplIso_hom_ι,
  faceSingletonIso_hom_comp_ι, δ_zero_comp_δ_zero_eq_const_two]

/-- The free endpoint of the first horn edge realises as `+1` on `[-1, 1]`. -/
lemma hornIcc_inl_δ_zero :
  SSet.toTop.map (SSet.stdSimplex.δ (0 : Fin 2)) >> hornIcc_inl =
    TopCat.const ((1, by norm_num) : Set.Icc (-1 : ℝ) 1) := by
ext x
have hx : SSet.stdSimplex.toTopObjIsoI.hom
  (SSet.toTop.map (SSet.stdSimplex.δ (0 : Fin 2)) x) = 1 := by
  simpa using congrArg (fun f : _ → TopCat.I.{0} => f x)
  toTop_map_δ_zero_comp_toTopObjIsoI
simp [hornIcc_inl, hx, TopCat.I.homeomorph_one]
rfl

/-- The free endpoint of the second horn edge realises as `-1` on `[-1, 1]`. -/
lemma hornIcc_inr_δ_zero :
  SSet.toTop.map (SSet.stdSimplex.δ (0 : Fin 2)) >> hornIcc_inr =
    TopCat.const ((-1, by norm_num) : Set.Icc (-1 : ℝ) 1) := by
ext x
have hx : SSet.stdSimplex.toTopObjIsoI.hom
  (SSet.toTop.map (SSet.stdSimplex.δ (0 : Fin 2)) x) = 1 := by
  simpa using congrArg (fun f : _ → TopCat.I.{0} => f x)
  toTop_map_δ_zero_comp_toTopObjIsoI

```

```

simp [hornIcc_inr, hx, TopCat.I.homeomorph_one]
rfl

/-- Vertex 1 of the horn realises as `+1` on `[-1, 1]`. -/
lemma horn_two_zero_toIcc_δ_zero_inl :
  SSet.toTop.map (SSet.stdSimplex.δ (0 : Fin 2)) >>
    SSet.toTop.map SSet.horn20.ℓ01 >> horn_two_zero_toIcc =
    TopCat.const (⟨1, by norm_num⟩ : Set.Icc (-1 : ℝ) 1) := by
  have h := toTop_horn20_isPushout.inl_desc hornIcc_inl hornIcc_inr
  hornIcc_inl_comp_δ
  have h' : SSet.toTop.map SSet.horn20.ℓ01 >> horn_two_zero_toIcc = hornIcc_inl := by
    change SSet.toTop.map SSet.horn20.ℓ01 >> horn_two_zero_toIcc = hornIcc_inl
    rwa [horn_two_zero_toIcc]
  rw [h', hornIcc_inl_δ_zero]

/-- Vertex 2 of the horn realises as `-1` on `[-1, 1]`. -/
lemma horn_two_zero_toIcc_δ_zero_inr :
  SSet.toTop.map (SSet.stdSimplex.δ (0 : Fin 2)) >>
    SSet.toTop.map SSet.horn20.ℓ02 >> horn_two_zero_toIcc =
    TopCat.const (⟨-1, by norm_num⟩ : Set.Icc (-1 : ℝ) 1) := by
  have h := toTop_horn20_isPushout.inr_desc hornIcc_inl hornIcc_inr
  hornIcc_inl_comp_δ
  have h' : SSet.toTop.map SSet.horn20.ℓ02 >> horn_two_zero_toIcc = hornIcc_inr := by
    change SSet.toTop.map SSet.horn20.ℓ02 >> horn_two_zero_toIcc = hornIcc_inr
    rwa [horn_two_zero_toIcc]
  rw [h', hornIcc_inr_δ_zero]

/-- Realization of the last edge of `∂Δ[2]` is the unit interval. -/
noncomputable def lastEdgeIsoI :
  |(boundaryTwoFirstFace : SSet.{0})| ≅ TopCat.I.{0} :=
  (SSet.toTop.mapIso
    (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).symm).trans
    SSet.stdSimplex.toTopObjIsoI

/-- Vertex 1 of the last edge realises as `0 ∈ I`. -/
lemma lastEdgeIsoI_vertex_one :
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
      SSet.Subcomplex.homOfLE face_one_le_boundaryTwoFirstFace) >>
    lastEdgeIsoI.hom =
  TopCat.const (0 : TopCat.I.{0}) := by
  rw [lastEdge_vertex_one, lastEdgeIsoI, Iso.trans_hom, SSet.toTop.map_comp]
  change SSet.toTop.map (SSet.stdSimplex.δ (1 : Fin 2)) >>
    SSet.toTop.map
      (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).hom >>
      SSet.toTop.map
        (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).inv >>
        SSet.stdSimplex.toTopObjIsoI.hom =
    TopCat.const (0 : TopCat.I.{0})
  have hcancel :
    SSet.toTop.map
      (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).hom >>
      SSet.toTop.map
        (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).inv =
    1 := by
    rw [← SSet.toTop.map_comp, Iso.hom_inv_id, SSet.toTop.map_id]
  conv_lhs =>
    enter [2]
    rw [← Category.assoc, hcancel, Category.id_comp]
  rw [toTop_map_δ_one_comp_toTopObjIsoI]

/-- Vertex 2 of the last edge realises as `1 ∈ I`. -/
lemma lastEdgeIsoI_vertex_two :

```

```

SSet.toTop.map
  ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
    SSet.Subcomplex.homOfLE face_two_le_boundaryTwoFirstFace) >>
  lastEdgeIsoI.hom =
  TopCat.const (1 : TopCat.I.{0}) := by
rw [lastEdge_vertex_two, lastEdgeIsoI, Iso.trans_hom, SSet.toTop.map_comp]
change SSet.toTop.map (SSet.stdSimplex.δ (0 : Fin 2)) >>
  SSet.toTop.map
    (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).hom >>
  SSet.toTop.map
    (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).inv >>
  SSet.stdSimplex.toTopObjIsoI.hom =
  TopCat.const (1 : TopCat.I.{0})
have hcancel :
  SSet.toTop.map
    (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).hom >>
  SSet.toTop.map
    (SSet.stdSimplex.faceSingletonComplIso (n := 1) (0 : Fin 3)).inv =
  ⋈ _ := by
  rw [← SSet.toTop.map_comp, Iso.hom_inv_id, SSet.toTop.map_id]
conv_lhs =>
  enter [2]
  rw [← Category.assoc, hcancel, Category.id_comp]
  rw [toTop_map_δ_zero_comp_toTopObjIsoI]

/-! ### The circle obtained from the last-edge decomposition -/

/-- A concrete circle of circumference three. -/
abbrev boundaryCircle : TopCat.{0} :=
  TopCat.of (AddCircle (3 : ℝ))

/-- Map the horn coordinate  $[-1, 1]$  to the arc  $[0, 2]$  of the circle. -/
def hornIccToBoundaryCircle : hornIcc → boundaryCircle :=
  TopCat.ofHom
    { toFun := fun t => ((t.1 + 1 : ℝ) : AddCircle (3 : ℝ))
      continuous_toFun :=
        (AddCircle.continuous_mk' (3 : ℝ)).comp
          (continuous_subtype_val.add continuous_const) }

/-- Map the last-edge coordinate  $I$  to the complementary arc  $[2, 3]$ . -/
def unitIntervalToBoundaryCircle : TopCat.I.{0} → boundaryCircle :=
  TopCat.ofHom
    { toFun := fun t =>
        ((2 + (TopCat.I.homeomorph t : ℝ) : ℝ) : AddCircle (3 : ℝ))
      continuous_toFun :=
        (AddCircle.continuous_mk' (3 : ℝ)).comp
          (continuous_const.add
            (continuous_subtype_val.comp TopCat.I.homeomorph.continuous)) }

/-- The horn as the first closed arc of the concrete circle. -/
noncomputable def hornToBoundaryCircle :
  |(Λ[2, 0] : SSet.{0})| → boundaryCircle :=
  horn_two_zero_toIcc >> hornIccToBoundaryCircle

/-- The last edge as the complementary closed arc of the concrete circle. -/
noncomputable def lastEdgeToBoundaryCircle :
  |(boundaryTwoFirstFace : SSet.{0})| → boundaryCircle :=
  lastEdgeIsoI.hom >> unitIntervalToBoundaryCircle

lemma hornIccToBoundaryCircle_one :
  (TopCat.const (⟨1, by norm_num⟩ : Set.Icc (-1 : ℝ) 1) :
    |(Δ[0] : SSet.{0})| → hornIcc) >>
  hornIccToBoundaryCircle =

```

```

    (TopCat.const ((2 : ℝ) : AddCircle (3 : ℝ)) :
      |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  boundaryCircle) := by
  ext x
  change (((1 : ℝ) + 1 : ℝ) : AddCircle (3 : ℝ)) = (2 : ℝ)
  norm_num

lemma hornIccToBoundaryCircle_neg_one :
  (TopCat.const (<-1, by norm_num) : Set.Icc (-1 : ℝ) 1) :
    |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  hornIcc  $\gg$ 
    hornIccToBoundaryCircle =
  (TopCat.const (0 : AddCircle (3 : ℝ)) :
    |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  boundaryCircle) := by
  ext x
  change (((-1 : ℝ) + 1 : ℝ) : AddCircle (3 : ℝ)) = 0
  norm_num

lemma unitIntervalToBoundaryCircle_zero :
  (TopCat.const (0 : TopCat.I.{0}) :
    |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  TopCat.I.{0})  $\gg$  unitIntervalToBoundaryCircle =
  (TopCat.const ((2 : ℝ) : AddCircle (3 : ℝ)) :
    |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  boundaryCircle) := by
  ext x
  change
    ((2 + (TopCat.I.homeomorph (0 : TopCat.I.{0}) : ℝ) : ℝ) :
      AddCircle (3 : ℝ)) = (2 : ℝ)
  simp

lemma unitIntervalToBoundaryCircle_one :
  (TopCat.const (1 : TopCat.I.{0}) :
    |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  TopCat.I.{0})  $\gg$  unitIntervalToBoundaryCircle =
  (TopCat.const (0 : AddCircle (3 : ℝ)) :
    |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  boundaryCircle) := by
  ext x
  change
    ((2 + (TopCat.I.homeomorph (1 : TopCat.I.{0}) : ℝ) : ℝ) :
      AddCircle (3 : ℝ)) = 0
  rw [TopCat.I.homeomorph_one]
  norm_num

lemma face_one_le_boundaryTwoRemainingFaces :
  SSet.stdSimplex.face ({1} : Finset (Fin 3))  $\leq$ 
  boundaryTwoRemainingFaces := by
  apply le_sup_of_le_right
  rw [SSet.stdSimplex.face_le_face_iff]
  decide

lemma face_two_le_boundaryTwoRemainingFaces :
  SSet.stdSimplex.face ({2} : Finset (Fin 3))  $\leq$ 
  boundaryTwoRemainingFaces := by
  apply le_sup_of_le_left
  rw [SSet.stdSimplex.face_le_face_iff]
  decide

/-- The identification of the two remaining faces with  $\Lambda[2,0]$ . -/
noncomputable def boundaryTwoRemainingFacesIsoHorn :
  (boundaryTwoRemainingFaces : SSet.{0})  $\cong$  ( $\Lambda[2, 0]$  : SSet.{0}) :=
  SSet.Subcomplex.eqToIso boundaryTwoRemainingFaces_eq_horn

lemma boundaryTwoRemainingFacesIsoHorn_hom_ι :
  boundaryTwoRemainingFacesIsoHorn.hom  $\gg$   $\Lambda[2, 0].\iota$  =
  boundaryTwoRemainingFaces. $\iota$  := by
  change
    SSet.Subcomplex.homOfLE boundaryTwoRemainingFaces_eq_horn.le  $\gg$ 

```

```

     $\Lambda[2, 0].\iota = \text{boundaryTwoRemainingFaces}.\iota$ 
  rfl

/-- Vertex 1 of the horn is the free endpoint of its first edge. -/
lemma horn_vertex_one :
  (SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
    SSet.Subcomplex.homOfLE face_one_le_boundaryTwoRemainingFaces >>
      boundaryTwoRemainingFacesIsoHorn.hom =
    SSet.stdSimplex. $\delta$  (0 : Fin 2) >> SSet.horn20. $\iota_{01}$  := by
  rw [← cancel_mono  $\Lambda[2, 0].\iota$ ]
  have h $\iota$  :
    SSet.Subcomplex.homOfLE face_one_le_boundaryTwoRemainingFaces >>
      boundaryTwoRemainingFaces. $\iota$  =
    (SSet.stdSimplex.face ({1} : Finset (Fin 3))). $\iota$  := rfl
  rw [Category.assoc, Category.assoc,
    boundaryTwoRemainingFacesIsoHorn_hom_ $\iota$ , h $\iota$ , Category.assoc,
    SSet.horn. $\iota$ . $\iota$ , faceSingletonIso_hom_comp_ $\iota$ ,
     $\delta$ _zero_comp_ $\delta$ _two_eq_const_one]

/-- Vertex 2 of the horn is the free endpoint of its second edge. -/
lemma horn_vertex_two :
  (SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
    SSet.Subcomplex.homOfLE face_two_le_boundaryTwoRemainingFaces >>
      boundaryTwoRemainingFacesIsoHorn.hom =
    SSet.stdSimplex. $\delta$  (0 : Fin 2) >> SSet.horn20. $\iota_{02}$  := by
  rw [← cancel_mono  $\Lambda[2, 0].\iota$ ]
  have h $\iota$  :
    SSet.Subcomplex.homOfLE face_two_le_boundaryTwoRemainingFaces >>
      boundaryTwoRemainingFaces. $\iota$  =
    (SSet.stdSimplex.face ({2} : Finset (Fin 3))). $\iota$  := rfl
  rw [Category.assoc, Category.assoc,
    boundaryTwoRemainingFacesIsoHorn_hom_ $\iota$ , h $\iota$ , Category.assoc,
    SSet.horn. $\iota$ . $\iota$ , faceSingletonIso_hom_comp_ $\iota$ ,
     $\delta$ _zero_comp_ $\delta$ _one_eq_const_two]

/-- The two vertex faces form a coproduct square under the empty
subcomplex. -/
lemma boundaryTwoVertices_isPushout :
  IsPushout
    (SSet.Subcomplex.homOfLE
      (bot_le :
        ( $\perp$  : (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex)  $\leq$ 
          SSet.stdSimplex.face ({1} : Finset (Fin 3))))
    (SSet.Subcomplex.homOfLE
      (bot_le :
        ( $\perp$  : (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex)  $\leq$ 
          SSet.stdSimplex.face ({2} : Finset (Fin 3))))
    (le_sup_left :
      SSet.stdSimplex.face ({1} : Finset (Fin 3))  $\leq$  boundaryTwoVertices)
    (SSet.Subcomplex.homOfLE
      (le_sup_right :
        SSet.stdSimplex.face ({2} : Finset (Fin 3))  $\leq$  boundaryTwoVertices)) :=
  SSet.Subcomplex.BicartSq.isPushout
  ({ sup_eq := rfl
    inf_eq := face_one_inf_face_two } :
    SSet.Subcomplex.BicartSq
      ( $\perp$  : (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex)
      (SSet.stdSimplex.face ({1} : Finset (Fin 3)))
      (SSet.stdSimplex.face ({2} : Finset (Fin 3)))
      boundaryTwoVertices)

/-- Realization preserves the coproduct decomposition of the two vertices. -/

```

```

lemma toTop_boundaryTwoVertices_isPushout :
  IsPushout
    (SSet.toTop.map
      (SSet.Subcomplex.homOfLE
        (bot_le :
          ( $\perp$  : (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex)  $\leq$ 
            SSet.stdSimplex.face ({1} : Finset (Fin 3)))))
    (SSet.toTop.map
      (SSet.Subcomplex.homOfLE
        (bot_le :
          ( $\perp$  : (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex)  $\leq$ 
            SSet.stdSimplex.face ({2} : Finset (Fin 3)))))
    (SSet.toTop.map
      (SSet.Subcomplex.homOfLE
        (le_sup_left :
          SSet.stdSimplex.face ({1} : Finset (Fin 3))  $\leq$  boundaryTwoVertices)))
    (SSet.toTop.map
      (SSet.Subcomplex.homOfLE
        (le_sup_right :
          SSet.stdSimplex.face ({2} : Finset (Fin 3))  $\leq$  boundaryTwoVertices)))) :=
  boundaryTwoVertices_isPushout.map SSet.toTop

/-- The two-edge horn, transported from `boundaryTwoRemainingFaces`, as the
first arc of the concrete circle. -/
noncomputable def boundaryTwoRemainingFacesToBoundaryCircle :
  |(boundaryTwoRemainingFaces : SSet.{0})|  $\longrightarrow$  boundaryCircle :=
  SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom  $\gg$ 
  hornToBoundaryCircle

lemma lastEdgeToBoundaryCircle_vertex_one :
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom  $\gg$ 
      SSet.Subcomplex.homOfLE face_one_le_boundaryTwoFirstFace)  $\gg$ 
  lastEdgeToBoundaryCircle =
  (TopCat.const ((2 :  $\mathbb{R}$ ) : AddCircle (3 :  $\mathbb{R}$ )) :
    |( $\Delta$ [0] : SSet.{0})|  $\longrightarrow$  boundaryCircle) := by
  rw [lastEdgeToBoundaryCircle,  $\leftarrow$  Category.assoc,
    lastEdgeIsoI_vertex_one, unitIntervalToBoundaryCircle_zero]

lemma lastEdgeToBoundaryCircle_vertex_two :
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom  $\gg$ 
      SSet.Subcomplex.homOfLE face_two_le_boundaryTwoFirstFace)  $\gg$ 
  lastEdgeToBoundaryCircle =
  (TopCat.const (0 : AddCircle (3 :  $\mathbb{R}$ )) :
    |( $\Delta$ [0] : SSet.{0})|  $\longrightarrow$  boundaryCircle) := by
  rw [lastEdgeToBoundaryCircle,  $\leftarrow$  Category.assoc,
    lastEdgeIsoI_vertex_two, unitIntervalToBoundaryCircle_one]

lemma boundaryTwoRemainingFacesToBoundaryCircle_vertex_one :
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom  $\gg$ 
      SSet.Subcomplex.homOfLE face_one_le_boundaryTwoRemainingFaces)  $\gg$ 
  boundaryTwoRemainingFacesToBoundaryCircle =
  (TopCat.const ((2 :  $\mathbb{R}$ ) : AddCircle (3 :  $\mathbb{R}$ )) :
    |( $\Delta$ [0] : SSet.{0})|  $\longrightarrow$  boundaryCircle) := by
  rw [boundaryTwoRemainingFacesToBoundaryCircle, hornToBoundaryCircle]
  rw [ $\leftarrow$  Category.assoc,  $\leftarrow$  SSet.toTop.map_comp]
  simp only [Category.assoc]
  rw [horn_vertex_one]
  have h := congrArg
    (fun f : |( $\Delta$ [0] : SSet.{0})|  $\longrightarrow$  hornIcc =>
      f  $\gg$  hornIccToBoundaryCircle)

```

```

    horn_two_zero_toIcc_δ_zero_inl
exact (by
  simp only [SSet.toTop.map_comp, Category.assoc] using
    h.trans hornIccToBoundaryCircle_one)

lemma boundaryTwoRemainingFacesToBoundaryCircle_vertex_two :
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
      SSet.Subcomplex.homOfLE face_two_le_boundaryTwoRemainingFaces) >>
    boundaryTwoRemainingFacesToBoundaryCircle =
  (TopCat.const (0 : AddCircle (3 : ℝ)) :
    |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  boundaryCircle) := by
  rw [boundaryTwoRemainingFacesToBoundaryCircle, hornToBoundaryCircle]
  rw [← Category.assoc, ← SSet.toTop.map_comp]
  simp only [Category.assoc]
  rw [horn_vertex_two]
  have h := congrArg
    (fun f : |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  hornIcc =>
      f >> hornIccToBoundaryCircle)
  horn_two_zero_toIcc_δ_zero_inr
exact (by
  simp only [SSet.toTop.map_comp, Category.assoc] using
    h.trans hornIccToBoundaryCircle_neg_one)

lemma boundaryTwoVertices_le_firstFace :
  boundaryTwoVertices ≤ boundaryTwoFirstFace :=
  sup_le face_one_le_boundaryTwoFirstFace face_two_le_boundaryTwoFirstFace

lemma boundaryTwoVertices_le_remainingFaces :
  boundaryTwoVertices ≤ boundaryTwoRemainingFaces :=
  sup_le face_one_le_boundaryTwoRemainingFaces
  face_two_le_boundaryTwoRemainingFaces

lemma faceOne_boundaryCircle_arc_agreement :
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE face_one_le_boundaryTwoFirstFace) >>
    lastEdgeToBoundaryCircle =
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE face_one_le_boundaryTwoRemainingFaces) >>
    boundaryTwoRemainingFacesToBoundaryCircle := by
  apply (cancel_epi
    (SSet.toTop.map (SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom)).1
  calc
    _ = SSet.toTop.map
      ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
        SSet.Subcomplex.homOfLE face_one_le_boundaryTwoFirstFace) >>
        lastEdgeToBoundaryCircle := by
      rw [SSet.toTop.map_comp, Category.assoc]
    _ = (TopCat.const ((2 : ℝ) : AddCircle (3 : ℝ)) :
      |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  boundaryCircle) :=
      lastEdgeToBoundaryCircle_vertex_one
    _ = SSet.toTop.map
      ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
        SSet.Subcomplex.homOfLE face_one_le_boundaryTwoRemainingFaces) >>
        boundaryTwoRemainingFacesToBoundaryCircle :=
      boundaryTwoRemainingFacesToBoundaryCircle_vertex_one.symm
    _ = _ := by
      rw [SSet.toTop.map_comp, Category.assoc]

lemma faceTwo_boundaryCircle_arc_agreement :
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE face_two_le_boundaryTwoFirstFace) >>
    lastEdgeToBoundaryCircle =

```



```

    SSet.toTop.map
      (SSet.Subcomplex.homOfLE face_two_le_boundaryTwoRemainingFaces) >>
      boundaryTwoRemainingFacesToBoundaryCircle := by
apply (cancel_epi
  (SSet.toTop.map (SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom)).1
calc
  _ = SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
      SSet.Subcomplex.homOfLE face_two_le_boundaryTwoFirstFace) >>
      lastEdgeToBoundaryCircle := by
  rw [SSet.toTop.map_comp, Category.assoc]
  _ = (TopCat.const (0 : AddCircle (3 : ℝ)) :
    |( $\Delta[0]$  : SSet.{0})|  $\rightarrow$  boundaryCircle) :=
    lastEdgeToBoundaryCircle_vertex_two
  _ = SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
      SSet.Subcomplex.homOfLE face_two_le_boundaryTwoRemainingFaces) >>
      boundaryTwoRemainingFacesToBoundaryCircle :=
      boundaryTwoRemainingFacesToBoundaryCircle_vertex_two.symm
  _ = _ := by
  rw [SSet.toTop.map_comp, Category.assoc]

/-- The two circle arcs agree on the two-vertex intersection. -/
lemma boundaryTwoVertices_boundaryCircle_arc_agreement :
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace) >>
    lastEdgeToBoundaryCircle =
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_remainingFaces) >>
    boundaryTwoRemainingFacesToBoundaryCircle := by
apply toTop_boundaryTwoVertices_isPushout.hom_ext
· simp only [← Category.assoc, ← SSet.toTop.map_comp]
  exact faceOne_boundaryCircle_arc_agreement
· simp only [← Category.assoc, ← SSet.toTop.map_comp]
  exact faceTwo_boundaryCircle_arc_agreement

/-- The actual last-edge/horn intersection identified with its two vertex
faces. -/
noncomputable def boundaryTwoIntersectionIsoVertices :
  ((boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces :
    (SSet.stdSimplex.obj  $\llbracket 2 \rrbracket$  : SSet.{0}).Subcomplex) : SSet.{0})  $\cong$ 
    (boundaryTwoVertices : SSet.{0}) :=
  SSet.Subcomplex.eqToIso boundaryTwoFirstFace_inf

lemma boundaryTwoIntersectionIsoVertices_inv_comp_first :
  boundaryTwoIntersectionIsoVertices.inv >>
  SSet.Subcomplex.homOfLE
    (inf_le_left :
      boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
      boundaryTwoFirstFace) =
  SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace := by
ext n x
rfl

lemma boundaryTwoIntersectionIsoVertices_inv_comp_remaining :
  boundaryTwoIntersectionIsoVertices.inv >>
  SSet.Subcomplex.homOfLE
    (inf_le_right :
      boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
      boundaryTwoRemainingFaces) =
  SSet.Subcomplex.homOfLE boundaryTwoVertices_le_remainingFaces := by
ext n x
rfl

```

```

/-- The last edge and horn circle maps agree on their actual intersection. -/
lemma boundaryTwo_boundaryCircle_arc_agreement :
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (inf_le_left :
        boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
        boundaryTwoFirstFace))  $\gg$ 
    lastEdgeToBoundaryCircle =
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (inf_le_right :
        boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
        boundaryTwoRemainingFaces))  $\gg$ 
    boundaryTwoRemainingFacesToBoundaryCircle := by
  apply (cancel_epi
    (SSet.toTop.map boundaryTwoIntersectionIsoVertices.inv)).1
  calc
    _ = SSet.toTop.map
      (boundaryTwoIntersectionIsoVertices.inv  $\gg$ 
        SSet.Subcomplex.homOfLE inf_le_left)  $\gg$ 
      lastEdgeToBoundaryCircle := by
    rw [SSet.toTop.map_comp, Category.assoc]
    _ = SSet.toTop.map
      (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace)  $\gg$ 
      lastEdgeToBoundaryCircle := by
    rw [boundaryTwoIntersectionIsoVertices_inv_comp_first]
    _ = SSet.toTop.map
      (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_remainingFaces)  $\gg$ 
      boundaryTwoRemainingFacesToBoundaryCircle :=
      boundaryTwoVertices_boundaryCircle_arc_agreement
    _ = SSet.toTop.map
      (boundaryTwoIntersectionIsoVertices.inv  $\gg$ 
        SSet.Subcomplex.homOfLE inf_le_right)  $\gg$ 
      boundaryTwoRemainingFacesToBoundaryCircle := by
    rw [boundaryTwoIntersectionIsoVertices_inv_comp_remaining]
    _ = _ := by
    rw [SSet.toTop.map_comp, Category.assoc]

/-- The map from the last-edge pushout to the concrete circle. -/
noncomputable def boundaryTwoSupToBoundaryCircle :
  |((boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces :
    (SSet.stdSimplex.obj  $\llbracket 2 \rrbracket$  : SSet.{0}).Subcomplex) : SSet.{0})|  $\longrightarrow$ 
    boundaryCircle :=
  toTop_boundaryTwo_isPushout.desc
  lastEdgeToBoundaryCircle
  boundaryTwoRemainingFacesToBoundaryCircle
  boundaryTwo_boundaryCircle_arc_agreement

/-- The boundary subcomplex identified with its last-edge decomposition. -/
noncomputable def boundaryTwoIsoSup :
  boundaryTwoSSet  $\cong$ 
    ((boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces :
      (SSet.stdSimplex.obj  $\llbracket 2 \rrbracket$  : SSet.{0}).Subcomplex) : SSet.{0}) :=
  SSet.Subcomplex.eqToIso boundary_two_eq_face_sup

/-- The canonical piecewise-linear map from  $|\partial\Delta[2]|$  to a concrete
additive circle. -/
noncomputable def boundaryTwoToBoundaryCircle :
  |boundaryTwoSSet|  $\longrightarrow$  boundaryCircle :=
  SSet.toTop.map boundaryTwoIsoSup.hom  $\gg$  boundaryTwoSupToBoundaryCircle

@[reassoc]

```

```

lemma boundaryTwoSupToBoundaryCircle_firstFace :
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (le_sup_left :
        boundaryTwoFirstFace ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces)) >>
    boundaryTwoSupToBoundaryCircle =
  lastEdgeToBoundaryCircle := by
  apply toTop_boundaryTwo_isPushout.inl_desc

@[reassoc]
lemma boundaryTwoSupToBoundaryCircle_remainingFaces :
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (le_sup_right :
        boundaryTwoRemainingFaces ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces)) >>
    boundaryTwoSupToBoundaryCircle =
  boundaryTwoRemainingFacesToBoundaryCircle := by
  apply toTop_boundaryTwo_isPushout.inr_desc

lemma lastEdgeIsoI_inv_zero :
  (TopCat.const (0 : TopCat.I.{0}) :
    | (Δ[0] : SSet.{0}) | → TopCat.I.{0}) >> lastEdgeIsoI.inv =
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
      SSet.Subcomplex.homOfLE face_one_le_boundaryTwoFirstFace) := by
  have h := congrArg
    (fun f : | (Δ[0] : SSet.{0}) | → TopCat.I.{0} => f >> lastEdgeIsoI.inv)
    lastEdgeIsoI.vertex_one
  simpa only [Category.assoc, Iso.hom_inv_id, Category.comp_id] using h.symm

lemma lastEdgeIsoI_inv_one :
  (TopCat.const (1 : TopCat.I.{0}) :
    | (Δ[0] : SSet.{0}) | → TopCat.I.{0}) >> lastEdgeIsoI.inv =
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
      SSet.Subcomplex.homOfLE face_two_le_boundaryTwoFirstFace) := by
  have h := congrArg
    (fun f : | (Δ[0] : SSet.{0}) | → TopCat.I.{0} => f >> lastEdgeIsoI.inv)
    lastEdgeIsoI.vertex_two
  simpa only [Category.assoc, Iso.hom_inv_id, Category.comp_id] using h.symm

lemma hornIcc_from_one :
  (TopCat.const (⟨1, by norm_num⟩ : Set.Icc (-1 : ℝ) 1) :
    | (Δ[0] : SSet.{0}) | → hornIcc) >> hornIcc_from =
  SSet.toTop.map
    (SSet.stdSimplex.δ (0 : Fin 2) >> SSet.horn20.ℓ01) := by
  have h := congrArg
    (fun f : | (Δ[0] : SSet.{0}) | → hornIcc => f >> hornIcc_from)
    horn_two_zero_toIcc_δ_zero_inl
  simpa only [SSet.toTop.map_comp, Category.assoc,
    horn_two_zero_toIcc_from, Category.comp_id] using h.symm

lemma hornIcc_from_neg_one :
  (TopCat.const (⟨-1, by norm_num⟩ : Set.Icc (-1 : ℝ) 1) :
    | (Δ[0] : SSet.{0}) | → hornIcc) >> hornIcc_from =
  SSet.toTop.map
    (SSet.stdSimplex.δ (0 : Fin 2) >> SSet.horn20.ℓ02) := by
  have h := congrArg
    (fun f : | (Δ[0] : SSet.{0}) | → hornIcc => f >> hornIcc_from)
    horn_two_zero_toIcc_δ_zero_inr
  simpa only [SSet.toTop.map_comp, Category.assoc,

```

```

horn_two_zero_toIcc_from, Category.comp_id] using h.symm

/-- Put the unit interval back into the last-edge summand of the pushout. -/
noncomputable def unitIntervalToBoundaryTwoSup :
  TopCat.I.{0} →
  |((boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces :
    (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) : SSet.{0})| :=
  lastEdgeIsoI.inv >>
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (le_sup_left :
        boundaryTwoFirstFace ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces))

/-- Put  $[-1,1]$  back into the horn summand of the pushout. -/
noncomputable def hornIccToBoundaryTwoSup :
  hornIcc →
  |((boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces :
    (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) : SSet.{0})| :=
  hornIcc_from >> SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv >>
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (le_sup_right :
        boundaryTwoRemainingFaces ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces))

lemma horn_vertex_one_comp_iso_inv :
  (SSet.stdSimplex.δ (0 : Fin 2) >> SSet.horn20.ι01) >>
  boundaryTwoRemainingFacesIsoHorn.inv =
  (SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
  SSet.Subcomplex.homOfLE face_one_le_boundaryTwoRemainingFaces := by
  have h := congrArg
    (fun f : (Δ[0] : SSet.{0}) → (Λ[2, 0] : SSet.{0}) =>
      f >> boundaryTwoRemainingFacesIsoHorn.inv)
  horn_vertex_one
  simpa only [Category.assoc, Iso.hom_inv_id, Category.comp_id] using h.symm

lemma horn_vertex_two_comp_iso_inv :
  (SSet.stdSimplex.δ (0 : Fin 2) >> SSet.horn20.ι02) >>
  boundaryTwoRemainingFacesIsoHorn.inv =
  (SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
  SSet.Subcomplex.homOfLE face_two_le_boundaryTwoRemainingFaces := by
  have h := congrArg
    (fun f : (Δ[0] : SSet.{0}) → (Λ[2, 0] : SSet.{0}) =>
      f >> boundaryTwoRemainingFacesIsoHorn.inv)
  horn_vertex_two
  simpa only [Category.assoc, Iso.hom_inv_id, Category.comp_id] using h.symm

/-- At vertex 1, the last-edge coordinate  $`0`$  and horn coordinate  $`1`$ 
give the same point of the pushout. -/
lemma unitIntervalToBoundaryTwoSup_zero :
  (TopCat.const (0 : TopCat.I.{0}) :
    | (Δ[0] : SSet.{0}) | → TopCat.I.{0}) >>
  unitIntervalToBoundaryTwoSup =
  (TopCat.const (⟨1, by norm_num⟩ : Set.Icc (-1 : ℝ) 1) :
    | (Δ[0] : SSet.{0}) | → hornIcc) >>
  hornIccToBoundaryTwoSup := by
  rw [unitIntervalToBoundaryTwoSup, hornIccToBoundaryTwoSup,
    ← Category.assoc, lastEdgeIsoI_inv_zero,
    ← Category.assoc, hornIcc_from_one]
  simp only [Category.assoc, ← SSet.toTop.map_comp]
  have hv := congrArg
    (fun f : (Δ[0] : SSet.{0}) →

```

```

    (boundaryTwoRemainingFaces : SSet.{0}) =>
  f >> SSet.Subcomplex.homOfLE
  (le_sup_right :
    boundaryTwoRemainingFaces ≤
      boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces))
  horn_vertex_one_comp_iso_inv
simp only [Category.assoc] at hv
rw [hv]
congr 1

/-- At vertex 2, the last-edge coordinate `1` and horn coordinate `-1`
give the same point of the pushout. -/
lemma unitIntervalToBoundaryTwoSup_one :
  (TopCat.const (1 : TopCat.I.{0}) :
    | (Δ[0] : SSet.{0}) | → TopCat.I.{0}) >>
    unitIntervalToBoundaryTwoSup =
  (TopCat.const (⟨-1, by norm_num⟩ : Set.Icc (-1 : ℝ) 1) :
    | (Δ[0] : SSet.{0}) | → hornIcc) >>
    hornIccToBoundaryTwoSup := by
  rw [unitIntervalToBoundaryTwoSup, hornIccToBoundaryTwoSup,
    ← Category.assoc, lastEdgeIsoI_inv_one,
    ← Category.assoc, hornIcc_from_neg_one]
simp only [Category.assoc, ← SSet.toTop.map_comp]
have hv := congrArg
  (fun f : (Δ[0] : SSet.{0}) →
    (boundaryTwoRemainingFaces : SSet.{0}) =>
      f >> SSet.Subcomplex.homOfLE
      (le_sup_right :
        boundaryTwoRemainingFaces ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces))
  horn_vertex_two_comp_iso_inv
simp only [Category.assoc] at hv
rw [hv]
congr 1

/-- A path through the two pushout arcs, parametrized by `[0,3]` and
extended continuously to all of `ℝ` using interval projections. -/
noncomputable def boundaryCircleLiftPath (x : ℝ) :
  | ((boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces :
    (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) : SSet.{0}) | :=
  if x ≤ 2 then
    hornIccToBoundaryTwoSup
    (Set.projIcc (-1 : ℝ) 1 (by norm_num) (x - 1))
  else
    unitIntervalToBoundaryTwoSup
    (TopCat.I.homeomorph.symm
      (Set.projIcc (0 : ℝ) 1 zero_le_one (x - 2)))

lemma continuous_boundaryCircleLiftPath :
  Continuous boundaryCircleLiftPath := by
  refine Continuous.if_le
  (f' := fun x : ℝ =>
    hornIccToBoundaryTwoSup
    (Set.projIcc (-1 : ℝ) 1 (by norm_num) (x - 1)))
  (g' := fun x : ℝ =>
    unitIntervalToBoundaryTwoSup
    (TopCat.I.homeomorph.symm
      (Set.projIcc (0 : ℝ) 1 zero_le_one (x - 2))))
  (f := fun x : ℝ => x) (g := fun _ => (2 : ℝ))
  ?_ ?_ continuous_id continuous_const ?_
· fun_prop
· fun_prop
· intro x hx

```

```

have hx2 : x = 2 := hx
subst x
have hpHorn :
  Set.projIcc (-1 : ℝ) 1 (by norm_num) (2 - 1) =
    ⟨1, by norm_num⟩ := by
      norm_num [Set.projIcc_right]
have hpEdge :
  Set.projIcc (0 : ℝ) 1 zero_le_one (2 - 2) =
    ⟨0, by norm_num⟩ := by
      norm_num [Set.projIcc_left]
change
  hornIccToBoundaryTwoSup
  (Set.projIcc (-1 : ℝ) 1 (by norm_num) (2 - 1)) =
    unitIntervalToBoundaryTwoSup
    (TopCat.I.homeomorph.symm
      (Set.projIcc (0 : ℝ) 1 zero_le_one (2 - 2)))
rw [hpHorn, hpEdge]
have h := congrArg
  (fun f : |(Δ[0] : SSet.{0})| →
    |((boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces :
      (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) : SSet.{0})| =>
      f (default : |(Δ[0] : SSet.{0})|)))
  unitIntervalToBoundaryTwoSup_zero
simpa using h.symm

lemma boundaryCircleLiftPath_zero_eq_three :
  boundaryCircleLiftPath 0 = boundaryCircleLiftPath 3 := by
  have hpHorn :
    Set.projIcc (-1 : ℝ) 1 (by norm_num) (0 - 1) =
      ⟨-1, by norm_num⟩ := by
        norm_num [Set.projIcc_left]
  have hpEdge :
    Set.projIcc (0 : ℝ) 1 zero_le_one (3 - 2) =
      ⟨1, by norm_num⟩ := by
        norm_num [Set.projIcc_right]
  rw [boundaryCircleLiftPath, if_pos (by norm_num), hpHorn,
    boundaryCircleLiftPath, if_neg (by norm_num), hpEdge]
  have h := congrArg
    (fun f : |(Δ[0] : SSet.{0})| →
      |((boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces :
        (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) : SSet.{0})| =>
        f (default : |(Δ[0] : SSet.{0})|)))
    unitIntervalToBoundaryTwoSup_one
  simpa using h.symm

/-- The inverse circle map obtained by traversing the horn and last edge. -/
noncomputable def boundaryCircleToBoundaryTwoSup :
  boundaryCircle →
  |((boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces :
    (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) : SSet.{0})| := by
  letI : Fact (0 < (3 : ℝ)) := ⟨by norm_num⟩
  exact TopCat.ofHom
    { toFun := AddCircle.liftIco (3 : ℝ) 0 boundaryCircleLiftPath
      continuous_toFun :=
        AddCircle.liftIco_zero_continuous boundaryCircleLiftPath_zero_eq_three
        continuous_boundaryCircleLiftPath.continuousOn }

lemma boundaryCircleToBoundaryTwoSup_hornIcc
  (t : Set.Icc (-1 : ℝ) 1) :
  boundaryCircleToBoundaryTwoSup (hornIccToBoundaryCircle t) =
    hornIccToBoundaryTwoSup t := by
  letI : Fact (0 < (3 : ℝ)) := ⟨by norm_num⟩
  have htIco : t.1 + 1 ∈ Set.Ico (0 : ℝ) 3 :=

```

```

    ⟨by linarith [t.2.1], by linarith [t.2.2]⟩
change
  AddCircle.liftIco (3 : ℝ) 0 boundaryCircleLiftPath
    ((t.1 + 1 : ℝ) : AddCircle (3 : ℝ)) =
    hornIccToBoundaryTwoSup t
rw [AddCircle.liftIco_zero_coe_apply htIco, boundaryCircleLiftPath,
  if_pos (by linarith [t.2.2])]
have hval : t.1 + 1 - 1 = t.1 := by ring
have hp :
  Set.projIcc (-1 : ℝ) 1 (by norm_num) (t.1 + 1 - 1) = t := by
  rw [hval]
  exact Set.projIcc_of_mem _ t.2
rw [hp]

lemma boundaryCircleToBoundaryTwoSup_unitInterval
  (t : TopCat.I.{0}) :
  boundaryCircleToBoundaryTwoSup (unitIntervalToBoundaryCircle t) =
  unitIntervalToBoundaryTwoSup t := by
letI : Fact (0 < (3 : ℝ)) := ⟨by norm_num⟩
let u := TopCat.I.homeomorph t
by_cases h1 : (u : ℝ) = 1
· have ht : t = 1 := by
  apply TopCat.I.homeomorph.injective
  apply Subtype.ext
  simpa [u, TopCat.I.homeomorph_one] using h1
subst t
have hcircle := congrArg
  (fun f : |(Δ[0] : SSet.{0})| → boundaryCircle =>
    boundaryCircleToBoundaryTwoSup
      (f (default : |(Δ[0] : SSet.{0})|)))
  unitIntervalToBoundaryCircle_one
have hglue := congrArg
  (fun f : |(Δ[0] : SSet.{0})| →
    |((boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces :
      (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) : SSet.{0})| =>
    f (default : |(Δ[0] : SSet.{0})|))
  unitIntervalToBoundaryTwoSup_one
change
  boundaryCircleToBoundaryTwoSup
    (unitIntervalToBoundaryCircle (1 : TopCat.I.{0})) =
    unitIntervalToBoundaryTwoSup (1 : TopCat.I.{0})
have hcircle' :
  boundaryCircleToBoundaryTwoSup
    (unitIntervalToBoundaryCircle (1 : TopCat.I.{0})) =
    boundaryCircleToBoundaryTwoSup (0 : AddCircle (3 : ℝ)) := by
  simpa using hcircle
rw [hcircle']
change
  AddCircle.liftIco (3 : ℝ) 0 boundaryCircleLiftPath
    ((0 : ℝ) : AddCircle (3 : ℝ)) =
    unitIntervalToBoundaryTwoSup (1 : TopCat.I.{0})
rw [AddCircle.liftIco_zero_coe_apply
  (p := (3 : ℝ)) (x := (0 : ℝ)) (by norm_num)]
rw [boundaryCircleLiftPath, if_pos (by norm_num)]
have hp :
  Set.projIcc (-1 : ℝ) 1 (by norm_num) (0 - 1) =
  ⟨-1, by norm_num⟩ := by
  norm_num [Set.projIcc_left]
rw [hp]
simpa using hglue.symm
· have hu_lt : (u : ℝ) < 1 := lt_of_le_of_ne u.2.2 h1
have hxIco : 2 + (u : ℝ) ∈ Set.Ico (0 : ℝ) 3 :=
  ⟨by linarith [u.2.1], by linarith⟩

```

```

change
  AddCircle.liftIco (3 : ℝ) 0 boundaryCircleLiftPath
    ((2 + (u : ℝ) : ℝ) : AddCircle (3 : ℝ)) =
      unitIntervalToBoundaryTwoSup t
rw [AddCircle.liftIco_zero_coe_apply hxIco, boundaryCircleLiftPath]
by_cases h0 : (u : ℝ) = 0
· have ht : t = 0 := by
  apply TopCat.I.homeomorph.injective
  apply Subtype.ext
  simpa [u, TopCat.I.homeomorph_zero] using h0
subst t
rw [if_pos (by simp [u])]
have hp :
  Set.projIcc (-1 : ℝ) 1 (by norm_num)
    (2 + (u : ℝ) - 1) =
    ⟨1, by norm_num⟩ := by
  rw [h0]
  norm_num [Set.projIcc_right]
rw [hp]
have hglue := congrArg
  (fun f : |(Δ[0] : SSet.{0})| →
    |((boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces :
      (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) : SSet.{0})| =>
      f (default : |(Δ[0] : SSet.{0})|)))
  unitIntervalToBoundaryTwoSup_zero
  simpa using hglue.symm
· have hu_pos : 0 < (u : ℝ) := lt_of_le_of_ne u.2.1 (Ne.symm h0)
  rw [if_neg (by linarith)]
  have hp :
    Set.projIcc (0 : ℝ) 1 zero_le_one
      (2 + (u : ℝ) - 2) = u := by
    have hval : 2 + (u : ℝ) - 2 = (u : ℝ) := by ring
    rw [hval]
    exact Set.projIcc_of_mem _ u.2
  rw [hp, Homeomorph.symm_apply_apply]

lemma lastEdgeToBoundaryCircle_comp_inverse :
  lastEdgeToBoundaryCircle >> boundaryCircleToBoundaryTwoSup =
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (le_sup_left :
        boundaryTwoFirstFace ≤
        boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces)) := by
ext x
change
  boundaryCircleToBoundaryTwoSup
    (unitIntervalToBoundaryCircle (lastEdgeIsoI.hom x)) =
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (le_sup_left :
        boundaryTwoFirstFace ≤
        boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces)) x
rw [boundaryCircleToBoundaryTwoSup_unitInterval]
change
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (le_sup_left :
        boundaryTwoFirstFace ≤
        boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces))
    (lastEdgeIsoI.inv (lastEdgeIsoI.hom x)) =
  -
have hx := congrArg
  (fun f : |(boundaryTwoFirstFace : SSet.{0})| →

```



```

      |(boundaryTwoFirstFace : SSet.{0})| => f x)
    lastEdgeIsoI.hom_inv_id
  have hx' : lastEdgeIsoI.inv (lastEdgeIsoI.hom x) = x := by
    change (lastEdgeIsoI.hom >> lastEdgeIsoI.inv) x = x
    exact hx
  rw [hx']

lemma boundaryTwoRemainingFacesToBoundaryCircle_comp_inverse :
  boundaryTwoRemainingFacesToBoundaryCircle >>
    boundaryCircleToBoundaryTwoSup =
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (le_sup_right :
        boundaryTwoRemainingFaces ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces)) := by
  ext x
  change
    boundaryCircleToBoundaryTwoSup
      (hornIccToBoundaryCircle
        (horn_two_zero_toIcc
          (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom x))) =
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (le_sup_right :
        boundaryTwoRemainingFaces ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces)) x
  rw [boundaryCircleToBoundaryTwoSup_hornIcc]
  change
    SSet.toTop.map
      (SSet.Subcomplex.homOfLE
        (le_sup_right :
          boundaryTwoRemainingFaces ≤
            boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces))
    (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv
      (hornIcc_from
        (horn_two_zero_toIcc
          (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom x))))) =
  -
  have hHorn := congrArg
    (fun f : |(Λ[2, 0] : SSet.{0})| → |(Λ[2, 0] : SSet.{0})| =>
      f (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom x))
    horn_two_zero_toIcc_from
  have hHorn' :
    hornIcc_from
      (horn_two_zero_toIcc
        (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom x)) =
    SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom x := by
    simpa using hHorn
  rw [hHorn']
  have hIsoCat :
    SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom >>
      SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv = ⌞ _ := by
    rw [← SSet.toTop.map_comp, Iso.hom_inv_id, SSet.toTop.map_id]
  have hIso := congrArg
    (fun f : |(boundaryTwoRemainingFaces : SSet.{0})| →
      |(boundaryTwoRemainingFaces : SSet.{0})| => f x)
    hIsoCat
  have hIso' :
    SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv
      (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom x) = x := by
    change
      (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom >>
        SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv) x = x

```

```

    exact hIso
  rw [hIso']

/-- The circle map followed by its path inverse is the identity on the
last-edge pushout. -/
lemma boundaryTwoSupToBoundaryCircle_comp_inverse :
  boundaryTwoSupToBoundaryCircle >> boundaryCircleToBoundaryTwoSup =  $\mathbb{1}$  _ := by
  apply toTop_boundaryTwo_isPushout.hom_ext
  · rw [boundaryTwoSupToBoundaryCircle_firstFace_assoc,
    lastEdgeToBoundaryCircle_comp_inverse, Category.comp_id]
  · rw [boundaryTwoSupToBoundaryCircle_remainingFaces_assoc,
    boundaryTwoRemainingFacesToBoundaryCircle_comp_inverse, Category.comp_id]

lemma boundaryTwoSupToBoundaryCircle_unitInterval
  (t : TopCat.I.{0}) :
  boundaryTwoSupToBoundaryCircle (unitIntervalToBoundaryTwoSup t) =
    unitIntervalToBoundaryCircle t := by
  have hFace := congrArg
    (fun f :
      |(boundaryTwoFirstFace : SSet.{0})| → boundaryCircle =>
      f (lastEdgeIsoI.inv t))
    boundaryTwoSupToBoundaryCircle_firstFace
  change
    boundaryTwoSupToBoundaryCircle
      (SSet.toTop.map
        (SSet.Subcomplex.homOfLE
          (le_sup_left :
            boundaryTwoFirstFace ≤
              boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces))
          (lastEdgeIsoI.inv t)) =
      unitIntervalToBoundaryCircle t
  rw [show boundaryTwoSupToBoundaryCircle
    (SSet.toTop.map
      (SSet.Subcomplex.homOfLE
        (le_sup_left :
          boundaryTwoFirstFace ≤
            boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces))
          (lastEdgeIsoI.inv t)) =
      lastEdgeToBoundaryCircle (lastEdgeIsoI.inv t) by simp using hFace]
  change unitIntervalToBoundaryCircle
    (lastEdgeIsoI.hom (lastEdgeIsoI.inv t)) =
    unitIntervalToBoundaryCircle t
  have ht := congrArg
    (fun f : TopCat.I.{0} → TopCat.I.{0} => f t)
    lastEdgeIsoI.inv_hom_id
  rw [show lastEdgeIsoI.hom (lastEdgeIsoI.inv t) = t by
    change (lastEdgeIsoI.inv >> lastEdgeIsoI.hom) t = t
    exact ht]

lemma boundaryTwoSupToBoundaryCircle_hornIcc
  (t : Set.Icc (-1 :  $\mathbb{R}$ ) 1) :
  boundaryTwoSupToBoundaryCircle (hornIccToBoundaryTwoSup t) =
    hornIccToBoundaryCircle t := by
  have hFace := congrArg
    (fun f :
      |(boundaryTwoRemainingFaces : SSet.{0})| → boundaryCircle =>
      f (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv
        (hornIcc_from t)))
    boundaryTwoSupToBoundaryCircle_remainingFaces
  change
    boundaryTwoSupToBoundaryCircle
      (SSet.toTop.map
        (SSet.Subcomplex.homOfLE

```

```

      (le_sup_right :
        boundaryTwoRemainingFaces ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces))
    (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv
      (hornIcc_from t))) =
  hornIccToBoundaryCircle t
rw [show boundaryTwoSupToBoundaryCircle
  (SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (le_sup_right :
        boundaryTwoRemainingFaces ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces))
    (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv
      (hornIcc_from t))) =
    boundaryTwoRemainingFacesToBoundaryCircle
    (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv
      (hornIcc_from t)) by simp using hFace]
change
  hornIccToBoundaryCircle
    (horn_two_zero_toIcc
      (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom
        (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv
          (hornIcc_from t)))) =
    hornIccToBoundaryCircle t
have hIsoCat :
  SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv >>
    SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom =  $\mathbb{K}$  _ := by
  rw [← SSet.toTop.map_comp, Iso.inv_hom_id, SSet.toTop.map_id]
have hIso := congrArg
  (fun f : |( $\Lambda$ [2, 0] : SSet.{0})| → |( $\Lambda$ [2, 0] : SSet.{0})| =>
    f (hornIcc_from t))
  hIsoCat
have hIso' :
  SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom
    (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv
      (hornIcc_from t)) =
    hornIcc_from t := by
  change
    (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.inv >>
      SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom)
      (hornIcc_from t) = hornIcc_from t
  exact hIso
rw [hIso']
have ht := congrArg (fun f : hornIcc → hornIcc => f t)
  hornIcc_from_toIcc
rw [show horn_two_zero_toIcc (hornIcc_from t) = t by
  change (hornIcc_from >> horn_two_zero_toIcc) t = t
  exact ht]

/-- The path inverse followed by the circle map is the identity. -/
lemma boundaryCircleToBoundaryTwoSup_comp_boundaryTwoSupToBoundaryCircle :
  boundaryCircleToBoundaryTwoSup >> boundaryTwoSupToBoundaryCircle =  $\mathbb{K}$  _ := by
  ext y
  letI : Fact (0 < (3 :  $\mathbb{R}$ )) := (by norm_num)
  let x := AddCircle.equivIco (3 :  $\mathbb{R}$ ) 0 y
  have hxIco : (x :  $\mathbb{R}$ ) ∈ Set.Ico (0 :  $\mathbb{R}$ ) 3 := by
    simp using x.2
  have hy : (((x :  $\mathbb{R}$ ) : AddCircle (3 :  $\mathbb{R}$ ))) = y :=
    AddCircle.coe_equivIco
  change
    boundaryTwoSupToBoundaryCircle
      (boundaryCircleToBoundaryTwoSup y) = y
  rw [← hy]

```

```

change
  boundaryTwoSupToBoundaryCircle
    (AddCircle.liftIco (3 : ℝ) 0 boundaryCircleLiftPath
      (((x : ℝ) : AddCircle (3 : ℝ)))) =
      (((x : ℝ) : AddCircle (3 : ℝ)))
rw [AddCircle.liftIco_zero_coe_apply hxIco, boundaryCircleLiftPath]
by_cases hx2 : (x : ℝ) ≤ 2
· rw [if_pos hx2, boundaryTwoSupToBoundaryCircle_hornIcc]
  have hmem : (x : ℝ) - 1 ∈ Set.Icc (-1 : ℝ) 1 :=
    ⟨by linarith [hxIco.1], by linarith⟩
  have hp :
    Set.projIcc (-1 : ℝ) 1 (by norm_num) ((x : ℝ) - 1) =
      ⟨(x : ℝ) - 1, hmem⟩ :=
    Set.projIcc_of_mem _ hmem
  rw [hp]
  change (((x : ℝ) - 1) + 1 : ℝ) : AddCircle (3 : ℝ) =
    ((x : ℝ) : AddCircle (3 : ℝ))
  congr 1
  ring
· rw [if_neg hx2, boundaryTwoSupToBoundaryCircle_unitInterval]
  have hmem : (x : ℝ) - 2 ∈ Set.Icc (0 : ℝ) 1 :=
    ⟨by linarith, by linarith [hxIco.2]⟩
  have hp :
    Set.projIcc (0 : ℝ) 1 zero_le_one ((x : ℝ) - 2) =
      ⟨(x : ℝ) - 2, hmem⟩ :=
    Set.projIcc_of_mem _ hmem
  rw [hp]
  change
    ((2 + (TopCat.I.homeomorph.{0}
      (TopCat.I.homeomorph.{0}.symm ⟨(x : ℝ) - 2, hmem⟩) : ℝ) : ℝ) :
      AddCircle (3 : ℝ)) =
      ((x : ℝ) : AddCircle (3 : ℝ))
  rw [Homeomorph.apply_symm_apply]
  change ((2 + ((x : ℝ) - 2) : ℝ) : AddCircle (3 : ℝ)) =
    ((x : ℝ) : AddCircle (3 : ℝ))
  congr 1
  ring

/-- The last-edge pushout is isomorphic to the concrete additive circle. -/
noncomputable def boundaryTwoSupIsoBoundaryCircle :
  |((boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces :
    (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) : SSet.{0})| ≅
  boundaryCircle where
hom := boundaryTwoSupToBoundaryCircle
inv := boundaryCircleToBoundaryTwoSup
hom_inv_id := boundaryTwoSupToBoundaryCircle_comp_inverse
inv_hom_id :=
  boundaryCircleToBoundaryTwoSup_comp_boundaryTwoSupToBoundaryCircle

/-- Geometric realization of  $\partial\Delta[2]$  is homeomorphic to an additive circle. -/
noncomputable def homeomorph_boundaryTwo_addCircle :
  |boundaryTwoSSet| ≅t AddCircle (3 : ℝ) :=
  TopCat.homeoOfIso
    ((SSet.toTop.mapIso boundaryTwoIsoSup).trans
      boundaryTwoSupIsoBoundaryCircle)

/-! ## A reusable attachment step -/

/-- The short-exact gluing step for the simplicial--singular comparison.

```

If simplicial chains and singular chains form compatible short exact sequences, and the comparison is a quasi-isomorphism on the first two terms, then it is a quasi-isomorphism on the glued third term. In an

attachment induction, the remaining geometric work is therefore exactly to construct these two short exact sequences and the compatibility squares. -/

```
theorem simplicialSingularComparison_quasiIso_of_shortExact
  (R : ModuleCat.{0} k) (X1 X2 X3 : SSet.{0})
  (a : X1.chainComplex R → X2.chainComplex R)
  (b : X2.chainComplex R → X3.chainComplex R)
  (a' : singularChainsOfRealization R X1 →
    singularChainsOfRealization R X2)
  (b' : singularChainsOfRealization R X2 →
    singularChainsOfRealization R X3)
  (hab : a >> b = 0) (hab' : a' >> b' = 0)
  (h1 : simplicialSingularComparison R X1 >> a' =
    a >> simplicialSingularComparison R X2)
  (h2 : simplicialSingularComparison R X2 >> b' =
    b >> simplicialSingularComparison R X3)
  (hsource : (ShortComplex.mk a b hab).ShortExact)
  (htarget : (ShortComplex.mk a' b' hab').ShortExact)
  [QuasiIso (simplicialSingularComparison R X1)]
  [QuasiIso (simplicialSingularComparison R X2)] :
  QuasiIso (simplicialSingularComparison R X3) := by
  let S := ShortComplex.mk a b hab
  let T := ShortComplex.mk a' b' hab'
  let φ : S → T := ShortComplex.Hom.mk
    (simplicialSingularComparison R X1)
    (simplicialSingularComparison R X2)
    (simplicialSingularComparison R X3) h1 h2
  exact HomologicalComplex.HomologySequence.quasiIso_τ3
  φ hsource htarget inferInstance inferInstance
```

/-! ## The zero-simplex base case -/

/-- The realization of the standard zero-simplex is terminal. -/

```
def isTerminal_toTop_stdSimplex_zero :
  CategoryTheory.Limits.IsTerminal |(SSet.stdSimplex.obj [0])| :=
  (TopCat.isTerminalPUnit.ofIso
    (TopCat.isoOfHomeo
      (Homeomorph.homeomorphOfUnique
        (SimplexCategory.toTop.obj [0]) PUnit)).symm).ofIso
    (SSet.toTopSimplex.app [0]).symm
```

/-- The adjunction unit is an isomorphism on the standard zero-simplex. -/

```
noncomputable instance isIso_sSetTopAdj_unit_stdSimplex_zero :
  IsIso (sSetTopAdj.unit.app (SSet.stdSimplex.obj [0])) := by
  let hsource := SSet.stdSimplex.isTerminalObj0
  let htarget :=
    isTerminal_toTop_stdSimplex_zero.isTerminalObj
    TopCat.toSSet |(SSet.stdSimplex.obj [0])|
  let e := hsource.uniqueUpToIso htarget
  have h :
    sSetTopAdj.unit.app (SSet.stdSimplex.obj [0]) = e.hom :=
    htarget.hom_ext _ _
  rw [h]
  infer_instance
```

/-- On the zero-simplex, the chain comparison is itself an isomorphism. -/

```
noncomputable instance (R : ModuleCat.{0} k) :
  IsIso (simplicialSingularComparison R (SSet.stdSimplex.obj [0])) := by
  let I : IsIso (sSetTopAdj.unit.app (SSet.stdSimplex.obj [0])) :=
    isIso_sSetTopAdj_unit_stdSimplex_zero
  change IsIso
    (((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map
      (sSetTopAdj.unit.app (SSet.stdSimplex.obj [0])))
```

```

exact Functor.map_isIso _ _

/-- The simplicial--singular comparison is a quasi-isomorphism for the
standard zero-simplex. -/
instance (R : ModuleCat.{0} k) :
  QuasiIso
    (simplicialSingularComparison R (SSet.stdSimplex.obj [0])) :=
  quasiIso_of_isIso _

/-! ## The disconnected two-vertex base case -/

/-- The realization of the empty subcomplex is initial. -/
def isInitial_toTop_boundaryTwoBot :
  IsInitial
    |(( $\perp$  : (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) : SSet.{0})| :=
  SSet.Subcomplex.isInitialBot.isInitialObj SSet.toTop

/-- The two realized vertices, labelled by `Bool`. -/
noncomputable def boundaryTwoVerticesToBool :
  |(boundaryTwoVertices : SSet.{0})|  $\longrightarrow$  TopCat.of Bool :=
  toTop_boundaryTwoVertices_isPushout.desc
  (TopCat.const false) (TopCat.const true)
  (isInitial_toTop_boundaryTwoBot.hom_ext _ _)

noncomputable def boundaryTwoVertexOnePoint :
  |(boundaryTwoVertices : SSet.{0})| :=
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom  $\gg$ 
      SSet.Subcomplex.homOfLE
        (le_sup_left :
          SSet.stdSimplex.face ({1} : Finset (Fin 3))  $\leq$  boundaryTwoVertices))
    (default : |( $\Delta$ [0] : SSet.{0})|)

noncomputable def boundaryTwoVertexTwoPoint :
  |(boundaryTwoVertices : SSet.{0})| :=
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom  $\gg$ 
      SSet.Subcomplex.homOfLE
        (le_sup_right :
          SSet.stdSimplex.face ({2} : Finset (Fin 3))  $\leq$  boundaryTwoVertices))
    (default : |( $\Delta$ [0] : SSet.{0})|)

/-- The inverse labelling map from the discrete two-point space. -/
noncomputable def boolToBoundaryTwoVertices :
  TopCat.of Bool  $\longrightarrow$  |(boundaryTwoVertices : SSet.{0})| :=
  TopCat.ofHom
    { toFun := fun b => if b then boundaryTwoVertexTwoPoint
      else boundaryTwoVertexOnePoint
      continuous_toFun := continuous_of_discreteTopology }

lemma boundaryTwoVerticesToBool_vertex_one :
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom  $\gg$ 
      SSet.Subcomplex.homOfLE
        (le_sup_left :
          SSet.stdSimplex.face ({1} : Finset (Fin 3))  $\leq$ 
            boundaryTwoVertices))  $\gg$ 
    boundaryTwoVerticesToBool =
  (TopCat.const false :
    |( $\Delta$ [0] : SSet.{0})|  $\longrightarrow$  TopCat.of Bool) := by
  unfold boundaryTwoVerticesToBool
  rw [SSet.toTop.map_comp]
  simp only [Category.assoc]

```

```

rw [
  toTop_boundaryTwoVertices_isPushout.inl_desc]
ext
rfl

lemma boundaryTwoVerticesToBool_vertex_two :
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
     SSet.Subcomplex.homOfLE
      (le_sup_right :
        SSet.stdSimplex.face ({2} : Finset (Fin 3)) ≤
        boundaryTwoVertices)) >>
    boundaryTwoVerticesToBool =
  (TopCat.const true :
    |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  TopCat.of Bool) := by
  unfold boundaryTwoVerticesToBool
  rw [SSet.toTop.map_comp]
  simp only [Category.assoc]
  rw [
    toTop_boundaryTwoVertices_isPushout.inr_desc]
  ext
  rfl

lemma boolToBoundaryTwoVertices_comp_boundaryTwoVerticesToBool :
  boolToBoundaryTwoVertices >> boundaryTwoVerticesToBool =  $\mathbb{K}$  _ := by
  ext b
  cases b
  · change boundaryTwoVerticesToBool boundaryTwoVertexOnePoint = false
    have h := congrArg
      (fun f : |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  TopCat.of Bool =>
        f (default : |( $\Delta[0]$  : SSet.{0})|))
      boundaryTwoVerticesToBool_vertex_one
    simp [boundaryTwoVertexOnePoint] using h
  · change boundaryTwoVerticesToBool boundaryTwoVertexTwoPoint = true
    have h := congrArg
      (fun f : |( $\Delta[0]$  : SSet.{0})|  $\longrightarrow$  TopCat.of Bool =>
        f (default : |( $\Delta[0]$  : SSet.{0})|))
      boundaryTwoVerticesToBool_vertex_two
    simp [boundaryTwoVertexTwoPoint] using h

lemma toTop_stdSimplex_zero_eq_default
  (x : |( $\Delta[0]$  : SSet.{0})|) : x = default := by
  have h := isTerminal_toTop_stdSimplex_zero.hom_ext
  (TopCat.const x : TopCat.of PUnit  $\longrightarrow$  |( $\Delta[0]$  : SSet.{0})|)
  (TopCat.const default : TopCat.of PUnit  $\longrightarrow$  |( $\Delta[0]$  : SSet.{0})|)
  simp using congrArg
  (fun f : TopCat.of PUnit  $\longrightarrow$  |( $\Delta[0]$  : SSet.{0})| => f PUnit.unit) h

lemma boundaryTwoVerticesToBool_comp_boolToBoundaryTwoVertices :
  boundaryTwoVerticesToBool >> boolToBoundaryTwoVertices =  $\mathbb{K}$  _ := by
  apply toTop_boundaryTwoVertices_isPushout.hom_ext
  · rw [← cancel_epi
      (SSet.toTop.map
        (SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom)]
    simp only [← Category.assoc]
    rw [← SSet.toTop.map_comp]
    rw [boundaryTwoVerticesToBool_vertex_one]
  ext x
  change boundaryTwoVertexOnePoint =
    SSet.toTop.map
      ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
       SSet.Subcomplex.homOfLE le_sup_left) x
  rw [toTop_stdSimplex_zero_eq_default x]

```

```

    rfl
  · rw [← cancel_epi
      (SSet.toTop.map
        (SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom)]
    simp only [← Category.assoc]
    rw [← SSet.toTop.map_comp]
    rw [boundaryTwoVerticesToBool_vertex_two]
    ext x
    change boundaryTwoVertexTwoPoint =
      SSet.toTop.map
        ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
          SSet.Subcomplex.homOfLE le_sup_right) x
    rw [toTop_stdSimplex_zero_eq_default x]
    rfl

/-- The two-vertex realization is a discrete two-point space. -/
noncomputable def homeomorph_boundaryTwoVertices_bool :
  |(boundaryTwoVertices : SSet.{0})|  $\simeq_t$  Bool :=
  TopCat.homeoOfIso
    { hom := boundaryTwoVerticesToBool
      inv := boolToBoundaryTwoVertices
      hom_inv_id := boundaryTwoVerticesToBool_comp_boolToBoundaryTwoVertices
      inv_hom_id := boolToBoundaryTwoVertices_comp_boundaryTwoVerticesToBool }

/-- The constant simplicial set on the two-element type. -/
abbrev boolSSet : SSet.{0} :=
  (Functor.const SimplexCategoryop).obj Bool

/-- Label the two vertex components simplicially. -/
noncomputable def boundaryTwoVerticesToBoolSSet :
  (boundaryTwoVertices : SSet.{0})  $\longrightarrow$  boolSSet :=
  boundaryTwoVertices_isPushout.desc
    (SSet.const false) (SSet.const true)
    (SSet.Subcomplex.isInitialBot.hom_ext _ _)

lemma boundaryTwoVerticesToBoolSSet_face_one :
  SSet.Subcomplex.homOfLE
    (le_sup_left :
      SSet.stdSimplex.face ({1} : Finset (Fin 3))  $\leq$  boundaryTwoVertices) >>
    boundaryTwoVerticesToBoolSSet =
  SSet.const false := by
  exact boundaryTwoVertices_isPushout.inl_desc _ _ _

lemma boundaryTwoVerticesToBoolSSet_face_two :
  SSet.Subcomplex.homOfLE
    (le_sup_right :
      SSet.stdSimplex.face ({2} : Finset (Fin 3))  $\leq$  boundaryTwoVertices) >>
    boundaryTwoVerticesToBoolSSet =
  SSet.const true := by
  exact boundaryTwoVertices_isPushout.inr_desc _ _ _

noncomputable def boundaryTwoVertexOneSimplex :
  (boundaryTwoVertices : SSet.{0})  $\llbracket 0 \rrbracket$  :=
  (((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
    SSet.Subcomplex.homOfLE le_sup_left).app _)
  (SSet.stdSimplex.obj0Equiv.symm 0)

noncomputable def boundaryTwoVertexTwoSimplex :
  (boundaryTwoVertices : SSet.{0})  $\llbracket 0 \rrbracket$  :=
  (((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
    SSet.Subcomplex.homOfLE le_sup_right).app _)
  (SSet.stdSimplex.obj0Equiv.symm 0)

```



```

/-- Reinsert a Boolean-labelled constant simplex as the corresponding
vertex of `boundaryTwoVertices`. -/
noncomputable def boolSSetToBoundaryTwoVertices :
  boolSSet → (boundaryTwoVertices : SSet.{0}) where
  app n := TypeCat.ofHom (fun b => by
    change Bool at b
    exact if b then
      (SSet.const (X := boolSSet) boundaryTwoVertexTwoSimplex).app n b
    else
      (SSet.const (X := boolSSet) boundaryTwoVertexOneSimplex).app n b)
  naturality X Y f := by
    ext b
    change Bool at b
    cases b
    · have h := congrArg
      (fun g => g false)
      ((SSet.const (X := boolSSet)
        boundaryTwoVertexOneSimplex).naturality f)
      simp using h
    · have h := congrArg
      (fun g => g true)
      ((SSet.const (X := boolSSet)
        boundaryTwoVertexTwoSimplex).naturality f)
      simp using h

lemma boolSSetToBoundaryTwoVertices_comp_boundaryTwoVerticesToBoolSSet :
  boolSSetToBoundaryTwoVertices >> boundaryTwoVerticesToBoolSSet =  $\mathbb{K}$  _ := by
  have h1 :
    SSet.const boundaryTwoVertexOneSimplex >>
      boundaryTwoVerticesToBoolSSet =
      (SSet.const false : boolSSet → boolSSet) := by
  rw [SSet.const_comp]
  congr 1
  have h := congrArg
    (fun f : (SSet.stdSimplex.face
      ({1} : Finset (Fin 3)) : SSet.{0}) → boolSSet =>
      f.app _ (((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom.app _))
        (SSet.stdSimplex.objEquiv.symm 0)))
    boundaryTwoVerticesToBoolSSet_face_one
  simp [boundaryTwoVertexOneSimplex] using h
  have h2 :
    SSet.const boundaryTwoVertexTwoSimplex >>
      boundaryTwoVerticesToBoolSSet =
      (SSet.const true : boolSSet → boolSSet) := by
  rw [SSet.const_comp]
  congr 1
  have h := congrArg
    (fun f : (SSet.stdSimplex.face
      ({2} : Finset (Fin 3)) : SSet.{0}) → boolSSet =>
      f.app _ (((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom.app _))
        (SSet.stdSimplex.objEquiv.symm 0)))
    boundaryTwoVerticesToBoolSSet_face_two
  simp [boundaryTwoVertexTwoSimplex] using h
  ext n b
  cases b
  · change
    ((SSet.const (X := boolSSet) boundaryTwoVertexOneSimplex >>
      boundaryTwoVerticesToBoolSSet).app n) false =
    (( $\mathbb{K}$  boolSSet) : boolSSet → boolSSet).app n) false
  rw [h1]
  rfl
  · change
    ((SSet.const (X := boolSSet) boundaryTwoVertexTwoSimplex >>

```

```

        boundaryTwoVerticesToBoolSSet).app n) true =
        ((! boolSSet) : boolSSet → boolSSet).app n) true
    rw [h₂]
    rfl

lemma boundaryTwoVerticesToBoolSSet_comp_boolSSetToBoundaryTwoVertices :
  boundaryTwoVerticesToBoolSSet >> boolSSetToBoundaryTwoVertices = ! _ := by
  apply boundaryTwoVertices_isPushout.hom_ext
  · rw [← Category.assoc, boundaryTwoVerticesToBoolSSet_face_one]
    rw [Category.comp_id]
    have hconst :
      (SSet.const false :
        (SSet.stdSimplex.face
          ({1} : Finset (Fin 3)) : SSet.{0}) → boolSSet) >>
        boolSSetToBoundaryTwoVertices =
        SSet.const boundaryTwoVertexOneSimplex := by
      rw [SSet.const_comp]
      congr 1
    rw [hconst]
    rw [← cancel_mono boundaryTwoVertices.ι,
      SSet.Subcomplex.homOfLE_ι, SSet.const_comp]
    rw [← cancel_epi
      (SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom]
    rw [SSet.comp_const, faceSingletonIso_hom_comp_ι]
    congr 1
  · rw [← Category.assoc, boundaryTwoVerticesToBoolSSet_face_two]
    rw [Category.comp_id]
    have hconst :
      (SSet.const true :
        (SSet.stdSimplex.face
          ({2} : Finset (Fin 3)) : SSet.{0}) → boolSSet) >>
        boolSSetToBoundaryTwoVertices =
        SSet.const boundaryTwoVertexTwoSimplex := by
      rw [SSet.const_comp]
      congr 1
    rw [hconst]
    rw [← cancel_mono boundaryTwoVertices.ι,
      SSet.Subcomplex.homOfLE_ι, SSet.const_comp]
    rw [← cancel_epi
      (SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom]
    rw [SSet.comp_const, faceSingletonIso_hom_comp_ι]
    congr 1

/-- The two-vertex simplicial set is the constant simplicial set on `Bool`. -/
noncomputable def boundaryTwoVerticesIsoBoolSSet :
  (boundaryTwoVertices : SSet.{0}) ≅ boolSSet where
  hom := boundaryTwoVerticesToBoolSSet
  inv := boolSSetToBoundaryTwoVertices
  hom_inv_id := boundaryTwoVerticesToBoolSSet_comp_boolSSetToBoundaryTwoVertices
  inv_hom_id := boolSSetToBoundaryTwoVertices_comp_boundaryTwoVerticesToBoolSSet

noncomputable instance boundaryTwoVertices_totallyDisconnectedSpace :
  TotallyDisconnectedSpace |(boundaryTwoVertices : SSet.{0})| :=
  homeomorph_boundaryTwoVertices_bool.symm.totallyDisconnectedSpace

/-- Identify the singular simplicial set of the realized two vertices with
the same constant Boolean simplicial set. -/
noncomputable def singularBoundaryTwoVerticesIsoBoolSSet :
  TopCat.toSSet.obj |(boundaryTwoVertices : SSet.{0})| ≅ boolSSet :=
  (TopCat.toSSet.mapIso
    (TopCat.isoOfHomeo homeomorph_boundaryTwoVertices_bool)).trans
  (TopCat.toSSetIsoConst (TopCat.of Bool))

```

```

lemma singularBoundaryTwoVerticesIsoBoolSSet_app_zero
  (q : (TopCat.toSSet.obj |(boundaryTwoVertices : SSet.{0})|) _[0]) :
  singularBoundaryTwoVerticesIsoBoolSSet.hom.app _ q =
    homeomorph_boundaryTwoVertices_bool
      (TopCat.toSSetObjEquiv q) := by
dsimp [singularBoundaryTwoVerticesIsoBoolSSet,
  TopCat.toSSetIsoConst]
change homeomorph_boundaryTwoVertices_bool
  ((TopCat.toSSetObjEquiv _ _ q) (Classical.arbitrary _)) =
    homeomorph_boundaryTwoVertices_bool
      ((TopCat.toSSetObjEquiv _ _ q) default)
rw [Subsingleton.elim (Classical.arbitrary _) default]

lemma toSSetObjEquiv_unit_app
  (X : SSet.{0}) (x : X _[0]) :
  TopCat.toSSetObjEquiv
    ((sSetTopAdj.unit.app X).app _ x) =
    SSet.toTop.map (SSet.yonedaEquiv.symm x)
    (default : |(Δ[0] : SSet.{0})|) := by
let f : (Δ[0] : SSet.{0}) → X := SSet.yonedaEquiv.symm x
have h :
  f >> sSetTopAdj.unit.app X =
    SSet.const (TopCat.toSSetObjEquiv.symm
      (SSet.toTop.map f (default : |(Δ[0] : SSet.{0})|))) := by
calc
  f >> sSetTopAdj.unit.app X =
    f >> (sSetTopAdj.homEquiv X |X|) (⋈ |X|) := by
      rw [Adjunction.homEquiv_unit]
      simp
  _ = (sSetTopAdj.homEquiv Δ[0] |X|)
    (SSet.toTop.map f >> ⋈ |X|) :=
    (sSetTopAdj.homEquiv_naturality_left f (⋈ |X|)).symm
  _ = (sSetTopAdj.homEquiv Δ[0] |X|)
    (SSet.toTop.map f) := by simp
  _ = _ := sSetTopAdj_homEquiv_stdSimplex_zero _
have hx := congrArg
  (fun g : (Δ[0] : SSet.{0}) → TopCat.toSSet.obj |X| =>
    g.app _ (SSet.stdSimplex.objEquiv.symm 0)) h
have hx' := congrArg TopCat.toSSetObjEquiv hx
simp [f] using hx'

lemma boundaryTwoVertices_unit_comp_bool :
  sSetTopAdj.unit.app (boundaryTwoVertices : SSet.{0}) >>
    singularBoundaryTwoVerticesIsoBoolSSet.hom =
    boundaryTwoVerticesIsoBoolSSet.hom := by
apply boundaryTwoVertices_isPushout.hom_ext
· change
  SSet.Subcomplex.homOfLE le_sup_left >>
    sSetTopAdj.unit.app (boundaryTwoVertices : SSet.{0}) >>
    singularBoundaryTwoVerticesIsoBoolSSet.hom =
    SSet.Subcomplex.homOfLE le_sup_left >>
    boundaryTwoVerticesToBoolSSet
rw [boundaryTwoVerticesToBoolSSet_face_one]
rw [← cancel_epi
  (SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom]
apply (SSet.yonedaEquiv (X := boolSSet) (n := [0])).injective
change
  (singularBoundaryTwoVerticesIsoBoolSSet.hom.app _)
    ((sSetTopAdj.unit.app (boundaryTwoVertices : SSet.{0})).app _
      boundaryTwoVertexOneSimplex) = false
rw [singularBoundaryTwoVerticesIsoBoolSSet_app_zero]
rw [toSSetObjEquiv_unit_app]
have hg :

```

```

    SSet.yonedaEquiv.symm boundaryTwoVertexOneSimplex =
      (SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
      SSet.Subcomplex.homOfLE le_sup_left := by
change
  (SSet.yonedaEquiv
    (X := (boundaryTwoVertices : SSet.{0})) (n := [0])).symm
    ((SSet.yonedaEquiv
      (X := (boundaryTwoVertices : SSet.{0})) (n := [0]))
      ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
        SSet.Subcomplex.homOfLE le_sup_left)) =
    -
    exact Equiv.symm_apply_apply _ _
rw [hg]
change boundaryTwoVerticesToBool boundaryTwoVertexOnePoint = false
have h := congrArg
  (fun f : |( $\Delta[0]$  : SSet.{0})|  $\rightarrow$  TopCat.of Bool =>
    f (default : |( $\Delta[0]$  : SSet.{0})|))
  boundaryTwoVerticesToBool_vertex_one
simp [boundaryTwoVertexOnePoint] using h
change
  SSet.Subcomplex.homOfLE le_sup_right >>
  sSetTopAdj.unit.app (boundaryTwoVertices : SSet.{0}) >>
  singularBoundaryTwoVerticesIsoBoolSSet.hom =
  SSet.Subcomplex.homOfLE le_sup_right >>
  boundaryTwoVerticesToBoolSSet
rw [boundaryTwoVerticesToBoolSSet_face_two]
rw [← cancel_epi
  (SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom]
apply (SSet.yonedaEquiv (X := boolSSet) (n := [0])).injective
change
  (singularBoundaryTwoVerticesIsoBoolSSet.hom.app _)
  ((sSetTopAdj.unit.app (boundaryTwoVertices : SSet.{0})).app _
    boundaryTwoVertexTwoSimplex) = true
rw [singularBoundaryTwoVerticesIsoBoolSSet_app_zero]
rw [toSSetObjEquiv_unit_app]
have hg :
  SSet.yonedaEquiv.symm boundaryTwoVertexTwoSimplex =
    (SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
    SSet.Subcomplex.homOfLE le_sup_right := by
change
  (SSet.yonedaEquiv
    (X := (boundaryTwoVertices : SSet.{0})) (n := [0])).symm
    ((SSet.yonedaEquiv
      (X := (boundaryTwoVertices : SSet.{0})) (n := [0]))
      ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
        SSet.Subcomplex.homOfLE le_sup_right)) =
    -
    exact Equiv.symm_apply_apply _ _
rw [hg]
change boundaryTwoVerticesToBool boundaryTwoVertexTwoPoint = true
have h := congrArg
  (fun f : |( $\Delta[0]$  : SSet.{0})|  $\rightarrow$  TopCat.of Bool =>
    f (default : |( $\Delta[0]$  : SSet.{0})|))
  boundaryTwoVerticesToBool_vertex_two
simp [boundaryTwoVertexTwoPoint] using h

/-- The adjunction unit is an isomorphism for the disconnected pair of
vertices. -/
noncomputable instance isIso_sSetTopAdj_unit_boundaryTwoVertices :
  IsIso (sSetTopAdj.unit.app (boundaryTwoVertices : SSet.{0})) := by
letI : IsIso
  (sSetTopAdj.unit.app (boundaryTwoVertices : SSet.{0}) >>
    singularBoundaryTwoVerticesIsoBoolSSet.hom) := by

```

```

    rw [boundaryTwoVertices_unit_comp_bool]
    infer_instance
exact IsIso.of_isIso_comp_right
  (sSetTopAdj.unit.app (boundaryTwoVertices : SSet.{0}))
  singularBoundaryTwoVerticesIsoBoolSSet.hom

/-- The canonical simplicial--singular comparison is a quasi-isomorphism
on the two endpoints of the last edge. -/
noncomputable instance simplicialSingularComparison_boundaryTwoVertices_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso
    (simplicialSingularComparison R (boundaryTwoVertices : SSet.{0})) := by
letI : IsIso
  (simplicialSingularComparison R
    (boundaryTwoVertices : SSet.{0})) := by
change IsIso
  (((SSet.chainComplexFunctor (ModuleCat.{0} k)).obj R).map
    (sSetTopAdj.unit.app (boundaryTwoVertices : SSet.{0})))
  exact Functor.map_isIso _ _
exact quasiIso_of_isIso _

/-- The same base case, transported to the actual edge--horn
intersection. -/
noncomputable instance
  simplicialSingularComparison_boundaryTwoIntersection_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso
    (simplicialSingularComparison R
      ((boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces : _) :
        SSet.{0})) := by
    rw [boundaryTwoFirstFace_inf]
  infer_instance

```

A.23 PlatonicSolids/SimplicialSingularBoundaryTwo.lean

Open-cover gluing proof of the canonical comparison for $\partial\Delta[2]$ (1541 lines).

```

import Mathlib.Analysis.Convex.Contractible
import Mathlib.Topology.OpenPartialHomeomorph.Basic
import PlatonicSolids.SimplicialSingularComparison
import PlatonicSolids.SingularExcision.OpenCoverQI

/-!
# The boundary-two comparison

This file supplies the open-cover bridge from the simplicial
Mayer--Vietoris sequence for  $\partial\Delta[2]$  to singular chains.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex Set
open scoped ContinuousMap Simplicial Topology

noncomputable section

universe u

variable {k : Type} [Ring k]

local instance boundaryTwoCirclePeriodPositive : Fact (0 < (3 :  $\mathbb{R}$ )) :=
  ⟨by norm_num⟩

/-- The puncture in the interior of the horn arc. -/
def hornInteriorPoint : AddCircle (3 :  $\mathbb{R}$ ) := (1 :  $\mathbb{R}$ )

```

```

/-- The puncture in the interior of the last-edge arc. -/
def lastEdgeInteriorPoint : AddCircle (3 : ℝ) := ((5 / 2 : ℝ) : AddCircle (3 : ℝ))

lemma hornInteriorPoint_ne_lastEdgeInteriorPoint :
  hornInteriorPoint ≠ lastEdgeInteriorPoint := by
  intro h
  have h' : (1 : ℝ) = 5 / 2 := by
    apply (AddCircle.coe_eq_coe_iff_of_mem_Ico
      (p := (3 : ℝ)) (a := (0 : ℝ))
      (x := (1 : ℝ)) (y := (5 / 2 : ℝ))
      (by norm_num) (by norm_num)).1
    exact h
  norm_num at h'

/-- An open neighborhood of the last closed edge. -/
def boundaryTwoLastEdgeOpen : Set |boundaryTwoSSet| :=
  boundaryTwoToBoundaryCircle -1 ({hornInteriorPoint}c)

/-- An open neighborhood of the two-edge horn. -/
def boundaryTwoHornOpen : Set |boundaryTwoSSet| :=
  boundaryTwoToBoundaryCircle -1 ({lastEdgeInteriorPoint}c)

lemma isOpen_boundaryTwoLastEdgeOpen : IsOpen boundaryTwoLastEdgeOpen :=
  isOpen_compl_singleton.preimage boundaryTwoToBoundaryCircle.hom.continuous

lemma isOpen_boundaryTwoHornOpen : IsOpen boundaryTwoHornOpen :=
  isOpen_compl_singleton.preimage boundaryTwoToBoundaryCircle.hom.continuous

lemma boundaryTwo_open_cover :
  boundaryTwoLastEdgeOpen ∪ boundaryTwoHornOpen = univ := by
  ext x
  simp only [boundaryTwoLastEdgeOpen, boundaryTwoHornOpen, mem_union,
    mem_preimage, mem_compl_iff, mem_singleton_iff, mem_univ, iff_true]
  by_contra h
  push_neg at h
  exact hornInteriorPoint_ne_lastEdgeInteriorPoint (h.1.symm.trans h.2)

/-- The punctured circle used around the last edge has an open-interval
coordinate. -/
noncomputable def boundaryTwoLastEdgeOpenHomeoIoo :
  ↑boundaryTwoLastEdgeOpen ≃t ↑(Ioo (1 : ℝ) 4) :=
  ((homeomorph_boundaryTwo_addCircle.subtype
    (p := fun x => x ∈ boundaryTwoLastEdgeOpen)
    (q := fun y => y ∈
      (AddCircle.openPartialHomeomorphCoe (3 : ℝ) (1 : ℝ)).target)
    (fun _ => Iff.rfl)).trans
    (AddCircle.openPartialHomeomorphCoe
      (3 : ℝ) (1 : ℝ)).toHomeomorphSourceTarget.symm).trans
    (Homeomorph.setCongr (by norm_num)))

/-- The punctured circle used around the horn has an open-interval
coordinate. -/
noncomputable def boundaryTwoHornOpenHomeoIoo :
  ↑boundaryTwoHornOpen ≃t ↑(Ioo (5 / 2 : ℝ) (11 / 2 : ℝ)) :=
  ((homeomorph_boundaryTwo_addCircle.subtype
    (p := fun x => x ∈ boundaryTwoHornOpen)
    (q := fun y => y ∈
      (AddCircle.openPartialHomeomorphCoe
        (3 : ℝ) (5 / 2 : ℝ)).target)
    (fun _ => Iff.rfl)).trans
    (AddCircle.openPartialHomeomorphCoe
      (3 : ℝ) (5 / 2 : ℝ)).toHomeomorphSourceTarget.symm).trans
    (Homeomorph.setCongr (by norm_num)))

```

```

(Homeomorph.setCongr (by norm_num))

noncomputable instance boundaryTwoLastEdgeOpen_contractible :
  ContractibleSpace  $\uparrow$ boundaryTwoLastEdgeOpen := by
  letI : ContractibleSpace  $\uparrow$ (Ioo (1 :  $\mathbb{R}$ ) 4) :=
    (convex_Ioo (1 :  $\mathbb{R}$ ) 4).contractibleSpace
    (2, by norm_num)
  exact boundaryTwoLastEdgeOpenHomeoIoo.contractibleSpace

noncomputable instance boundaryTwoHornOpen_contractible :
  ContractibleSpace  $\uparrow$ boundaryTwoHornOpen := by
  letI : ContractibleSpace  $\uparrow$ (Ioo (5 / 2 :  $\mathbb{R}$ ) (11 / 2 :  $\mathbb{R}$ )) :=
    (convex_Ioo (5 / 2 :  $\mathbb{R}$ ) (11 / 2 :  $\mathbb{R}$ )).contractibleSpace
    (3, by norm_num)
  exact boundaryTwoHornOpenHomeoIoo.contractibleSpace

/-- Inclusion of the last edge into the whole simplicial boundary. -/
def boundaryTwoLastEdgeIncl :
  |(boundaryTwoFirstFace : SSet.{0})|  $\rightarrow$  |boundaryTwoSSet| :=
  SSet.toTop.map (SSet.Subcomplex.homOfLE
    (by
      rw [boundary_two_eq_face_sup]
      exact (le_sup_left :
        boundaryTwoFirstFace  $\leq$ 
        boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces)))

/-- Inclusion of the horn into the whole simplicial boundary. -/
def boundaryTwoHornIncl :
  |(boundaryTwoRemainingFaces : SSet.{0})|  $\rightarrow$  |boundaryTwoSSet| :=
  SSet.toTop.map (SSet.Subcomplex.homOfLE
    (by
      rw [boundary_two_eq_face_sup]
      exact (le_sup_right :
        boundaryTwoRemainingFaces  $\leq$ 
        boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces)))

lemma boundaryTwoLastEdgeIncl_circle :
  boundaryTwoLastEdgeIncl  $\gg$  boundaryTwoToBoundaryCircle =
  lastEdgeToBoundaryCircle := by
  change
  SSet.toTop.map (SSet.Subcomplex.homOfLE _)  $\gg$ 
  SSet.toTop.map boundaryTwoIsoSup.hom  $\gg$ 
  boundaryTwoSupToBoundaryCircle =
  lastEdgeToBoundaryCircle
  rw [ $\leftarrow$  Category.assoc,  $\leftarrow$  SSet.toTop.map_comp]
  have h :
    SSet.Subcomplex.homOfLE
    (show boundaryTwoFirstFace  $\leq$ 
      (SSet.boundary 2 :
        (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) by
      rw [boundary_two_eq_face_sup]
      exact le_sup_left)  $\gg$ 
    boundaryTwoIsoSup.hom =
    SSet.Subcomplex.homOfLE
    (le_sup_left : boundaryTwoFirstFace  $\leq$ 
      boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces) := rfl
  rw [h, boundaryTwoSupToBoundaryCircle_firstFace]

lemma boundaryTwoHornIncl_circle :
  boundaryTwoHornIncl  $\gg$  boundaryTwoToBoundaryCircle =
  boundaryTwoRemainingFacesToBoundaryCircle := by
  change
  SSet.toTop.map (SSet.Subcomplex.homOfLE _)  $\gg$ 

```

```

      SSet.toTop.map boundaryTwoIsoSup.hom >>
        boundaryTwoSupToBoundaryCircle =
        boundaryTwoRemainingFacesToBoundaryCircle
rw [← Category.assoc, ← SSet.toTop.map_comp]
have h :
  SSet.Subcomplex.homOfLE
    (show boundaryTwoRemainingFaces ≤
      (SSet.boundary 2 :
        (SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex) by
        rw [boundary_two_eq_face_sup]
        exact le_sup_right) >>
      boundaryTwoIsoSup.hom =
    SSet.Subcomplex.homOfLE
      (le_sup_right : boundaryTwoRemainingFaces ≤
        boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces) := rfl
rw [h, boundaryTwoSupToBoundaryCircle_remainingFaces]

lemma lastEdgeToBoundaryCircle_ne_hornInteriorPoint
  (x : |(boundaryTwoFirstFace : SSet.{0})|) :
  lastEdgeToBoundaryCircle x ≠ hornInteriorPoint := by
  intro h
  let t := lastEdgeIsoI.hom x
  have hmem : 2 + (TopCat.I.homeomorph t : ℝ) ∈ Ico (1 : ℝ) 4 :=
    ⟨by linarith [(TopCat.I.homeomorph t).property.1],
      by linarith [(TopCat.I.homeomorph t).property.2]⟩
  have hmem' : 2 + (TopCat.I.homeomorph t : ℝ) ∈
    Ico (1 : ℝ) (1 + 3) := by
    convert hmem using 1 <.> norm_num
  have hone : (1 : ℝ) ∈ Ico (1 : ℝ) (1 + 3) := by norm_num
  have hr : 2 + (TopCat.I.homeomorph t : ℝ) = 1 :=
    (AddCircle.coe_eq_coe_iff_of_mem_Ico
      (p := (3 : ℝ)) (a := (1 : ℝ)) hmem' hone).1 h
  linarith [(TopCat.I.homeomorph t).property.1]

lemma boundaryTwoRemainingFacesToBoundaryCircle_ne_lastEdgeInteriorPoint
  (x : |(boundaryTwoRemainingFaces : SSet.{0})|) :
  boundaryTwoRemainingFacesToBoundaryCircle x ≠ lastEdgeInteriorPoint := by
  intro h
  let t := horn_two_zero_toIcc
    (SSet.toTop.map boundaryTwoRemainingFacesIsoHorn.hom x)
  have ht : (t : ℝ) ∈ Icc (-1 : ℝ) 1 := t.property
  have hmem : (t : ℝ) + 1 ∈ Ico (0 : ℝ) 3 :=
    ⟨by linarith [ht.1], by linarith [ht.2]⟩
  have hp : (5 / 2 : ℝ) ∈ Ico (0 : ℝ) 3 := by norm_num
  have hmem' : (t : ℝ) + 1 ∈ Ico (0 : ℝ) (0 + 3) := by
    simpa using hmem
  have hp' : (5 / 2 : ℝ) ∈ Ico (0 : ℝ) (0 + 3) := by
    simpa using hp
  have hr : (t : ℝ) + 1 = 5 / 2 :=
    (AddCircle.coe_eq_coe_iff_of_mem_Ico
      (p := (3 : ℝ)) (a := (0 : ℝ)) hmem' hp').1 h
  linarith [ht.2]

/-- The last edge lands in its punctured-circle neighborhood. -/
def boundaryTwoLastEdgeToOpen :
  |(boundaryTwoFirstFace : SSet.{0})| → TopCat.of ↑boundaryTwoLastEdgeOpen :=
  TopCat.ofHom
  { toFun := fun x => ⟨boundaryTwoLastEdgeIncl x, by
    change boundaryTwoToBoundaryCircle (boundaryTwoLastEdgeIncl x) ≠
      hornInteriorPoint
    rw [← CategoryTheory.comp_apply, boundaryTwoLastEdgeIncl_circle]
    exact lastEdgeToBoundaryCircle_ne_hornInteriorPoint x⟩
    continuous_toFun := by fun_prop }

```



```

/-- The horn lands in its punctured-circle neighborhood. -/
def boundaryTwoHornToOpen :
  |(boundaryTwoRemainingFaces : SSet.{0})| → TopCat.of ↑boundaryTwoHornOpen :=
  TopCat.ofHom
  { toFun := fun x => ⟨boundaryTwoHornIncl x, by
    change boundaryTwoToBoundaryCircle (boundaryTwoHornIncl x) ≠
    lastEdgeInteriorPoint
    rw [← CategoryTheory.comp_apply, boundaryTwoHornIncl_circle]
    exact boundaryTwoRemainingFacesToBoundaryCircle_ne_lastEdgeInteriorPoint x⟩
    continuous_toFun := by fun_prop }

/-! ## Quasi-isomorphisms for the two contractible pieces -/

lemma singularChainMap_quasiIso_of_contractible
  (R : ModuleCat.{0} k) {X Y : TopCat.{0}}
  [ContractibleSpace X] [ContractibleSpace Y] (f : X → Y) :
  QuasiIso
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map f) := by
  letI : PathConnectedSpace X :=
    ContractibleSpace.instPathConnectedSpace
  letI : PathConnectedSpace Y :=
    ContractibleSpace.instPathConnectedSpace
  rw [quasiIso_iff]
  intro n
  rw [quasiIsoAt_iff_isIso_homologyMap]
  by_cases hn : n = 0
  · subst n
    letI hXiso : IsIso (X.singularHomology0 R) := by infer_instance
    letI hYiso : IsIso (Y.singularHomology0 R) := by infer_instance
    have hnat := singularHomology0_natural f R
    let  $\varphi$  := ((singularChainComplexFunctor
      (ModuleCat.{0} k)).obj R).map f
    change IsIso (homologyMap  $\varphi$  0)
    letI hcomp : IsIso
      (homologyMap  $\varphi$  0  $\gg$  Y.singularHomology0 R) := by
      rw [hnat]
      exact hXiso
    exact @IsIso.of_isIso_comp_right _ _ _ _
      (homologyMap  $\varphi$  0) (Y.singularHomology0 R) hYiso hcomp
  · apply IsZero.isIso
    (isZero_singularHomology_of_contractible R hn)
    exact isZero_singularHomology_of_contractible R hn

noncomputable instance boundaryTwoFirstFace_realization_contractible :
  ContractibleSpace |(boundaryTwoFirstFace : SSet.{0})| := by
  letI : ContractibleSpace _root_.unitInterval :=
    (convex_Icc (0 : ℝ) 1).contractibleSpace ⟨0, by norm_num⟩
  letI : ContractibleSpace TopCat.I :=
    TopCat.I.homeomorph.contractibleSpace
  exact (TopCat.homeoOfIso lastEdgeIsoI).contractibleSpace

noncomputable instance boundaryTwoRemainingFaces_realization_contractible :
  ContractibleSpace |(boundaryTwoRemainingFaces : SSet.{0})| := by
  letI : ContractibleSpace |( $\wedge$ [2, 0] : SSet.{0})| :=
    realization_horn_two_zero_contractible
  exact (TopCat.homeoOfIso
    (SSet.toTop.mapIso boundaryTwoRemainingFacesIsoHorn)).contractibleSpace

/-- The comparison from the last-edge simplicial chains to singular chains
of its open neighborhood. -/
noncomputable def boundaryTwoLastEdgeOpenComparison
  (R : ModuleCat.{0} k) :

```

```

(boundaryTwoFirstFace : SSet.{0}).chainComplex R →
  ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
  (TopCat.of ↑boundaryTwoLastEdgeOpen) :=
simplicialSingularComparison R (boundaryTwoFirstFace : SSet.{0}) >>
  ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
  boundaryTwoLastEdgeToOpen

noncomputable instance boundaryTwoLastEdgeOpenComparison_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso (boundaryTwoLastEdgeOpenComparison R) := by
dsimp [boundaryTwoLastEdgeOpenComparison]
letI : QuasiIso
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
   boundaryTwoLastEdgeToOpen) :=
  singularChainMap_quasiIso_of_contractible R _
infer_instance

/-- The comparison from the horn simplicial chains to singular chains of
its open neighborhood. -/
noncomputable def boundaryTwoHornOpenComparison
  (R : ModuleCat.{0} k) :
  (boundaryTwoRemainingFaces : SSet.{0}).chainComplex R →
  ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
  (TopCat.of ↑boundaryTwoHornOpen) :=
simplicialSingularComparison R (boundaryTwoRemainingFaces : SSet.{0}) >>
  ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
  boundaryTwoHornToOpen

noncomputable instance boundaryTwoHornOpenComparison_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso (boundaryTwoHornOpenComparison R) := by
dsimp [boundaryTwoHornOpenComparison]
letI : QuasiIso
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
   boundaryTwoHornToOpen) :=
  singularChainMap_quasiIso_of_contractible R _
infer_instance

/-! ## The two-component open intersection -/

abbrev boundaryTwoOpenIntersection : Set |boundaryTwoSSet| :=
  boundaryTwoLastEdgeOpen ∩ boundaryTwoHornOpen

/-- Inclusion of the two simplicial vertices in the whole boundary. -/
def boundaryTwoVerticesInclBoundary :
  |(boundaryTwoVertices : SSet.{0})| → |boundaryTwoSSet| :=
  SSet.toTop.map (SSet.Subcomplex.homOfLE
    (boundaryTwoVertices_le_firstFace.trans
      (by
        rw [boundary_two_eq_face_sup]
        exact le_sup_left))))

lemma boundaryTwoVerticesInclBoundary_circle :
  boundaryTwoVerticesInclBoundary >> boundaryTwoToBoundaryCircle =
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace) >>
    lastEdgeToBoundaryCircle := by
  rw [← boundaryTwoLastEdgeIncl_circle]
  simp only [← Category.assoc]
  congr 1
  unfold boundaryTwoVerticesInclBoundary boundaryTwoLastEdgeIncl
  rw [← SSet.toTop.map_comp]
  rfl

```

```

lemma boundaryTwoVerticesInclBoundary_circle_vertex_one :
  boundaryTwoToBoundaryCircle
    (boundaryTwoVerticesInclBoundary boundaryTwoVertexOnePoint) =
    ((2 : ℝ) : AddCircle (3 : ℝ)) := by
  have h := congrArg
    (fun f : |(boundaryTwoVertices : SSet.{0})| → boundaryCircle =>
      f boundaryTwoVertexOnePoint)
  boundaryTwoVerticesInclBoundary_circle
  change
    boundaryTwoToBoundaryCircle
      (boundaryTwoVerticesInclBoundary boundaryTwoVertexOnePoint) =
      lastEdgeToBoundaryCircle
        (SSet.toTop.map
          (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace)
            boundaryTwoVertexOnePoint) at h
  rw [h]
  have hmap :
    SSet.toTop.map
      ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
        SSet.Subcomplex.homOfLE
          (le_sup_left :
            SSet.stdSimplex.face ({1} : Finset (Fin 3)) ≤
              boundaryTwoVertices)) >>
        SSet.toTop.map
          (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace) =
    SSet.toTop.map
      ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
        SSet.Subcomplex.homOfLE face_one_le_boundaryTwoFirstFace) := by
  simp only [← SSet.toTop.map_comp]
  rfl
  have heval := congrArg
    (fun f : |(Δ[0] : SSet.{0})| →
      |(boundaryTwoFirstFace : SSet.{0})| =>
        f (default : |(Δ[0] : SSet.{0})|)) hmap
  change
    SSet.toTop.map
      (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace)
        boundaryTwoVertexOnePoint =
    SSet.toTop.map
      ((SSet.stdSimplex.faceSingletonIso (1 : Fin 3)).hom >>
        SSet.Subcomplex.homOfLE face_one_le_boundaryTwoFirstFace)
        default at heval
  rw [heval]
  have hv := congrArg
    (fun f : |(Δ[0] : SSet.{0})| → boundaryCircle =>
      f (default : |(Δ[0] : SSet.{0})|))
  lastEdgeToBoundaryCircle_vertex_one
  simpa [boundaryTwoVertexOnePoint, SSet.toTop.map_comp,
    CategoryTheory.comp_apply] using hv

lemma boundaryTwoVerticesInclBoundary_circle_vertex_two :
  boundaryTwoToBoundaryCircle
    (boundaryTwoVerticesInclBoundary boundaryTwoVertexTwoPoint) =
    (0 : AddCircle (3 : ℝ)) := by
  have h := congrArg
    (fun f : |(boundaryTwoVertices : SSet.{0})| → boundaryCircle =>
      f boundaryTwoVertexTwoPoint)
  boundaryTwoVerticesInclBoundary_circle
  change
    boundaryTwoToBoundaryCircle
      (boundaryTwoVerticesInclBoundary boundaryTwoVertexTwoPoint) =
      lastEdgeToBoundaryCircle

```

```

      (SSet.toTop.map
        (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace)
        boundaryTwoVertexTwoPoint) at h
rw [h]
have hmap :
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
      SSet.Subcomplex.homOfLE
        (le_sup_right :
          SSet.stdSimplex.face ({2} : Finset (Fin 3)) ≤
            boundaryTwoVertices)) >>
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace) =
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
      SSet.Subcomplex.homOfLE face_two_le_boundaryTwoFirstFace) := by
simp only [← SSet.toTop.map_comp]
rfl
have heval := congrArg
  (fun f : |(Δ[0] : SSet.{0})| →
    |(boundaryTwoFirstFace : SSet.{0})| =>
      f (default : |(Δ[0] : SSet.{0})|)) hmap
change
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace)
    boundaryTwoVertexTwoPoint =
  SSet.toTop.map
    ((SSet.stdSimplex.faceSingletonIso (2 : Fin 3)).hom >>
      SSet.Subcomplex.homOfLE face_two_le_boundaryTwoFirstFace)
    default at heval
rw [heval]
have hv := congrArg
  (fun f : |(Δ[0] : SSet.{0})| → boundaryCircle =>
    f (default : |(Δ[0] : SSet.{0})|))
  lastEdgeToBoundaryCircle_vertex_two
simpa [boundaryTwoVertexTwoPoint, SSet.toTop.map_comp,
  CategoryTheory.comp_apply] using hv

lemma boundaryTwoVerticesInclBoundary_mem_intersection
  (x : |(boundaryTwoVertices : SSet.{0})|) :
  boundaryTwoVerticesInclBoundary x ∈ boundaryTwoOpenIntersection := by
have hx := congrArg
  (fun f : |(boundaryTwoVertices : SSet.{0})| →
    |(boundaryTwoVertices : SSet.{0})| => f x)
  boundaryTwoVerticesToBool_comp_boolToBoundaryTwoVertices
change boolToBoundaryTwoVertices
  (boundaryTwoVerticesToBool x) = x at hx
constructor
· change boundaryTwoToBoundaryCircle
  (boundaryTwoVerticesInclBoundary x) ≠ hornInteriorPoint
rw [← hx]
cases h : boundaryTwoVerticesToBool x
· change boundaryTwoToBoundaryCircle
  (boundaryTwoVerticesInclBoundary boundaryTwoVertexOnePoint) ≠
  hornInteriorPoint
rw [boundaryTwoVerticesInclBoundary_circle_vertex_one]
intro hbad
have : (2 : ℝ) = 1 := by
  apply (AddCircle.coe_eq_coe_iff_of_mem_Ico
    (p := (3 : ℝ)) (a := (0 : ℝ))
    (x := (2 : ℝ)) (y := (1 : ℝ))
    (by norm_num) (by norm_num)).1
  exact hbad

```

```

    norm_num at this
  · change boundaryTwoToBoundaryCircle
    (boundaryTwoVerticesInclBoundary boundaryTwoVertexTwoPoint) ≠
      hornInteriorPoint
  rw [boundaryTwoVerticesInclBoundary_circle_vertex_two]
  intro hbad
  have : (0 : ℝ) = 1 := by
    apply (AddCircle.coe_eq_coe_iff_of_mem_Ico
      (p := (3 : ℝ)) (a := (0 : ℝ))
      (x := (0 : ℝ)) (y := (1 : ℝ))
      (by norm_num) (by norm_num)).1
    exact hbad
  norm_num at this
  · change boundaryTwoToBoundaryCircle
    (boundaryTwoVerticesInclBoundary x) ≠ lastEdgeInteriorPoint
  rw [← hx]
  cases h : boundaryTwoVerticesToBool x
  · change boundaryTwoToBoundaryCircle
    (boundaryTwoVerticesInclBoundary boundaryTwoVertexOnePoint) ≠
      lastEdgeInteriorPoint
  rw [boundaryTwoVerticesInclBoundary_circle_vertex_one]
  intro hbad
  have : (2 : ℝ) = 5 / 2 := by
    apply (AddCircle.coe_eq_coe_iff_of_mem_Ico
      (p := (3 : ℝ)) (a := (0 : ℝ))
      (x := (2 : ℝ)) (y := (5 / 2 : ℝ))
      (by norm_num) (by norm_num)).1
    exact hbad
  norm_num at this
  · change boundaryTwoToBoundaryCircle
    (boundaryTwoVerticesInclBoundary boundaryTwoVertexTwoPoint) ≠
      lastEdgeInteriorPoint
  rw [boundaryTwoVerticesInclBoundary_circle_vertex_two]
  intro hbad
  have : (0 : ℝ) = 5 / 2 := by
    apply (AddCircle.coe_eq_coe_iff_of_mem_Ico
      (p := (3 : ℝ)) (a := (0 : ℝ))
      (x := (0 : ℝ)) (y := (5 / 2 : ℝ))
      (by norm_num) (by norm_num)).1
    exact hbad
  norm_num at this

/-- The realized two vertices land in the two components of the open
intersection. -/
def boundaryTwoVerticesToOpenIntersection :
  |(boundaryTwoVertices : SSet.{0})| →
  TopCat.of ↑boundaryTwoOpenIntersection :=
TopCat.ofHom
{ toFun := fun x =>
  ⟨boundaryTwoVerticesInclBoundary x,
    boundaryTwoVerticesInclBoundary_mem_intersection x⟩
  continuous_toFun := by fun_prop }

/-- A linear coordinate model for a circle punctured at `1` and `5/2`. -/
abbrev puncturedBoundaryIoo :=
  {x : ℝ // x ∈ Ioo (1 : ℝ) 4 ∧ x ≠ 5 / 2}

def puncturedBoundaryBase (b : Bool) : puncturedBoundaryIoo :=
  if b then
    ⟨3, by constructor <;> norm_num⟩
  else
    ⟨2, by constructor <;> norm_num⟩

```

```

def puncturedBoundarySide (x : puncturedBoundaryIoo) : Bool :=
  if 5 / 2 < (x : ℝ) then true else false

lemma continuous_puncturedBoundarySide :
  Continuous puncturedBoundarySide := by
  rw [continuous_iff_continuousAt]
  intro x
  by_cases hx : (x : ℝ) < 5 / 2
  · apply (continuousAt_const :
    ContinuousAt (fun _ : puncturedBoundaryIoo => false) x).congr_of_eventuallyEq
    filter_upwards
      [(isOpen_lt continuous_subtype_val continuous_const).mem_nhds hx]
      with y hy
    simp [puncturedBoundarySide, not_lt.mpr (le_of_lt hy)]
  · have hx' : 5 / 2 < (x : ℝ) :=
    lt_of_le_of_ne (le_of_not_gt hx) (Ne.symm x.property.2)
    apply (continuousAt_const :
    ContinuousAt (fun _ : puncturedBoundaryIoo => true) x).congr_of_eventuallyEq
    filter_upwards
      [(isOpen_lt continuous_const continuous_subtype_val).mem_nhds hx']
      with y hy
    simp [puncturedBoundarySide, hy]

def puncturedBoundarySideMap : C(puncturedBoundaryIoo, Bool) :=
  ⟨puncturedBoundarySide, continuous_puncturedBoundarySide⟩

def puncturedBoundaryBaseMap : C(Bool, puncturedBoundaryIoo) :=
  ⟨puncturedBoundaryBase, continuous_of_discreteTopology⟩

lemma puncturedBoundarySide_base (b : Bool) :
  puncturedBoundarySide (puncturedBoundaryBase b) = b := by
  cases b <|> simp [puncturedBoundarySide, puncturedBoundaryBase] <|> norm_num

/-- Contract each of the two punctured-interval components to its selected
endpoint vertex. -/
def puncturedBoundaryContraction :
  ContinuousMap.Homotopy
    (puncturedBoundaryBaseMap.comp puncturedBoundarySideMap)
    (ContinuousMap.id puncturedBoundaryIoo) where
toFun tx :=
  let t : ℝ := tx.1
  let x : ℝ := tx.2
  let b : ℝ := puncturedBoundaryBase
    (puncturedBoundarySide tx.2)
  ⟨(1 - t) * b + t * x, by
    have ht0 : 0 ≤ t := tx.1.property.1
    have ht1 : t ≤ 1 := tx.1.property.2
    have hx1 : 1 < x := tx.2.property.1.1
    have hx4 : x < 4 := tx.2.property.1.2
    constructor
  · constructor
    · by_cases hs : 5 / 2 < x
      · have hb : b = 3 := by
        change 5 / 2 < (tx.2 : ℝ) at hs
        simp [b, puncturedBoundarySide, puncturedBoundaryBase, hs]
        rw [hb]
        nlinarith
      · have hb : b = 2 := by
        change ¬5 / 2 < (tx.2 : ℝ) at hs
        simp [b, puncturedBoundarySide, puncturedBoundaryBase, hs]
        rw [hb]
        nlinarith
    · by_cases hs : 5 / 2 < x

```

```

    · have hb : b = 3 := by
      change 5 / 2 < (tx.2 : ℝ) at hs
      simp [b, puncturedBoundarySide, puncturedBoundaryBase, hs]
      rw [hb]
      nlinarith
    · have hb : b = 2 := by
      change -5 / 2 < (tx.2 : ℝ) at hs
      simp [b, puncturedBoundarySide, puncturedBoundaryBase, hs]
      rw [hb]
      nlinarith
  · by_cases hs : 5 / 2 < x
    · have hb : b = 3 := by
      change 5 / 2 < (tx.2 : ℝ) at hs
      simp [b, puncturedBoundarySide, puncturedBoundaryBase, hs]
      rw [hb]
      have hx : 5 / 2 < x := hs
      nlinarith
    · have hx : x < 5 / 2 :=
      lt_of_le_of_ne (le_of_not_gt hs) tx.2.property.2
      have hb : b = 2 := by
        change -5 / 2 < (tx.2 : ℝ) at hs
        simp [b, puncturedBoundarySide, puncturedBoundaryBase, hs]
        rw [hb]
        nlinarith
continuous_toFun := by
  apply Continuous.subtype_mk
  dsimp
  have ht : Continuous
    (fun tx : _root_.unitInterval × puncturedBoundaryIoo =>
      (tx.1 : ℝ)) :=
    continuous_subtype_val.comp continuous_fst
  have hx : Continuous
    (fun tx : _root_.unitInterval × puncturedBoundaryIoo =>
      (tx.2 : ℝ)) :=
    continuous_subtype_val.comp continuous_snd
  have hbsub : Continuous
    (fun tx : _root_.unitInterval × puncturedBoundaryIoo =>
      (puncturedBoundaryBaseMap.comp puncturedBoundarySideMap) tx.2) :=
    (puncturedBoundaryBaseMap.comp
      puncturedBoundarySideMap).continuous.comp continuous_snd
  have hb : Continuous
    (fun tx : _root_.unitInterval × puncturedBoundaryIoo =>
      ((puncturedBoundaryBaseMap.comp
        puncturedBoundarySideMap) tx.2).val) :=
    continuous_subtype_val.comp hbsub
  exact ((continuous_const.sub ht).mul hb).add (ht.mul hx)
map_zero_left x := by
  ext
  simp [puncturedBoundaryBaseMap, puncturedBoundarySideMap]
map_one_left x := by
  ext
  simp
/-- The punctured interval has exactly the two contractible components
selected by `Bool`. -/
def puncturedBoundaryHomotopyEquivBool :
  Bool ≃h puncturedBoundaryIoo where
  toFun := puncturedBoundaryBaseMap
  invFun := puncturedBoundarySideMap
  left_inv := by
    have h :
      puncturedBoundarySideMap.comp puncturedBoundaryBaseMap =
        ContinuousMap.id Bool := by

```

```

    ext b
    exact puncturedBoundarySide_base b
    rw [h]
    right_inv := ⟨puncturedBoundaryContraction⟩

lemma boundaryTwoLastEdgeOpenHomeoIoo_coe
  (x : ↑boundaryTwoLastEdgeOpen) :
  (((boundaryTwoLastEdgeOpenHomeoIoo x : Ioo (1 : ℝ) 4) : ℝ) :
    AddCircle (3 : ℝ)) =
    homeomorph_boundaryTwo_addCircle x.1 := by
change
  (((AddCircle.equivIco (3 : ℝ) (1 : ℝ)
    (homeomorph_boundaryTwo_addCircle x.1) : ℝ)) :
    AddCircle (3 : ℝ)) =
    homeomorph_boundaryTwo_addCircle x.1
exact AddCircle.coe_equivIco

/-- Reassociate the subtype representing the intersection of the two
open neighborhoods. -/
def boundaryTwoIntersectionNestedHomeo :
  ↑boundaryTwoOpenIntersection  $\simeq_t$ 
  {x : ↑boundaryTwoLastEdgeOpen // x.1 ∈ boundaryTwoHornOpen} where
toFun x := ⟨⟨x.1, x.2.1⟩, x.2.2⟩
invFun x := ⟨x.1.1, x.1.2, x.2⟩
left_inv _ := rfl
right_inv _ := rfl
continuous_toFun := by fun_prop
continuous_invFun := by fun_prop

def nestedPuncturedIooHomeo :
  {x : ↑(Ioo (1 : ℝ) 4) // (x : ℝ) ≠ 5 / 2}  $\simeq_t$ 
  puncturedBoundaryIoo where
toFun x := ⟨x.1.1, x.1.2, x.2⟩
invFun x := ⟨⟨x.1, x.2.1⟩, x.2.2⟩
left_inv _ := rfl
right_inv _ := rfl
continuous_toFun := by fun_prop
continuous_invFun := by fun_prop

/-- The open intersection is the interval `(1,4)` with its midpoint
puncture `5/2` removed. -/
noncomputable def boundaryTwoOpenIntersectionHomeoPuncturedIoo :
  ↑boundaryTwoOpenIntersection  $\simeq_t$  puncturedBoundaryIoo :=
  boundaryTwoIntersectionNestedHomeo.trans
  ((boundaryTwoLastEdgeOpenHomeoIoo.subtype
    (p := fun x : ↑boundaryTwoLastEdgeOpen =>
      x.1 ∈ boundaryTwoHornOpen)
    (q := fun x : ↑(Ioo (1 : ℝ) 4) => (x : ℝ) ≠ 5 / 2)
    (fun x => by
      change homeomorph_boundaryTwo_addCircle x.1 ≠
        ((5 / 2 : ℝ) : AddCircle (3 : ℝ)) ↔
        (boundaryTwoLastEdgeOpenHomeoIoo x : ℝ) ≠ 5 / 2
      rw [← boundaryTwoLastEdgeOpenHomeoIoo_coe]
      have hm :
        (boundaryTwoLastEdgeOpenHomeoIoo x : ℝ) ∈
          Ioo (1 : ℝ) (1 + 3) := by
        convert (boundaryTwoLastEdgeOpenHomeoIoo x).property using 1 <;>
          norm_num
      exact not_congr (AddCircle.coe_eq_coe_iff_of_mem_Ico
        (p := (3 : ℝ)) (a := (1 : ℝ))
        (Ioo_subset_Ico_self hm)
        (by norm_num))))).trans nestedPuncturedIooHomeo)

```



```

lemma homeomorph_boundaryTwo_addCircle_apply
  (x : |boundaryTwoSSet|) :
  homeomorph_boundaryTwo_addCircle x =
    boundaryTwoToBoundaryCircle x := rfl

lemma boundaryTwoOpenIntersectionHomeo_vertex_one :
  boundaryTwoOpenIntersectionHomeoPuncturedIoo
    (boundaryTwoVerticesToOpenIntersection boundaryTwoVertexOnePoint) =
    puncturedBoundaryBase false := by
  apply Subtype.ext
  change
    ((AddCircle.equivIco (3 : ℝ) (1 : ℝ))
      (homeomorph_boundaryTwo_addCircle
        (boundaryTwoVerticesInclBoundary boundaryTwoVertexOnePoint))).1 = 2
  rw [homeomorph_boundaryTwo_addCircle_apply]
  rw [boundaryTwoVerticesInclBoundary_circle_vertex_one]
  rw [AddCircle.equivIco_coe_eq (by norm_num)]

lemma boundaryTwoOpenIntersectionHomeo_vertex_two :
  boundaryTwoOpenIntersectionHomeoPuncturedIoo
    (boundaryTwoVerticesToOpenIntersection boundaryTwoVertexTwoPoint) =
    puncturedBoundaryBase true := by
  apply Subtype.ext
  change
    ((AddCircle.equivIco (3 : ℝ) (1 : ℝ))
      (homeomorph_boundaryTwo_addCircle
        (boundaryTwoVerticesInclBoundary boundaryTwoVertexTwoPoint))).1 = 3
  rw [homeomorph_boundaryTwo_addCircle_apply]
  rw [boundaryTwoVerticesInclBoundary_circle_vertex_two]
  have hzero : (0 : AddCircle (3 : ℝ)) = ((3 : ℝ) : AddCircle (3 : ℝ)) := by
    exact (AddCircle.coe_period (3 : ℝ)).symm
  rw [hzero, AddCircle.equivIco_coe_eq (by norm_num)]

/-- The open intersection deformation-retracts onto the two realized
vertices. -/
noncomputable def boundaryTwoVerticesOpenIntersectionHomotopyEquiv :
  |(boundaryTwoVertices : SSet.{0})|  $\simeq_h$ 
   $\uparrow$ boundaryTwoOpenIntersection := by
  let e :
    |(boundaryTwoVertices : SSet.{0})|  $\simeq_h$ 
     $\uparrow$ boundaryTwoOpenIntersection :=
    homeomorph_boundaryTwoVertices_bool.toHomotopyEquiv.trans
      (puncturedBoundaryHomotopyEquivBool.trans
        boundaryTwoOpenIntersectionHomeoPuncturedIoo.toHomotopyEquiv.symm)
  have hfun :
    e.toFun =
      boundaryTwoVerticesToOpenIntersection.hom := by
  apply ContinuousMap.ext
  intro x
  apply boundaryTwoOpenIntersectionHomeoPuncturedIoo.injective
  have hx := congrArg
    (fun f : |(boundaryTwoVertices : SSet.{0})|  $\longrightarrow$ 
      |(boundaryTwoVertices : SSet.{0})|  $\Rightarrow$  f x)
    boundaryTwoVerticesToBool_comp_boolToBoundaryTwoVertices
  change boolToBoundaryTwoVertices
    (boundaryTwoVerticesToBool x) = x at hx
  rw [← hx]
  cases h : boundaryTwoVerticesToBool x
  · have hs := congrArg
      (fun f : TopCat.of Bool  $\longrightarrow$  TopCat.of Bool  $\Rightarrow$  f false)
      boolToBoundaryTwoVertices_comp_boundaryTwoVerticesToBool
    change homeomorph_boundaryTwoVertices_bool
      (boolToBoundaryTwoVertices false) = false at hs

```

```

calc
  boundaryTwoOpenIntersectionHomeoPuncturedIoo
    (e.toFun (boolToBoundaryTwoVertices false)) =
  puncturedBoundaryBase
    (homeomorph_boundaryTwoVertices_bool
      (boolToBoundaryTwoVertices false)) := by
  change boundaryTwoOpenIntersectionHomeoPuncturedIoo
    (boundaryTwoOpenIntersectionHomeoPuncturedIoo.symm
      (puncturedBoundaryBase
        (homeomorph_boundaryTwoVertices_bool
          (boolToBoundaryTwoVertices false)))) =
  puncturedBoundaryBase
    (homeomorph_boundaryTwoVertices_bool
      (boolToBoundaryTwoVertices false))
  rw [Homeomorph.apply_symm_apply]
  _ = puncturedBoundaryBase false := by rw [hs]
  _ = boundaryTwoOpenIntersectionHomeoPuncturedIoo
    (boundaryTwoVerticesToOpenIntersection
      (boolToBoundaryTwoVertices false)) := by
    simpa using boundaryTwoOpenIntersectionHomeo_vertex_one.symm
· have hs := congrArg
  (fun f : TopCat.of Bool → TopCat.of Bool => f true)
  boolToBoundaryTwoVertices_comp_boundaryTwoVerticesToBool
change homeomorph_boundaryTwoVertices_bool
  (boolToBoundaryTwoVertices true) = true at hs
calc
  boundaryTwoOpenIntersectionHomeoPuncturedIoo
    (e.toFun (boolToBoundaryTwoVertices true)) =
  puncturedBoundaryBase
    (homeomorph_boundaryTwoVertices_bool
      (boolToBoundaryTwoVertices true)) := by
  change boundaryTwoOpenIntersectionHomeoPuncturedIoo
    (boundaryTwoOpenIntersectionHomeoPuncturedIoo.symm
      (puncturedBoundaryBase
        (homeomorph_boundaryTwoVertices_bool
          (boolToBoundaryTwoVertices true)))) =
  puncturedBoundaryBase
    (homeomorph_boundaryTwoVertices_bool
      (boolToBoundaryTwoVertices true))
  rw [Homeomorph.apply_symm_apply]
  _ = puncturedBoundaryBase true := by rw [hs]
  _ = boundaryTwoOpenIntersectionHomeoPuncturedIoo
    (boundaryTwoVerticesToOpenIntersection
      (boolToBoundaryTwoVertices true)) := by
    simpa using boundaryTwoOpenIntersectionHomeo_vertex_two.symm
exact
{ toFun := boundaryTwoVerticesToOpenIntersection.hom
  invFun := e.invFun
  left_inv := by
    rw [← hfun]
    exact e.left_inv
  right_inv := by
    rw [← hfun]
    exact e.right_inv }

noncomputable instance boundaryTwoVerticesToOpenIntersection_quasiIso
(R : ModuleCat.{0} k) :
QuasiIso
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
    boundaryTwoVerticesToOpenIntersection) := by
change QuasiIso
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
    (TopCat.ofHom boundaryTwoVerticesToOpenIntersectionHomotopyEquiv.toFun))

```

```

exact
  (singularChainHomotopyEquiv
    boundaryTwoVerticesOpenIntersectionHomotopyEquiv R).quasiIso_hom

/-- The comparison from the simplicial edge--horn intersection to
singular chains of the open intersection. -/
noncomputable def boundaryTwoOpenIntersectionComparison
  (R : ModuleCat.{0} k) :
  (boundaryTwoVertices : SSet.{0}).chainComplex R →
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
      (TopCat.of  $\uparrow$ boundaryTwoOpenIntersection) :=
  simplicialSingularComparison R (boundaryTwoVertices : SSet.{0}) >>
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
      boundaryTwoVerticesToOpenIntersection

noncomputable instance boundaryTwoOpenIntersectionComparison_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso (boundaryTwoOpenIntersectionComparison R) := by
  dsimp [boundaryTwoOpenIntersectionComparison]
  infer_instance

/-! ## Compatibility with the simplicial Mayer--Vietoris cover -/

/-- The realized edge--horn intersection lands in the intersection of the
two open neighborhoods. -/
noncomputable def boundaryTwoIntersectionToOpenIntersection :
  |((boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces : _) : SSet.{0})| →
    TopCat.of boundaryTwoOpenIntersection :=
  SSet.toTop.map boundaryTwoIntersectionIsoVertices.hom >>
    boundaryTwoVerticesToOpenIntersection

/-- The intersection landing map agrees with the last-edge landing map. -/
lemma boundaryTwoIntersectionToOpenIntersection_left :
  boundaryTwoIntersectionToOpenIntersection >>
    interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen =
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (inf_le_left :
        boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
          boundaryTwoFirstFace)) >>
    boundaryTwoLastEdgeToOpen := by
  apply (cancel_epi
    (SSet.toTop.map boundaryTwoIntersectionIsoVertices.inv)).1
  simp only [boundaryTwoIntersectionToOpenIntersection,  $\leftarrow$  Category.assoc,
     $\leftarrow$  SSet.toTop.map_comp, Iso.inv_hom_id, SSet.toTop.map_id,
    Category.id_comp, boundaryTwoIntersectionIsoVertices_inv_comp_first]
  apply TopCat.hom_ext
  ext x
  change boundaryTwoVerticesInclBoundary x =
    boundaryTwoLastEdgeIncl
      (SSet.toTop.map
        (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_firstFace) x)
  unfold boundaryTwoVerticesInclBoundary boundaryTwoLastEdgeIncl
  rw [ $\leftarrow$  CategoryTheory.comp_apply,  $\leftarrow$  SSet.toTop.map_comp]
  rfl

/-- The intersection landing map agrees with the horn landing map. -/
lemma boundaryTwoIntersectionToOpenIntersection_right :
  boundaryTwoIntersectionToOpenIntersection >>
    interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen =
  SSet.toTop.map
    (SSet.Subcomplex.homOfLE
      (inf_le_right :

```

```

        boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
        boundaryTwoRemainingFaces))  $\gg$ 
    boundaryTwoHornToOpen := by
    apply (cancel_epi
      (SSet.toTop.map boundaryTwoIntersectionIsoVertices.inv)).1
    simp only [boundaryTwoIntersectionToOpenIntersection,  $\leftarrow$  Category.assoc,
       $\leftarrow$  SSet.toTop.map_comp, Iso.inv_hom_id, SSet.toTop.map_id,
      Category.id_comp, boundaryTwoIntersectionIsoVertices_inv_comp_remaining]
    apply TopCat.hom_ext
    ext x
    change boundaryTwoVerticesInclBoundary x =
      boundaryTwoHornIncl
      (SSet.toTop.map
        (SSet.Subcomplex.homOfLE boundaryTwoVertices_le_remainingFaces) x)
    unfold boundaryTwoVerticesInclBoundary boundaryTwoHornIncl
    rw [ $\leftarrow$  CategoryTheory.comp_apply,  $\leftarrow$  SSet.toTop.map_comp]
    rfl

/-- The realized simplicial intersection is homotopy equivalent to the
open-cover intersection. -/
noncomputable def boundaryTwoIntersectionOpenHomotopyEquiv :
  |((boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces : _) : SSet.{0})|  $\simeq_h$ 
  TopCat.of boundaryTwoOpenIntersection :=
  (TopCat.homeoOfIso
    (SSet.toTop.mapIso boundaryTwoIntersectionIsoVertices)).toHomotopyEquiv.trans
    boundaryTwoVerticesOpenIntersectionHomotopyEquiv

/-- The singular-chain map induced by the intersection landing map is a
quasi-isomorphism. -/
noncomputable instance boundaryTwoIntersectionToOpenIntersection_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso
    (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
      boundaryTwoIntersectionToOpenIntersection) := by
  change QuasiIso
    (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
      (TopCat.ofHom boundaryTwoIntersectionOpenHomotopyEquiv.toFun))
  exact
    (singularChainHomotopyEquiv
      boundaryTwoIntersectionOpenHomotopyEquiv R).quasiIso_hom

/-- Comparison on the actual edge--horn intersection, factored through the
open-cover intersection. -/
noncomputable def boundaryTwoIntersectionOpenComparison
  (R : ModuleCat.{0} k) :
  ((boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces : _) : SSet.{0}).chainComplex R  $\longrightarrow$ 
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (TopCat.of boundaryTwoOpenIntersection) :=
  simplicialSingularComparison R
    ((boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces : _) : SSet.{0})  $\gg$ 
    (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
      boundaryTwoIntersectionToOpenIntersection

noncomputable instance boundaryTwoIntersectionOpenComparison_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso (boundaryTwoIntersectionOpenComparison R) := by
  dsimp [boundaryTwoIntersectionOpenComparison]
  infer_instance

lemma boundaryTwoIntersectionOpenComparison_left
  (R : ModuleCat.{0} k) :
  boundaryTwoIntersectionOpenComparison R  $\gg$ 
    (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map

```

```

      (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen) =
      SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE
      (inf_le_left :
      boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
      boundaryTwoFirstFace)) R  $\gg$ 
      boundaryTwoLastEdgeOpenComparison R := by
rw [boundaryTwoIntersectionOpenComparison,
    boundaryTwoLastEdgeOpenComparison]
simp only [Category.assoc,  $\leftarrow$  Functor.map_comp]
rw [boundaryTwoIntersectionToOpenIntersection_left]
simp only [Functor.map_comp,  $\leftarrow$  Category.assoc]
rw [simplicialSingularComparison_naturality]
rfl

lemma boundaryTwoIntersectionOpenComparison_right
  (R : ModuleCat.{0} k) :
  boundaryTwoIntersectionOpenComparison R  $\gg$ 
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
    (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen) =
  SSet.chainComplexMap
  (SSet.Subcomplex.homOfLE
  (inf_le_right :
  boundaryTwoFirstFace  $\sqcap$  boundaryTwoRemainingFaces  $\leq$ 
  boundaryTwoRemainingFaces)) R  $\gg$ 
  boundaryTwoHornOpenComparison R := by
rw [boundaryTwoIntersectionOpenComparison,
    boundaryTwoHornOpenComparison]
simp only [Category.assoc,  $\leftarrow$  Functor.map_comp]
rw [boundaryTwoIntersectionToOpenIntersection_right]
simp only [Functor.map_comp,  $\leftarrow$  Category.assoc]
rw [simplicialSingularComparison_naturality]
rfl

/-- The direct sum of the comparison maps for the edge and horn pieces. -/
noncomputable def boundaryTwoPieceOpenComparison
  (R : ModuleCat.{0} k) :
  (boundaryTwoFirstFace : SSet.{0}).chainComplex R  $\boxplus$ 
    (boundaryTwoRemainingFaces : SSet.{0}).chainComplex R  $\longrightarrow$ 
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (TopCat.of boundaryTwoLastEdgeOpen)  $\boxplus$ 
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (TopCat.of boundaryTwoHornOpen) :=
  biprod.map
    (boundaryTwoLastEdgeOpenComparison R)
    (boundaryTwoHornOpenComparison R)

/-- A binary biproduct of quasi-isomorphisms is a quasi-isomorphism. -/
lemma quasiIso_biprod_map
  {K1 K2 L1 L2 : ChainComplex (ModuleCat.{0} k)  $\mathbb{N}$ }
  (f : K1  $\longrightarrow$  L1) (g : K2  $\longrightarrow$  L2) [QuasiIso f] [QuasiIso g] :
  QuasiIso (biprod.map f g) := by
rw [quasiIso_iff]
intro n
rw [quasiIsoAt_iff_isIso_homologyMap]
let F := homologyFunctor (ModuleCat.{0} k) (ComplexShape.down  $\mathbb{N}$ ) n
let I : PreservesBinaryBiproduct K1 K2 F :=
  preservesBinaryBiproduct_sset_homologyFunctor n K1 K2
let I : PreservesBinaryBiproduct L1 L2 F :=
  preservesBinaryBiproduct_sset_homologyFunctor n L1 L2
let eK := F.mapBiprod K1 K2
let eL := F.mapBiprod L1 L2
have h :

```

```

homologyMap (biprod.map f g) n >> eL.hom =
  eK.hom >> biprod.map (homologyMap f n) (homologyMap g n) := by
change F.map (biprod.map f g) >> eL.hom =
  eK.hom >> biprod.map (F.map f) (F.map g)
apply biprod.hom_ext
· calc
  (F.map (biprod.map f g) >> eL.hom) >> biprod.fst =
    F.map (biprod.map f g) >> (eL.hom >> biprod.fst) := by
      rw [Category.assoc]
  _ = F.map (biprod.map f g) >> F.map biprod.fst := by
    change _ >>
      ((F.mapBiprod L1 L2).hom >> biprod.fst) = _
      rw [Functor.mapBiprod_hom, biprod.lift_fst]
  _ = F.map (biprod.map f g >> biprod.fst) := by
    rw [F.map_comp]
  _ = F.map (biprod.fst >> f) := by rw [biprod.map_fst]
  _ = F.map biprod.fst >> F.map f := F.map_comp _ _
  _ = (eK.hom >> biprod.fst) >> F.map f := by
    change _ = (((F.mapBiprod K1 K2).hom >> biprod.fst) >> _)
    rw [Functor.mapBiprod_hom, biprod.lift_fst]
  _ = eK.hom >> (biprod.fst >> F.map f) := Category.assoc _ _ _
  _ = eK.hom >>
    (biprod.map (F.map f) (F.map g) >> biprod.fst) := by
      rw [biprod.map_fst]
  _ = (eK.hom >> biprod.map (F.map f) (F.map g)) >> biprod.fst :=
    (Category.assoc _ _ _).symm
· calc
  (F.map (biprod.map f g) >> eL.hom) >> biprod.snd =
    F.map (biprod.map f g) >> (eL.hom >> biprod.snd) := by
      rw [Category.assoc]
  _ = F.map (biprod.map f g) >> F.map biprod.snd := by
    change _ >>
      ((F.mapBiprod L1 L2).hom >> biprod.snd) = _
      rw [Functor.mapBiprod_hom, biprod.lift_snd]
  _ = F.map (biprod.map f g >> biprod.snd) := by
    rw [F.map_comp]
  _ = F.map (biprod.snd >> g) := by rw [biprod.map_snd]
  _ = F.map biprod.snd >> F.map g := F.map_comp _ _
  _ = (eK.hom >> biprod.snd) >> F.map g := by
    change _ = (((F.mapBiprod K1 K2).hom >> biprod.snd) >> _)
    rw [Functor.mapBiprod_hom, biprod.lift_snd]
  _ = eK.hom >> (biprod.snd >> F.map g) := Category.assoc _ _ _
  _ = eK.hom >>
    (biprod.map (F.map f) (F.map g) >> biprod.snd) := by
      rw [biprod.map_snd]
  _ = (eK.hom >> biprod.map (F.map f) (F.map g)) >> biprod.snd :=
    (Category.assoc _ _ _).symm
let ef := asIso (homologyMap f n)
let eg := asIso (homologyMap g n)
let I : IsIso (biprod.map (homologyMap f n) (homologyMap g n)) := by
  change IsIso (biprod.map ef.hom eg.hom)
  exact (biprod.mapIso ef eg).isIso_hom
have I : IsIso
  (homologyMap (biprod.map f g) n >> eL.hom) := by
  rw [h]
  change IsIso ((eK.trans (biprod.mapIso ef eg)).hom)
  exact (eK.trans (biprod.mapIso ef eg)).isIso_hom
exact IsIso.of_isIso_comp_right _ eL.hom

noncomputable instance boundaryTwoPieceOpenComparison_quasiIso
(R : ModuleCat.{0} k) :
  QuasiIso (boundaryTwoPieceOpenComparison R) := by
dsimp [boundaryTwoPieceOpenComparison]

```

```

exact quasiIso_biprod_map _ _

/-- The source and target Mayer--Vietoris inclusion maps commute with the
three factored comparison maps. -/
lemma boundaryTwo_openCover_square_f
  (R : ModuleCat.{0} k) :
  (subcomplexChainShortComplex R boundaryTwoFirstFace
    boundaryTwoRemainingFaces).f >>
    boundaryTwoPieceOpenComparison R =
  boundaryTwoIntersectionOpenComparison R >>
    mvIncl (C := ModuleCat.{0} k) R
    (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
    (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen) := by
change
  biprod.lift
    (SSet.chainComplexMap (SSet.Subcomplex.homOfLE inf_le_left) R)
    (-SSet.chainComplexMap (SSet.Subcomplex.homOfLE inf_le_right) R) >>
    biprod.map
      (boundaryTwoLastEdgeOpenComparison R)
      (boundaryTwoHornOpenComparison R) =
  boundaryTwoIntersectionOpenComparison R >>
    biprod.lift
      (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
        (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen))
      (-((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
        (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen))
  apply biprod.hom_ext
  · calc
    _ = SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE inf_le_left) R >>
      boundaryTwoLastEdgeOpenComparison R := by
      rw [Category.assoc, biprod.map_fst, ← Category.assoc,
        biprod.lift_fst]
    _ = boundaryTwoIntersectionOpenComparison R >>
      ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
        (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen) :=
      (boundaryTwoIntersectionOpenComparison_left R).symm
    _ = _ := by rw [Category.assoc, biprod.lift_fst]
  · calc
    _ = -(SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE inf_le_right) R >>
      boundaryTwoHornOpenComparison R) := by
      rw [Category.assoc, biprod.map_snd, ← Category.assoc,
        biprod.lift_snd, Preadditive.neg_comp]
    _ = -(boundaryTwoIntersectionOpenComparison R >>
      ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
        (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)) :=
      congrArg Neg.neg (boundaryTwoIntersectionOpenComparison_right R).symm
    _ = _ := by
      rw [Category.assoc, biprod.lift_snd, Preadditive.comp_neg]

/-- The singular-chain pushout associated to the two open neighborhoods. -/
abbrev boundaryTwoCoverChains (R : ModuleCat.{0} k) :
  ChainComplex (ModuleCat.{0} k)  $\mathbb{N}$  :=
  coverChains (C := ModuleCat.{0} k) R
    (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
    (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)

/-- The map from the simplicial-chain pushout to the singular open-cover
pushout. -/
noncomputable def boundaryTwoUnionToCoverChains
  (R : ModuleCat.{0} k) :
  ((boundaryTwoFirstFace  $\sqcup$  boundaryTwoRemainingFaces : _) :
```

```

      SSet.{0}).chainComplex R →
      boundaryTwoCoverChains R :=
(boundaryTwoChainIsPushout R).desc
(boundaryTwoLastEdgeOpenComparison R >> biprod.inl >>
  cokernel.π
  (mvIncl (C := ModuleCat.{0} k) R
    (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
    (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)))
(boundaryTwoHornOpenComparison R >> biprod.inr >>
  cokernel.π
  (mvIncl (C := ModuleCat.{0} k) R
    (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
    (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)))
(by
  let iL := ((singularChainComplexFunctor
    (ModuleCat.{0} k)).obj R).map
    (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
  let iR := ((singularChainComplexFunctor
    (ModuleCat.{0} k)).obj R).map
    (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
  let htarget := SingularExcision.cokernelIsPushout iL iR
  calc
    SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE inf_le_left) R >>
      boundaryTwoLastEdgeOpenComparison R >> biprod.inl >>
      cokernel.π (mvIncl (C := ModuleCat.{0} k) R
        (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
        (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)) =
      boundaryTwoIntersectionOpenComparison R >> iL >> biprod.inl >>
      cokernel.π (mvIncl (C := ModuleCat.{0} k) R
        (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
        (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)) := by
      simp only [← Category.assoc]
      rw [boundaryTwoIntersectionOpenComparison_left]
    _ = boundaryTwoIntersectionOpenComparison R >> iR >> biprod.inr >>
      cokernel.π (mvIncl (C := ModuleCat.{0} k) R
        (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
        (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)) := by
      simpa only [htarget, iL, iR,
        SingularExcision.cokernelPushoutInl,
        SingularExcision.cokernelPushoutInr,
        SingularExcision.mvIncl_eq_pushoutRelation,
        Category.assoc] using
      congrArg
      (fun q => boundaryTwoIntersectionOpenComparison R >> q)
      htarget.w
    _ = SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE inf_le_right) R >>
      boundaryTwoHornOpenComparison R >> biprod.inr >>
      cokernel.π (mvIncl (C := ModuleCat.{0} k) R
        (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
        (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)) := by
      simp only [← Category.assoc]
      rw [boundaryTwoIntersectionOpenComparison_right])

/-- The quotient maps in the source and target Mayer--Vietoris short
complexes commute. -/
lemma boundaryTwo_openCover_square_g
  (R : ModuleCat.{0} k) :
  (subcomplexChainShortComplex R boundaryTwoFirstFace
    boundaryTwoRemainingFaces).g >>
    boundaryTwoUnionToCoverChains R =
    boundaryTwoPieceOpenComparison R >>

```



```

    cokernel.π
    (mvIncl (C := ModuleCat.{0} k) R
      (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
      (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)) := by
apply biprod.hom_ext'
· change
  (biprod.inl >>
    biprod.desc
      (SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE le_sup_left) R)
      (SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE le_sup_right) R)) >>
    boundaryTwoUnionToCoverChains R =
  (biprod.inl >>
    biprod.map
      (boundaryTwoLastEdgeOpenComparison R)
      (boundaryTwoHornOpenComparison R)) >>
    cokernel.π
    (mvIncl (C := ModuleCat.{0} k) R
      (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
      (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen))
rw [biprod.inl_desc, biprod.inl_map]
exact (boundaryTwoChainIsPushout R).inl_desc _ _ _
· change
  (biprod.inr >>
    biprod.desc
      (SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE le_sup_left) R)
      (SSet.chainComplexMap
        (SSet.Subcomplex.homOfLE le_sup_right) R)) >>
    boundaryTwoUnionToCoverChains R =
  (biprod.inr >>
    biprod.map
      (boundaryTwoLastEdgeOpenComparison R)
      (boundaryTwoHornOpenComparison R)) >>
    cokernel.π
    (mvIncl (C := ModuleCat.{0} k) R
      (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
      (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen))
rw [biprod.inr_desc, biprod.inr_map]
exact (boundaryTwoChainIsPushout R).inr_desc _ _ _

/-- Morphism from the simplicial Mayer--Vietoris short complex to the
singular open-cover short complex. -/
noncomputable def boundaryTwoOpenCoverShortComplexHom
  (R : ModuleCat.{0} k) :
  subcomplexChainShortComplex R boundaryTwoFirstFace
    boundaryTwoRemainingFaces →
  triadShortComplex (C := ModuleCat.{0} k) R
    (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
    (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen) :=
ShortComplex.Hom.mk
  (boundaryTwoIntersectionOpenComparison R)
  (boundaryTwoPieceOpenComparison R)
  (boundaryTwoUnionToCoverChains R)
  (boundaryTwo_openCover_square_f R).symm
  (boundaryTwo_openCover_square_g R).symm

/-- The map from the simplicial union to open-cover chains is a
quasi-isomorphism. -/
noncomputable instance boundaryTwoUnionToCoverChains_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso (boundaryTwoUnionToCoverChains R) := by

```

```

letI : QuasiIso (boundaryTwoOpenCoverShortComplexHom R). $\tau_1$  := by
  change QuasiIso (boundaryTwoIntersectionOpenComparison R)
  infer_instance
letI : QuasiIso (boundaryTwoOpenCoverShortComplexHom R). $\tau_2$  := by
  change QuasiIso (boundaryTwoPieceOpenComparison R)
  infer_instance
exact HomologicalComplex.HomologySequence.quasiIso_ $\tau_3$ 
  (boundaryTwoOpenCoverShortComplexHom R)
  (boundaryTwoChainShortExact R)
  (triad_shortExact (C := ModuleCat.{0} k) R
    (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
    (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen))
  inferInstance inferInstance

/-- Open-cover chains map canonically to singular chains of the whole
boundary. -/
noncomputable def boundaryTwoCoverComparison
  (R : ModuleCat.{0} k) :
  boundaryTwoCoverChains R  $\longrightarrow$  singularChainsOfRealization R boundaryTwoSSet :=
coverComparison (C := ModuleCat.{0} k) R
  (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
  (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
  (subtypeIncl |boundaryTwoSSet| boundaryTwoLastEdgeOpen)
  (subtypeIncl |boundaryTwoSSet| boundaryTwoHornOpen)
  (subset_square_comm boundaryTwoLastEdgeOpen boundaryTwoHornOpen)

noncomputable instance boundaryTwoCoverComparison_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso (boundaryTwoCoverComparison R) := by
  exact SingularExcision.IsExcisiveSubspaces_of_openCover (R := R)
  isOpen_boundaryTwoLastEdgeOpen isOpen_boundaryTwoHornOpen
  boundaryTwo_open_cover

lemma boundaryTwoLastEdgeToOpen_subtype :
  boundaryTwoLastEdgeToOpen  $\gg$ 
    subtypeIncl |boundaryTwoSSet| boundaryTwoLastEdgeOpen =
    boundaryTwoLastEdgeIncl := by
  ext x
  rfl

lemma boundaryTwoHornToOpen_subtype :
  boundaryTwoHornToOpen  $\gg$ 
    subtypeIncl |boundaryTwoSSet| boundaryTwoHornOpen =
    boundaryTwoHornIncl := by
  ext x
  rfl

@[reassoc]
lemma boundaryTwoCoverComparison_ $\pi$ 
  (R : ModuleCat.{0} k) :
  cokernel. $\pi$ 
    (mvIncl (C := ModuleCat.{0} k) R
      (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
      (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen))  $\gg$ 
    boundaryTwoCoverComparison R =
  coverToX (C := ModuleCat.{0} k) R
    (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
    (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
    (subtypeIncl |boundaryTwoSSet| boundaryTwoLastEdgeOpen)
    (subtypeIncl |boundaryTwoSSet| boundaryTwoHornOpen)
    (subset_square_comm boundaryTwoLastEdgeOpen boundaryTwoHornOpen) := by
  unfold boundaryTwoCoverComparison coverComparison
  exact cokernel. $\pi$ _desc _ _ _

```

```

lemma boundaryTwoUnionToCoverComparison_first
  (R : ModuleCat.{0} k) :
  SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE
      (le_sup_left :
        boundaryTwoFirstFace ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces)) R >>
    boundaryTwoUnionToCoverChains R >>
    boundaryTwoCoverComparison R =
  boundaryTwoLastEdgeOpenComparison R >>
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
      (subtypeIncl |boundaryTwoSSet| boundaryTwoLastEdgeOpen) := by
  unfold boundaryTwoUnionToCoverChains
  rw [← Category.assoc,
    (boundaryTwoChainIsPushout R).inl_desc]
  calc
    _ = boundaryTwoLastEdgeOpenComparison R >> biprod.inl >>
      coverToX (C := ModuleCat.{0} k) R
        (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
        (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
        (subtypeIncl |boundaryTwoSSet| boundaryTwoLastEdgeOpen)
        (subtypeIncl |boundaryTwoSSet| boundaryTwoHornOpen)
        (subset_square_comm boundaryTwoLastEdgeOpen
          boundaryTwoHornOpen) := by
    simp only [Category.assoc] using congrArg
    (fun q => boundaryTwoLastEdgeOpenComparison R >> biprod.inl >> q)
    (boundaryTwoCoverComparison_π R)
  _ = _ := by
    rw [coverToX, biprod.inl_desc]

lemma boundaryTwoUnionToCoverComparison_remaining
  (R : ModuleCat.{0} k) :
  SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE
      (le_sup_right :
        boundaryTwoRemainingFaces ≤
          boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces)) R >>
    boundaryTwoUnionToCoverChains R >>
    boundaryTwoCoverComparison R =
  boundaryTwoHornOpenComparison R >>
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
      (subtypeIncl |boundaryTwoSSet| boundaryTwoHornOpen) := by
  unfold boundaryTwoUnionToCoverChains
  rw [← Category.assoc,
    (boundaryTwoChainIsPushout R).inr_desc]
  calc
    _ = boundaryTwoHornOpenComparison R >> biprod.inr >>
      coverToX (C := ModuleCat.{0} k) R
        (interInclLeft boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
        (interInclRight boundaryTwoLastEdgeOpen boundaryTwoHornOpen)
        (subtypeIncl |boundaryTwoSSet| boundaryTwoLastEdgeOpen)
        (subtypeIncl |boundaryTwoSSet| boundaryTwoHornOpen)
        (subset_square_comm boundaryTwoLastEdgeOpen
          boundaryTwoHornOpen) := by
    simp only [Category.assoc] using congrArg
    (fun q => boundaryTwoHornOpenComparison R >> biprod.inr >> q)
    (boundaryTwoCoverComparison_π R)
  _ = _ := by
    rw [coverToX, biprod.inr_desc]

/-- Simplicial inclusion of the last edge into the whole boundary. -/
def boundaryTwoFirstFaceIncl :

```

```

    (boundaryTwoFirstFace : SSet.{0}) → boundaryTwoSSet :=
SSet.Subcomplex.homOfLE (by
  rw [boundary_two_eq_face_sup]
  exact (le_sup_left :
    boundaryTwoFirstFace ≤
      boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces))

/-- Simplicial inclusion of the horn into the whole boundary. -/
def boundaryTwoRemainingFacesIncl :
  (boundaryTwoRemainingFaces : SSet.{0}) → boundaryTwoSSet :=
SSet.Subcomplex.homOfLE (by
  rw [boundary_two_eq_face_sup]
  exact (le_sup_right :
    boundaryTwoRemainingFaces ≤
      boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces))

lemma boundaryTwoFirstFace_sup_comp_iso_inv :
  SSet.Subcomplex.homOfLE
    (le_sup_left :
      boundaryTwoFirstFace ≤
        boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces) >>
    boundaryTwoIsoSup.inv =
  boundaryTwoFirstFaceIncl := rfl

lemma boundaryTwoRemainingFaces_sup_comp_iso_inv :
  SSet.Subcomplex.homOfLE
    (le_sup_right :
      boundaryTwoRemainingFaces ≤
        boundaryTwoFirstFace ⊔ boundaryTwoRemainingFaces) >>
    boundaryTwoIsoSup.inv =
  boundaryTwoRemainingFacesIncl := rfl

/-- The open-cover factorization recombines to the canonical comparison on
the simplicial boundary. -/
lemma boundaryTwoUnion_cover_factor
  (R : ModuleCat.{0} k) :
  boundaryTwoUnionToCoverChains R >> boundaryTwoCoverComparison R =
  SSet.chainComplexMap boundaryTwoIsoSup.inv R >>
  simplicialSingularComparison R boundaryTwoSSet := by
  apply (boundaryTwoChainIsPushout R).hom_ext
  · calc
    SSet.chainComplexMap
      (SSet.Subcomplex.homOfLE le_sup_left) R >>
      boundaryTwoUnionToCoverChains R >>
      boundaryTwoCoverComparison R =
      boundaryTwoLastEdgeOpenComparison R >>
      ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
        (subtypeIncl |boundaryTwoSSet| boundaryTwoLastEdgeOpen) :=
      boundaryTwoUnionToCoverComparison_first R
    _ = simplicialSingularComparison R
      (boundaryTwoFirstFace : SSet.{0}) >>
      ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
        (boundaryTwoLastEdgeToOpen >>
          subtypeIncl |boundaryTwoSSet| boundaryTwoLastEdgeOpen) := by
      rw [boundaryTwoLastEdgeOpenComparison, Category.assoc,
        Functor.map_comp]
    _ = simplicialSingularComparison R
      (boundaryTwoFirstFace : SSet.{0}) >>
      ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
        boundaryTwoLastEdgeIncl := by
      rw [boundaryTwoLastEdgeToOpen_subtype]
    _ = SSet.chainComplexMap boundaryTwoFirstFaceIncl R >>
      simplicialSingularComparison R boundaryTwoSSet := by

```

```

    rw [simplicialSingularComparison_naturality]
    rfl
  _ = SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE le_sup_left) R >>
    SSet.chainComplexMap boundaryTwoIsoSup.inv R >>
    simplicialSingularComparison R boundaryTwoSSet := by
  rw [← Category.assoc, ← Functor.map_comp,
    boundaryTwoFirstFace_sup_comp_iso_inv]
· calc
  SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE le_sup_right) R >>
    boundaryTwoUnionToCoverChains R >>
    boundaryTwoCoverComparison R =
  boundaryTwoHornOpenComparison R >>
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
    (subtypeIncl |boundaryTwoSSet| boundaryTwoHornOpen) :=
  boundaryTwoUnionToCoverComparison_remaining R
  _ = simplicialSingularComparison R
    (boundaryTwoRemainingFaces : SSet.{0}) >>
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
    (boundaryTwoHornToOpen >>
      subtypeIncl |boundaryTwoSSet| boundaryTwoHornOpen) := by
  rw [boundaryTwoHornOpenComparison, Category.assoc,
    Functor.map_comp]
  _ = simplicialSingularComparison R
    (boundaryTwoRemainingFaces : SSet.{0}) >>
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
    boundaryTwoHornIncl := by
  rw [boundaryTwoHornToOpen_subtype]
  _ = SSet.chainComplexMap boundaryTwoRemainingFacesIncl R >>
    simplicialSingularComparison R boundaryTwoSSet := by
  rw [simplicialSingularComparison_naturality]
  rfl
  _ = SSet.chainComplexMap
    (SSet.Subcomplex.homOfLE le_sup_right) R >>
    SSet.chainComplexMap boundaryTwoIsoSup.inv R >>
    simplicialSingularComparison R boundaryTwoSSet := by
  rw [← Category.assoc, ← Functor.map_comp,
    boundaryTwoRemainingFaces_sup_comp_iso_inv]

/-- The canonical adjunction-unit comparison is a quasi-isomorphism for
the boundary of the standard two-simplex. -/
theorem simplicialSingularComparison_boundaryTwo_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso (simplicialSingularComparison R boundaryTwoSSet) := by
let I : QuasiIso
  (boundaryTwoUnionToCoverChains R >> boundaryTwoCoverComparison R) := by
infer_instance
let I : QuasiIso
  (SSet.chainComplexMap boundaryTwoIsoSup.inv R >>
    simplicialSingularComparison R boundaryTwoSSet) := by
  rw [← boundaryTwoUnion_cover_factor]
  infer_instance
exact quasiIso_of_comp_left
  (SSet.chainComplexMap boundaryTwoIsoSup.inv R)
  (simplicialSingularComparison R boundaryTwoSSet)

/-- Supporting normalized-chain version of the boundary comparison. -/
theorem normalizedSimplicialSingularComparison_boundaryTwo_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso
    (normalizedSimplicialSingularComparison R boundaryTwoSSet) :=
  (normalizedSimplicialSingularComparison_quasiIso_iff R

```

```
boundaryTwoSSet).2
(simplicialSingularComparison_boundaryTwo_quasiIso R)
```

A.24 PlatonicSolids/SphereSingularHomology.lean

Stereographic $U \cap V \simeq_h S^{n-1}$ and $H_{n+1}(S^{n+1}) \cong H_n(S^n)$ (967 lines).

```
/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Category.ModuleCat.Abelian
import Mathlib.Algebra.Homology.HomologicalComplexBiprod
import Mathlib.Analysis.Convex.Contractible
import Mathlib.Analysis.Normed.Module.Ball.RadialEquiv
import PlatonicSolids.CellularHomology
import PlatonicSolids.Incidence
import PlatonicSolids.SingularExcision.OpenCoverQI

/-!
Singular  $H_n(S^n) \cong \mathbb{R}$  for  $n > 0$  from the stereographic
Mayer-Vietoris cover, after the open-cover quasi-isomorphism. The
intersection of the charts is homotopy-equivalent to  $S^{n-1}$ ; the
connecting map is an isomorphism in positive degrees;  $H_1(S^1)$  is
identified with  $\ker \epsilon$  of the two-point space. The optional Euler
bridge platonic_solids_3d_of_sphere derives the Betti numbers 1, 0, 1
from a quasi-isomorphism to singular chains of a model homotopy-equivalent
to  $S^2$ ; it does not change Palomar compared types.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex Metric Set ContinuousMap
open scoped Topology InnerProductSpace

set_option linter.unusedSectionVars false

variable {k : Type} [Ring k] (R : ModuleCat.{0} k)
variable [HasCoproducts.{0} (ModuleCat.{0} k)]
variable [HasPullbacks (ModuleCat.{0} k)]

noncomputable section

/-! ## Stereographic intersection homotopy-equivalent to a sphere -/

lemma southPole_eq_neg_northPole (n : ℕ) :
  (southPole n : EuclideanSpace ℝ (Fin (n + 1))) =
    (-1 : ℝ) • (northPole n : EuclideanSpace ℝ (Fin (n + 1))) := by
  simp [southPole, northPole]

lemma stereoToFun_north_south (n : ℕ) :
  stereoToFun (northPole n : EuclideanSpace ℝ (Fin (n + 1)))
    (southPole n : EuclideanSpace ℝ (Fin (n + 1))) = 0 := by
  have hz :
    ((ℝ · (northPole n : EuclideanSpace ℝ (Fin (n + 1))))).orthogonal).orthogonalProjection
      (southPole n : EuclideanSpace ℝ (Fin (n + 1))) = 0 := by
  rw [southPole_eq_neg_northPole, map_smul,
      Submodule.orthogonalProjection_orthogonalComplement_singleton_eq_zero]
  simp
  rw [stereoToFun_apply, hz, smul_zero]

lemma homeoPuncturedNorth_apply (n : ℕ) (x : ↑(puncturedNorth n)) :
  homeoPuncturedNorth n x =
    stereoToFun (northPole n : EuclideanSpace ℝ (Fin (n + 1))) (x : _) := by
  apply Subtype.ext
```

```

dsimp [homeoPuncturedNorth]
rfl

lemma homeoPuncturedNorth_south (n : ℕ) :
  homeoPuncturedNorth n ⟨southPole n, by
    simp [puncturedNorth] using (northPole_ne_southPole n).symm⟩ = 0 := by
  rw [homeoPuncturedNorth_apply, stereoToFun_north_south]

lemma homeoPuncturedNorth_symm_zero (n : ℕ) :
  (homeoPuncturedNorth n).symm 0 =
    ⟨southPole n, by simp [puncturedNorth] using (northPole_ne_southPole n).symm⟩ := by
  apply (homeoPuncturedNorth n).injective
  simp [homeoPuncturedNorth_south]

lemma homeoPuncturedNorth_eq_zero_iff (n : ℕ) (x : ↑(puncturedNorth n)) :
  homeoPuncturedNorth n x = 0 ↔ (x : MetricSphere n) = southPole n := by
  constructor
  · intro hx
    have hx' : x = (homeoPuncturedNorth n).symm 0 := by
      rw [← hx, Homeomorph.symm_apply_apply]
    rw [hx', homeoPuncturedNorth_symm_zero]
  · rintro h
    have hx : x = ⟨southPole n, by
      simp [puncturedNorth] using (northPole_ne_southPole n).symm⟩ :=
      Subtype.ext h
    rw [hx, homeoPuncturedNorth_south]

/-- Stereographic projection from the north pole identifies `U ∩ V` with
the punctured hyperplane. -/
def homeoPuncturedInter (n : ℕ) :
  ↑(puncturedNorth n ∩ puncturedSouth n) ≃ₜ
    ({0}ᶜ : Set ((ℝ · (northPole n : EuclideanSpace ℝ (Fin (n + 1)))).orthogonal)) where
toFun p :=
  ⟨homeoPuncturedNorth n ⟨p.1, p.2.1⟩,
    mt (homeoPuncturedNorth_eq_zero_iff n ⟨p.1, p.2.1⟩).1 p.2.2⟩
invFun w :=
  ⟨(homeoPuncturedNorth n).symm w.1, by
    refine ⟨((homeoPuncturedNorth n).symm w.1).2, ?_⟩
    exact mt (homeoPuncturedNorth_eq_zero_iff n ((homeoPuncturedNorth n).symm w.1)).2
    (by rw [(homeoPuncturedNorth n).apply_symm_apply]; exact w.2)⟩
left_inv p := by
  ext
  simp
right_inv w := by
  ext
  simp
continuous_toFun := by
  rw [continuous_induced_rng]
  exact (homeoPuncturedNorth n).continuous.comp
    (Continuous.subtype_mk continuous_subtype_val _)
continuous_invFun := by
  rw [continuous_induced_rng]
  convert (continuous_subtype_val.comp
    ((homeoPuncturedNorth n).symm.continuous.comp continuous_subtype_val)) using 1

instance contractibleSpace_Ioi_zero : ContractibleSpace (Ioi (0 : ℝ)) :=
  (convex_Ioi (0 : ℝ)).contractibleSpace nonempty_Ioi

/-- A product with a contractible space is homotopy-equivalent to the first factor. -/
def homotopyEquiv_prod_contractible
  {X Y : Type} [TopologicalSpace X] [TopologicalSpace Y] [ContractibleSpace Y] :
  (X × Y) ≃ₕ X :=
  (ContinuousMap.HomotopyEquiv.prodCongr (ContinuousMap.HomotopyEquiv.refl X)

```

```

    (Classical.choice (ContractibleSpace.hequiv_unit Y))).trans
    (Homeomorph.prodUnique X Unit).toHomotopyEquiv

/-- Polar coordinates: the punctured hyperplane is homotopy-equivalent to
its unit sphere. -/
def homotopyEquiv_puncturedHyperplane_sphere (n : ℕ) :
  ({0}^c : Set ((ℝ · (northPole n : EuclideanSpace ℝ (Fin (n + 1)))).orthogonal)) ≃_h
  ↑(sphere (0 : ((ℝ · (northPole n : EuclideanSpace ℝ (Fin (n + 1)))).orthogonal)) 1) :=
  (homeomorphUnitSphereProd
    ((ℝ · (northPole n : EuclideanSpace ℝ (Fin (n + 1)))).orthogonal)).toHomotopyEquiv.trans
    homotopyEquiv_prod_contractible

/-- The unit sphere in the polar hyperplane of `S^{n+1}` is homeomorphic to `S^n`. -/
def homeoHyperplaneSphere (n : ℕ) :
  ↑(sphere (0 : ((ℝ · (northPole (n + 1) : EuclideanSpace ℝ (Fin (n + 2)))).orthogonal)) 1)
  ≃_t
  MetricSphere n :=
  haveI : Fact (Module.finrank ℝ (EuclideanSpace ℝ (Fin (n + 2))) = n + 1 + 1) :=
    ⟨by simp⟩
  have hv : (northPole (n + 1) : EuclideanSpace ℝ (Fin (n + 2))) ≠ 0 := by
    simp [northPole]
  let b := OrthonormalBasis.fromOrthogonalSpanSingleton (n + 1) hv
  have himg : b.repr.toHomeomorph '' sphere 0 1 =
    sphere (0 : EuclideanSpace ℝ (Fin (n + 1))) 1 := by
    rw [LinearIsometryEquiv.coe_toHomeomorph, LinearIsometryEquiv.image_sphere]
    simp
  (b.repr.toHomeomorph.image (sphere 0 1)).trans (Homeomorph.setCongr himg)

/-- The stereographic intersection of `S^{n+1}` is homotopy-equivalent to `S^n`. -/
def homotopyEquiv_puncturedInter_sphere (n : ℕ) :
  ↑(puncturedNorth (n + 1) ∩ puncturedSouth (n + 1)) ≃_h MetricSphere n :=
  (homeoPuncturedInter (n + 1)).toHomotopyEquiv.trans
  ((homotopyEquiv_puncturedHyperplane_sphere (n + 1)).trans
    (homeoHyperplaneSphere n).toHomotopyEquiv)

/-! ## Mayer-Vietoris connecting map, hex discharged -/

/-- Connecting map of the stereographic cover, with excision supplied by
the open-cover quasi-isomorphism. -/
def singularHomology_sphere_mayerVietorisδ (n : ℕ) :
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (.of (MetricSphere (n + 1)))) .homology (n + 1) →
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (.of ↑(puncturedNorth (n + 1) ∩ puncturedSouth (n + 1)))) .homology n :=
  mayerVietorisδ (C := ModuleCat.{0} k) R
  (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
  (interInclRight (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
  (subtypeIncl (MetricSphere (n + 1)) (puncturedNorth (n + 1)))
  (subtypeIncl (MetricSphere (n + 1)) (puncturedSouth (n + 1)))
  (subset_square_comm _ _)
  (SingularExcision.IsExcisiveSubspaces_puncturedSphere (R := R) (n + 1)) n

lemma isZero_homology_chainBiprod
  {U V : TopCat.{0}} {n : ℕ}
  (hU : IsZero (((singularHomologyFunctor (ModuleCat.{0} k) n).obj R).obj U))
  (hV : IsZero (((singularHomologyFunctor (ModuleCat.{0} k) n).obj R).obj V)) :
  IsZero (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj U ⊕
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj V)) .homology n := by
  let KU := ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj U
  let KV := ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj V
  change IsZero ((KU ⊕ KV).homology n)
  rw [IsZero.iff_id_eq_zero, ← homologyMap_id (KU ⊕ KV) n, ← biprod.total]
  rw [homologyMap_add, homologyMap_comp, homologyMap_comp]

```



```

have hf : homologyMap (biprod.fst : KU  $\boxplus$  KV  $\longrightarrow$  KU) n = 0 := hU.eq_of_tgt _
have hs : homologyMap (biprod.snd : KU  $\boxplus$  KV  $\longrightarrow$  KV) n = 0 := hV.eq_of_tgt _
simp [hf, hs]

lemma isZero_homology_chartSum (n m :  $\mathbb{N}$ ) (hm : m  $\neq$  0) :
  IsZero (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (.of  $\uparrow$ (puncturedNorth n))  $\boxplus$ 
    ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (.of  $\uparrow$ (puncturedSouth n))))).homology m :=
  isZero_homology_chainBiprod (R := R)
  (isZero_singularHomology_puncturedNorth (C := ModuleCat.{0} k) R n hm)
  (isZero_singularHomology_puncturedSouth (C := ModuleCat.{0} k) R n hm)

lemma triad $\delta$ _sphere_isIso {n :  $\mathbb{N}$ } (hn : 0 < n) :
  IsIso (triad $\delta$  (C := ModuleCat.{0} k) R
    (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
    (interInclRight (puncturedNorth (n + 1)) (puncturedSouth (n + 1))) n) :=
  (triad_shortExact (C := ModuleCat.{0} k) R
    (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
    (interInclRight (puncturedNorth (n + 1)) (puncturedSouth (n + 1))))).isIso_ $\delta$ 
  (n + 1) n (by simp)
  (isZero_homology_chartSum (R := R) (n + 1) (n + 1) (Nat.succ_ne_zero n))
  (isZero_homology_chartSum (R := R) (n + 1) n hn.ne')

lemma singularHomology_sphere_mayerVietoris $\delta$ _isIso {n :  $\mathbb{N}$ } (hn : 0 < n) :
  IsIso (singularHomology_sphere_mayerVietoris $\delta$  (R := R) n) := by
  dsimp [singularHomology_sphere_mayerVietoris $\delta$ , mayerVietoris $\delta$ ]
  haveI := triad $\delta$ _sphere_isIso (R := R) hn
  infer_instance

/-- In positive degrees the stereographic connecting map identifies
`H_{n+1}(S^{n+1})` with `H_n` of the chart intersection. -/
def singularHomology_sphere_mayerVietoris $\delta$ _iso {n :  $\mathbb{N}$ } (hn : 0 < n) :
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (.of (MetricSphere (n + 1))))).homology (n + 1)  $\cong$ 
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (.of  $\uparrow$ (puncturedNorth (n + 1)  $\cap$  puncturedSouth (n + 1)))).homology n :=
  haveI := singularHomology_sphere_mayerVietoris $\delta$ _isIso (R := R) hn
  asIso (singularHomology_sphere_mayerVietoris $\delta$  (R := R) n)

/-- For `n  $\geq$  1`, `H_{n+1}(S^{n+1})` is isomorphic to `H_n(S^n)`. -/
def singularHomology_sphere_succ_iso {n :  $\mathbb{N}$ } (hn : 0 < n) :
  (((singularHomologyFunctor (ModuleCat.{0} k) (n + 1)).obj R).obj
    (.of (MetricSphere (n + 1))))  $\cong$ 
  (((singularHomologyFunctor (ModuleCat.{0} k) n).obj R).obj
    (.of (MetricSphere n)))) :=
  (singularHomology_sphere_mayerVietoris $\delta$ _iso (R := R) hn).trans
  (singularHomologyIso_of_homotopyEquiv
    (homotopyEquiv_puncturedInter_sphere n) R n)

/-- Iterated connecting isomorphism: for `n  $\geq$  1`,
`H_{n+j}(S^{n+j})` is isomorphic to `H_n(S^n)`. -/
def singularHomology_sphere_iterate_iso
  {n :  $\mathbb{N}$ } (hn : 0 < n) : (j :  $\mathbb{N}$ )  $\rightarrow$ 
  (((singularHomologyFunctor (ModuleCat.{0} k) (n + j)).obj R).obj
    (.of (MetricSphere (n + j))))  $\cong$ 
  (((singularHomologyFunctor (ModuleCat.{0} k) n).obj R).obj
    (.of (MetricSphere n))))
| 0 => Iso.refl _
| j + 1 =>
  (singularHomology_sphere_succ_iso (R := R) (n := n + j) (by omega)).trans
  (singularHomology_sphere_iterate_iso hn j)

```

```

/-! ## `H_*(S^n)` in all degrees -/

/-- Connecting map of the stereographic cover of `S^{n+1}` in any degree. -/
def singularHomology_sphere_mayerVietorisδ_deg (n m : ℕ) :
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (.of (MetricSphere (n + 1))))).homology (m + 1) →
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (.of (↑(puncturedNorth (n + 1) ∩ puncturedSouth (n + 1))))).homology m :=
  mayerVietorisδ (C := ModuleCat.{0} k) R
  (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
  (interInclRight (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
  (subtypeIncl (MetricSphere (n + 1)) (puncturedNorth (n + 1)))
  (subtypeIncl (MetricSphere (n + 1)) (puncturedSouth (n + 1)))
  (subset_square_comm _ _)
  (SingularExcision.IsExcisiveSubspaces_puncturedSphere (R := R) (n + 1)) m

lemma triadδ_sphere_isIso_of_pos (n m : ℕ) (hm : 0 < m) :
  IsIso (triadδ (C := ModuleCat.{0} k) R
    (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
    (interInclRight (puncturedNorth (n + 1)) (puncturedSouth (n + 1))) m) :=
  (triad_shortExact (C := ModuleCat.{0} k) R
    (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
    (interInclRight (puncturedNorth (n + 1)) (puncturedSouth (n + 1))))).isIso_δ
  (m + 1) m (by simp)
  (isZero_homology_chartSum (R := R) (n + 1) (m + 1) (Nat.succ_ne_zero m))
  (isZero_homology_chartSum (R := R) (n + 1) m hm.ne')

lemma singularHomology_sphere_mayerVietorisδ_deg_isIso
  (n : ℕ) {m : ℕ} (hm : 0 < m) :
  IsIso (singularHomology_sphere_mayerVietorisδ_deg (R := R) n m) := by
  dsimp [singularHomology_sphere_mayerVietorisδ_deg, mayerVietorisδ]
  haveI := triadδ_sphere_isIso_of_pos (R := R) n m hm
  infer_instance

/-- For `m ≥ 1`, `H_{m+1}(S^{n+1})` is isomorphic to `H_m(S^n)`. -/
def singularHomology_sphere_succ_iso_of_pos {m : ℕ} (hm : 0 < m) (n : ℕ) :
  (((singularHomologyFunctor (ModuleCat.{0} k) (m + 1)).obj R).obj
    (.of (MetricSphere (n + 1)))) ≅
  (((singularHomologyFunctor (ModuleCat.{0} k) m).obj R).obj
    (.of (MetricSphere n))) :=
  haveI := singularHomology_sphere_mayerVietorisδ_deg_isIso (R := R) n hm
  (asIso (singularHomology_sphere_mayerVietorisδ_deg (R := R) n m)).trans
  (singularHomologyIso_of_homotopyEquiv
    (homotopyEquiv_puncturedInter_sphere n) R m)

/-- The 0-sphere in `EuclideanSpace ℝ (Fin 1)` is homeomorphic to `{±1} ⊂ ℝ`. -/
def homeoMetricSphere0 :
  MetricSphere 0 ≃t (↑(Metric.sphere (0 : ℝ) 1)) :=
  let e : EuclideanSpace ℝ (Fin 1) ≃t ℝ :=
    (PiLp.homeomorph 2 fun _ : Fin 1 ↦ ℝ).trans (Homeomorph.funUnique (Fin 1) ℝ)
  have himg : e '' sphere 0 1 = sphere (0 : ℝ) 1 := by
    ext x
    constructor
    · rintro ⟨y, hy, rfl⟩
      have hy1 : ||y|| = 1 := by simpa [mem_sphere_iff_norm] using hy
      have : |e y| = ||y|| := by
        simp [e, EuclideanSpace.norm_eq, Real.norm_eq_abs, Finset.univ_unique,
          Fin.default_eq_zero, Real.sqrt_sq_eq_abs, PiLp.homeomorph]
      simpa [mem_sphere_iff_norm, Real.norm_eq_abs, this] using hy1
    · intro hx
      refine ⟨e.symm x, ?_, e.apply_symm_apply x⟩
      have hx1 : |x| = 1 := by simpa [mem_sphere_iff_norm, Real.norm_eq_abs] using hx
      have : ||e.symm x|| = |x| := by

```

```

simp [e, EuclideanSpace.norm_eq, Real.norm_eq_abs, Finset.univ_unique,
      Fin.default_eq_zero, Real.sqrt_sq_eq_abs, PiLp.homeomorph]
simp [mem_sphere_iff_norm, this] using hx1
(e.image (sphere 0 1)).trans (Homeomorph.setCongr himg)

lemma isZero_singularHomology_metricSphere0 {m : ℕ} (hm : m ≠ 0) :
  IsZero (((singularHomologyFunctor (ModuleCat.{0} k) m).obj R).obj
    (.of (MetricSphere 0))) :=
  (isZero_singularHomology_sphere0 (C := ModuleCat.{0} k) R hm).of_iso
  (singularHomologyIso_of_homeomorph (homeoMetricSphere0) R m)

/-- `H_m(S^1) = 0` for `m ≥ 2`. -/
lemma isZero_singularHomology_sphere1_of_ge_two {m : ℕ} (hm : 2 ≤ m) :
  IsZero (((singularHomologyFunctor (ModuleCat.{0} k) m).obj R).obj
    (.of (MetricSphere 1))) := by
  have hm0 : 0 < m - 1 := Nat.sub_pos_of_lt hm
  convert (isZero_singularHomology_metricSphere0 (R := R) hm0.ne').of_iso
    (singularHomology_sphere_succ_iso_of_pos (R := R) hm0 0)
  exact (Nat.sub_add_cancel (Nat.le_trans (by norm_num : 1 ≤ 2) hm)).symm

lemma triadδ_sphere_mono_zero (n : ℕ) :
  Mono (triadδ (C := ModuleCat.{0} k) R
    (interInclLeft (puncturedNorth n) (puncturedSouth n))
    (interInclRight (puncturedNorth n) (puncturedSouth n)) 0) :=
  (triad_shortExact (C := ModuleCat.{0} k) R
    (interInclLeft (puncturedNorth n) (puncturedSouth n))
    (interInclRight (puncturedNorth n) (puncturedSouth n))) .mono_δ
  1 0 (by simp) (isZero_homology_chartSum (R := R) n 1 one_ne_zero)

instance pathConnectedSpace_puncturedInter {n : ℕ} [NeZero n] :
  PathConnectedSpace ↑(puncturedNorth (n + 1) ∩ puncturedSouth (n + 1)) :=
  pathConnectedSpace_of_homotopyEquiv (homotopyEquiv_puncturedInter_sphere n)

lemma homologyMap_mvIncl_zero_mono
  {n : ℕ} [NeZero n] :
  Mono (homologyMap
    (mvIncl (C := ModuleCat.{0} k) R
      (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
      (interInclRight (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))) 0) := by
  have : IsIso (homologyMap
    (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
      (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))) 0) :=
    inferInstance
  have hfst :
    homologyMap
      (mvIncl (C := ModuleCat.{0} k) R
        (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
        (interInclRight (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))) 0 >>
    homologyMap
      (biprod.fst :
        ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
          (.of ↑(puncturedNorth (n + 1))) ⊕
        ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
          (.of ↑(puncturedSouth (n + 1))) →
        ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
          (.of ↑(puncturedNorth (n + 1)))) 0 =
    homologyMap
      (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
        (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))) 0 := by
  rw [← homologyMap_comp]
  congr 1
  exact biprod.lift_fst _ _
  exact mono_of_mono_fac hfst

```

```

/-- `H_1(S^n) = 0` for `n ≥ 2`. -/
lemma isZero_singularHomology_sphere_one {n : ℕ} (hn : 2 ≤ n) :
  IsZero (((singularHomologyFunctor (ModuleCat.{0} k) 1).obj R).obj
    (.of (MetricSphere n))) := by
  have hn1 : n - 1 + 1 = n := Nat.sub_add_cancel (by omega)
  haveI : NeZero (n - 1) := ⟨by omega⟩
  haveI : PathConnectedSpace ↑(puncturedNorth n ∩ puncturedSouth n) := by
    rw [show n = n - 1 + 1 from hn1.symm]
    infer_instance
  have hmono : Mono (homologyMap
    (mvIncl (C := ModuleCat.{0} k) R
      (interInclLeft (puncturedNorth n) (puncturedSouth n))
      (interInclRight (puncturedNorth n) (puncturedSouth n))) 0) := by
  have hfst :
    homologyMap
      (mvIncl (C := ModuleCat.{0} k) R
        (interInclLeft (puncturedNorth n) (puncturedSouth n))
        (interInclRight (puncturedNorth n) (puncturedSouth n))) 0 >>
    homologyMap
      (biprod.fst :
        ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
          (.of ↑(puncturedNorth n)) ⊞
          ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
            (.of ↑(puncturedSouth n)) →
          ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
            (.of ↑(puncturedNorth n))) 0 =
        homologyMap
          (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map
            (interInclLeft (puncturedNorth n) (puncturedSouth n))) 0 := by
    rw [← homologyMap_comp]
    congr 1
    exact biprod.lift_fst _ _
  exact mono_of_mono_fac hfst
  have hz := (triad_shortExact (C := ModuleCat.{0} k) R
    (interInclLeft (puncturedNorth n) (puncturedSouth n))
    (interInclRight (puncturedNorth n) (puncturedSouth n))).δ_comp
    1 0 (by simp)
  have hδ0 : triadδ (C := ModuleCat.{0} k) R
    (interInclLeft (puncturedNorth n) (puncturedSouth n))
    (interInclRight (puncturedNorth n) (puncturedSouth n)) 0 = 0 := by
  refine (cancel_mono (homologyMap
    (mvIncl (C := ModuleCat.{0} k) R
      (interInclLeft (puncturedNorth n) (puncturedSouth n))
      (interInclRight (puncturedNorth n) (puncturedSouth n))) 0)).1 ?_
  erw [hz]
  exact (Limits.zero_comp (f := homologyMap
    (mvIncl (C := ModuleCat.{0} k) R
      (interInclLeft (puncturedNorth n) (puncturedSouth n))
      (interInclRight (puncturedNorth n) (puncturedSouth n))) 0)).symm
  haveI := triadδ_sphere_mono_zero (R := R) n
  refine IsZero.of_iso ?_
    (excisiveHomologyIso (C := ModuleCat.{0} k) R
      (interInclLeft (puncturedNorth n) (puncturedSouth n))
      (interInclRight (puncturedNorth n) (puncturedSouth n))
      (subtypeIncl (MetricSphere n) (puncturedNorth n))
      (subtypeIncl (MetricSphere n) (puncturedSouth n))
      (subset_square_comm _ _))
    (SingularExcision.IsExcisiveSubspaces_puncturedSphere (R := R) n 1).symm
  rw [IsZero.iff_id_eq_zero]
  refine (cancel_mono (triadδ (C := ModuleCat.{0} k) R
    (interInclLeft (puncturedNorth n) (puncturedSouth n))
    (interInclRight (puncturedNorth n) (puncturedSouth n)) 0)).1 ?_

```

```

rw [hδ0]
simp

/-- Drop one dimension:  $H_a(S^b) \cong H_{a-1}(S^{b-1})$  when  $a \geq 2$  and  $b \geq 1$ . -/
def singularHomology_sphere_down_iso
  {a b : ℕ} (ha : 2 ≤ a) (hb : 1 ≤ b) :
  (((singularHomologyFunctor (ModuleCat.{0} k) a).obj R).obj
    (.of (MetricSphere b))) ≅
    (((singularHomologyFunctor (ModuleCat.{0} k) (a - 1)).obj R).obj
      (.of (MetricSphere (b - 1))))) :=
  by
    have hm : 0 < a - 1 := Nat.sub_pos_of_lt ha
    have ha' : a - 1 + 1 = a := Nat.sub_add_cancel (by omega)
    have hb' : b - 1 + 1 = b := Nat.sub_add_cancel hb
    have type_eq :
      (((singularHomologyFunctor (ModuleCat.{0} k) a).obj R).obj
        (.of (MetricSphere b))) =
        (((singularHomologyFunctor (ModuleCat.{0} k) (a - 1 + 1)).obj R).obj
          (.of (MetricSphere (b - 1 + 1))))) := by
      rw [ha', hb']
    exact (eqToIso type_eq).trans
      (singularHomology_sphere_succ_iso_of_pos (R := R) hm (b - 1))

/-- For  $n \geq 1$ ,  $H_n(S^n)$  is isomorphic to  $H_1(S^1)$ . -/
def singularHomology_sphere_iso_sphere1 {n : ℕ} (hn : 0 < n) :
  (((singularHomologyFunctor (ModuleCat.{0} k) n).obj R).obj
    (.of (MetricSphere n))) ≅
    (((singularHomologyFunctor (ModuleCat.{0} k) 1).obj R).obj
      (.of (MetricSphere 1))) :=
  by
    have hn' : 1 + (n - 1) = n := Nat.add_sub_cancel' hn
    have type_eq :
      (((singularHomologyFunctor (ModuleCat.{0} k) n).obj R).obj
        (.of (MetricSphere n))) =
        (((singularHomologyFunctor (ModuleCat.{0} k) (1 + (n - 1))).obj R).obj
          (.of (MetricSphere (1 + (n - 1))))) := by
      rw [hn']
    exact (eqToIso type_eq).trans
      (singularHomology_sphere_iterate_iso (R := R) (n := 1)
        (by norm_num : 0 < 1) (n - 1))

/-- Positive-degree singular homology of  $S^n$  vanishes off the top degree. -/
lemma isZero_singularHomology_sphere
  {n m : ℕ} (hn : 0 < n) (hm : m ≠ 0) (hne : m ≠ n) :
  IsZero (((singularHomologyFunctor (ModuleCat.{0} k) m).obj R).obj
    (.of (MetricSphere n))) := by
  rcases lt_or_gt_of_ne hne with hlt | hgt
  · --  $0 < m < n$ : induct on  $n$ , reducing to  $H_1$  of a sphere of dimension  $\geq 2$ .
    induction n generalizing m with
    | zero => exact (hn.ne rfl).elim
    | succ n ih =>
      have hn0 : n ≠ 0 := by omega
      cases m with
      | zero => exact (hm rfl).elim
      | succ m =>
        cases m with
        | zero =>
          exact isZero_singularHomology_sphere_one (R := R) (by omega)
        | succ m =>
          refine (ih (m := m + 1) (by omega) (by omega) (by omega)
            (by omega)).of_iso
            (singularHomology_sphere_down_iso (R := R) (a := m + 2) (b := n + 1)
              (by omega) (by omega))

```

```

· -- `m > n ≥ 1`: induct on `n`, reducing to `H_*(S^1)` with degree ≥ 2.
induction n generalizing m with
| zero => exact (hn.ne rfl).elim
| succ n ih =>
  by_cases hn0 : n = 0
  · subst hn0
    exact isZero_singularHomology_sphere1_of_ge_two (R := R) (by omega)
  · refine (ih (m := m - 1) (by omega) (by omega) (by omega))
    (by omega).of_iso
    (singularHomology_sphere_down_iso (R := R) (a := m) (b := n + 1)
    (by omega) (by omega))

/-! ## `H_1(S^1) ≅ R` and the full calculation of `H_*(S^n)` -/

instance preservesBinaryBiproduct_homologyFunctor (n : ℕ)
  (K L : ChainComplex (ModuleCat.{0} k) ℕ) :
  PreservesBinaryBiproduct K L
  (homologyFunctor (ModuleCat.{0} k) (ComplexShape.down ℕ) n) :=
  preservesBinaryBiproduct_of_preservesBiproduct _ K L

lemma homologyMap_mvIncl
  {W U V : TopCat.{0}} (iU : W → U) (iV : W → V) (n : ℕ) :
  homologyMap (mvIncl (C := ModuleCat.{0} k) R iU iV) n ≫
  ((homologyFunctor (ModuleCat.{0} k) (ComplexShape.down ℕ) n).mapBiprod
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj U)
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj V)).hom =
  biprod.lift
  (homologyMap (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map iU) n)
  (-homologyMap (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map iV) n) := by
let F := homologyFunctor (ModuleCat.{0} k) (ComplexShape.down ℕ) n
let KU := ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj U
let KV := ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj V
have h := biprod.map_lift_mapBiprod (F := F) (X := KU) (Y := KV)
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map iU)
  (-(((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map iV))
simp [mvIncl, homologyMap_neg] using h

lemma biprod_lift_id_neg_id_mono (X : ModuleCat.{0} k) :
  Mono (biprod.lift (⊣ X) (¬⊣ X)) :=
  mono_of_mono_fac (biprod.lift_fst (⊣ X) (¬⊣ X))

lemma biprod_lift_comp_map {W X Y X' Y' : ModuleCat.{0} k}
  (f : W → X) (g : W → Y) (a : X → X') (b : Y → Y') :
  biprod.lift f g ≫ biprod.map a b = biprod.lift (f ≫ a) (g ≫ b) := by
  refine biprod.hom_ext _ _ ?_ ?_
  · simp
  · simp

lemma biprod_comp_lift {W' W X Y : ModuleCat.{0} k}
  (h : W' → W) (f : W → X) (g : W → Y) :
  h ≫ biprod.lift f g = biprod.lift (h ≫ f) (h ≫ g) := by
  refine biprod.hom_ext _ _ ?_ ?_
  · simp
  · simp

lemma homologyMap_mvIncl_comp_aug
  {W U V : TopCat.{0}} [PathConnectedSpace U] [PathConnectedSpace V]
  (iU : W → U) (iV : W → V) :
  homologyMap (mvIncl (C := ModuleCat.{0} k) R iU iV) 0 ≫
  ((homologyFunctor (ModuleCat.{0} k) (ComplexShape.down ℕ) 0).mapBiprod
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj U)
  (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj V)).hom ≫
  biprod.map (U.singularHomology_0ε R) (V.singularHomology_0ε R) =

```

```

W.singularHomology0ε R ≫ biprod.lift (⊗ R) (⊗ R) := by
have hmv := homologyMap_mvIncl (R := R) iU iV 0
have hU := singularHomology0ε_natural iU R
have hV := singularHomology0ε_natural iV R
have hneg :
  (-homologyMap (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map iV) 0) ≫
    V.singularHomology0ε R =
  -W.singularHomology0ε R := by
  rw [Preadditive.neg_comp, hV]; rfl
calc
homologyMap (mvIncl (C := ModuleCat.{0} k) R iU iV) 0 ≫
  ((homologyFunctor (ModuleCat.{0} k) (ComplexShape.down N) 0).mapBiprod
    (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj U)
    (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj V)).hom ≫
  biprod.map (U.singularHomology0ε R) (V.singularHomology0ε R) =
  biprod.lift
    (homologyMap (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map iU) 0)
    (-homologyMap (((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).map iV) 0) ≫
  biprod.map (U.singularHomology0ε R) (V.singularHomology0ε R) := by
  rw [← Category.assoc, hmv]; rfl
_ = biprod.lift (W.singularHomology0ε R) (-W.singularHomology0ε R) := by
  rw [biprod_lift_comp_map, hU, hneg]; rfl
_ = W.singularHomology0ε R ≫ biprod.lift (⊗ R) (⊗ R) := by
  rw [biprod_comp_lift, Category.comp_id, Preadditive.comp_neg, Category.comp_id]

/-- Connecting map of the triad is a kernel of `mvIncl_*` when it is mono. -/
noncomputable def triadδ_isoKernel
  {W U V : TopCat.{0}} (iU : W → U) (iV : W → V) [Mono iU]
  [Mono (triadδ (C := ModuleCat.{0} k) R iU iV 0)] :
  (coverChains (C := ModuleCat.{0} k) R iU iV).homology 1 ≅
  kernel (homologyMap (mvIncl (C := ModuleCat.{0} k) R iU iV) 0) :=
  (limit.isoLimitCone
    (KernelFork.of
      (triadδ (C := ModuleCat.{0} k) R iU iV 0)
      (by
        have h := (triad_shortExact (C := ModuleCat.{0} k) R iU iV).δ_comp
          1 0 (by simp)
        exact h),
      (triad_exact_at_W (C := ModuleCat.{0} k) R iU iV 0).fIsKernel)).symm

def sphere0_pos : ↑(Metric.sphere (0 : ℝ) 1) :=
  ⟨1, by simp [mem_sphere_iff_norm]⟩

def sphere0_neg : ↑(Metric.sphere (0 : ℝ) 1) :=
  ⟨-1, by simp [mem_sphere_iff_norm]⟩

lemma sphere0_eq_pos_or_neg (x : ↑(Metric.sphere (0 : ℝ) 1)) :
  x = sphere0_pos ∨ x = sphere0_neg := by
  have hx : |(x : ℝ)| = 1 := by
  have hmem := x.2
  rw [mem_sphere_iff_norm, Real.norm_eq_abs, sub_zero] at hmem
  exact hmem
  rcases (abs_eq (show (0 : ℝ) ≤ 1 from by norm_num)).mp hx with h | h
  · exact Or.inl (Subtype.ext h)
  · exact Or.inr (Subtype.ext h)

lemma not_joined_sphere0_pos_neg :
  ¬ Joined sphere0_pos sphere0_neg := by
  intro h
  have h0 : h.somePath 0 = sphere0_pos := Path.source _
  have h1 : h.somePath 1 = sphere0_neg := Path.target _
  have heq := TotallyDisconnectedSpace.eq_of_continuous
    (h.somePath : unitInterval → ↑(Metric.sphere (0 : ℝ) 1))

```

```

    h.somePath.continuous (0 : unitInterval) (1 : unitInterval)
  rw [h0, h1] at heq
  have : (1 : ℝ) = -1 := congrArg Subtype.val heq
  norm_num at this

lemma zerothHomotopy_sphere0_pos_ne_neg :
  ZerothHomotopy.mk sphere0_pos ≠ ZerothHomotopy.mk sphere0_neg := by
  intro h
  exact not_joined_sphere0_pos_neg (Quotient.exact h)

lemma zerothHomotopy_sphere0_eq (c : ZerothHomotopy ↑(Metric.sphere (0 : ℝ) 1)) :
  c = ZerothHomotopy.mk sphere0_pos ∨ c = ZerothHomotopy.mk sphere0_neg := by
  obtain ⟨x, rfl⟩ := ZerothHomotopy.mk_surjective c
  rcases sphere0_eq_pos_or_neg x with hx | hx
  · exact Or.inl (congrArg _ hx)
  · exact Or.inr (congrArg _ hx)

noncomputable def equivZerothHomotopy_sphere0 :
  ZerothHomotopy ↑(Metric.sphere (0 : ℝ) 1) ≃ Bool := by
  classical
  refine
  { toFun := fun c => decide (c = ZerothHomotopy.mk sphere0_pos)
    invFun := fun b => bif b then ZerothHomotopy.mk sphere0_pos
      else ZerothHomotopy.mk sphere0_neg
    left_inv := ?_
    right_inv := ?_ }
  · intro c
    rcases zerothHomotopy_sphere0_eq c with hc | hc
    · simp [hc]
    · have hne : ¬ ZerothHomotopy.mk sphere0_neg = ZerothHomotopy.mk sphere0_pos :=
        zerothHomotopy_sphere0_pos_ne_neg.symm
      simp [hc, hne]
  · intro b
    cases b with
    | true => simp
    | false =>
      have hne : ¬ ZerothHomotopy.mk sphere0_neg = ZerothHomotopy.mk sphere0_pos :=
        zerothHomotopy_sphere0_pos_ne_neg.symm
      simp [hne]

/-- Coproduct of two copies of `R`, indexed by `Bool`, is the biproduct. -/
noncomputable def sigmaBoolIsoBiprod :
  (∐ fun _ : Bool => R) ≅ R ⊕ R where
  hom := Sigma.desc fun b => bif b then biprod.inl else biprod.inr
  inv := biprod.desc (Sigma.ι (fun _ : Bool => R) true) (Sigma.ι (fun _ : Bool => R) false)
  hom_inv_id := by
    refine Sigma.hom_ext _ _ ?_
    intro b
    cases b with
    | false =>
      rw [Sigma.ι_desc_assoc]
      change biprod.inr >> biprod.desc _ _ = _
      rw [biprod.inr_desc, Category.comp_id]
    | true =>
      rw [Sigma.ι_desc_assoc]
      change biprod.inl >> biprod.desc _ _ = _
      rw [biprod.inl_desc, Category.comp_id]
  inv_hom_id := by
    refine biprod.hom_ext' _ _ ?_ ?_
    · rw [biprod.inl_desc_assoc, Sigma.ι_desc, Category.comp_id]
      rfl
    · rw [biprod.inr_desc_assoc, Sigma.ι_desc, Category.comp_id]
      rfl

```



```

/-- Shear  $(x, y) \mapsto (x, x+y)$  on a biproduct. -/
noncomputable def biprodShearIso (X : ModuleCat.{0} k) : X  $\boxplus$  X  $\cong$  X  $\boxplus$  X where
  hom := biprod.lift biprod.fst (biprod.fst + biprod.snd)
  inv := biprod.lift biprod.fst (biprod.snd + (-biprod.fst))
  hom_inv_id := by
    refine biprod.hom_ext _ _ ?_ ?_
    · simp
    · simp [sub_eq_add_neg, add_assoc, add_neg_cancel_comm_assoc, add_neg_cancel]
  inv_hom_id := by
    refine biprod.hom_ext _ _ ?_ ?_
    · simp
    · simp [sub_eq_add_neg, add_assoc, add_neg_cancel_comm_assoc, add_neg_cancel]

lemma biprodShearIso_hom_snd (X : ModuleCat.{0} k) :
  (biprodShearIso X).hom  $\gg$  biprod.snd = biprod.fst + biprod.snd :=
  biprod.lift_snd _ _

lemma biprod_fst_add_snd_eq_desc (X : ModuleCat.{0} k) :
  (biprod.fst : X  $\boxplus$  X  $\longrightarrow$  X) + biprod.snd = biprod.desc ( $\not\vdash$  X) ( $\not\vdash$  X) := by
  refine biprod.hom_ext' _ _ ?_ ?_
  · simp
  · simp

lemma sigmaBoolIsoBiprod_hom_desc :
  (sigmaBoolIsoBiprod (R := R)).hom  $\gg$  biprod.desc ( $\not\vdash$  R) ( $\not\vdash$  R) =
  Sigma.desc fun _ : Bool  $\mapsto$   $\not\vdash$  R := by
  refine Sigma.hom_ext _ _ ?_
  intro b
  cases b with
  | false =>
    dsimp [sigmaBoolIsoBiprod]
    rw [Sigma.ι_desc_assoc]
    change biprod.inr  $\gg$  biprod.desc ( $\not\vdash$  R) ( $\not\vdash$  R) = _
    rw [biprod.inr_desc, Sigma.ι_desc]
  | true =>
    dsimp [sigmaBoolIsoBiprod]
    rw [Sigma.ι_desc_assoc]
    change biprod.inl  $\gg$  biprod.desc ( $\not\vdash$  R) ( $\not\vdash$  R) = _
    rw [biprod.inl_desc, Sigma.ι_desc]

lemma sigma_desc_id_sphere0 :
  (Sigma.reindex equivZerothHomotopy_sphere0
    (fun _ : Bool  $\mapsto$  R)).hom  $\gg$ 
  Sigma.desc (fun _ : Bool  $\mapsto$   $\not\vdash$  R) =
  Sigma.desc fun _ : ZerothHomotopy  $\uparrow$  (Metric.sphere (0 :  $\mathbb{R}$ ) 1)  $\mapsto$   $\not\vdash$  R := by
  refine Sigma.hom_ext _ _ ?_
  intro c
  rw [Sigma.ι_reindex_hom_assoc]
  refine Eq.trans (Sigma.ι_desc (fun _ : Bool  $\mapsto$   $\not\vdash$  R)
    (equivZerothHomotopy_sphere0 c)) ?_
  exact (Sigma.ι_desc (fun _ : ZerothHomotopy  $\uparrow$  (Metric.sphere (0 :  $\mathbb{R}$ ) 1)  $\mapsto$   $\not\vdash$  R) c).symm

/--  $\ker(\epsilon : H_0(S^0) \rightarrow R) \cong R$ . -/
noncomputable def kernel_singularHomology0_ε_sphere0 :
  kernel ((TopCat.of  $\uparrow$  (Metric.sphere (0 :  $\mathbb{R}$ ) 1)).singularHomology0_ε R)  $\cong$  R :=
  let e0 := TopCat.singularHomologyIso (.of  $\uparrow$  (Metric.sphere (0 :  $\mathbb{R}$ ) 1)) R
  let ere : ( $\coprod$  fun _ : ZerothHomotopy  $\uparrow$  (Metric.sphere (0 :  $\mathbb{R}$ ) 1)  $\mapsto$  R)  $\cong$ 
    ( $\coprod$  fun _ : Bool  $\mapsto$  R) :=
    Sigma.reindex equivZerothHomotopy_sphere0 (fun _ : Bool  $\mapsto$  R)
  have hEps : (TopCat.of  $\uparrow$  (Metric.sphere (0 :  $\mathbb{R}$ ) 1)).singularHomology0_ε R =
    e0.hom  $\gg$  ere.hom  $\gg$  (sigmaBoolIsoBiprod (R := R)).hom  $\gg$ 
    (biprodShearIso R).hom  $\gg$  biprod.snd := by

```

```

have h1 := TopCat.singularHomologyIso_sigma_desc_id
  (.of ↑(Metric.sphere (0 : ℝ) 1)) R
have h2 := sigma_desc_id_sphere0 (R := R)
have h3 := sigmaBoolIsoBiprod_hom_desc (R := R)
have h4 := biprodShearIso_hom_snd (X := R)
have h5 := biprod_fst_add_snd_eq_desc (X := R)
calc
  (TopCat.of ↑(Metric.sphere (0 : ℝ) 1)).singularHomologyIso R =
    e0.hom >> Sigma.desc (fun _ ↦ R) := h1.symm
  _ = e0.hom >> ere.hom >> Sigma.desc fun _ : Bool ↦ R := by
    rw [← h2]; rfl
  _ = e0.hom >> ere.hom >> (sigmaBoolIsoBiprod (R := R)).hom >>
    biprod.desc (R) (R) := by
    rw [← h3]
  _ = e0.hom >> ere.hom >> (sigmaBoolIsoBiprod (R := R)).hom >>
    (biprod.fst + biprod.snd) := by
    rw [h5]
  _ = e0.hom >> ere.hom >> (sigmaBoolIsoBiprod (R := R)).hom >>
    (biprodShearIso R).hom >> biprod.snd := by
    rw [← h4]
(kernel.congr _ _ hEps).trans <|
  (kernelIsoComp (e0.hom >> ere.hom >> (sigmaBoolIsoBiprod (R := R)).hom >>
    (biprodShearIso R).hom) biprod.snd).trans
  kernelBiprodSndIso

/-- Stereographic intersection of `S^1` is homotopy-equivalent to `S^0 ⊂ ℝ`. -/
def homotopyEquiv_puncturedInter_sphere0 :
  ↑(puncturedNorth 1 ∩ puncturedSouth 1) ≃h ↑(Metric.sphere (0 : ℝ) 1) :=
  (homotopyEquiv_puncturedInter_sphere0).trans homeoMetricSphere0.toHomotopyEquiv

/-- `H_1(S^1; R) ≅ R`. -/
noncomputable def singularHomology_sphere1 :
  (((singularHomologyFunctor (ModuleCat.{0} k) 1).obj R).obj
    (.of (MetricSphere 1))) ≅ R :=
  let iU := interInclLeft (puncturedNorth 1) (puncturedSouth 1)
  let iV := interInclRight (puncturedNorth 1) (puncturedSouth 1)
  haveI : Mono (triadδ (C := ModuleCat.{0} k) R iU iV 0) :=
    triadδ_sphere_mono_zero (R := R) 1
  have hcmp := homologyMap_mvIncl_comp_aug (R := R) iU iV
  let F := homologyFunctor (ModuleCat.{0} k) (ComplexShape.down N) 0
  let KU := ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (.of ↑(puncturedNorth 1))
  let KV := ((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj
    (.of ↑(puncturedSouth 1))
  let epsU := (TopCat.of ↑(puncturedNorth 1)).singularHomologyIso R
  let epsV := (TopCat.of ↑(puncturedSouth 1)).singularHomologyIso R
  haveI : IsIso epsU := inferInstance
  haveI : IsIso epsV := inferInstance
  let thetaIso :
    (((singularHomologyFunctor (ModuleCat.{0} k) 0).obj R).obj
      (.of ↑(puncturedNorth 1))) ⊕
      (((singularHomologyFunctor (ModuleCat.{0} k) 0).obj R).obj
        (.of ↑(puncturedSouth 1))) ≅ R ⊕ R :=
  { hom := biprod.map epsU epsV
    inv := biprod.map (inv epsU) (inv epsV)
    hom_inv_id := by
      refine biprod.hom_ext _ _ ?_ ?_
      · simp
      · simp
    inv_hom_id := by
      refine biprod.hom_ext _ _ ?_ ?_
      · simp
      · simp }

```

```

let q := (F.mapBiprod KU KV).trans thetaIso
haveI : Mono (biprod.lift (⊗ R) (⊗ R)) := biprod_lift_id_neg_id_mono R
let eW := singularHomologyIso_of_homotopyEquiv
(homotopyEquiv_puncturedInter_sphere0) R 0
have hnats : eW.hom >>
  (TopCat.of ↑(Metric.sphere (0 : ℝ) 1)).singularHomology_0ε R =
  (TopCat.of ↑(puncturedNorth 1 ∩ puncturedSouth 1)).singularHomology_0ε R :=
  singularHomology_0ε_natural (TopCat.ofHom homotopyEquiv_puncturedInter_sphere0.toFun) R
(excisiveHomologyIso (C := ModuleCat.{0} k) R iU iV
  (subtypeIncl (MetricSphere 1) (puncturedNorth 1))
  (subtypeIncl (MetricSphere 1) (puncturedSouth 1))
  (subset_square_comm _ _))
(SingularExcision.IsExcisiveSubspaces_puncturedSphere (R := R) 1) 1).symm.trans <|
(triadδ_isoKernel (R := R) iU iV).trans <|
(kernel.mapIso
  (f := homologyMap (mvIncl (C := ModuleCat.{0} k) R iU iV) 0)
  ((TopCat.of ↑(puncturedNorth 1 ∩ puncturedSouth 1)).singularHomology_0ε R >>
    biprod.lift (⊗ R) (⊗ R))
  (Iso.refl _) q (by
    simp using hcmp)).trans <|
(kernelCompMono
  ((TopCat.of ↑(puncturedNorth 1 ∩ puncturedSouth 1)).singularHomology_0ε R)
  (biprod.lift (⊗ R) (⊗ R))).trans <|
(kernel.congr _ _ hnats.symm).trans <|
(kernelIsIsoComp eW.hom
  ((TopCat.of ↑(Metric.sphere (0 : ℝ) 1)).singularHomology_0ε R)).trans
  (kernel_singularHomology_0ε_sphere0 (R := R))

/-- For `n > 0`, `H_n(S^n; R) ≅ R`. -/
noncomputable def singularHomology_sphere {n : ℕ} (hn : 0 < n) :
  (((singularHomologyFunctor (ModuleCat.{0} k) n).obj R).obj
    (.of (MetricSphere n))) ≅ R :=
  (singularHomology_sphere_iso_sphere1 (R := R) hn).trans
  (singularHomology_sphere1 (R := R))

/-- `H_0(S^2; R) ≅ R` by path-connectedness. -/
noncomputable def singularHomology_sphere2_zero :
  (((singularHomologyFunctor (ModuleCat.{0} k) 0).obj R).obj
    (.of (MetricSphere 2))) ≅ R :=
  haveI : NeZero (2 : ℕ) := ⟨by decide⟩
  singularHomology_0Iso_pathConnected _ R

/-- `H_1(S^2; R) = 0`. -/
lemma isZero_singularHomology_sphere2_one :
  IsZero (((singularHomologyFunctor (ModuleCat.{0} k) 1).obj R).obj
    (.of (MetricSphere 2))) :=
  isZero_singularHomology_sphere (R := R) (n := 2) (m := 1)
  (by decide) (by decide) (by decide)

/-- `H_2(S^2; R) ≅ R`. -/
noncomputable def singularHomology_sphere2 :
  (((singularHomologyFunctor (ModuleCat.{0} k) 2).obj R).obj
    (.of (MetricSphere 2))) ≅ R :=
  singularHomology_sphere (R := R) (n := 2) (by decide)

/-- The singular zeroth homology of `S^2` has dimension one over a field. -/
theorem singularHomology_sphere2_finrank_zero
  (K : Type) [Field K] :
  Module.finrank K
    (((singularHomologyFunctor (ModuleCat.{0} K) 0).obj
      (ModuleCat.of K K)).obj (.of (MetricSphere 2))) = 1 := by
  simp using ModuleCat.finrank_eq_of_iso
  (singularHomology_sphere2_zero (R := ModuleCat.of K K))

```

```

/-- The singular first homology of  $S^2$  has dimension zero over a field. -/
theorem singularHomology_sphere2_finrank_one
  (K : Type) [Field K] :
  Module.finrank K
    (((singularHomologyFunctor (ModuleCat.{0} K) 1).obj
      (ModuleCat.of K K)).obj (.of (MetricSphere 2))) = 0 := by
  letI := ModuleCat.subsingleton_of_isZero
  (isZero_singularHomology_sphere2_one (R := ModuleCat.of K K))
  exact Module.finrank_zero_of_subsingleton

/-- The singular second homology of  $S^2$  has dimension one over a field. -/
theorem singularHomology_sphere2_finrank_two
  (K : Type) [Field K] :
  Module.finrank K
    (((singularHomologyFunctor (ModuleCat.{0} K) 2).obj
      (ModuleCat.of K K)).obj (.of (MetricSphere 2))) = 1 := by
  simp using ModuleCat.finrank_eq_of_iso
  (singularHomology_sphere2 (R := ModuleCat.of K K))

/-- A finite cellular  $2$ -complex quasi-isomorphic to the singular chains of
a space homotopy-equivalent to  $S^2$  has Betti profile  $(1, 0, 1)$ . -/
theorem cellular_betti_profile_of_quasiIso_sphere2
  {K : Type} [Field K]
  {C2 C1 C0 : Type}
  [AddCommGroup C2] [Module K C2] [FiniteDimensional K C2]
  [AddCommGroup C1] [Module K C1] [FiniteDimensional K C1]
  [AddCommGroup C0] [Module K C0] [FiniteDimensional K C0]
  {X : Type} [TopologicalSpace X]
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0)
  (hcomp : d1.comp d2 = 0)
  (φ : cellularChainComplex d2 d1 hcomp →
    ((singularChainComplexFunctor (ModuleCat.{0} K)).obj
      (ModuleCat.of K K)).obj (.of X))
  [QuasiIso φ]
  (e : X ≃h MetricSphere 2) :
  betti0 d1 = 1 ∧ betti1 d2 d1 = 0 ∧ betti2 d2 = 1 := by
  have hfinrank (n : ℕ) :
    Module.finrank K ((cellularChainComplex d2 d1 hcomp).homology n) =
    Module.finrank K
      (((singularHomologyFunctor (ModuleCat.{0} K) n).obj
        (ModuleCat.of K K)).obj (.of (MetricSphere 2))) := by
  calc
    Module.finrank K ((cellularChainComplex d2 d1 hcomp).homology n) =
      Module.finrank K
        (((singularChainComplexFunctor (ModuleCat.{0} K)).obj
          (ModuleCat.of K K)).obj (.of X)).homology n :=
        homology_finrank_eq_of_quasiIso φ n
    _ = Module.finrank K
      (((singularHomologyFunctor (ModuleCat.{0} K) n).obj
        (ModuleCat.of K K)).obj (.of (MetricSphere 2))) :=
      ModuleCat.finrank_eq_of_iso
      (singularHomologyIso_of_homotopyEquiv e (ModuleCat.of K K) n)
  refine {?, ?, ?}
  · rw [← cellularChainComplex_homology_finrank_zero d2 d1 hcomp]
    exact_mod_cast (hfinrank 0).trans (singularHomology_sphere2_finrank_zero K)
  · rw [← cellularChainComplex_homology_finrank_one d2 d1 hcomp]
    exact_mod_cast (hfinrank 1).trans (singularHomology_sphere2_finrank_one K)
  · rw [← cellularChainComplex_homology_finrank_two d2 d1 hcomp]
    exact_mod_cast (hfinrank 2).trans (singularHomology_sphere2_finrank_two K)

/-- The 3D classification from a cellular  $2$ -complex identified, by a
quasi-isomorphism, with the singular chains of a space homotopy-equivalent

```

```

to `S^2`. Euler-Poincaré supplies `V+F=E+2`; regularity then forces a
Platonic pair. Palomar compared types are unchanged. -/
theorem platonic_solids_3d_of_sphere
  {K : Type} [Field K]
  {C2 C1 C0 : Type}
  [AddCommGroup C2] [Module K C2] [FiniteDimensional K C2]
  [AddCommGroup C1] [Module K C1] [FiniteDimensional K C1]
  [AddCommGroup C0] [Module K C0] [FiniteDimensional K C0]
  (d2 : C2 →1 [K] C1) (d1 : C1 →1 [K] C0)
  (V E F p q : ℕ)
  (hp : 3 ≤ p) (hq : 3 ≤ q)
  (h_edges_faces : p * F = 2 * E)
  (h_edges_verts : q * V = 2 * E)
  (hV : Module.finrank K C0 = V)
  (hE : Module.finrank K C1 = E)
  (hF : Module.finrank K C2 = F)
  (hcomp : d1.comp d2 = 0)
  {X : Type} [TopologicalSpace X]
  (φ : cellularChainComplex d2 d1 hcomp →
    ((singularChainComplexFunctor (ModuleCat.{0} K)).obj
      (ModuleCat.of K K)).obj (.of X))
  [QuasiIso φ]
  (e : X ≃h MetricSphere 2) :
  IsPlatonicPair p q := by
obtain ⟨hs0, hs1, hs2⟩ :=
  cellular_betti_profile_of_quasiIso_sphere2 d2 d1 hcomp φ e
exact platonic_solids_3d_of_homology d2 d1 V E F p q hp hq
  h_edges_faces h_edges_verts hV hE hF hcomp hs0 hs1 hs2

end

```

A.25 PlatonicSolids/Sphere2Homology.lean

Cellular $H_*(S^2; \mathbb{Q})$ of the tetrahedron surface (129 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.LinearAlgebra.Matrix.Notation
import Mathlib.LinearAlgebra.Matrix.Rank
import Mathlib.LinearAlgebra.Matrix.ToLin
import Mathlib.Tactic
import PlatonicSolids.EulerPoincare
import PlatonicSolids.Incidence

/-!
Paper §2.1. Cellular homology of the standard simplicial  $(S^2)$ :
the boundary of a tetrahedron (4 vertices, 6 edges, 4 triangles).

This is the usual CW / simplicial calculation
`H0 ≅ ℚ`, `H1 = 0`, `H2 ≅ ℚ`. Singular `H0` of `Metric.sphere`
and `H_*(S^0)` are in `PlatonicSolids/SingularHomology.lean`;
the  $S^3$ -sphere calculation is `PlatonicSolids/Sphere3Homology.lean`.
-/

open scoped Matrix
open Matrix Module

/-- Oriented incidence vertices ← edges: columns `01,02,03,12,13,23`. -/
def tetra_d1 : Matrix (Fin 4) (Fin 6) ℚ :=
  !![ -1, -1, -1, 0, 0, 0;
      1, 0, 0, -1, -1, 0;

```

```

0, 1, 0, 1, 0, -1;
0, 0, 1, 0, 1, 1]

/-- Oriented incidence edges ← faces: columns `012,013,023,123`. -/
def tetra_d2 : Matrix (Fin 6) (Fin 4) ℚ :=
  !![ 1, 1, 0, 0;
    -1, 0, 1, 0;
    0, -1, -1, 0;
    1, 0, 0, 1;
    0, 1, 0, -1;
    0, 0, 1, 1]

lemma tetra_comp : tetra_d1 * tetra_d2 = 0 := by
  ext i j
  fin_cases i <;> fin_cases j <;> simp [tetra_d1, tetra_d2]

noncomputable def tetra_del1 : (Fin 6 → ℚ) →1 [ℚ] (Fin 4 → ℚ) := tetra_d1.toLin'
noncomputable def tetra_del2 : (Fin 4 → ℚ) →1 [ℚ] (Fin 6 → ℚ) := tetra_d2.toLin'

lemma tetra_del_comp : tetra_del1.comp tetra_del2 = 0 := by
  rw [tetra_del1, tetra_del2, ← toLin'_mul, tetra_comp]
  exact map_zero _

lemma tetra_d1_rank_ge : 3 ≤ tetra_d1.rank := by
  have hI : ((1 : Matrix (Fin 3) (Fin 3) ℚ).rank) = 3 := rank_one
  have hsub :
    tetra_d1.submatrix ![1, 2, 3] ![0, 1, 2] = (1 : Matrix (Fin 3) (Fin 3) ℚ) := by
    ext i j
    fin_cases i <;> fin_cases j <;> simp [tetra_d1]
  have : (tetra_d1.submatrix ![1, 2, 3] ![0, 1, 2]).rank = 3 := by
    rw [hsub]; exact hI
  exact this.symm.trans_le (rank_submatrix_le _ _ _)

lemma tetra_d2_rank_ge : 3 ≤ tetra_d2.rank := by
  have hI : ((1 : Matrix (Fin 3) (Fin 3) ℚ).rank) = 3 := rank_one
  have hsub :
    tetra_d2.submatrix ![3, 4, 5] ![0, 1, 2] = (1 : Matrix (Fin 3) (Fin 3) ℚ) := by
    ext i j
    fin_cases i <;> fin_cases j <;> simp [tetra_d2]
  have : (tetra_d2.submatrix ![3, 4, 5] ![0, 1, 2]).rank = 3 := by
    rw [hsub]; exact hI
  exact this.symm.trans_le (rank_submatrix_le _ _ _)

lemma tetra_d1_rank : tetra_d1.rank = 3 := by
  have hsum := rank_add_rank_le_card_of_mul_eq_zero tetra_comp
  have h1 := tetra_d1_rank_ge
  have h2 := tetra_d2_rank_ge
  simp only [Fintype.card_fin] at hsum
  omega

lemma tetra_d2_rank : tetra_d2.rank = 3 := by
  have hsum := rank_add_rank_le_card_of_mul_eq_zero tetra_comp
  have h1 := tetra_d1_rank_ge
  have h2 := tetra_d2_rank_ge
  simp only [Fintype.card_fin] at hsum
  omega

lemma tetra_del1_range : finrank ℚ (LinearMap.range tetra_del1) = 3 := by
  simp [tetra_del1, toLin'_apply] using tetra_d1_rank

lemma tetra_del2_range : finrank ℚ (LinearMap.range tetra_del2) = 3 := by
  simp [tetra_del2, toLin'_apply] using tetra_d2_rank

```

```

/-- `H0(S2; ℚ) ≅ ℚ`. -/
theorem sphere2_betti0 : betti0 tetra_del1 = 1 := by
  simp [betti0, tetra_del1_range, finrank_fintype_fun_eq_card]

/-- `H1(S2; ℚ) = 0`. -/
theorem sphere2_betti1 : betti1 tetra_del2 tetra_del1 = 0 := by
  have hr := tetra_del1.finrank_range_add_finrank_ker
  simp [betti1, tetra_del2_range, tetra_del1_range, finrank_fintype_fun_eq_card] at hr ⊢
  omega

/-- `H2(S2; ℚ) ≅ ℚ`. -/
theorem sphere2_betti2 : betti2 tetra_del2 = 1 := by
  have hr := tetra_del2.finrank_range_add_finrank_ker
  simp [betti2, tetra_del2_range, finrank_fintype_fun_eq_card] at hr ⊢
  omega

/-- Cellular homology of the standard simplicial 2-sphere. -/
theorem homology_sphere2 :
  betti0 tetra_del1 = 1 ∧ betti1 tetra_del2 tetra_del1 = 0 ∧ betti2 tetra_del2 = 1 :=
  ⟨sphere2_betti0, sphere2_betti1, sphere2_betti2⟩

/-- Euler's formula on the tetrahedron, from its computed Betti numbers. -/
theorem euler_tetrahedron : (4 : ℕ) + 4 = 6 + 2 :=
  euler_formula_of_sphere2 tetra_del2 tetra_del1
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  tetra_del_comp sphere2_betti0 sphere2_betti1 sphere2_betti2

/-- The tetrahedron incidence structure is Platonic, using homology not a
raw Euler hypothesis. -/
theorem tetrahedron_platonic : IsPlatonicPair 3 3 :=
  platonic_solids_3d_of_homology tetra_del2 tetra_del1 4 6 4 3 3
  (by decide) (by decide) (by decide) (by decide)
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  tetra_del_comp sphere2_betti0 sphere2_betti1 sphere2_betti2

```

A.26 PlatonicSolids/Sphere3Homology.lean

Cellular $H_*(S^3; \mathbb{Q})$ of the 4-simplex boundary (195 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.LinearAlgebra.Matrix.Notation
import Mathlib.LinearAlgebra.Matrix.Rank
import Mathlib.LinearAlgebra.Matrix.ToLin
import Mathlib.Tactic
import PlatonicSolids.EulerPoincare

/-!
Paper §3.1. Cellular homology of the standard simplicial  $(S^3)$ :
the boundary of a 4-simplex (5 vertices, 10 edges, 10 triangles,
5 tetrahedra).

Ranks are  $\langle 4, 6, 4 \rangle$ , so  $b_0 = 1$ ,  $b_1 = 0$ ,  $b_2 = 0$ ,  $b_3 = 1$  and
 $\chi = 0$ . This is the combinatorial 3-sphere the paper uses; Mathlib
does not compute singular homology of Metric.sphere in positive
degree.
-/

```

```

open scoped Matrix
open Matrix Module

/-- Vertices `0..4` ← edges `01,02,03,04,12,13,14,23,24,34`. -/
def pentatope_d1 : Matrix (Fin 5) (Fin 10) ℚ :=
  !![ -1, -1, -1, -1, 0, 0, 0, 0, 0, 0;
      1, 0, 0, 0, -1, -1, -1, 0, 0, 0;
      0, 1, 0, 0, 1, 0, 0, -1, -1, 0;
      0, 0, 1, 0, 0, 1, 0, 1, 0, -1;
      0, 0, 0, 1, 0, 0, 1, 0, 1, 1]

/-- Edges ← triangles
`012,013,014,023,024,034,123,124,134,234`. -/
def pentatope_d2 : Matrix (Fin 10) (Fin 10) ℚ :=
  !![ 1, 1, 1, 0, 0, 0, 0, 0, 0, 0;
      -1, 0, 0, 1, 1, 0, 0, 0, 0, 0;
      0, -1, 0, -1, 0, 1, 0, 0, 0, 0;
      0, 0, -1, 0, -1, -1, 0, 0, 0, 0;
      1, 0, 0, 0, 0, 0, 1, 1, 0, 0;
      0, 1, 0, 0, 0, 0, -1, 0, 1, 0;
      0, 0, 1, 0, 0, 0, 0, -1, -1, 0;
      0, 0, 0, 1, 0, 0, 1, 0, 0, 1;
      0, 0, 0, 0, 1, 0, 0, 1, 0, -1;
      0, 0, 0, 0, 0, 1, 0, 0, 1, 1]

/-- Triangles ← tetrahedra `0123,0124,0134,0234,1234`. -/
def pentatope_d3 : Matrix (Fin 10) (Fin 5) ℚ :=
  !![ -1, -1, 0, 0, 0;
      1, 0, -1, 0, 0;
      0, 1, 1, 0, 0;
      -1, 0, 0, -1, 0;
      0, -1, 0, 1, 0;
      0, 0, -1, -1, 0;
      1, 0, 0, 0, -1;
      0, 1, 0, 0, 1;
      0, 0, 1, 0, -1;
      0, 0, 0, 1, 1]

lemma pentatope_comp21 : pentatope_d1 * pentatope_d2 = 0 := by
  native_decide

lemma pentatope_comp32 : pentatope_d2 * pentatope_d3 = 0 := by
  native_decide

noncomputable def pentatope_del1 : (Fin 10 → ℚ) →1[ℚ] (Fin 5 → ℚ) :=
  pentatope_d1.toLin'
noncomputable def pentatope_del2 : (Fin 10 → ℚ) →1[ℚ] (Fin 10 → ℚ) :=
  pentatope_d2.toLin'
noncomputable def pentatope_del3 : (Fin 5 → ℚ) →1[ℚ] (Fin 10 → ℚ) :=
  pentatope_d3.toLin'

lemma pentatope_del_comp21 : pentatope_del1.comp pentatope_del2 = 0 := by
  rw [pentatope_del1, pentatope_del2, ← toLin'_mul, pentatope_comp21]
  exact map_zero _

lemma pentatope_del_comp32 : pentatope_del2.comp pentatope_del3 = 0 := by
  rw [pentatope_del2, pentatope_del3, ← toLin'_mul, pentatope_comp32]
  exact map_zero _

lemma pentatope_d1_rank_ge : 4 ≤ pentatope_d1.rank := by
  have h1 : ((1 : Matrix (Fin 4) (Fin 4) ℚ).rank) = 4 := rank_one
  have hsub :

```



```

    pentatope_d1.submatrix ![1, 2, 3, 4] ![0, 1, 2, 3] =
      (1 : Matrix (Fin 4) (Fin 4)  $\mathbb{Q}$ ) := by
    ext i j
    fin_cases i <;> fin_cases j <;> simp [pentatope_d1]
  have : (pentatope_d1.submatrix ![1, 2, 3, 4] ![0, 1, 2, 3]).rank = 4 := by
    rw [hsub]; exact hI
  exact this.symm.trans_le (rank_submatrix_le _ _ _)

lemma pentatope_d3_rank_ge : 4 ≤ pentatope_d3.rank := by
  have hI : ((1 : Matrix (Fin 4) (Fin 4)  $\mathbb{Q}$ )).rank = 4 := rank_one
  have hsub :
    pentatope_d3.submatrix ![6, 7, 8, 9] ![0, 1, 2, 3] =
      (1 : Matrix (Fin 4) (Fin 4)  $\mathbb{Q}$ ) := by
    ext i j
    fin_cases i <;> fin_cases j <;> simp [pentatope_d3]
  have : (pentatope_d3.submatrix ![6, 7, 8, 9] ![0, 1, 2, 3]).rank = 4 := by
    rw [hsub]; exact hI
  exact this.symm.trans_le (rank_submatrix_le _ _ _)

/-- A `6 × 6` minor of `d2` with determinant `1`. -/
def pentatope_d2_minor : Matrix (Fin 6) (Fin 6)  $\mathbb{Q}$  :=
  !![ 1, 1, 1, 0, 0, 0;
    -1, 0, 0, 1, 1, 0;
     0, -1, 0, -1, 0, 1;
     1, 0, 0, 0, 0, 0;
     0, 1, 0, 0, 0, 0;
     0, 0, 0, 1, 0, 0]

lemma pentatope_d2_minor_eq :
  pentatope_d2.submatrix ![0, 1, 2, 4, 5, 7] ![0, 1, 2, 3, 4, 5] =
    pentatope_d2_minor := by
  ext i j
  fin_cases i <;> fin_cases j <;> simp [pentatope_d2, pentatope_d2_minor]

lemma pentatope_d2_minor_det : pentatope_d2_minor.det = 1 := by
  native_decide

lemma pentatope_d2_rank_ge : 6 ≤ pentatope_d2.rank := by
  have hU : IsUnit pentatope_d2_minor :=
    (Matrix.isUnit_iff_isUnit_det _).2 (by simp [pentatope_d2_minor_det])
  have : pentatope_d2_minor.rank = 6 := by
    simpa using rank_of_isUnit pentatope_d2_minor hU
  rw [← pentatope_d2_minor_eq] at this
  exact this.symm.trans_le (rank_submatrix_le _ _ _)

lemma pentatope_d1_rank : pentatope_d1.rank = 4 := by
  have hsum := rank_add_rank_le_card_of_mul_eq_zero pentatope_comp21
  have h1 := pentatope_d1_rank_ge
  have h2 := pentatope_d2_rank_ge
  simp only [Fintype.card_fin] at hsum
  omega

lemma pentatope_d2_rank : pentatope_d2.rank = 6 := by
  have hsum := rank_add_rank_le_card_of_mul_eq_zero pentatope_comp21
  have h1 := pentatope_d1_rank_ge
  have h2 := pentatope_d2_rank_ge
  simp only [Fintype.card_fin] at hsum
  omega

lemma pentatope_d3_rank : pentatope_d3.rank = 4 := by
  have hsum := rank_add_rank_le_card_of_mul_eq_zero pentatope_comp32
  have h2 := pentatope_d2_rank
  have h3 := pentatope_d3_rank_ge

```

```

simp only [Fintype.card_fin] at hsum
omega

lemma pentatope_del1_range : finrank  $\mathbb{Q}$  (LinearMap.range pentatope_del1) = 4 := by
  simpa [pentatope_del1, toLin'_apply'] using pentatope_d1_rank

lemma pentatope_del2_range : finrank  $\mathbb{Q}$  (LinearMap.range pentatope_del2) = 6 := by
  simpa [pentatope_del2, toLin'_apply'] using pentatope_d2_rank

lemma pentatope_del3_range : finrank  $\mathbb{Q}$  (LinearMap.range pentatope_del3) = 4 := by
  simpa [pentatope_del3, toLin'_apply'] using pentatope_d3_rank

theorem sphere3_betti0 : betti0 pentatope_del1 = 1 := by
  simp [betti0, pentatope_del1_range, finrank_fintype_fun_eq_card]

theorem sphere3_betti1 : betti1 pentatope_del2 pentatope_del1 = 0 := by
  have hr := pentatope_del1.finrank_range_add_finrank_ker
  simp [betti1, pentatope_del2_range, pentatope_del1_range,
    finrank_fintype_fun_eq_card] at hr ⊢
  omega

theorem sphere3_betti2 : betti2Rel pentatope_del3 pentatope_del2 = 0 := by
  have hr := pentatope_del2.finrank_range_add_finrank_ker
  simp [betti2Rel, pentatope_del3_range, pentatope_del2_range,
    finrank_fintype_fun_eq_card] at hr ⊢
  omega

theorem sphere3_betti3 : betti3 pentatope_del3 = 1 := by
  have hr := pentatope_del3.finrank_range_add_finrank_ker
  simp [betti3, pentatope_del3_range, finrank_fintype_fun_eq_card] at hr ⊢
  omega

/-- Cellular homology of the standard simplicial 3-sphere. -/
theorem homology_sphere3 :
  betti0 pentatope_del1 = 1 ∧
  betti1 pentatope_del2 pentatope_del1 = 0 ∧
  betti2Rel pentatope_del3 pentatope_del2 = 0 ∧
  betti3 pentatope_del3 = 1 :=
  ⟨sphere3_betti0, sphere3_betti1, sphere3_betti2, sphere3_betti3⟩

/--  $\chi(S^3) = 0$  from the computed Betti numbers of the 4-simplex boundary. -/
theorem eulerChar_sphere3 : ((5 :  $\mathbb{N}$ ) :  $\mathbb{Z}$ ) - 10 + 10 - 5 = 0 :=
  euler_formula_of_sphere3 pentatope_del3 pentatope_del2 pentatope_del1
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  pentatope_del_comp32 pentatope_del_comp21
  sphere3_betti0 sphere3_betti1 sphere3_betti2 sphere3_betti3

```

A.27 PlatonicSolids/EulerBetti.lean

Betti arithmetic of $\chi(S^2) = 2$ and $\chi(S^3) = 0$ (24 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Tactic

/-!
Paper §2.1 / §3.1. Betti arithmetic of  $\chi(S^2)$  and  $\chi(S^3)$ .

```

The algebraic Euler-Poincaré identities are ``euler_poincare`` and ``euler_poincare3`` in ``PlatonicSolids/EulerPoincare.lean``. The numbers ``b0 = 1``, ``b1 = 0``, ``b2 = 1`` for the standard simplicial ``S2`` are ``homology_sphere2``; the numbers ``b0 = 1``, ``b1 = 0``, ``b2 = 0``, ``b3 = 1`` for the standard simplicial ``S3`` are ``homology_sphere3``. Singular ``H0`` of ``Metric.sphere`` in dimension ``n ≥ 1`` and the full calculation of ``H*(S0)`` are in ``PlatonicSolids/SingularHomology.lean``. These lemmas record the resulting ``χ(S2) = 2`` and ``χ(S3) = 0``.
`-/`

```
/-- χ(S2) = b0 - b1 + b2 = 1 - 0 + 1. -/
lemma eulerChar_sphere2_betti : (1 : ℤ) - 0 + 1 = 2 := by norm_num

/-- χ(S3) = b0 - b1 + b2 - b3 = 1 - 0 + 0 - 1. -/
lemma eulerChar_sphere3_betti : (1 : ℤ) - 0 + 0 - 1 = 0 := by norm_num
```

A.28 PlatonicSolids/PlatonicPair.lean

The five solutions of the Platonic Diophantine inequality (38 lines).

```
/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Tactic

/-!
Paper §2.3. The five solutions of `(p-2)(q-2) < 4` for `p, q ≥ 3`.
-/

/-- Schläfli pairs of the five Platonic solids. -/
def IsPlatonicPair (p q : ℕ) : Prop :=
  (p = 3 ∧ q = 3) ∨
  (p = 3 ∧ q = 4) ∨
  (p = 4 ∧ q = 3) ∨
  (p = 3 ∧ q = 5) ∨
  (p = 5 ∧ q = 3)

lemma pq_ge_two_p_add_two_q {p q : ℕ} (hp : 4 ≤ p) (hq : 4 ≤ q) :
  2 * p + 2 * q ≤ p * q := by
  zify
  nlinarith

/-- The inequality `pq < 2p + 2q` for `p, q ≥ 3` has exactly the five Platonic solutions. -/
theorem platonic_pairs_of_inequality {p q : ℕ}
  (hp : 3 ≤ p) (hq : 3 ≤ q) (h : p * q < 2 * p + 2 * q) :
  IsPlatonicPair p q := by
  rcases show p = 3 ∨ q = 3 ∨ (4 ≤ p ∧ 4 ≤ q) by omega with rfl | rfl | ⟨hp4, hq4⟩
  · rcases show q = 3 ∨ q = 4 ∨ q = 5 by omega with rfl | rfl | rfl <=> simp [IsPlatonicPair]
  · rcases show p = 3 ∨ p = 4 ∨ p = 5 by omega with rfl | rfl | rfl <=> simp [IsPlatonicPair]
  · exact (lt_irrefl _ (h.trans_le (pq_ge_two_p_add_two_q hp4 hq4))).elim

theorem IsPlatonicPair.ge_three {p q : ℕ} (h : IsPlatonicPair p q) : 3 ≤ p ∧ 3 ≤ q := by
  rcases h with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ <=> decide

theorem IsPlatonicPair.inequality {p q : ℕ} (h : IsPlatonicPair p q) :
  p * q < 2 * p + 2 * q := by
  rcases h with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ <=> decide
```

A.29 PlatonicSolids/Reciprocal.lean

$1/p + 1/q > 1/2$ iff the integer inequality (44 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Data.Real.Basic
import PlatonicSolids.PlatonicPair

/-!
Paper §2.3 / §3.3.  $\frac{1}{p} + \frac{1}{q} > \frac{1}{2}$  is the reciprocal form of the
Platonic Diophantine inequality, and is the condition  $D_3 > 0$ .
-/

lemma reciprocal_iff {p q : ℕ} (hp : 0 < p) (hq : 0 < q) :
  (1 / (p : ℝ) + 1 / q > 1 / 2) ↔ p * q < 2 * p + 2 * q := by
  have : (0 : ℝ) < p := Nat.cast_pos.mpr hp
  have : (0 : ℝ) < q := Nat.cast_pos.mpr hq
  constructor
  · intro h
    have h' : (p + q : ℝ) / (p * q) > 1 / 2 := by
      convert h using 1
      field_simp
      ring
    rw [gt_iff_lt, lt_div_iff₀ (by positivity)] at h'
    exact_mod_cast (by linarith : (p * q : ℝ) < 2 * p + 2 * q)
  · intro h
    have hR : (p * q : ℝ) < 2 * p + 2 * q := mod_cast h
    have h' : (p + q : ℝ) / (p * q) > 1 / 2 := by
      rw [gt_iff_lt, lt_div_iff₀ (by positivity)]
      linarith
    convert h' using 1
    field_simp
    ring

theorem platonic_pair_iff {p q : ℕ} :
  IsPlatonicPair p q ↔ 3 ≤ p ∧ 3 ≤ q ∧ p * q < 2 * p + 2 * q := by
  constructor
  · intro h
    exact ⟨h.ge_three.1, h.ge_three.2, h.inequality⟩
  · exact fun ⟨hp, hq, h⟩ => platonic_pairs_of_inequality hp hq h

theorem platonic_pair_iff_reciprocal {p q : ℕ} (hp : 3 ≤ p) (hq : 3 ≤ q) :
  IsPlatonicPair p q ↔ 1 / (p : ℝ) + 1 / q > 1 / 2 := by
  rw [platonic_pair_iff, reciprocal_iff (by omega) (by omega)]
  simp [hp, hq]

```

A.30 PlatonicSolids/Incidence.lean

Regularity incidence plus Euler force a Platonic pair and $E > 0$ (84 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.EulerPoincare
import PlatonicSolids.PlatonicPair

/-!
Paper §2.2-2.4. Regularity incidence  $pF = 2E$ ,  $qV = 2E$  and Euler
 $V + F = E + 2$  force a Platonic pair, and force  $E > 0$  with no extra
non-degeneracy axiom.
-/

/-- The Euler + regularity identities algebraically prohibit  $E = 0$ . -/

```

```

theorem edges_pos_of_regular
  (V E F p q : ℕ)
  (hp : 3 ≤ p) (hq : 3 ≤ q)
  (h_edges_faces : p * F = 2 * E)
  (h_edges_verts : q * V = 2 * E)
  (h_euler : V + F = E + 2) :
  0 < E := by
have h_lhs : p * q * (V + F) = (2 * p + 2 * q) * E := calc
  p * q * (V + F) = p * (q * V) + q * (p * F) := by ring
  _ = p * (2 * E) + q * (2 * E) := by rw [h_edges_verts, h_edges_faces]
  _ = (2 * p + 2 * q) * E := by ring
have h_rhs : p * q * (V + F) = p * q * E + 2 * p * q := calc
  p * q * (V + F) = p * q * (E + 2) := by rw [h_euler]
  _ = p * q * E + 2 * p * q := by ring
have h_eq : (2 * p + 2 * q) * E = p * q * E + 2 * p * q := by rw [← h_lhs, h_rhs]
by_contra hz
have hE0 : E = 0 := Nat.eq_zero_of_not_pos hz
subst hE0
simp only [mul_zero, zero_add] at h_eq
exact (Nat.pos_iff_ne_zero.mp (by positivity : 0 < 2 * p * q)) h_eq.symm

/-- Any regular incidence structure with  $\chi = 2$  is a Platonic pair. -/
theorem platonic_solids_3d
  (V E F p q : ℕ)
  (hp : 3 ≤ p) (hq : 3 ≤ q)
  (h_edges_faces : p * F = 2 * E)
  (h_edges_verts : q * V = 2 * E)
  (h_euler : V + F = E + 2) :
  IsPlatonicPair p q := by
have h_lhs : p * q * (V + F) = (2 * p + 2 * q) * E := calc
  p * q * (V + F) = p * (q * V) + q * (p * F) := by ring
  _ = p * (2 * E) + q * (2 * E) := by rw [h_edges_verts, h_edges_faces]
  _ = (2 * p + 2 * q) * E := by ring
have h_eq : (2 * p + 2 * q) * E = p * q * E + 2 * p * q := by
  calc (2 * p + 2 * q) * E = p * q * (V + F) := h_lhs.symm
  _ = p * q * (E + 2) := by rw [h_euler]
  _ = p * q * E + 2 * p * q := by ring
have h_ineq : p * q < 2 * p + 2 * q := by
  obtain hlt | hge := lt_or_ge (p * q) (2 * p + 2 * q)
  · exact hlt
  · have : (2 * p + 2 * q) * E ≤ p * q * E := Nat.mul_le_mul_right E hge
    have : 0 < 2 * p * q := by positivity
    omega
exact platonic_pairs_of_inequality hp hq h_ineq

/-- The 3D classification from a cellular 2-complex with Betti numbers
`1, 0, 1`. Euler-Poincaré (`euler_formula_of_sphere2`) supplies
`V + F = E + 2`; regularity incidence then forces a Platonic pair.
The optional form `platonic_solids_3d_of_sphere` in
`SphereSingularHomology.lean` derives these numbers from a
quasi-isomorphism to singular chains of a model of `S^2`. -/
theorem platonic_solids_3d_of_homology
  {K : Type*} [Field K]
  {C₂ C₁ C₀ : Type*}
  [AddCommGroup C₂] [Module K C₂] [FiniteDimensional K C₂]
  [AddCommGroup C₁] [Module K C₁] [FiniteDimensional K C₁]
  [AddCommGroup C₀] [Module K C₀] [FiniteDimensional K C₀]
  (d₂ : C₂ →₁[K] C₁) (d₁ : C₁ →₁[K] C₀)
  (V E F p q : ℕ)
  (hp : 3 ≤ p) (hq : 3 ≤ q)
  (h_edges_faces : p * F = 2 * E)
  (h_edges_verts : q * V = 2 * E)
  (hV : Module.finrank K C₀ = V)

```

```

(hE : Module.finrank K C1 = E)
(hF : Module.finrank K C2 = F)
(hcomp : d1.comp d2 = 0)
(hb0 : betti0 d1 = 1) (hb1 : betti1 d2 d1 = 0) (hb2 : betti2 d2 = 1) :
IsPlatonicPair p q :=
platonic_solids_3d V E F p q hp hq h_edges_faces h_edges_verts
(euler_formula_of_sphere2 d2 d1 hV hE hF hcomp hb0 hb1 hb2)

```

A.31 PlatonicSolids/Angles.lean

$\pi/n \in (0, \pi/2)$ so sine and cosine are positive (28 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic

/-!
Paper §3.3. For  $n \geq 3$ , the Schläfli angle  $\pi/n$  lies in  $(0, \pi/2)$ ,
so  $\sin$  and  $\cos$  are positive.
-/

open Real

lemma angle_pos {n : ℕ} (hn : 1 ≤ n) : 0 < π / n := by
  have : (0 : ℝ) < n := Nat.cast_pos.mpr hn
  positivity

lemma angle_lt_pi_div_two {n : ℕ} (hn : 3 ≤ n) : π / n < π / 2 :=
  div_lt_div_of_pos_left pi_pos (by positivity : (0 : ℝ) < 2)
  (by exact_mod_cast (lt_of_lt_of_le (by decide : (2 : ℕ) < 3) hn))

lemma sin_angle_pos {n : ℕ} (hn : 3 ≤ n) : 0 < sin (π / n) :=
  sin_pos_of_pos_of_lt_pi (angle_pos (Nat.succ_le_of_lt (Nat.zero_lt_of_lt hn)))
  (angle_lt_pi_div_two hn |>.trans (by linarith [pi_pos]))

lemma cos_angle_pos {n : ℕ} (hn : 3 ≤ n) : 0 < cos (π / n) :=
  cos_pos_of_mem_Ioo (by linarith [angle_pos (Nat.succ_le_of_lt (Nat.zero_lt_of_lt hn)), pi_pos
  ],
  angle_lt_pi_div_two hn)

```

A.32 PlatonicSolids/GramMatrix.lean

Schläfli Gram matrix and the scalar Δ (30 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic
import Mathlib.LinearAlgebra.Matrix.Hermitian
import Mathlib.LinearAlgebra.Matrix.Notation

/-!
Paper §3.2. The Schläfli Gram matrix of  $\{p, q, r\}$  and the scalar
 $\Delta(p, q, r)$  that will equal its determinant.
-/

open scoped Matrix
open Real Matrix

```

```

/-- Canonical bilinear form of the rank-4 Schläfli Coxeter system `{p, q, r}`. -/
noncomputable def schlafligram (p q r : ℕ) : Matrix (Fin 4) (Fin 4) ℝ :=
  !![1, -cos (π / p), 0, 0;
    -cos (π / p), 1, -cos (π / q), 0;
    0, -cos (π / q), 1, -cos (π / r);
    0, 0, -cos (π / r), 1]

/--  $\Delta(p, q, r) = \sin^2(\pi/p) \sin^2(\pi/r) - \cos^2(\pi/q)$ . -/
noncomputable def schlaflidet (p q r : ℕ) : ℝ :=
  sin (π / p) ^ 2 * sin (π / r) ^ 2 - cos (π / q) ^ 2

lemma schlafligram_isHermitian (p q r : ℕ) : (schlafligram p q r).IsHermitian := by
  ext i j
  fin_cases i <;> fin_cases j <;> simp [schlafligram, conjTranspose]

```

A.33 PlatonicSolids/LeadingMinors.lean

Closed forms of the leading minors D_1, D_2, D_3 (34 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.LinearAlgebra.Matrix.Determinant.Basic
import PlatonicSolids.GramMatrix

/-!
Paper §3.3-3.4. Leading principal minors `D₁`, `D₂`, `D₃` of the Gram
matrix, matching the paper's closed forms.
-/

open scoped Matrix
open Real Matrix

def leadingSubmatrix (k : ℕ) (hk : k ≤ 4) (A : Matrix (Fin 4) (Fin 4) ℝ) :
  Matrix (Fin k) (Fin k) ℝ :=
  A.submatrix (Fin.castLE hk) (Fin.castLE hk)

noncomputable def leadingMinor (k : ℕ) (hk : k ≤ 4) (p q r : ℕ) : ℝ :=
  (leadingSubmatrix k hk (schlafligram p q r)).det

lemma leadingMinor_one (p q r : ℕ) : leadingMinor 1 (by decide) p q r = 1 := by
  simp [leadingMinor, leadingSubmatrix, schlafligram]

lemma leadingMinor_two (p q r : ℕ) :
  leadingMinor 2 (by decide) p q r = sin (π / p) ^ 2 := by
  simp [leadingMinor, leadingSubmatrix, schlafligram, det_fin_two]
  linarith [sin_sq_add_cos_sq (π / p)]

lemma leadingMinor_three (p q r : ℕ) :
  leadingMinor 3 (by decide) p q r = sin (π / p) ^ 2 - cos (π / q) ^ 2 := by
  simp [leadingMinor, leadingSubmatrix, schlafligram, det_fin_three]
  linarith [sin_sq_add_cos_sq (π / p)]

```

A.34 PlatonicSolids/TrailingMinor.lean

Trailing 3×3 minor is the vertex-figure cell minor (22 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/

```

```

import Mathlib.LinearAlgebra.Matrix.Determinant.Basic
import PlatonicSolids.GramMatrix

/-!
Paper §3.3. The trailing 3×3 principal minor is the cell minor of the
vertex figure  $\{q, r\}$ .
-/

open scoped Matrix
open Real Matrix

noncomputable def trailing3 (p q r : ℕ) : Matrix (Fin 3) (Fin 3) ℝ :=
  (schlafliGram p q r).submatrix (fun i : Fin 3 => i.succ) (fun i => i.succ)

lemma trailing3_det (p q r : ℕ) :
  (trailing3 p q r).det = sin (π / q) ^ 2 - cos (π / r) ^ 2 := by
  simp [trailing3, schlafliGram, det_fin_three]
  linarith [sin_sq_add_cos_sq (π / q)]

```

A.35 PlatonicSolids/D3IffPlatonic.lean

$D_3 > 0$ iff the cell is Platonic (58 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/

import PlatonicSolids.Angles
import PlatonicSolids.LeadingMinors
import PlatonicSolids.Reciprocal
import PlatonicSolids.TrailingMinor

/-!
Paper §3.3.  $D_3 > 0$  iff the cell is Platonic; the trailing 3×3 minor
is the same statement for the vertex figure.
-/

open Real

lemma angle_mem_Icc {n : ℕ} (hn : 3 ≤ n) :
  π / n ∈ Set.Icc (-(π / 2)) (π / 2) :=
  ⟨by linarith [angle_pos (Nat.succ_le_of_lt (Nat.zero_lt_of_lt hn)), pi_pos],
    (angle_lt_pi_div_two hn).le⟩

lemma complement_mem_Icc {n : ℕ} (hn : 3 ≤ n) :
  π / 2 - π / n ∈ Set.Icc (-(π / 2)) (π / 2) :=
  ⟨by linarith [angle_lt_pi_div_two hn, pi_pos],
    by linarith [angle_pos (Nat.succ_le_of_lt (Nat.zero_lt_of_lt hn))]⟩

/--  $\sin(\pi/p) > \cos(\pi/q)$  iff  $1/p + 1/q > 1/2$ . -/
lemma sin_gt_cos_iff_reciprocal {p q : ℕ} (hp : 3 ≤ p) (hq : 3 ≤ q) :
  sin (π / p) > cos (π / q) ↔ 1 / (p : ℝ) + 1 / q > 1 / 2 := by
  have hp0 : (0 : ℝ) < p := Nat.cast_pos.mpr (Nat.zero_lt_of_lt hp)
  have hq0 : (0 : ℝ) < q := Nat.cast_pos.mpr (Nat.zero_lt_of_lt hq)
  rw [← sin_pi_div_two_sub (π / q), gt_iff_lt, gt_iff_lt]
  rw [strictMonoOn_sin.lt_iff_lt (complement_mem_Icc hq) (angle_mem_Icc hp)]
  constructor
  · intro h
    have hsum : π / p + π / q > π / 2 := by linarith
    have hmul : π * (1 / (p : ℝ) + 1 / q) > π * (1 / 2) := by
      convert hsum using 1 <=> field_simp
    exact lt_of_mul_lt_mul_left hmul pi_pos.le
  · intro h

```



```

have hmul :  $\pi * (1 / (p : \mathbb{R}) + 1 / q) > \pi * (1 / 2) :=$ 
  mul_lt_mul_of_pos_left h pi_pos
have hsum :  $\pi / p + \pi / q > \pi / 2 :=$  by
  convert hmul using 1 <=> field_simp
  linarith

/-- Leading 3×3 minor  $D_3 > 0$  iff  $\{p, q\}$  is Platonic. -/
theorem leadingMinor_three_pos_iff {p q r :  $\mathbb{N}$ } (hp :  $3 \leq p$ ) (hq :  $3 \leq q$ ) :
   $0 < \text{leadingMinor } 3$  (by decide) p q r  $\leftrightarrow$  IsPlatonicPair p q := by
  rw [leadingMinor_three, platonic_pair_iff_reciprocal hp hq, sub_pos,
    sq_lt_sq0 (cos_angle_pos hq).le (sin_angle_pos hp).le]
  exact sin_gt_cos_iff_reciprocal hp hq

/-- Trailing 3×3 minor  $> 0$  iff the vertex figure  $\{q, r\}$  is Platonic. -/
theorem trailing3_pos_iff {p q r :  $\mathbb{N}$ } (hq :  $3 \leq q$ ) (hr :  $3 \leq r$ ) :
   $0 < (\text{trailing3 } p q r).det \leftrightarrow$  IsPlatonicPair q r := by
  rw [trailing3_det,  $\leftarrow$  leadingMinor_three (p := q) (q := r) (r := p)]
  exact leadingMinor_three_pos_iff hq hr

```

A.36 PlatonicSolids/QuadForm.lean

Quadratic form of the Gram matrix (26 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.GramMatrix

/-!
Paper §3.3. The quadratic form of the Schläfli Gram matrix.
-/

open scoped Matrix
open Real Matrix

private lemma sum_fin4 (f : Fin 4  $\rightarrow \mathbb{R}$ ) :  $\sum i, f i = f 0 + f 1 + f 2 + f 3 :=$  by
  rw [Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_succ]
  simp
  abel

lemma schlaefliGram_quad (p q r :  $\mathbb{N}$ ) (x : Fin 4  $\rightarrow \mathbb{R}$ ) :
  star x .v (schlaefliGram p q r *v x) =
    x 0 2 + x 1 2 + x 2 2 + x 3 2
    - 2 * cos ( $\pi / p$ ) * x 0 * x 1
    - 2 * cos ( $\pi / q$ ) * x 1 * x 2
    - 2 * cos ( $\pi / r$ ) * x 2 * x 3 := by
  simp [schlaefliGram, dotProduct, mulVec, sum_fin4]
  ring

```

A.37 PlatonicSolids/CompletingSquare.lean

LDL / completing-the-square identity (55 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Tactic

/-!
Paper §3.3. Completing the square (LDL) for the rank-4 tridiagonal

```

```

quadratic form that the Schläfli Gram matrix induces.
-/

/-- Weighted sum of squares after the triangular change of variables. -/
lemma band_quad_squares (α β γ : ℝ) (x0 x1 x2 x3 : ℝ)
  (h2 : 1 - α ^ 2 ≠ 0) (h3 : 1 - α ^ 2 - β ^ 2 ≠ 0) :
  x0 ^ 2 + x1 ^ 2 + x2 ^ 2 + x3 ^ 2 + 2 * α * x0 * x1 + 2 * β * x1 * x2 +
    2 * γ * x2 * x3 =
    (x0 + α * x1) ^ 2
    + (1 - α ^ 2) * (x1 + β / (1 - α ^ 2) * x2) ^ 2
    + ((1 - α ^ 2 - β ^ 2) / (1 - α ^ 2)) *
      (x2 + γ * (1 - α ^ 2) / (1 - α ^ 2 - β ^ 2) * x3) ^ 2
    + ((1 - α ^ 2 - β ^ 2 - γ ^ 2 * (1 - α ^ 2)) / (1 - α ^ 2 - β ^ 2)) *
      x3 ^ 2 := by
  field_simp [h2, h3]
  ring

lemma weighted_squares_pos {a b c d c1 c2 c3 : ℝ}
  (hc1 : 0 < c1) (hc2 : 0 < c2) (hc3 : 0 < c3)
  (h : a ≠ 0 ∨ b ≠ 0 ∨ c ≠ 0 ∨ d ≠ 0) :
  0 < a ^ 2 + c1 * b ^ 2 + c2 * c ^ 2 + c3 * d ^ 2 := by
  rcases h with ha | hb | hc | hd
  · nlinarith [sq_pos_of_ne_zero ha, sq_nonneg b, sq_nonneg c, sq_nonneg d]
  · nlinarith [sq_pos_of_ne_zero hb, sq_nonneg a, sq_nonneg c, sq_nonneg d]
  · nlinarith [sq_pos_of_ne_zero hc, sq_nonneg a, sq_nonneg b, sq_nonneg d]
  · nlinarith [sq_pos_of_ne_zero hd, sq_nonneg a, sq_nonneg b, sq_nonneg c]

/-- The triangular change of variables is invertible. -/
lemma ldl_coords_eq_zero {α β γ x0 x1 x2 x3 : ℝ}
  (_h2 : 1 - α ^ 2 ≠ 0) (_h3 : 1 - α ^ 2 - β ^ 2 ≠ 0)
  (h01 : x0 + α * x1 = 0)
  (h12 : x1 + β / (1 - α ^ 2) * x2 = 0)
  (h23 : x2 + γ * (1 - α ^ 2) / (1 - α ^ 2 - β ^ 2) * x3 = 0)
  (h3z : x3 = 0) :
  x0 = 0 ∧ x1 = 0 ∧ x2 = 0 ∧ x3 = 0 := by
  have hx3 : x3 = 0 := h3z
  have hx2 : x2 = 0 := by
    rw [hx3] at h23
    simp using h23
  have hx1 : x1 = 0 := by
    rw [hx2] at h12
    simp at h12
    exact h12
  have hx0 : x0 = 0 := by
    rw [hx1] at h01
    simp using h01
  exact ⟨hx0, hx1, hx2, hx3⟩

```

A.38 PlatonicSolids/DetFormula.lean

$\det M = \Delta$ (43 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.LinearAlgebra.Matrix.Determinant.Basic
import PlatonicSolids.GramMatrix

/-!
Paper §3.4. The determinant of the rank-4 Schläfli Gram matrix is
 $\Delta = \sin^2(\pi/p) \sin^2(\pi/r) - \cos^2(\pi/q)$ .
-/

```

```

open scoped Matrix
open Real Matrix

private lemma sum_fin4 (f : Fin 4 → ℝ) :  $\sum i, f i = f 0 + f 1 + f 2 + f 3$  := by
  rw [Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_succ]
  simp
  abel

/-- Determinant of the tridiagonal Gram pattern. -/
lemma det_band (a b c : ℝ) :
  (!! [1, a, 0, 0;
    a, 1, b, 0;
    0, b, 1, c;
    0, 0, c, 1] : Matrix (Fin 4) (Fin 4) ℝ).det =
  (1 - a ^ 2) * (1 - c ^ 2) - b ^ 2 := by
  rw [det_succ_row_zero, sum_fin4]
  simp [det_fin_three, Fin.succAbove]
  ring

lemma schlafliGram_det (p q r : ℕ) :
  (schlafliGram p q r).det = schlafliDet p q r := by
  unfold schlafliGram schlafliDet
  rw [det_band]
  have hp : 1 - (-cos (π / p)) ^ 2 = sin (π / p) ^ 2 := by linarith [sin_sq_add_cos_sq (π / p)]
  have hr : 1 - (-cos (π / r)) ^ 2 = sin (π / r) ^ 2 := by linarith [sin_sq_add_cos_sq (π / r)]
  rw [hp, hr]
  ring

lemma leadingMinor_four (p q r : ℕ) :
  (schlafliGram p q r).det = schlafliDet p q r :=
  schlafliGram_det p q r

```

A.39 PlatonicSolids/Sylvester.lean

Sylvester: positive definite iff leading minors are positive (100 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Analysis.Matrix.PosDef
import PlatonicSolids.CompletingSquare
import PlatonicSolids.DetFormula
import PlatonicSolids.LleadingMinors
import PlatonicSolids.QuadForm

/-!
Paper §3.3. Sylvester's criterion for the Schläfli Gram matrix:
positive-definite iff the four leading principal minors are positive.
`PosDef →` minors uses Mathlib (`PosDef.submatrix`, `PosDef.det_pos`);
the converse is the completing-the-square identity.
-/

open scoped Matrix
open Real Matrix

lemma leadingMinors_pos_of_posDef {p q r : ℕ} (h : (schlafliGram p q r).PosDef) :
  0 < leadingMinor 1 (by decide) p q r ∧
  0 < leadingMinor 2 (by decide) p q r ∧
  0 < leadingMinor 3 (by decide) p q r ∧
  0 < (schlafliGram p q r).det := by
  have h1 := h.submatrix (Fin.castLE_injective (by decide : 1 ≤ 4))

```

```

have h2 := h.submatrix (Fin.castLE_injective (by decide : 2 ≤ 4))
have h3 := h.submatrix (Fin.castLE_injective (by decide : 3 ≤ 4))
refine {?, ?, ?, PosDef.det_pos h}
· simp [leadingMinor, leadingSubmatrix] using PosDef.det_pos h1
· simp [leadingMinor, leadingSubmatrix] using PosDef.det_pos h2
· simp [leadingMinor, leadingSubmatrix] using PosDef.det_pos h3

/-- Converse Sylvester for this Gram family: positive leading minors  $\Rightarrow$  `PosDef`. -/
theorem schlafliGram_posDef_of_minors {p q r : ℕ}
  (h1 : 0 < leadingMinor 1 (by decide) p q r)
  (h2 : 0 < leadingMinor 2 (by decide) p q r)
  (h3 : 0 < leadingMinor 3 (by decide) p q r)
  (h4 : 0 < (schlafliGram p q r).det) :
  (schlafliGram p q r).PosDef := by
  have hD1 : leadingMinor 1 (by decide) p q r = 1 := leadingMinor_one p q r
  have : (0 : ℝ) < 1 := hD1 ▸ h1
  clear this
  refine posDef_iff_dotProduct_mulVec.mpr ⟨schlafliGram_isHermitian p q r, ?_⟩
  intro x hx
  set α := -cos (π / p)
  set β := -cos (π / q)
  set γ := -cos (π / r)
  have hα : 1 - α ^ 2 = sin (π / p) ^ 2 := by
    simp [α]; linarith [sin_sq_add_cos_sq (π / p)]
  have hβ : 1 - α ^ 2 - β ^ 2 = sin (π / p) ^ 2 - cos (π / q) ^ 2 := by
    simp [α, β]; linarith [sin_sq_add_cos_sq (π / p)]
  have hD2 : leadingMinor 2 (by decide) p q r = sin (π / p) ^ 2 := leadingMinor_two p q r
  have hD3 : leadingMinor 3 (by decide) p q r =
    sin (π / p) ^ 2 - cos (π / q) ^ 2 := leadingMinor_three p q r
  have h2ne : 1 - α ^ 2 ≠ 0 := by
    rw [hα]; exact ne_of_gt (hD2 ▸ h2)
  have h3ne : 1 - α ^ 2 - β ^ 2 ≠ 0 := by
    rw [hβ]; exact ne_of_gt (hD3 ▸ h3)
  have hc1 : 0 < 1 - α ^ 2 := by rw [hα]; exact hD2 ▸ h2
  have hc2 : 0 < 1 - α ^ 2 - β ^ 2 := by rw [hβ]; exact hD3 ▸ h3
  have hγ : 1 - γ ^ 2 = sin (π / r) ^ 2 := by
    simp [γ]; linarith [sin_sq_add_cos_sq (π / r)]
  have hc3 : 0 < 1 - α ^ 2 - β ^ 2 - γ ^ 2 * (1 - α ^ 2) := by
    have : 1 - α ^ 2 - β ^ 2 - γ ^ 2 * (1 - α ^ 2) =
      (1 - α ^ 2) * (1 - γ ^ 2) - β ^ 2 := by ring
    rw [this, hα, hγ]
  have hβsq : β ^ 2 = cos (π / q) ^ 2 := by simp [β]
  rw [hβsq]
  have hdet : sin (π / p) ^ 2 * sin (π / r) ^ 2 - cos (π / q) ^ 2 =
    (schlafliGram p q r).det := by
    rw [schlafliGram_det, schlafliDet]
  rw [hdet]
  exact h4
  have hQ : star x .v (schlafliGram p q r *v x) =
    (x 0 + α * x 1) ^ 2
    + (1 - α ^ 2) * (x 1 + β / (1 - α ^ 2) * x 2) ^ 2
    + ((1 - α ^ 2 - β ^ 2) / (1 - α ^ 2)) *
      (x 2 + γ * (1 - α ^ 2) / (1 - α ^ 2 - β ^ 2) * x 3) ^ 2
    + ((1 - α ^ 2 - β ^ 2 - γ ^ 2 * (1 - α ^ 2)) / (1 - α ^ 2 - β ^ 2)) *
      x 3 ^ 2 := by
    calc star x .v (schlafliGram p q r *v x)
      = x 0 ^ 2 + x 1 ^ 2 + x 2 ^ 2 + x 3 ^ 2
        - 2 * cos (π / p) * x 0 * x 1
        - 2 * cos (π / q) * x 1 * x 2
        - 2 * cos (π / r) * x 2 * x 3 := schlafliGram_quad p q r x
      _ = x 0 ^ 2 + x 1 ^ 2 + x 2 ^ 2 + x 3 ^ 2
        + 2 * α * x 0 * x 1 + 2 * β * x 1 * x 2 + 2 * γ * x 2 * x 3 := by
    simp [α, β, γ]; ring

```

```

    _ = _ := band_quad_squares  $\alpha$   $\beta$   $\gamma$  (x 0) (x 1) (x 2) (x 3) h2ne h3ne
  rw [hQ]
  refine weighted_squares_pos hc1 (div_pos hc2 hc1) (div_pos hc3 hc2) ?_
  by_contra hforms
  push Not at hforms
  obtain ⟨h01, h12, h23, h3z⟩ := hforms
  obtain ⟨hx0, hx1, hx2, hx3⟩ := ldl_coords_eq_zero h2ne h3ne h01 h12 h23 h3z
  have : x = 0 := by
    ext i
    fin_cases i <|> simp [hx0, hx1, hx2, hx3]
  exact hx this

```

A.40 PlatonicSolids/TrigValues.lean

Exact

$\sin^2(\pi/n)$ and
 $\cos^2(\pi/n)$ for n
 in
 3, 4, 5 (63 lines).

/-

Copyright (c) 2026 Lars Warren Ericson.

Released under Apache 2.0 license as described in the file LICENSE.

-/

import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic

/-!

Paper §4. Exact $\sin^2(\pi/n)$ and $\cos^2(\pi/n)$ for $n \in \{3, 4, 5\}$, from
 Mathlib's $\sin_pi_div_*$ / $\cos_pi_div_*$.

-/

open Real

```

lemma sq_cos_pi_div_three :  $\cos(\pi / 3)^2 = 1 / 4$  := by
  rw [cos_pi_div_three]; norm_num

```

```

lemma sq_sin_pi_div_four :  $\sin(\pi / 4)^2 = 1 / 2$  := by
  rw [sin_pi_div_four, div_pow, sq_sqrt (by norm_num : (0 : ℝ) ≤ 2)]; norm_num

```

```

lemma sq_cos_pi_div_four :  $\cos(\pi / 4)^2 = 1 / 2$  := by
  rw [cos_pi_div_four, div_pow, sq_sqrt (by norm_num : (0 : ℝ) ≤ 2)]; norm_num

```

```

lemma sq_cos_pi_div_five :  $\cos(\pi / 5)^2 = (3 + \sqrt{5}) / 8$  := by
  rw [cos_pi_div_five]
  have :  $(1 + \sqrt{5})^2 = 6 + 2 * \sqrt{5}$  := by
    ring_nf
    rw [sq_sqrt (by norm_num : (0 : ℝ) ≤ 5)]
    ring
  field_simp
  rw [this]
  ring

```

```

lemma sq_sin_pi_div_five :  $\sin(\pi / 5)^2 = (5 - \sqrt{5}) / 8$  := by
  have h :  $\sin(\pi / 5)^2 = 1 - \cos(\pi / 5)^2$  := by
    linarith [sin_sq_add_cos_sq (pi / 5)]
  rw [h, sq_cos_pi_div_five]
  field_simp
  ring

```

```

lemma sqrt5_bounds : 2 <  $\sqrt{5}$   $\wedge$   $\sqrt{5}$  < 3 := by
  constructor
  · rw [lt_sqrt (by norm_num)]; norm_num
  · rw [sqrt_lt' (by norm_num)]; norm_num

lemma three_mul_sqrt5_lt_seven : 3 *  $\sqrt{5}$  < 7 := by
  have h : (3 *  $\sqrt{5}$ ) ^ 2 < (7 :  $\mathbb{R}$ ) ^ 2 := by
    calc (3 *  $\sqrt{5}$ ) ^ 2 = 9 * ( $\sqrt{5}$  ^ 2) := by ring
    _ = 9 * 5 := by rw [sq_sqrt (by norm_num)]
    _ = 45 := by norm_num
    _ < 49 := by norm_num
    _ = (7 :  $\mathbb{R}$ ) ^ 2 := by norm_num
  have : 0 ≤ 3 *  $\sqrt{5}$  := by positivity
  nlinarith

lemma two_mul_sqrt5_gt_three : 3 < 2 *  $\sqrt{5}$  := by
  have h : (3 :  $\mathbb{R}$ ) ^ 2 < (2 *  $\sqrt{5}$ ) ^ 2 := by
    calc (3 :  $\mathbb{R}$ ) ^ 2 = 9 := by norm_num
    _ < 20 := by norm_num
    _ = 4 * 5 := by norm_num
    _ = 4 * ( $\sqrt{5}$  ^ 2) := by rw [sq_sqrt (by norm_num)]
    _ = (2 *  $\sqrt{5}$ ) ^ 2 := by ring
  have : 0 ≤ 2 *  $\sqrt{5}$  := by positivity
  nlinarith

```

A.41 PlatonicSolids/CompatibleTriples.lean

The eleven Platonic-compatible Schläfli triples (64 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.PlatonicPair

/-!
Paper §4. The eleven Schläfli triples whose cell and vertex figure
are both Platonic.
-/

/-- Compatible middle index: 11 of the 25 Platonic-pair products survive. -/
theorem platonic_compatible_eleven {p q r :  $\mathbb{N}$ }
  (hc : IsPlatonicPair p q) (hv : IsPlatonicPair q r) :
  p = 3  $\wedge$  q = 3  $\wedge$  r = 3  $\vee$ 
  p = 3  $\wedge$  q = 3  $\wedge$  r = 4  $\vee$ 
  p = 3  $\wedge$  q = 3  $\wedge$  r = 5  $\vee$ 
  p = 3  $\wedge$  q = 4  $\wedge$  r = 3  $\vee$ 
  p = 4  $\wedge$  q = 3  $\wedge$  r = 3  $\vee$ 
  p = 4  $\wedge$  q = 3  $\wedge$  r = 4  $\vee$ 
  p = 4  $\wedge$  q = 3  $\wedge$  r = 5  $\vee$ 
  p = 3  $\wedge$  q = 5  $\wedge$  r = 3  $\vee$ 
  p = 5  $\wedge$  q = 3  $\wedge$  r = 3  $\vee$ 
  p = 5  $\wedge$  q = 3  $\wedge$  r = 4  $\vee$ 
  p = 5  $\wedge$  q = 3  $\wedge$  r = 5 := by
  rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
  · -- cell {3,3}
    rcases hv with ⟨_, rfl⟩ | ⟨_, rfl⟩ | ⟨h, _⟩ | ⟨_, rfl⟩ | ⟨h, _⟩
    · exact .inl ⟨rfl, rfl, rfl⟩
    · exact .inr <| .inl ⟨rfl, rfl, rfl⟩
    · cases h
    · exact .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
    · cases h

```

```

· -- cell {3,4}
  rcases hv with ⟨h, _⟩ | ⟨h, _⟩ | ⟨_, rfl⟩ | ⟨h, _⟩ | ⟨h, _⟩
  · cases h
  · cases h
  · exact .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · cases h
  · cases h
· -- cell {4,3}
  rcases hv with ⟨_, rfl⟩ | ⟨_, rfl⟩ | ⟨h, _⟩ | ⟨_, rfl⟩ | ⟨h, _⟩
  · exact .inr <| .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · cases h
  · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · cases h
· -- cell {3,5}
  rcases hv with ⟨h, _⟩ | ⟨h, _⟩ | ⟨h, _⟩ | ⟨h, _⟩ | ⟨_, rfl⟩
  · cases h
  · cases h
  · cases h
  · cases h
  · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
· -- cell {5,3}
  rcases hv with ⟨_, rfl⟩ | ⟨_, rfl⟩ | ⟨h, _⟩ | ⟨_, rfl⟩ | ⟨h, _⟩
  · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inr <|
    .inl ⟨rfl, rfl, rfl⟩
  · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inr <|
    .inr <| .inl ⟨rfl, rfl, rfl⟩
  · cases h
  · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inr <|
    .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · cases h

```

A.42 PlatonicSolids/SignTable.lean

Sign of Δ on those eleven triples (121 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.GramMatrix
import PlatonicSolids.TrigValues

/-!
Paper §4. Sign of  $\Delta$  on the eleven Platonic-compatible triples.
-/

open Real

private lemma unfold_det (p q r : ℕ) :
  schlafliDet p q r = sin (π / p) ^ 2 * sin (π / r) ^ 2 - cos (π / q) ^ 2 :=
  rfl

private lemma cast_angles :
  sin (π / (3 : ℕ)) ^ 2 = sin (π / 3) ^ 2 ∧
  sin (π / (4 : ℕ)) ^ 2 = sin (π / 4) ^ 2 ∧
  sin (π / (5 : ℕ)) ^ 2 = sin (π / 5) ^ 2 ∧
  cos (π / (3 : ℕ)) ^ 2 = cos (π / 3) ^ 2 ∧
  cos (π / (4 : ℕ)) ^ 2 = cos (π / 4) ^ 2 ∧
  cos (π / (5 : ℕ)) ^ 2 = cos (π / 5) ^ 2 := by
  simp [Nat.cast_ofNat]

lemma schlafliDet_333 : schlafliDet 3 3 3 = 5 / 16 := by

```

```

rw [unfold_det, cast_angles.1, cast_angles.2.2.2.1, sq_sin_pi_div_three, sq_cos_pi_div_three]
norm_num

lemma schlafliDet_334 : schlafliDet 3 3 4 = 1 / 8 := by
  rw [unfold_det, cast_angles.1, cast_angles.2.1, cast_angles.2.2.2.1,
    sq_sin_pi_div_three, sq_sin_pi_div_four, sq_cos_pi_div_three]
  norm_num

lemma schlafliDet_433 : schlafliDet 4 3 3 = 1 / 8 := by
  rw [unfold_det, cast_angles.2.1, cast_angles.1, cast_angles.2.2.2.1,
    sq_sin_pi_div_four, sq_sin_pi_div_three, sq_cos_pi_div_three]
  norm_num

lemma schlafliDet_343 : schlafliDet 3 4 3 = 1 / 16 := by
  rw [unfold_det, cast_angles.1, cast_angles.2.2.2.2.1, sq_sin_pi_div_three, sq_cos_pi_div_four
  ]
  norm_num

lemma schlafliDet_335 : schlafliDet 3 3 5 = (7 - 3 *  $\sqrt{5}$ ) / 32 := by
  rw [unfold_det, cast_angles.1, cast_angles.2.2.1, cast_angles.2.2.2.1,
    sq_sin_pi_div_three, sq_sin_pi_div_five, sq_cos_pi_div_three]
  ring

lemma schlafliDet_533 : schlafliDet 5 3 3 = (7 - 3 *  $\sqrt{5}$ ) / 32 := by
  rw [unfold_det, cast_angles.2.2.1, cast_angles.1, cast_angles.2.2.2.1,
    sq_sin_pi_div_five, sq_sin_pi_div_three, sq_cos_pi_div_three]
  ring

lemma schlafliDet_434 : schlafliDet 4 3 4 = 0 := by
  rw [unfold_det, cast_angles.2.1, cast_angles.2.2.2.1, sq_sin_pi_div_four, sq_cos_pi_div_three
  ]
  norm_num

lemma schlafliDet_353 : schlafliDet 3 5 3 = (3 - 2 *  $\sqrt{5}$ ) / 16 := by
  rw [unfold_det, cast_angles.1, cast_angles.2.2.2.2.2, sq_sin_pi_div_three, sq_cos_pi_div_five
  ]
  ring

lemma schlafliDet_435 : schlafliDet 4 3 5 = (1 -  $\sqrt{5}$ ) / 16 := by
  rw [unfold_det, cast_angles.2.1, cast_angles.2.2.1, cast_angles.2.2.2.1,
    sq_sin_pi_div_four, sq_sin_pi_div_five, sq_cos_pi_div_three]
  ring

lemma schlafliDet_534 : schlafliDet 5 3 4 = (1 -  $\sqrt{5}$ ) / 16 := by
  rw [unfold_det, cast_angles.2.2.1, cast_angles.2.1, cast_angles.2.2.2.1,
    sq_sin_pi_div_five, sq_sin_pi_div_four, sq_cos_pi_div_three]
  ring

lemma schlafliDet_535 : schlafliDet 5 3 5 = (14 - 10 *  $\sqrt{5}$ ) / 64 := by
  rw [unfold_det, cast_angles.2.2.1, cast_angles.2.2.2.1, sq_sin_pi_div_five,
    sq_cos_pi_div_three]
  have : ((5 -  $\sqrt{5}$ ) / 8) ^ 2 = (30 - 10 *  $\sqrt{5}$ ) / 64 := by
    field_simp
    ring_nf
    rw [sq_sqrt (by norm_num : (0 :  $\mathbb{R}$ ) ≤ 5)]
    ring
  rw [← pow_two, this]
  ring

lemma schlafliDet_333_pos : 0 < schlafliDet 3 3 3 := by rw [schlafliDet_333]; norm_num
lemma schlafliDet_334_pos : 0 < schlafliDet 3 3 4 := by rw [schlafliDet_334]; norm_num
lemma schlafliDet_433_pos : 0 < schlafliDet 4 3 3 := by rw [schlafliDet_433]; norm_num

```



```

lemma schlafliDet_343_pos : 0 < schlafliDet 3 4 3 := by rw [schlafliDet_343]; norm_num

lemma schlafliDet_335_pos : 0 < schlafliDet 3 3 5 := by
  rw [schlafliDet_335]
  have : 0 < 7 - 3 *  $\sqrt{5}$  := by linarith [three_mul_sqrt5_lt_seven]
  positivity

lemma schlafliDet_533_pos : 0 < schlafliDet 5 3 3 := by
  rw [schlafliDet_533]
  have : 0 < 7 - 3 *  $\sqrt{5}$  := by linarith [three_mul_sqrt5_lt_seven]
  positivity

lemma schlafliDet_434_nonpos : schlafliDet 4 3 4  $\leq$  0 := by
  rw [schlafliDet_434]

lemma schlafliDet_353_neg : schlafliDet 3 5 3 < 0 := by
  rw [schlafliDet_353]
  have : 3 - 2 *  $\sqrt{5}$  < 0 := by linarith [two_mul_sqrt5_gt_three]
  exact div_neg_of_neg_of_pos this (by positivity)

lemma schlafliDet_435_neg : schlafliDet 4 3 5 < 0 := by
  rw [schlafliDet_435]
  have : 1 -  $\sqrt{5}$  < 0 := by linarith [sqrt5_bounds.1]
  exact div_neg_of_neg_of_pos this (by positivity)

lemma schlafliDet_534_neg : schlafliDet 5 3 4 < 0 := by
  rw [schlafliDet_534]
  have : 1 -  $\sqrt{5}$  < 0 := by linarith [sqrt5_bounds.1]
  exact div_neg_of_neg_of_pos this (by positivity)

lemma schlafliDet_535_neg : schlafliDet 5 3 5 < 0 := by
  rw [schlafliDet_535]
  have : 14 - 10 *  $\sqrt{5}$  < 0 := by
    have : 14 < 10 *  $\sqrt{5}$  := by nlinarith [sqrt5_bounds.1]
    linarith
  exact div_neg_of_neg_of_pos this (by positivity)

```

A.43 PlatonicSolids/Regular4.lean

The six regular convex 4-polytopes (28 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.PlatonicPair

/-!
Paper §4, Group I. The six regular convex 4-polytopes.
-/

/-- Schläfli triples of the six regular convex 4-polytopes. -/
def IsRegular4Polytope (p q r : ℕ) : Prop :=
  (p = 3 ∧ q = 3 ∧ r = 3) ∨
  (p = 4 ∧ q = 3 ∧ r = 3) ∨
  (p = 3 ∧ q = 3 ∧ r = 4) ∨
  (p = 3 ∧ q = 4 ∧ r = 3) ∨
  (p = 5 ∧ q = 3 ∧ r = 3) ∨
  (p = 3 ∧ q = 3 ∧ r = 5)

theorem IsRegular4Polytope.ge_three {p q r : ℕ} (h : IsRegular4Polytope p q r) :
  3 ≤ p ∧ 3 ≤ q ∧ 3 ≤ r := by

```

```

rcases h with ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
| ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ <;> decide

theorem IsRegular4Polytope.platonic {p q r : ℕ} (h : IsRegular4Polytope p q r) :
  IsPlatonicPair p q ∧ IsPlatonicPair q r := by
  rcases h with ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ <;> simp [IsPlatonicPair]

```

A.44 PlatonicSolids/Classification.lean

$\Delta > 0$ (equivalently M positive definite) iff one of the six (84 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/

import PlatonicSolids.Angles
import PlatonicSolids.CompatibleTriples
import PlatonicSolids.D3IffPlatonic
import PlatonicSolids.Regular4
import PlatonicSolids.SignTable
import PlatonicSolids.Sylvester
import PlatonicSolids.TrailingMinor

/-!
Paper §4. The six regular convex 4-polytopes are exactly the
Platonic-compatible triples with  $\Delta > 0$ , equivalently those whose
Schläfli Gram matrix is positive definite.
-/

open Real Matrix

/-- Palomar compared 4D theorem: Platonic cell and vertex figure with
 $\Delta > 0$  force one of the six regular convex 4-polytopes. -/
theorem regular_polychora_classification (p q r : ℕ)
  (h_cell : IsPlatonicPair p q)
  (h_vf : IsPlatonicPair q r)
  (h_det : 0 < schlaflDet p q r) :
  IsRegular4Polytope p q r := by
  rcases platonic_compatible_eleven h_cell h_vf with
  ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  · exact .inl ⟨rfl, rfl, rfl⟩
  · exact .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · exact .inr <| .inr <| .inr <| .inr <| .inr ⟨rfl, rfl, rfl⟩
  · exact .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · exact .inr <| .inl ⟨rfl, rfl, rfl⟩
  · exact (not_lt_of_ge schlaflDet_434_nonpos h_det).elim
  · exact (lt_asymm schlaflDet_435_neg h_det).elim
  · exact (lt_asymm schlaflDet_353_neg h_det).elim
  · exact .inr <| .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · exact (lt_asymm schlaflDet_534_neg h_det).elim
  · exact (lt_asymm schlaflDet_535_neg h_det).elim

theorem schlaflDet_pos_of_regular {p q r : ℕ} (h : IsRegular4Polytope p q r) :
  0 < schlaflDet p q r := by
  rcases h with ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  · exact schlaflDet_333_pos
  · exact schlaflDet_433_pos
  · exact schlaflDet_334_pos
  · exact schlaflDet_343_pos

```

```

· exact schlaflDet_533_pos
· exact schlaflDet_335_pos

theorem regular_of_posDef {p q r : ℕ} (hp : 3 ≤ p) (hq : 3 ≤ q) (hr : 3 ≤ r)
  (h : (schlaflGram p q r).PosDef) :
  IsRegular4Polytope p q r := by
  obtain ⟨_, _, h3, h4⟩ := leadingMinors_pos_of_posDef h
  have hcell : IsPlatonicPair p q := (leadingMinor_three_pos_iff hp hq).mp h3
  have hvf : IsPlatonicPair q r := by
    have hs := h.submatrix (Fin.succ_injective (n := 3))
    have : 0 < (trailing3 p q r).det := by
      simpa [trailing3] using PosDef.det_pos hs
    exact (trailing3_pos_iff hq hr).mp this
  have hdet : 0 < schlaflDet p q r := by
    rw [← schlaflGram_det]
    exact PosDef.det_pos h
  exact regular_polychora_classification p q r hcell hvf hdet

theorem posDef_of_regular {p q r : ℕ} (h : IsRegular4Polytope p q r) :
  (schlaflGram p q r).PosDef := by
  obtain ⟨hp, hq, _hr⟩ := h.ge_three
  obtain ⟨hcell, _hvf⟩ := h.platonic
  refine schlaflGram_posDef_of_minors ?_ ?_ ?_ ?_
  · rw [leadingMinor_one]; norm_num
  · rw [leadingMinor_two]; exact sq_pos_of_pos (sin_angle_pos hp)
  · exact (leadingMinor_three_pos_iff hp hq).mpr hcell
  · rw [schlaflGram_det]; exact schlaflDet_pos_of_regular h

/-- Finite regular convex 4-polytopes are exactly the triples `{p,q,r}`
with `p,q,r ≥ 3` whose Schläfli Gram matrix is positive definite. -/
theorem regular_polychora_iff {p q r : ℕ} (hp : 3 ≤ p) (hq : 3 ≤ q) (hr : 3 ≤ r) :
  (schlaflGram p q r).PosDef ↔ IsRegular4Polytope p q r :=
  ⟨regular_of_posDef hp hq hr, posDef_of_regular⟩

```

A.45 Challenge.lean

Palomar statement of record (deliberate sorries) (158 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic
import Mathlib.Algebra.Category.ModuleCat.Abelian
import Mathlib.Algebra.Category.ModuleCat.Colimits
import Mathlib.Algebra.Homology.QuasiIso
import Mathlib.AlgebraicTopology.SimplicialSet.Boundary
import Mathlib.AlgebraicTopology.SimplicialSet.TopAdj
import Mathlib.AlgebraicTopology.SingularHomology.Basic
import Mathlib.LinearAlgebra.Matrix.Notation
import Mathlib.Tactic

/-!
# Palomar statement of record (Platonic solids, regular polychora,
# and the simplicial-circle comparison)

```

This module states the **compared Palomar family** and the Mathlib type surface it depends on. Challenge may use ``sorry``; Solution supplies the proofs.

The selected theorems are:

1. ``platonic_solids_3d`` - Euler $\backslash(V-E+F=2\backslash)$ plus regularity incidence $\backslash(pF=2E\backslash)$, $\backslash(qV=2E\backslash)$ force a Platonic pair $\backslash((p,q)\backslash)$.
2. ``edges_pos_of_regular`` - the same identities algebraically force $\backslash(E>0\backslash)$; non-degeneracy is a theorem, not an axiom.
3. ``regular_polychora_classification`` - Platonic cells and vertex figures with positive Schläfli Gram determinant force one of the six regular convex 4-polytopes (the PosDef equivalence is ``regular_polychora_iff``, not compared).
4. ``simplicialSingularComparison_boundaryTwo_quasiIso`` - the canonical realization--singular adjunction-unit map is a quasi-isomorphism for the simplicial circle ``∂Δ[2]``.

How to read this file

The definitions below are the vocabulary of the claim. A reader who wants to check *what* has been proved should read this file and need not read anything else. ``Solution.lean`` imports the sorry-free development in ``PlatonicSolids.lean``. Each supporting lemma lives in a section-sized module under ``PlatonicSolids/``, meant to be read next to the matching paragraph of ``PlatonicSolids.md``.
-/

```
open scoped Matrix
open Real Matrix
open AlgebraicTopology CategoryTheory HomologicalComplex
open scoped Simplicial
```

noncomputable section

/-- The 5 Platonic pairs (p, q):

```
(3, 3): Tetrahedron
(3, 4): Octahedron
(4, 3): Cube
(3, 5): Icosahedron
(5, 3): Dodecahedron -/
```

def IsPlatonicPair (p q : ℕ) : Prop :=

```
(p = 3 ∧ q = 3) ∨
(p = 3 ∧ q = 4) ∨
(p = 4 ∧ q = 3) ∨
(p = 3 ∧ q = 5) ∨
(p = 5 ∧ q = 3)
```

/-- Canonical bilinear form of the rank-4 Schläfli Coxeter system $\{p, q, r\}$. -/

noncomputable def schlaflGram (p q r : ℕ) : Matrix (Fin 4) (Fin 4) ℝ :=

```
!![1, -cos (π / p), 0, 0;
   -cos (π / p), 1, -cos (π / q), 0;
   0, -cos (π / q), 1, -cos (π / r);
   0, 0, -cos (π / r), 1]
```

/-- $\Delta(p, q, r) = \sin^2(\pi/p) \sin^2(\pi/r) - \cos^2(\pi/q)$. -/

noncomputable def schlaflDet (p q r : ℕ) : ℝ :=

```
sin (π / p) ^ 2 * sin (π / r) ^ 2 - cos (π / q) ^ 2
```

/-- The 6 regular convex 4-polytopes (Schläfli symbols {p, q, r}):

```
{3, 3, 3}: 5-cell (4D Simplex)
{4, 3, 3}: 8-cell (Tesseract / Hypercube)
{3, 3, 4}: 16-cell (Cross-polytope)
{3, 4, 3}: 24-cell (Exceptional 4D regular polytope)
{5, 3, 3}: 120-cell
{3, 3, 5}: 600-cell -/
```

def IsRegular4Polytope (p q r : ℕ) : Prop :=

```
(p = 3 ∧ q = 3 ∧ r = 3) ∨
(p = 4 ∧ q = 3 ∧ r = 3) ∨
```

```

(p = 3 ∧ q = 3 ∧ r = 4) ∨
(p = 3 ∧ q = 4 ∧ r = 3) ∨
(p = 5 ∧ q = 3 ∧ r = 3) ∨
(p = 3 ∧ q = 3 ∧ r = 5)

/-- Theorem: A Platonic configuration cannot have zero edges.
The topological Euler characteristic ( $\chi = 2$ ) and regularity conditions ( $p, q \geq 3$ )
algebraically force  $E \geq 1$  without requiring any geometric non-degeneracy axiom. -/
theorem edges_pos_of_regular
(V E F p q : ℕ)
(hp : 3 ≤ p) (hq : 3 ≤ q)
(h_edges_faces : p * F = 2 * E)
(h_edges_verts : q * V = 2 * E)
(h_euler : V + F = E + 2) :
0 < E := by
sorry

/-- The 3D Main Theorem:
Any polyhedron satisfying Euler's formula ( $V - E + F = 2$ )
and regularity incidence relations ( $pF = 2E, qV = 2E$ )
must have (p, q) belonging to the 5 Platonic solids. -/
theorem platonic_solids_3d
(V E F p q : ℕ)
(hp : 3 ≤ p) (hq : 3 ≤ q)
(h_edges_faces : p * F = 2 * E)
(h_edges_verts : q * V = 2 * E)
(h_euler : V + F = E + 2) :
IsPlatonicPair p q := by
sorry

/-- The compared 4D classification: a triple {p, q, r} whose 3D cells {p, q}
and vertex figures {q, r} are Platonic and whose Schläfli determinant
satisfies  $\Delta > 0$  is one of the 6 regular convex 4-polytopes.
(Not an if-and-only-if; that is `regular_polychora_iff`.) -/
theorem regular_polychora_classification (p q r : ℕ)
(h_cell : IsPlatonicPair p q)
(h_vf : IsPlatonicPair q r)
(h_det : 0 < schlaefliDet p q r) :
IsRegular4Polytope p q r := by
sorry

/-! ### Canonical simplicial--singular comparison for  $\partial\Delta[2]$  -/

variable {k : Type} [Ring k]

/-- The simplicial set underlying the boundary of the standard
two-simplex. -/
abbrev boundaryTwoSSet : SSet.{0} :=
(SSet.boundary 2 :
(SSet.stdSimplex.obj [2] : SSet.{0}).Subcomplex)

/-- Singular chains on the geometric realization of  $X$ . -/
abbrev singularChainsOfRealization
(R : ModuleCat.{0} k) (X : SSet.{0}) :
ChainComplex (ModuleCat.{0} k) ℕ :=
((singularChainComplexFunctor (ModuleCat.{0} k)).obj R).obj |X|

/-- The canonical chain map induced by the
realization--singular adjunction unit. -/
def simplicialSingularComparison
(R : ModuleCat.{0} k) (X : SSet.{0}) :
X.chainComplex R → singularChainsOfRealization R X :=
SSet.chainComplexMap (sSetTopAdj.unit.app X) R

```

```

/-- The canonical comparison is a quasi-isomorphism for the simplicial
circle  $\partial\Delta[2]$ . -/
theorem simplicialSingularComparison_boundaryTwo_quasiIso
  (R : ModuleCat.{0} k) :
  QuasiIso (simplicialSingularComparison R boundaryTwoSSet) := by
  sorry

```

A.46 Solution.lean

Palomar solution module: imports PlatonicSolids (25 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/
import PlatonicSolids

/-!
# Solutions to the Challenge

The declarations of `Challenge.lean`, proved. Importing `PlatonicSolids`
supplies `platonic_solids_3d`, `edges_pos_of_regular`,
`regular_polychora_classification`,
`simplicialSingularComparison_boundaryTwo_quasiIso`, and the definitions
they depend on (`IsPlatonicPair`, `IsRegular4Polytope`, `schlafliGram`,
`schlafliDet`, `boundaryTwoSSet`, `singularChainsOfRealization`,
`simplicialSingularComparison`), with the same names and types as in the
Challenge module. Comparator compares those declarations. The original
three combinatorial types are unchanged; the fourth theorem is the
canonical adjunction-unit comparison for  $\partial\Delta[2]$ , not a general
finite-cellulation result.

The proofs are split across section-sized modules under `PlatonicSolids/`.
There is no `sorry` in those files.
-/

```